

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



SPECIALIZED PROJECT

REPORT

aBridgeAI: An AI-Powered Career Readiness Learning Management System

Advisor(s): Assoc. Prof. Vo Thi Ngoc Chau, PhD

Student(s): Le Thanh Duy 2352178

Vo Hoang Ngan 2353373

Nguyen Hoang Nam 2252502

HO CHI MINH CITY, SEPTEMBER 2026

Instructor's Signature

Approved by:

Signature

Date

Assoc. Prof. Vo Thi Ngoc Chau, Ph.D.

Associate Professor

Faculty of Computer Science and Engineering

Ho Chi Minh City University of Technology (HCMUT)

Declaration of Authenticity

We hereby certify that this project is our original work, conducted under the supervision of **Assoc. Prof. Vo Thi Ngoc Chau** at the Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (VNU-HCM):

- All external sources, data, and literature used in this report have been appropriately acknowledged and cited.
- We understand that any form of plagiarism will result in disciplinary action according to the university's regulations.
- The University shall not be held responsible for any copyright violations arising from this research.

Ho Chi Minh City, May 2026

Le Thanh Duy

Vo Hoang Ngan

Nguyen Hoang Nam

Acknowledgement

This project would not have been possible without the support and contribution of many individuals.

First and foremost, we would like to express our deepest gratitude to our supervisor, **Assoc. Prof. Vo Thi Ngoc Chau**. Her profound expertise, insightful feedback, and unwavering support have been the cornerstone of this research. We are truly honored to have been guided by her intellectual mentorship throughout the development process.

We also extend our sincere appreciation to the faculty members of the Computer Science and Engineering department. Their constructive comments and keen insights into multimodal models have significantly refined our methodology and broadened our academic perspective.

Finally, a special thank you goes to our families and friends. Their constant encouragement and belief in our potential have been our greatest source of motivation and resilience, especially during the most challenging phases of this work.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Goals	2
1.3	Scope	3
1.4	Significance of the Project	4
1.4.1	Scientific Perspectives	4
1.4.2	Practical Perspectives	5
1.5	Report Structure	6
2	Related Works	8
2.1	The Bottlenecks of Traditional Spaced Repetition (SRS) Algorithms	8
2.2	Methodological Constraints in Retrieval Practice Applications	10
2.3	The Deficit in Contextual and Cognitive Adaptivity	12
2.4	The Path Toward aBridgeAI	16
3	Theoretical Background	17
3.1	Cognitive Foundations of Learning	17
3.1.1	Cognitive Load Theory (CLT)	17
3.1.2	Handling Overly Complex Lessons	18
3.2	Algorithmic Learner Modeling and SR	19
3.2.1	The SM-2 Algorithm	19
3.2.2	Memory Models: Stability and Retrievability	22
3.2.3	Knowledge Tracing and Learner Modeling	24
3.3	Core Artificial Intelligence Technologies	28
3.3.1	LLMs in Education	28
3.3.2	Relational Reasoning: Overcoming Document Silos	29
3.3.3	Vector Databases and High-Dimensional Search	30
3.4	Technical Foundations and System Design Rationale	30
3.4.1	Component-Based Interface Design and Cognitive Load	30
3.4.2	Integration of the Adaptive Loop	31
3.5	Summary about the Theoretical Background	31

4	System Requirements and Analysis	33
4.1	System Overview	33
4.1.1	Purpose	33
4.1.2	Stakeholders and Users of the System	33
4.2	Business Process	35
4.3	Requirements	36
4.3.1	User Requirements	36
4.3.2	Functional Requirements	38
4.3.3	Non-Functional Requirements	43
4.3.4	Data Requirements	44
4.4	System Modeling	50
4.4.1	Overall Use Case Diagram	50
4.4.2	Detailed Use Case Diagrams and Use Case Specifications	51
4.4.3	Activity Diagrams	60
5	System Design	63
5.1	Technology Stack Selection	63
5.1.1	Backend Framework	63
5.1.2	Frontend Framework	64
5.1.3	Real-Time Communication	65
5.1.4	Primary Database & Vector Search	66
5.1.5	Graph Database	66
5.1.6	Caching & Task Queue	67
5.1.7	Final Stack Summary	69
5.2	System Architecture	69
5.2.1	Actors and External Environment	69
5.2.2	Internal System Components	70
5.2.3	System Boundary Diagram	72
5.2.4	Deployment Architecture	72
5.3	Algorithm Design	73
5.3.1	Hybrid SM-2 and Q Derivation	73
5.3.2	Individualised Learner Modeling	76
5.3.3	Student Performance Metrics	78
5.4	API Design	81
5.4.1	Architecture & Design Philosophy	81
5.4.2	Authentication Strategy	83

5.4.3	Error Handling	85
5.4.4	Endpoint Surface (overview)	85
5.4.5	Pagination, Filtering and Idempotency	86
5.4.6	API Design Summary	87
5.5	Class Diagrams	88
5.6	Sequence Diagrams	89
5.7	Database Design	92
5.7.1	Entity-Relationship Diagram	93
5.7.2	Logical Database Design	94
5.7.3	Physical Database Design	94
5.8	UI/UX Design	95
5.8.1	Design Philosophy	95
5.8.2	Design Tokens	96
5.8.3	Key Screens	97
5.8.4	Navigation Architecture	99
5.9	Site Map	99
5.10	Test Case Design	101
6	Implementation and Testing	103
6.1	Development Environment and Toolchain	103
6.2	Project Structure	104
6.3	Authentication and Identity	106
6.4	Material Ingestion Pipeline	107
6.5	AI Generation Pipelines	109
6.5.1	Hybrid Retrieval	109
6.5.2	Quiz Generation Pipeline	110
6.5.3	Interview Generation Pipeline	111
6.5.4	LLM Gateway and Audit	112
6.6	Spaced Repetition Implementation	112
6.7	Access Control	114
6.8	Testing	116
6.8.1	Authentication and Access Control	116
6.8.2	Course and Material Management	117
6.8.3	Quiz Generation and Taking	117
6.8.4	CI Pipeline	118

7	Evaluation	119
7.1	Frontend Evaluation	119
7.2	Backend Performance	121
7.2.1	Test Environment	121
7.2.2	Test Scenarios	121
7.2.3	Results	121
7.2.4	Analysis against NFR Targets	122
7.3	AI Model Comparison for Quiz Generation	122
7.3.1	Methodology	122
7.3.2	Quality Results	124
7.3.3	Latency and Cost	124
7.3.4	Failure Analysis	125
7.3.5	Recommendation	125
8	Conclusion	126
8.1	Achievements	126
8.2	Limitations	126
8.3	Future Work	127
8.4	Concluding Remarks	130
	Appendices	136
A	Functional Requirements Specification	136
B	Non-Functional Requirements Specification	180
C	Use Case Specification	185
C.1	Authentication and Access Control Module	185
C.2	Organization and User Management Module	191
C.3	Course and Material Management Module	200
C.4	Learning Loop and Adaptive Quiz Module	208
C.5	Application Loop and Mock Interview Module	218
C.6	Analytics, Reporting and Notifications Module	225
C.7	Learning Programs and Path Governance Module	229
C.8	Platform Configuration and Policy Documents Module	236

D	Activity Diagrams Specification	242
D.1	Authentication Flow	242
D.2	Smart Content Ingestion Flow	243
D.3	Human-in-the-Loop Quiz Generation Flow	243
D.4	Context-Aware Mock Interview Flow	244
E	Sequence Diagrams Specification	246
E.0.1	Authentication	246
E.0.2	Smart Content Ingestion	248
E.0.3	Human-in-the-Loop Quiz Generation	248
E.0.4	Context-Aware Mock Interview Session	249
F	API Designs Specification	252
F.1	Architecture & Design Philosophy	252
F.2	Authentication Strategy	253
F.3	Multi-Factor Authentication	255
F.4	Error Handling	256
F.5	Pagination, Filtering and Idempotency	257
F.6	Endpoint Catalogue	258
F.6.1	Identity — Authentication & Profile	258
F.6.2	Access Control — Roles, Grants & Organisations	259
F.6.3	Courses — Learner	262
F.6.4	Courses — Authoring (Teacher)	262
F.6.5	Courses — Department & Administration	264
F.6.6	Materials	266
F.6.7	Quizzes	268
F.6.8	Interviews	270
F.6.9	Progress	272
F.6.10	Career Paths	273
F.6.11	Spaced Repetition	274
F.6.12	Notifications	275
F.6.13	Admin Operations	276
F.6.14	Health	277
F.7	API Design Summary	278
F.8	Swagger UI Screenshot Catalogue	279

G Site Map Specification	296
G.1 Public Routes (Unauthenticated)	296
G.2 Student Routes	296
G.3 Teacher / Instructor Routes	298
G.4 Educational Manager Routes	299
G.5 IT Administrator Routes	300
G.6 Route Summary	301
H UI/UX Design Specification	302
H.1 UI/UX Design	302
H.1.1 Design Philosophy	302
H.1.2 Internationalisation (i18n)	302
H.1.3 Design Rules & Constraints	303
H.1.4 Key Screen Designs	307
H.1.5 Navigation Architecture	320
H.1.6 Design System Summary	321
I Test Case Specifications	324

List of Figures

3.1	Forgetting curves applied in SR learning. Source: [36].	27
4.1	Overall Use Case Diagram of the aBridgeAI System	51
4.2	Detailed Use Case Diagram: Authentication and Access Control Module . .	52
4.3	Detailed Use Case Diagram: Organization and User Management Module .	53
4.4	Detailed Use Case Diagram: Course and Material Management Module . .	55
4.5	Detailed Use Case Diagram: Learning Loop and Adaptive Quiz Module . .	57
4.6	Detailed Use Case Diagram: Application Loop and Mock Interview Module	59
4.7	Detailed Use Case Diagram: Analytics, Reporting and Notifications Module	60
4.8	Activity Diagram: HITL Quiz Generation and Execution	61
4.9	Activity Diagram: Smart Content Ingestion and AI Parsing	61
5.1	aBridgeAI System Architecture Diagram	72
5.2	Live Swagger UI at /docs (rendered from the FastAPI /openapi.json): title, authorise control, and the <code>auth</code> audience-tag section.	83
5.3	UML Class Diagram representing the core Domain Entities of aBridgeAI .	89
5.4	Sequence Diagram: Authentication	90
5.5	Sequence Diagram: Smart Content Ingestion	91
5.6	Sequence Diagram: HITL Quiz Generation and Execution	92
5.7	Entity-Relationship Diagram	93
5.8	Logical Database Design	94
5.9	Physical Database Design	94
5.10	Application sitemap — colour-coded by role group: grey (public), blue (student), teal (instructor), violet (educational manager), amber (administrator). Dynamic URL segments are highlighted in amber as <code>\$param</code>	100
6.1	Top-level directory layout of the aBridgeAI backend.	105
7.1	Lighthouse audit results for the Login page.	119
7.2	Lighthouse audit results for the Student Dashboard.	120
8.1	Gantt chart of planned future work over 15 weeks.	130
D.1	Activity Diagram: User Login with 2FA	242
D.2	Activity Diagram: Smart Content Ingestion and AI Parsing	243
D.3	Activity Diagram: HITL Quiz Generation and Execution	244
D.4	Activity Diagram: Context-Aware Mock Interview	245
E.1	Sequence Diagram: Authentication	247
E.2	Sequence Diagram: Smart Content Ingestion	248

E.3	Sequence Diagram: HITL Quiz Generation and Execution	249
E.4	Sequence Diagram: Mock Interview (Phase 1)	250
E.5	Sequence Diagram: Mock Interview (Phase 2)	250
E.6	Sequence Diagram: Mock Interview (Phase 3)	251
F.1	Swagger UI — Authentication endpoints. Google OAuth login/callback, re- fresh, logout, and TOTP-based MFA enrolment, challenge, verify, recovery- code regeneration, and disable.	280
F.2	Swagger UI — MFA management. Self-service multi-factor authentication endpoints under <code>/auth/me/mfa</code>	280
F.3	Swagger UI — Administration — part 1. First half of the administrative endpoint surface (user disable/enable, audit subset).	281
F.4	Swagger UI — Administration — part 2. Remaining administrative end- points (continued from previous figure).	282
F.5	Swagger UI — Access control & roles. Role assignments, permission grants, organisation-scoped role queries.	283
F.6	Swagger UI — Organisations. Organisational unit CRUD, hierarchy walk, member listing.	284
F.7	Swagger UI — Courses — learner. Published-course catalogue, enrolled- course lookup, and lesson tree for the calling student.	284
F.8	Swagger UI — Courses — authoring. Instructor course creation, mod- ule/lesson editing, publish/archive lifecycle, roster, prerequisites, item re- ordering.	285
F.9	Swagger UI — Courses — department & administration. Department- scoped course assignment to teachers, cross-organisation course listing, soft- delete restoration, and audit views.	286
F.10	Swagger UI — Materials. Two-phase upload protocol (presigned init / mul- tipart parts / complete), processing summaries, learner-facing presigned streaming URLs.	286
F.11	Swagger UI — Quizzes. Authoring (draft/generate/review/publish) and learner (start attempt, submit, results) endpoints; AI generation through <code>generation-runs/{run_id}</code> polling.	287
F.12	Swagger UI — Interviews. Mock-interview authoring (slots, question banks, regeneration) and learner endpoints (start/submit/feedback).	288
F.13	Swagger UI — Enrollments. Learner self-enrolment and department- assigned enrolment management.	288

F.14	Swagger UI — Progress. Learner progress tracking and instructor progress dashboards.	289
F.15	Swagger UI — Career paths. Published career-path catalogue (learner) and authoring routes (instructor/admin).	289
F.16	Swagger UI — Notifications & stats. Learner notification inbox (cursor-paginated by <code>created_at</code>) and admin platform-wide statistics.	290
F.17	Swagger UI — Audit & processing. Cross-feature audit log queries and AI-processing run inspection (admin).	290
F.18	Swagger UI — Spaced repetition. Learner card-due queue, review submission, lesson-unlock checks, and instructor analytics.	291
F.19	Swagger UI — AI costs & health. AI-cost telemetry endpoints and Kubernetes liveness/readiness probes (<code>/healthz</code> , <code>/readyz</code>).	292
F.20	Swagger UI — Schemas — part 1. Pydantic v2 schemas exposed in <code>OpenAPI components.schemas</code> (first third of 223 schemas).	293
F.21	Swagger UI — Schemas — part 2. Pydantic v2 schemas (continued).	294
F.22	Swagger UI — Schemas — part 3. Pydantic v2 schemas (final third).	295
H.1	Landing Page — Marketing entry point with hero section and call-to-action.	308
H.2	Login — Google SSO entry point.	309
H.3	Student Dashboard — Overview of enrolled courses, due reviews, and recent activity.	310
H.4	Course Catalog — Search, filter and enrollment status across the full course library.	311
H.5	Course View — Curriculum accordion with lesson state icons.	312
H.6	Quiz Session — Timed multiple-choice interface with question navigation.	313
H.7	Teacher Dashboard — Operational overview with course health stats and quick course creation.	314
H.8	Course Manage — Module and lesson tree with drag-handle reordering and inline quick-add.	315
H.9	Lesson Editor — Markdown content editor with attached resources and right-hand settings panel.	316
H.10	Quiz Editor — Question list with bulk timing controls and an AI-generation side panel.	317
H.11	AI Quiz Generation modal — Source-lesson selection, question count, mode and append/replace controls.	318

H.12 Admin Processing Queue — Background-job visibility with retry actions for failed runs.	319
H.13 Admin AI Costs — Token consumption and spend across pipeline roles and stages.	320

List of Tables

2.1	Comparative analysis of educational technologies across pedagogical dimensions.	15
3.1	Quality response scale used in the SM-2 SR algorithm [5].	21
4.1	Core functional requirements by module.	39
4.2	Representative non-functional requirements by domain.	44
4.3	Representative Use Case Specification: Manage Tenant Organisations . . .	53
4.4	Representative Use Case Specification: Upload and Process Learning Materials	55
4.5	Representative Use Case Specification: Configure and Take Adaptive Quiz	57
5.1	Backend Framework Comparison	63
5.2	Frontend Framework Comparison	64
5.3	Real-Time Communication Comparison	65
5.4	Primary Database & Vector Search Comparison	66
5.5	Graph Database Comparison	67
5.6	Caching & Task Queue Comparison	68
5.7	Selected Technology Stack — aBridgeAI	69
5.8	Q value assignment rules based on correctness, hint usage, and time ratio ρ . Boundaries at $\rho = 1$ and $\rho = 2$ correspond to the teacher-defined expected response time and twice that value respectively; the boundary at $\rho = 0.5$ corresponds to half the expected time.	75
5.9	Summary of student performance metrics and their roles within the aBridgeAI learner modeling layer.	80
5.10	Token pair issued on successful Google login — backend storage.	83
5.11	Feature breakdown of the API surface.	85
5.12	API design summary — strategies and rationale.	87
5.13	Core design tokens.	96
5.14	Representative Test Cases: Authentication & Access Control	101
5.15	Representative Test Cases: Learning Loop & Adaptive Quiz	101
5.16	Representative Test Cases: Application Loop & Mock Interview	102
6.1	Test Execution Summary by Module (Iteration 1)	116
6.2	Test Results: Authentication and Access Control	116
6.3	Test Results: Course and Material Management	117
6.4	Test Results: Quiz Generation and Taking	117

7.1	Lighthouse Audit Results (Mobile, Simulated Throttling)	119
7.2	Backend Load Test Results (100 VUs, 30s sustained)	122
7.3	Quiz Generation Quality by Model (judge: <code>gpt-5-chat-latest</code> , reference-guided CoT)	124
7.4	Quiz Generation Latency and Cost by Model	124
8.1	Planned future work with assignment and timeline (15 weeks total).	128
A.1	Functional Requirements for Authentication & Access Control	136
A.2	Functional Requirements for Organization & User Management	138
A.3	Functional Requirements for Course & Material Management	144
A.4	Functional Requirements for Learning Loop & Adaptive Quiz	151
A.5	Functional Requirements for the Application Loop & Mock Interview	160
A.6	Functional Requirements for Analytics, Reporting & Notifications	165
A.7	Functional Requirements for Learning Programs & Path Governance	169
A.8	Functional Requirements for Platform Configuration & Policy Documents	176
B.1	Non-Functional Requirements for Performance & Latency	180
B.2	Non-Functional Requirements for Scalability & Reliability	181
B.3	Non-Functional Requirements for Security & Compliance	182
B.4	Non-Functional Requirements for Usability & Accessibility	184
C.1	Use Case Specification: Authenticate and Log Out	185
C.2	Use Case Specification: Manage Multi-Factor Authentication	187
C.3	Use Case Specification: Access Protected Features	189
C.4	Use Case Specification: Manage User Account Access	190
C.5	Use Case Specification: Manage Tenant Organisations	192
C.6	Use Case Specification: Manage Users and Memberships	193
C.7	Use Case Specification: Manage Career Pathways and Teacher Assignments	194
C.8	Use Case Specification: Organise Faculties and Staff Courses	196
C.9	Use Case Specification: Manage Course Enrolment and Invitation Codes	198
C.10	Use Case Specification: Manage Course Structure and Lifecycle	200
C.11	Use Case Specification: Upload and Process Learning Materials	201
C.12	Use Case Specification: Browse Courses and Access Materials	203
C.13	Use Case Specification: Curate Ingested Content and Reuse a Course	204
C.14	Use Case Specification: Participate in a Lesson Discussion	206
C.15	Use Case Specification: Author and Review Quiz Questions	208
C.16	Use Case Specification: Configure and Take Adaptive Quiz	209
C.17	Use Case Specification: View Learning Loop Dashboards and Notifications	211

C.18 Use Case Specification: Grade Open Responses and Maintain the Gradebook	212
C.19 Use Case Specification: Administer Assessment Delivery and Integrity . . .	215
C.20 Use Case Specification: Configure Mock Interview	218
C.21 Use Case Specification: Take Mock Interview	219
C.22 Use Case Specification: Review Interview Result	221
C.23 Use Case Specification: Curate the Interview Question Bank	222
C.24 Use Case Specification: Review the Interview Gap Report	224
C.25 Use Case Specification: Manage Notifications	225
C.26 Use Case Specification: View Progress and Career Readiness Reports . . .	226
C.27 Use Case Specification: Monitor Platform Operations	227
C.28 Use Case Specification: Define and Publish a Learning Programme	229
C.29 Use Case Specification: Enrol Students onto a Learning Programme	231
C.30 Use Case Specification: Select and Change Career Pathway	233
C.31 Use Case Specification: Tune Runtime Configuration with Preview and Rollback	237
C.32 Use Case Specification: Publish and Read Policy Documents	239
F.1 Token Pair — backend storage	254
F.2 MFA endpoints	255
F.3 Common HTTP status codes	257
F.4 Identity endpoints (Google OAuth, refresh, logout, profile).	259
F.5 Access-control endpoints (admin-scoped).	260
F.6 Course-learner endpoints (the calling user's own enrolments).	262
F.7 Course-authoring endpoints (teacher role).	263
F.8 Course department-assignment and administration endpoints.	265
F.9 Materials endpoints.	266
F.10 Quizzes endpoints.	268
F.11 Interview endpoints (AI mock-interview surface).	270
F.12 Progress endpoints.	273
F.13 Career-path endpoints.	274
F.14 Spaced-repetition endpoints (FSRS scheduler).	274
F.15 Notification endpoints.	275
F.16 Admin operations endpoints.	276
F.17 Health-check endpoints.	277
F.18 API design summary — chosen strategies and their rationale.	278
G.1 Public (unauthenticated) routes.	296

G.2	Student-facing routes.	296
G.3	Teacher and instructor routes.	298
G.4	Educational Manager routes.	299
G.5	IT Administrator routes.	300
G.6	Route count by role.	301
H.1	Colour System Rules	303
H.2	Typography Rules	304
H.3	Z-Index Layering Rules	306
H.4	Navigation Components by Context	321
H.5	Design System Tokens	322
I.1	Test Cases: Authentication & Access Control	324
I.2	Test Cases: Organization & User Management	327
I.3	Test Cases: Course & Material Management	330
I.4	Test Cases: Learning Loop & Adaptive Quiz	332
I.5	Test Cases: Application Loop & Mock Interview	334
I.6	Test Cases: Analytics, Reporting & Notifications	336
I.7	Test Cases: Non-Functional Requirements	338

1 Introduction

In chapter 1, the motivation, goals, and scopes of the project are illustrated. The outline of the report is also presented.

1.1 Motivation

In the evolving landscape of higher education and vocational training, the demand for job-ready graduates is experiencing rapid growth, catering to diverse industry requirements. However, as the gap between academic theory and practical application widens, students often find themselves spending considerable effort struggling to transition from theoretical learning to actual career readiness. This transition is further complicated by a modern cognitive crisis described as the "age of interruption," where the constant barrage of digital stimuli from smartphones and social media leads to "continuous partial attention". In this state, learners perpetually shift their focus across multiple stimuli without fully immersing themselves in any single task, resulting in a superficial understanding of information. Research indicates that students in the presence of technological distractors can often only maintain sustained focus for a mere six minutes before being diverted. Consequently, the inefficiency of conventional Learning Management Systems (LMS) is not merely a structural issue but a psychological one; "attentional overload" occurs when environmental demands exceed an individual's finite attentional capacity, hindering the deep processing required for career-critical skills[1].

By introducing an AI-powered Career Readiness Learning Management System, we aim to revolutionize the way educational institutions prepare students for the workforce. A core objective of aBridgeAI is to mitigate the risk of "AI as Autopilot," a failure mode where generative tools provide complete solutions that reduce "germane load"—the productive mental work needed to build mastery—and thus prevent actual learning. Instead of doing the thinking for the student, aBridgeAI utilizes Large Language Models (LLMs) to provide "fading support" scaffolding, which transitions learners from fully solved worked examples to independent practice as their competence grows. The system is designed to force "System-2 level engagement" by prompting students to explain their reasoning in steps, identify underlying assumptions, or solve problems using alternative methods. To further respect the constraints of human working memory, the platform employs "segmenting," breaking high-intrinsic-load curriculum into bite-size, learner-controlled segments. This ensures that the learner has sufficient time to organize and integrate information from one portion of the lesson before the next segment begins, preventing cognitive overload

during the acquisition of complex technical skills[2, 3].

Moreover, the development of such a system addresses the pressing need for automation and scalable assessment in the educational sector while actively countering "digital dementia." This term refers to the cognitive decline and memory loss caused by overreliance on digital devices to store information that would otherwise be held in long-term memory. aBridgeAI combats this by integrating "active recall" and hybrid spacing algorithms that identify the optimal intervals for review just before information is likely to be forgotten. As students gain expertise, the system dynamically fades guidance to avoid the "expertise reversal effect," where redundant instructional support becomes an "extraneous load" that interferes with the schemas already constructed by the learner. This automated and context-aware approach allows educators to track authentic progress and provide personalized remediation, ensuring that mastery is internalized rather than offloaded to a tool. Ultimately, aBridgeAI answers the critical need to guide learners from being merely "Book Smart" to becoming "Job-Ready" candidates whose professional competencies are automated and resilient[4, 5].

1.2 Goals

The primary goal of this project is to research, design, implement and evaluate aBridgeAI, an intelligent LMS that bridges the gap between theoretical knowledge acquisition and practical career readiness. To achieve this aim, the project focuses on the following objectives:

- **Developing a Smart Content Ingestion and Retrieval Pipeline:** To engineer an automated system capable of processing multimodal educational materials. The system will transcribe, extract, and index these materials into a vector database for subsequent retrieval tasks.
- **Developing an Adaptive Theoretical Assessment Module:** To build an AI-driven quiz generation engine that dynamically creates context-specific questions based strictly on the ingested curriculum. This goal includes designing an adaptive feedback loop that directs struggling students back to precise timestamps or document sections for targeted remediation.
- **Providing Students with a Context-Aware Mock Interview System:** To build a conversational AI interviewer utilizing LLMs and Retrieval-Augmented Generation (RAG). The core innovation lies in dynamically generating technical and

behavioral interview scenarios strictly grounded in both the teacher’s specific lecture content and the student’s personal resume, ensuring an authentic and highly relevant practical assessment.

- **Constructing a Comprehensive Analytics and Gap-Reporting Engine:** To develop an automated evaluation rubric that assesses technical accuracy, communication clarity, and problem-solving skills. The system will synthesize data from both the learning loop (quizzes) and the application loop (interviews) to generate a personalized "Gap Report," highlighting discrepancies between theoretical knowledge and practical articulation.

1.3 Scope

To ensure the feasibility of the aBridgeAI, the scope is constrained across five dimensions:

- **Technological Scope:** The system is developed as a responsive web application to ensure cross-platform compatibility. Based on the evaluation of real-time AI requirements, the technology stack encompasses React.js for the frontend, FastAPI (Python) for backend and AI orchestration, a hybrid database topology utilizing PostgreSQL (with pgvector) and MongoDB, LiveKit for WebRTC-based ultra-low latency audio streaming, and Google OAuth 2.0 / OpenID Connect for delegated authentication with in-platform second-factor (TOTP) enrolment and session management.
- **Temporal Scope:** The development, testing, and deployment lifecycle of this Minimum Viable Product (MVP) is strictly bounded within Semester 252, commencing in early February and concluding in June.
- **Spatial and Contextual Scope:** The system’s operational context is designed specifically for academic and structured learning environments. It targets higher education institutions, university departments, and coding bootcamps, focusing exclusively on bridging the gap between theoretical academic curricula and practical industry interview requirements.
- **Functional Scope:** The system’s core functionalities are limited to four main interactive pillars:
 1. *Smart Content Ingestion:* Parsing uploaded documents (PDF, DOCX) and media (MP4) to construct a localized Course Knowledge Graph.

2. *The Learning Loop*: Providing automated, adaptive quizzes based on the ingested content.
3. *The Application Loop*: Executing context-aware mock interviews supporting text-based and asynchronous/WebRTC voice-based interactions.
4. *Assessment Analytics*: Generating automated Gap Reports and tracking student performance matrices.

The following capabilities are explicit non-goals of the MVP and are excluded from this phase: synchronous human-to-human tutoring and live video conferencing; in-platform email/password registration, password reset, “remember me” device-trust, and per-device session pinning. Authentication is delegated exclusively to Google OAuth, and self-service password recovery is therefore handled by Google’s identity infrastructure rather than in-platform.

- **Data Scope**: The system is scoped to process, store, and manage standard educational materials, user profiles, and assessment logs. For AI operations, the system limits data processing to generating and storing vector embeddings (via pgvector) strictly derived from the institution’s uploaded materials to prevent LLM hallucinations. Live audio streams during voice interviews are processed transiently for Speech-to-Text (STT) and Text-to-Speech (TTS) purposes and are not persistently archived as raw audio files, thereby optimizing cloud storage overhead.

1.4 Significance of the Project

1.4.1 Scientific Perspectives

From a scientific standpoint, aBridgeAI makes several contributions to the intersection of educational technology, cognitive science, and applied machine learning.

The primary scientific contribution is the formalisation of an algorithmically derived quality response value for the SM-2 spaced repetition algorithm in institutional deployment contexts. Standard SM-2 relies on learner self-assessment as its sole input signal, an assumption that holds in solo study environments but breaks down when the quality score influences externally visible progression gates. This work derives Q from three objective, system-observable signals — response correctness, hint usage, and a dimensionless time ratio $\rho = T_{\text{actual}}/T_{\text{exp}}$ — replacing a subjective input with a deterministic computation grounded in response latency theory [6]. The threshold boundaries in the Q assignment rule are derived as natural multiples of the teacher-defined expected response time rather

than as independently calibrated constants, eliminating arbitrary parameterisation from the scheduling model.

A secondary contribution is the introduction of a calibration weight α applied asymmetrically to early EF increments, addressing the known cold-start overestimation problem in SM-2 initialisation. By dampening positive EF drift during the first three repetitions while applying negative increments at full magnitude, the model converges onto a student’s true retention profile faster than the standard initialisation assumption permits. This mechanism is formalised within the SM-2 framework without requiring the full parametric complexity of successor algorithms such as FSRS [ye2022fsrs].

At the assessment layer, the work contributes a teacher-defined outcome verification architecture for AI-conducted mock interviews that decouples question generation from answer evaluation. Rather than scoring open-ended responses against a model answer key — a task that is both computationally expensive and epistemically fragile — the system reduces the AI’s role to binary outcome verification against a teacher-specified checklist. This design is shown to be more robust, auditable, and cost-efficient than real-time generative assessment while preserving the open-ended nature of the interview interaction.

Finally, the system introduces three formally defined learner-facing metrics — the Knowledge Retention Estimate $\hat{R}_{s,\ell}$, Progression Readiness, and Review Compliance Rate $\rho_{s,\ell}$ — that together characterise a student’s cognitive standing within a lesson from complementary diagnostic, gate, and engagement perspectives. The formal definitions of these metrics, their derivation from per-student SM-2 state, and their relationship to the no-decay design decision constitute a contribution to the literature on competency-based progression systems in spaced repetition contexts.

1.4.2 Practical Perspectives

From a practical standpoint, aBridgeAI addresses a concrete and growing problem in institutional education: the inability of existing tools to bridge the gap between passive knowledge consumption and verified competency demonstration at scale.

For **teachers and instructors**, the platform eliminates the manual overhead of quiz creation by automating question generation from uploaded course materials, while preserving full editorial control through a mandatory review and configuration phase. Teachers can define expected response times and EF thresholds per card, configure lesson unlock criteria, and specify interview outcome requirements — all without requiring knowledge of the underlying scheduling algorithm. The monitoring dashboard surfaces per-student and cohort-level retention metrics, enabling data-informed intervention rather than reliance

on periodic examination snapshots.

For **students**, the platform replaces fixed assessment schedules with individually paced review cycles governed by demonstrated memory stability. Students are notified when cards are due for review at algorithmically optimal intervals, reducing the cognitive burden of self-scheduling while reinforcing retention through spaced practice. The interview module provides repeated access to structured practice sessions in a low-stakes environment, building the practical communication and problem-solving skills that quiz-based assessment cannot evaluate. Career Readiness reporting gives students a transparent, continuously updated signal of their progression toward professional competency targets.

For **organisations and educational managers**, the system provides institution-level visibility into cohort progression through career-scoped reporting. Career Readiness scores aggregated across course completion percentages give managers a quantitative basis for identifying at-risk cohorts, evaluating curriculum effectiveness, and demonstrating learning outcomes to accreditation bodies or industry partners — without requiring manual data collection or post-hoc grade analysis.

From a broader industry perspective, aBridgeAI demonstrates that the cost and complexity barriers associated with AI-assisted personalised learning are addressable through careful architectural constraint: by pre-generating question banks, restricting real-time AI involvement to a single post-session batch call, and grounding scheduling in a well-understood deterministic algorithm, the system achieves meaningful personalisation at a token cost profile compatible with institutional deployment at scale.

1.5 Report Structure

This report is organized as follows:

- **Chapter 1 – Introduction:** Introduces the motivation behind aBridgeAI, identifying the limitations of traditional educational systems in delivering personalised, competency-driven learning at institutional scale. Defines the project goals, delimits the system scope, and articulates the academic and practical significance of the proposed solution.
- **Chapter 2 – Related Works:** Reviews existing educational technologies and mock interview methodologies, identifying their current limitations and establishing the theoretical need for an integrated, context-aware solution.
- **Chapter 3 – Theoretical Background:** Establishes the theoretical foundation

for the core technologies employed in this project, including LLMs, RAG pipelines, Vector Databases, and modern web development frameworks.

- **Chapter 4 – System Analysis and Requirements:** Details the systematic extraction of software requirements. It encompasses the definition of user roles, comprehensive Functional Requirements (FRs), quantifiable Non-Functional Requirements (NFRs), and Unified Modeling Language (UML) artifacts such as Use Case and Activity diagrams.
- **Chapter 5 – System Design:** Presents the high-level architectural blueprint of the system. It covers the system topology, the hybrid database schema design (integrating PostgreSQL for relational data, MongoDB for content, and Qdrant for vector embeddings), and the User Interface/User Experience (UI/UX) wireframes.
- **Chapter 6 – Implementation and Testing:** Discusses the practical realization of the proposed pipeline. This chapter highlights the core algorithms for context-aware question generation and empirically evaluates the system’s performance, latency, and scalability against the defined non-functional metrics.
- **Chapter 7 – Evaluation:** Empirically evaluates the system’s performance, latency, and scalability against the defined non-functional metrics.
- **Chapter 8 – Conclusion:** Summarizes the technical contributions and educational impacts of the project, acknowledges current system limitations, and outlines strategic directions for future enhancements.

2 Related Works

The intersection of cognitive science and artificial intelligence has birthed powerful tools for learning and knowledge management. However, current implementations—ranging from SRS algorithms to RAG suffer from systemic fragmentation. This essay explores the critical limitations within these domains, establishing the theoretical necessity for a context-aware, integrated knowledge system like aBridgeAI.

2.1 The Bottlenecks of Traditional Spaced Repetition (SRS) Algorithms

While SRS is a cornerstone of evidence-based learning, traditional algorithms fail to account for the complexity of human cognition. Recent work such as LECTOR [7] highlights the need for concept-aware and test-oriented adaptive scheduling mechanisms.

Semantic Isolation and Interference.

Most popular algorithms, including SM-2, HLR, and even the more recent FSRS (Free Spaced Repetition Scheduler), treat flashcards as independent data points. This “atomistic” approach ignores the semantic relationships between concepts. In practice, this leads to semantic interference, where similar concepts (e.g., nuanced vocabulary in standardized testing) become muddled because the system fails to highlight their distinctions or shared contexts [7].

These theoretical limitations are directly observable in today’s most widely adopted educational platforms. For instance, Anki, a standard utility for language acquisition and medical studies, relies heavily on SM-2 and FSRS to evaluate cards in strict isolation. When a user studies closely related terms, the system schedules them independently, failing to detect their semantic proximity to present them together for necessary contrastive learning. Similarly, Duolingo’s foundational Half-Life Regression (HLR) models the decay rate of individual grammatical structures in a vacuum. If a learner struggles to distinguish between two similar concepts, the model merely decreases their respective half-lives, prompting more frequent, isolated reviews rather than recognizing the relationship itself as the cognitive bottleneck. Commercial platforms like Quizlet further exacerbate this issue by prioritizing rapid, item-by-item testing that strips concepts of their broader contextual frameworks[8].

Consequently, learners often experience retrieval failure when confronted with real-world applications that require distinguishing between closely related ideas. When ac-

quiring new language structures or technical definitions, studying synonymous terms on entirely separate review schedules prevents the brain from forming necessary contrastive distinctions. The learner might remember the general meaning but fail to grasp the specific usage constraints. To mitigate this, next-generation schedulers must incorporate structural architectures that map semantic proximities. By actively identifying clusters of related information, an advanced algorithm could intelligently group these items for side-by-side contrastive review, or strategically distance them to prevent memory overwriting. This shift would transform a fragmented, item-by-item database into a cohesive, associative web of knowledge that mirrors human cognition.

The Legacy of Limited Data.

The SM-2 algorithm, the default for platforms like Anki, was developed in 1987 based on a very narrow experimental dataset. Its reliance on fixed mathematical formulas makes it brittle” and incapable of adapting to the non-linear learning curves of modern learners. While machine learning variants exist, they often lack the granularity” required for complex conceptual mapping. Consequently, users often find themselves trapped in “review hell”, where the algorithm stubbornly schedules repetitions based on superficial recall rather than deep comprehension. A foreign language vocabulary word, a historical timeline, and a complex syntax rule are all subjected to the exact same rigid decay functions[9].

In practice, this rigidity is acutely felt by users who attempt to manage diverse, multi-disciplinary knowledge bases. For example, a software developer preparing for technical interviews might use standard SRS to memorize both simple programming syntax and complex, multi-layered frameworks like WCAG accessibility standards. Because the legacy algorithm lacks structural granularity, it cannot distinguish between the cognitive effort required for a simple rote fact versus a heavily interdependent concept. It simply subjects both to the same mathematical decay, frequently burying the user under an overwhelming backlog of due cards and forcing brute-force memorization over true understanding.

To move beyond these archaic constraints, next-generation SRS must evolve from isolated item schedulers into context-aware engines. By leveraging deep neural networks trained on vast, diverse sets of user interaction data, future algorithms could dynamically adjust intervals based on the intrinsic semantic difficulty of the material. Ultimately, a truly modern system would recognize the interconnected nature of knowledge, mapping how the mastery of one concept accelerates the retention of another, thereby replacing a one-size-fits-all mathematical formula with a highly personalized cognitive model.

The Personalization Gap.

Traditional systems focus almost exclusively on temporal scheduling (when to review) rather than cognitive profiling (how to review). They do not dynamically adjust based on a learner’s specific preparation scenarios, such as the high-pressure environment of a professional certification or the nuanced needs of language acquisition. For instance, the cognitive load required to distinguish visually similar logograms—such as complex Kanji characters—differs drastically from the effort needed to master basic phonetic scripts or standard vocabulary. Yet, legacy algorithms treat these distinct cognitive tasks uniformly. Furthermore, a robust cognitive profile should account for the learner’s real-time state, including mental fatigue and specific error patterns, rather than relying solely on blunt, self-reported pass/fail metrics.

This gap is immediately evident in the user interfaces of major SRS platforms, which rely entirely on subjective feedback buttons (e.g., “Anki’s Again, Hard, Good, Easy”). If a learner studying for a standardized language exam, such as the JLPT, repeatedly confuses visually similar Kanji, clicking “Again” merely resets the temporal interval. The system cannot diagnose the underlying nature of the error—whether it stems from semantic interference, a fundamental misunderstanding of the character’s radical, or simply cognitive exhaustion. Furthermore, current platforms fail to adapt to the learner’s physical and digital environment; a user squeezing in quick reviews on a mobile device receives the exact same cognitively demanding prompts as a user engaged in deep, focused study on a desktop machine.

By measuring multidimensional variables like response latency and contextual interference, a truly adaptive system could pivot its strategy on the fly. It might, for example, shift from demanding active recall to providing targeted recognition tasks when cognitive exhaustion is detected, transforming a rigid review schedule into an intuitive, highly personalized learning environment.

2.2 Methodological Constraints in Retrieval Practice Applications

Retrieval practice—the act of recalling information—is one of the most robust findings in educational psychology. Yet, its application in software remains superficial.

Confounded Experimental Control.

Many educational applications claim to leverage the “testing effect”, but their implementations are often methodologically noisy. It is frequently unclear whether a learner’s

progress stems from the act of retrieval itself or from the feedback provided afterward. Without isolating these variables, systems cannot optimize the specific level of “cognitive effort” required for durable long-term retention. Furthermore, when platforms intertwine testing with immediate, high-information feedback, the assessment essentially transforms into an additional study trial. This conflation masks whether a memory trace was strengthened by the active, internal generation of the answer or merely refreshed by the external re-exposure to the correct information. Consequently, the concept of “desirable difficulties”—the foundational principle that learning should be suitably challenging to promote deep processing—is often compromised.

This methodological conflation extends far beyond mobile flashcard apps, deeply permeating both enterprise and academic LMS. In corporate or organizational training environments powered by platforms like Docebo, TalentLMS, 360Learning, and Absorb LMS, assessments are frequently designed for rapid compliance rather than long-term retention. Quizzes on these platforms often provide instant remediation; if a user fails a question, the system immediately highlights the correct protocol, effectively turning a test of recall into a passive reading exercise.

Similarly, massive open online course (MOOC) providers like Coursera and Khan Academy, alongside foundational open-source architectures like Moodle, heavily utilize immediate, high-information feedback to maintain student momentum. For instance, when a learner is tested on complex technical concepts—such as implementing WCAG accessibility standards—these platforms frequently offer progressive hints, automated corrections, or infinite module retakes with explicit error breakdowns. While platform creators like Gurucan incentivize this to scale course completion rates, this excessive scaffolding artificially inflates short-term performance. It inherently reduces the productive struggle necessary to forge durable neural pathways. To genuinely harness the testing effect, future iterations of educational technology must employ rigorous, factorial experimental designs that decouple retrieval difficulty from feedback mechanisms, thereby allowing algorithms to dynamically calibrate to a learner’s optimal zone of cognitive friction [10].

The Monotony of Format.

There is a heavy over-reliance on Multiple-Choice Questions (MCQs) due to their ease of automation and objective grading scalability. However, research suggests that MCQs are often significantly less effective than Short-Answer Questions (SAQs) or production-based tasks for cultivating higher-level conceptual mastery at the university level. Because MCQs inherently present the correct answer alongside distractors, they primarily stimulate recognition memory rather than the more cognitively demanding process of active,

free recall. This paradigm trains learners to identify plausible answers instead of synthesizing and articulating knowledge independently, which frequently results in brittle, inert knowledge that fails to transfer to complex, real-world applications.

This structural bottleneck is ubiquitous across modern educational infrastructure. Global university LMS, such as Canvas and Blackboard, alongside massive MOOC platforms like edX and Udemy, rely almost exclusively on auto-graded MCQs to handle large student volumes. When these platforms do attempt to automate SAQs or fill-in-the-blank exercises, their legacy assessment engines typically rely on rigid regular expressions (Regex) or exact string matching. Consequently, a student who deeply understands a concept—such as explaining the core loop of an algorithm or the nuance of a Japanese grammar rule—but phrases it uniquely is often penalized. This forces instructors and course designers to default back to the safety and scalability of MCQs.

Furthermore, current educational systems largely lack the generative and evaluative sophistication necessary to seamlessly produce diverse assessment formats capable of probing deep structural understanding. While modern artificial intelligence has made strides in natural language processing, reliably evaluating open-ended, production-based tasks requires nuanced semantic parsing to distinguish between genuine conceptual comprehension and mere keyword mimicry. Until automated assessment platforms can robustly evaluate these complex, constructed responses with human-like accuracy, the pedagogical landscape will remain bottlenecked by formats that prioritize superficial recognition over genuine cognitive generation.

2.3 The Deficit in Contextual and Cognitive Adaptivity

A significant bottleneck in current educational and automation frameworks is their inability to dynamically modulate the “pace” and “intensity” of interaction based on the user’s fluctuating cognitive state. This lack of adaptivity manifests across several dimensions.

Static Control Architectures vs. High-Entropy Environments.

Most conventional learning systems are built upon implicitly Linear-Time-Invariant (LTI) operating assumptions. They presuppose a predictable, incremental trajectory of knowledge acquisition. However, human learning is inherently non-linear and high-entropy: learners experience moments of rapid conceptual integration followed by extended consolidation plateaus. Because existing systems cannot dynamically pivot their instructional strategy in response to these shifting probabilistic states, they

remain rigid—failing to capitalize on periods of high receptivity or provide calibrated intervention during unexpected cognitive stalls. To truly model the stochastic nature of human cognition, these architectures must transition away from deterministic, rule-based sequencing toward probabilistic control frameworks, such as Partially Observable Markov Decision Processes (POMDPs) or deep Bayesian Knowledge Tracing. In a high-entropy cognitive state—characterized by high working memory load or conceptual confusion—a rigid system blindly pushes the next scheduled module, often exacerbating cognitive overload and precipitating algorithmic fatigue. Conversely, during periods of low entropy and rapid assimilation, the system fails to accelerate the pacing or increase the structural complexity of the tasks, leading to boredom and disengagement. Achieving genuine adaptivity requires continuous, real-time telemetry—analyzing micro-interactions, response latencies, and specific error typologies—to accurately infer the user’s hidden cognitive state. Only by closing this complex, multi-dimensional feedback loop can an educational platform dynamically modulate the “bandwidth” of information delivery, evolving from a static content repository into a highly responsive cognitive symbiote.

Failure to Maintain the “Goldilocks Zone” of Flow.

Effective knowledge construction requires maintaining the learner within the “Goldilocks zone”—a state of optimal challenge formally associated with Lev Vygotsky’s Zone of Proximal Development and Mihaly Csikszentmihalyi’s psychological theory of Flow. Current educational tools and automated tutors typically rely on fixed, deterministic review schedules or simplistic, performance-based branching mechanisms that only measure binary outcomes, such as whether a final answer is correct or incorrect. Without a more granular, continuous understanding of the user’s internal affective state—derived from subtler telemetry like response latency, error self-correction rates, or dwell time—these platforms frequently drift into two counterproductive extremes:

- **Cognitive Overload (Anxiety):** Delivering complex material or requiring high-dimensional reasoning too rapidly. When systemic demands significantly exceed the learner’s current schema capacity, it triggers working memory saturation and high affective resistance, ultimately resulting in frustration, algorithmic fatigue, and system abandonment.
- **Cognitive Underload (Complacency):** Providing overly simplistic tasks, excessive scaffolding, or highly predictable generative outputs that fail to stimulate semantic processing. Prolonged exposure to this state not only induces boredom and reduces immediate engagement, but also precipitates severe automation bias;

the learner becomes passively dependent on the system’s external cognitive labor, gradually eroding their internal critical reasoning and independent problem-solving faculties.

To sustain this delicate equilibrium, next-generation architectures must integrate dynamic difficulty adjustment (DDA) algorithms capable of micro-modulating pedagogical friction. By dynamically increasing abstraction when rapid mastery is detected and providing targeted, localized interventions during genuine struggle, these systems can actively engineer an enduring, personalized flow channel.

Synthesis of Limitations

These collective limitations—ranging from semantic isolation in SR and retrieval interference in rigid assessment formats, to structural fragmentation in RAG systems and the absence of dynamic cognitive feedback—reveal a deeply segmented technological landscape. Current frameworks treat knowledge as a collection of discrete artifacts rather than a living, interdependent cognitive network. This fragmentation underscores the necessity for an integrated, context-aware, and cognitively aligned architecture such as *aBridgeAI*, which aims to unify adaptive scheduling, relational reasoning, and real-time cognitive modulation into a cohesive intelligence partner.

Table 2.1: Comparative analysis of educational technologies across pedagogical dimensions.

Feature / Pedagogical Dimension	Traditional SRS (Anki, Quizlet) [11, 12]	Enterprise / Academic LMS (Canvas, Moodle) [13, 14]	Gamified EdTech (Duolingo) [15]	aBridgeAI
Knowledge Retrieval	Atomistic / Isolated flash-cards	Modular / Static document silos	Linear / Skill-tree based	Semantic retrieval over structured learning materials (vector-indexed knowledge units)
Assessment Format	Binary recall (pass / fail)	MCQs, RegEx short answers	Drag-and-drop, translation	Algorithmically graded quiz with hint penalty; outcome-verified mock interviews
Adaptive Scheduling	Fixed multiplicative interval scheduling (SM-2, FSRs)	None (instructor scheduled)	HLR	Hybrid SM-2 with algorithmically derived Q via response latency normalisation and per-student EF isolation
Content Creation	Manual user generation	Manual instructor creation	Studio-produced content	Instructor-uploaded materials with AI-assisted breakdown; full manual creation and editorial override supported
Feedback Mechanism	Instant answer exposure	Instant remediation / scaffolding	Immediate error correction	Hint-penalised Q scoring discouraging passive retrieval; binary interview outcome without rubric exposure
Support applying knowledge and skills in practice	None (pure theory)	Limited (sandbox coding)	None (standardised translation)	Teacher-defined outcome verification via context-aware technical and behavioural mock interviews

2.4 The Path Toward aBridgeAI

The limitations outlined above—semantic isolation in SRS, insufficient experimental control in retrieval practice implementations, relational blindness in RAG, and the maintenance fragility of large-scale Knowledge Graph Construction (KGC)—converge toward a singular conclusion. Contemporary systems continue to treat knowledge as a collection of isolated data points rather than as a dynamic, interdependent cognitive ecosystem.

By integrating context-aware algorithms with rigorous dependency management and cognitively grounded retrieval strategies, *aBridgeAI* directly addresses these structural deficiencies. The system advances beyond simple recall optimization toward deep structural understanding, offering a unified and adaptive framework capable of supporting both individual learners and enterprise-scale knowledge infrastructures.

3 Theoretical Background

To address the structural deficiencies and semantic isolation of traditional educational systems identified in Chapter 2, aBridgeAI relies on a rigorous synthesis of cognitive psychology, algorithmic modeling, and advanced artificial intelligence. This chapter establishes the theoretical and technological foundations that empower the system to treat knowledge as a dynamic, interdependent cognitive ecosystem.

First, the chapter explores the baseline constraints of human memory through CLT, establishing the pedagogical necessity for adaptive knowledge structuring. These psychological principles are then translated into computational mechanics through the design of a hybrid SM-2 SR algorithm and individualized learner evaluation metrics. Finally, the chapter examines the sophisticated technological infrastructure required to execute these models at scale—specifically LLMs, RAG pipelines, Vector Databases, and modern web development frameworks. Together, these theoretical pillars form the architectural blueprint for aBridgeAI’s context-aware learning environment.

3.1 Cognitive Foundations of Learning

3.1.1 Cognitive Load Theory (CLT)

Human cognitive architecture is characterized by a working memory that has severe limitations in capacity and duration when processing novel information. Working memory typically holds only about seven chunks of information at a time and is restricted in its ability to simultaneously process and organize interacting elements. All information must first pass through this limited working memory before it can be stored in the unlimited long-term memory as permanent schemas. When instructional materials exceed these processing constraints, cognitive overload occurs, leading the learner to lose focus and stop processing the content deeply. Consequently, total cognitive load—the sum of intrinsic, extraneous, and germane loads—must be managed to ensure that the learner’s cognitive capacity is not overwhelmed[16].

Intrinsic Load

Intrinsic cognitive load refers to the inherent complexity of the material being learned, which is determined by its "element interactivity". This load is generated by the number of elements that a learner must attend to simultaneously to understand the subject matter. Because experts can use previously stored schemas in long-term memory to process multiple interacting squiggles or concepts as a single chunk, intrinsic load is always

relative to the learner's level of prior knowledge. It cannot be reduced by instructional design unless the learning objective itself is simplified or broken down. Therefore, a task that is high-load for a novice may impose minimal load on an expert due to their ability to activate existing knowledge structures in working memory[17].

Extraneous Load

Extraneous cognitive load is unnecessary mental strain caused by the way information is presented or the activities a learner is required to perform. Poor design choices, such as forcing learners to mentally integrate spatially separated text and diagrams (the split-attention effect), increase this bad form of load. Extraneous load consumes working memory resources that could otherwise be used for meaningful learning, thereby hindering mastery and schema construction. Instructional interventions aim to minimize this load by clarifying wording, simplifying navigation, and avoiding redundant information. Effective system design ensures that the cognitive capacity is focused on the essential content rather than on managing a messy user interface or unclear instructions[2].

Germane Load

Germane cognitive load refers to the productive mental effort that learners expend to construct schemas and build a permanent store of knowledge. Modern conceptualizations view germane load as a redistributive function that moves working memory resources away from extraneous activities and toward the intrinsic aspects of the learning task. Well-structured instructional methods increase germane load by prompting learners to engage in reflection, retrieval practice, and self-explanation. This type of load is essential for deep understanding, as it represents the actual mental work required to automate skills and integrate new information with prior knowledge. By minimizing extraneous distractions, systems can maximize the cognitive resources available for this critical germane processing[18].

3.1.2 Handling Overly Complex Lessons

To mitigate cognitive overload, the system employs dynamic chunking and segmenting to break high-intrinsic-load material into digestible, bite-size nodes. This "segmenting effect" ensures that learners have sufficient time and capacity to organize and integrate information from one portion of the lesson before the next segment begins. By breaking a continuous unit into learner-controlled segments, the system prevents the "zoning out" that happens when novel information is presented at a pace too fast for deep processing. This approach essentially creates interconnected nodes of knowledge, allowing the working

memory to focus on building component models before attempting to understand the entire complex causal system[3].

The system also utilizes isolated elements sequencing, where individual interacting components are presented sequentially before being integrated into a fully complex form. This strategy is particularly effective for very high-element interactivity tasks that would otherwise exceed working memory limits if all interactions were taught at once. Once these isolated elements are stored in long-term memory, the learner only needs to learn the interactions between them, which significantly reduces the working memory load during the final integration phase. By managing the order of presentation, the system ensures that the intrinsic complexity of the subject matter is introduced at a rate that aligns with the recovery and limits of working memory resources[16].

Finally, aBridgeAI provides "fading support" or scaffolding to transition learners from studying worked examples to independent practice. Initially, the system provides fully solved examples to reduce extraneous load, as solving a conventional problem requires high-load strategies like means-ends analysis which leave little capacity for learning. As the learner's expertise increases, the system provides partially worked examples and completion tasks, eventually requiring independent problem solving. This adaptive approach avoids the "expertise reversal effect," where redundant guidance becomes an extraneous load for learners who have already acquired sufficient domain knowledge[4].

3.2 Algorithmic Learner Modeling and SR

SR is a memory consolidation technique grounded in the psychological phenomenon of the spacing effect, which holds that information reviewed across distributed intervals is retained more durably than information reviewed in massed practice [19, 20]. Algorithmic implementations of SR automate the scheduling of these intervals by maintaining a model of each learner's memory state and adjusting review frequency accordingly. This subsection surveys the foundational algorithm underpinning aBridgeAI's learner modeling layer, its theoretical memory model, and its known limitations in institutional deployment contexts.

3.2.1 The SM-2 Algorithm

The SuperMemo-2 (SM-2) algorithm, introduced by Wozniak in 1987 and subsequently refined [5], is among the most widely adopted SR scheduling algorithms in educational software. It introduced item-level difficulty differentiation through a scalar parameter termed the Easiness Factor (EF), which governs the rate at which review intervals grow

for each knowledge item independently. This allowed the algorithm to adapt review schedules to the relative difficulty of individual items rather than applying a uniform spacing function across all material.

Interval Scheduling

SM-2 schedules reviews through a recursive interval function. The first two intervals are fixed:

$$I(1) = 1 \text{ day}, \quad I(2) = 6 \text{ days.} \quad (3.1)$$

All subsequent intervals are computed as:

$$I(n) = I(n-1) \cdot EF, \quad n > 2, \quad (3.2)$$

where EF is initialised at $EF_0 = 2.5$ for all items and bounded below by $EF_{\min} = 1.3$ to prevent pathologically short intervals on items of high difficulty. The interval therefore grows multiplicatively with each successful review, reflecting the empirical observation that successfully recalled items can tolerate progressively longer delays before the next review without meaningful loss of retention [5].

Easiness Factor Update

After each review, EF is updated according to the learner’s reported quality response value $q \in \{0, 1, 2, 3, 4, 5\}$:

$$EF_{\text{new}} = EF_{\text{old}} + (0.1 - (5 - q)(0.08 + (5 - q) \cdot 0.02)). \quad (3.3)$$

This update rule penalises EF for low q values and rewards it for high q values, with the neutral point at $q = 4$ producing approximately zero net change. Items recalled with difficulty accumulate lower EF values and are therefore scheduled more frequently; items recalled effortlessly accumulate higher EF values and are scheduled less frequently. The EF floor of 1.3 ensures that even the most difficult items eventually stabilise at a reviewable cadence rather than collapsing into daily repetition indefinitely.

Failure and Reset

If $q < 3$, the item is considered failed. The interval resets to $I = 1$ day, the repetition counter returns to zero, and the item re-enters the learning queue regardless of its prior history. The EF penalty is nonetheless applied, ensuring that repeated failures progressively reduce the item’s interval growth rate even as the absolute interval resets.

Quality Response Scale

The quality response value q is the sole input the algorithm receives from the learner per review. In the original SM-2 specification, q is assigned entirely through learner self-assessment after the correct answer has been revealed. The semantic definitions of each value are summarised in Table 3.1.

Table 3.1: Quality response scale used in the SM-2 SR algorithm [5].

q	Response Quality Description
5	Perfect response; the correct answer is recalled immediately and without effort.
4	Correct response after a brief hesitation.
3	Correct response recalled with noticeable difficulty.
2	Incorrect response; the correct answer feels familiar upon seeing it.
1	Incorrect response; the correct answer is only partially recognised after review.
0	Complete blackout; no recognition of the correct answer whatsoever.

Known Limitations

Although SM-2 has demonstrated practical effectiveness across decades of deployment in self-study tools [21], several structural limitations bear directly on its applicability in institutional learning environments.

First, the algorithm treats q as a honest self-assessment, assuming the learner has no incentive to misreport. This assumption is well-founded in solo study contexts but breaks down when q accumulates into metrics that influence externally visible outcomes such as lesson progression or instructor-facing dashboards.

Second, interval growth is governed solely by EF , which encodes only the historical pattern of q values for a given item. The algorithm applies no model of item-level response demand — a brief factual question and a complex multi-step problem receive identical scheduling treatment once their EF values converge, despite the substantially different cognitive effort required to answer each.

Third, SM-2 does not model the passage of time between reviews beyond the binary distinction of whether the review occurred before or after the scheduled date. A student

who reviews every card one day late receives identical scheduling to one who reviews on time, and a student who never reviews an overdue card suffers no algorithmic consequence — their EF remains frozen at its last recorded value indefinitely [5].

These limitations motivate the design adaptations described in Section 5.3, where aBridgeAI’s hybrid scheduling model is specified.

3.2.2 Memory Models: Stability and Retrievability

Whilst SM-2 provides a practical scheduling heuristic, its interval recurrence function does not derive from an explicit model of how memory traces form, strengthen, or decay over time. A more theoretically grounded account of memory dynamics is provided by the Two-Component Model of Memory, which characterises the state of any memorised item through two independent variables: memory stability and memory retrievability [22].

The Forgetting Curve

The empirical foundation of SR theory is Ebbinghaus’s forgetting curve, first described in 1885 [23]. Ebbinghaus demonstrated through systematic self-experimentation that the probability of successful recall declines as a negatively exponential function of elapsed time since the last successful review:

$$R(t) = e^{-\frac{t}{S}} \tag{3.4}$$

where $R(t) \in [0, 1]$ denotes the probability of recall at time t days after the last review, and S denotes a stability parameter governing the rate of decay. Items with higher stability retain acceptable recall probability over longer intervals; items with lower stability decay rapidly toward chance-level performance. This formulation implies that forgetting is not a discrete event but a continuous process, and that any fixed review interval will produce different retention outcomes for items of differing stability.

Memory Stability

Memory stability S represents the durability of a memory trace — formally, the inter-repetition interval at which the probability of successful recall equals a target retention threshold, typically set at $R = 0.9$ [22]. Stability is not an intrinsic property of the material alone but reflects the interaction between item difficulty, the learner’s prior knowledge, and the history of prior retrievals. Each successful retrieval increases stability, a phenomenon termed the retrieval practice effect [24], with the magnitude of the stability increase depending on the retrievability at the moment of practice: retrievals attempted

when recall probability is low produce larger stability gains than retrievals attempted when recall probability is high [25]. This is the theoretical basis for spacing reviews at intervals that allow some forgetting to occur before the next retrieval rather than reviewing while recall is still effortless.

Memory Retrievability

Memory retrievability R represents the probability of successful recall at a specific point in time, computed from the forgetting curve given the current stability S and the elapsed time since the last review t . Unlike stability, which changes only at the moment of review, retrievability decays continuously between reviews. At the moment of a successful review, retrievability resets to 1.0 and then begins declining again according to the exponential function above. Retrievability thus provides a real-time estimate of a learner’s current knowledge state for any given item, independent of when the next scheduled review falls [22].

Stability Increase and the Role of Difficulty

The Two-Component Model further specifies that the increase in stability ΔS produced by a successful retrieval is governed by an Optimum Factor (OF), which encodes the relationship between retrievability at the time of review and the resulting stability gain. Items reviewed at low retrievability — that is, items that the learner has nearly forgotten — yield disproportionately large stability increases, consistent with the desirable difficulties framework described by Bjork [25]. This property motivates scheduling reviews near the point of forgetting rather than significantly before or after it, which is the theoretical justification for the interval lengths produced by SM-2 and its successors.

Absolute item difficulty, termed the A-Factor in SuperMemo’s formulation [5], modulates stability growth independently of retrievability. High A-Factor items require more repetitions to achieve equivalent stability to low A-Factor items, reflecting genuine differences in material complexity that persist across learners. Separating item difficulty from learner-specific memory dynamics allows the model to distinguish between an item that is universally difficult and a learner who is individually struggling, a distinction that has practical consequences for both scheduling and content design.

Relationship to SM-2

SM-2 can be understood as an approximation of the Two-Component Model operating without explicit stability or retrievability estimates. The Easiness Factor EF functions as a proxy for stability — items with high EF grow intervals rapidly, consistent with high stability — but it encodes no continuous retrievability estimate and applies no forgetting

curve between reviews. The consequence is that SM-2 responds only to review outcomes, not to the passage of time between reviews. An item due for review three months ago carries the same EF as the day it was last reviewed, despite the intervening decay in its true retrievability. More recent algorithms, including SM-17 and the FSRS. [26], address this by maintaining explicit stability and retrievability estimates updated continuously over time, at the cost of greater computational and parametric complexity. The theoretical properties of stability and retrievability nonetheless inform the design of aBridgeAI’s learner-facing performance metrics, as described in Section 5.3.

3.2.3 Knowledge Tracing and Learner Modeling

SR algorithms model memory at the level of individual items, scheduling each card independently based on its own retrieval history. A complementary line of research, termed knowledge tracing, addresses a broader question: given a sequence of observed practice attempts, what is the probability that a learner has mastered an underlying knowledge component at any given point in time? Where SR optimises *when* to review, knowledge tracing estimates *whether* mastery has been achieved [27].

Bayesian Knowledge Tracing

The foundational model in this field is Bayesian Knowledge Tracing (BKT), introduced by Corbett and Anderson [27] in the context of the ACT programming tutor. BKT models each knowledge component as a latent binary variable — either mastered or not mastered — and updates the probability of mastery after each practice attempt using Bayes’ theorem. The model is parameterised by four values:

- $P(L_0)$: the prior probability that the learner has already mastered the knowledge component before any practice.
- $P(T)$: the probability of transitioning from unmastered to mastered after a single practice opportunity.
- $P(S)$: the probability of a slip — answering incorrectly despite having mastered the component.
- $P(G)$: the probability of a guess — answering correctly without having mastered the component.

After each observed response, the posterior probability of mastery $P(L_n)$ is updated as:

$$P(L_n \mid \text{correct}) = \frac{P(L_{n-1})(1 - P(S))}{P(L_{n-1})(1 - P(S)) + (1 - P(L_{n-1})) \cdot P(G)}, \quad (3.5)$$

$$P(L_n \mid \text{incorrect}) = \frac{P(L_{n-1}) \cdot P(S)}{P(L_{n-1}) \cdot P(S) + (1 - P(L_{n-1}))(1 - P(G))}. \quad (3.6)$$

The updated posterior is then carried forward as the prior for the next practice attempt, allowing the model to track mastery probability as a continuous trajectory over the learner’s practice history. A mastery threshold — typically $P(L_n) \geq 0.95$ — determines when a learner is considered to have sufficiently mastered a component and may progress [27].

BKT established several principles that remain influential in educational data mining: the separation of latent knowledge state from observed performance, the explicit modeling of slip and guess probabilities as noise sources, and the use of a mastery threshold as a progression gate. However, BKT operates on discrete knowledge components modeled independently, with no mechanism for capturing relationships between components or for modeling the continuous decay of knowledge over time [28].

Deep Knowledge Tracing

Piech et al. [29] extended knowledge tracing to sequence modeling using recurrent neural networks, introducing Deep Knowledge Tracing (DKT). Rather than maintaining an explicit probability for each knowledge component, DKT encodes the learner’s entire interaction history as a hidden state vector within a long short-term memory (LSTM) network, and predicts the probability of correct response on any future knowledge component from this compressed representation. This allows the model to implicitly capture relationships between knowledge components — for instance, that mastery of TCP flow control is predictive of performance on TCP congestion control — without requiring these relationships to be explicitly specified by a domain expert.

DKT demonstrated substantially improved predictive accuracy over BKT on large-scale educational interaction datasets [29], and subsequent variants have incorporated attention mechanisms, graph structures, and forgetting functions to further improve performance [30]. However, DKT and its successors require substantial interaction data to train reliably, operate as largely opaque models offering limited interpretability, and are computationally intensive relative to the closed-form updates of BKT or SM-2 [31].

Per-Student State Isolation

A principle common to both BKT and SM-2 is that knowledge state is modeled at the level of the individual learner. Each student maintains an independent set of parameters — mastery probabilities in BKT, Easiness Factors in SM-2 — that evolve according to their own practice history without influence from peer performance. This isolation is not merely a computational convenience but a theoretical necessity: the same observable response pattern may reflect different underlying knowledge states in different learners depending on their prior experience, learning strategies, and domain familiarity [27, 32]. Aggregating or sharing state across learners would systematically misrepresent individual knowledge trajectories and undermine the validity of any progression gate derived from the model.

This principle of per-student state isolation is directly adopted in aBridgeAI’s learner modeling design, as elaborated in Section 5.3.

Item-Level Demand and Latency Normalisation

A challenge in using raw response latency as a cognitive signal is that observed latency confounds retrieval effort with item-level reading and processing demand. A learner responding to a lengthy, multi-clause technical question will necessarily exhibit higher latency than a learner responding to a brief factual question, regardless of their underlying memory state for the relevant knowledge. Meaningful interpretation of response latency therefore requires normalisation against an item-specific baseline that accounts for the inherent processing demand of the question itself [33].

Several approaches to this normalisation have been proposed in the literature. Standardisation against population response time distributions — dividing individual latency by the mean or median latency for that item across all learners — removes item-level demand while preserving individual differences in retrieval speed. Where population data is unavailable, item-level demand can be estimated from surface features of the stimulus: reading time models based on word count and syntactic complexity provide principled baselines against which individual latency can be compared. Technical content specifically warrants a conservative reading rate estimate, as domain-specific vocabulary and conceptual density slow comprehension relative to general prose. Difficulty ratings assigned by domain experts provide a further adjustment factor, capturing the additional cognitive demand imposed by items that require not only reading but active problem construction or multi-step reasoning [16].

Response Time in Automated Assessment

The use of response time as a formal scoring input in automated assessment systems has gained traction alongside the growth of computer-based testing. Item Response Theory extensions incorporating response time — most notably the hierarchical model of van der Linden [34] — treat latency as a second observed variable alongside accuracy, jointly informing estimates of both item difficulty and learner ability. In adaptive learning systems, response time has been used to adjust item selection, to detect disengaged or random responding, and to calibrate difficulty estimates in the absence of sufficient historical response data [35]. These applications share the common principle that response latency, when properly normalised, provides diagnostic information about the learner’s cognitive state that accuracy-only models systematically discard.

The theoretical basis established here — that normalised response latency reflects memory strength beyond what correctness alone captures, and that item-level demand must be accounted for in any latency-based signal — directly motivates the expected time formulation and Q derivation model specified in Section 5.3.1.

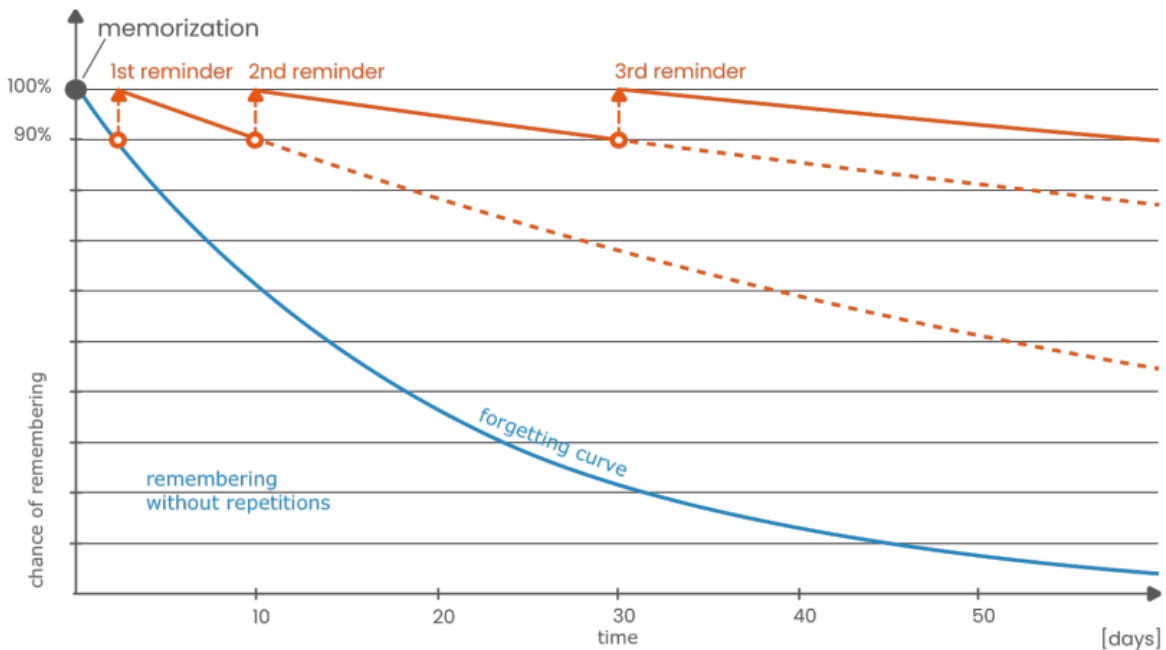


Figure 3.1: Forgetting curves applied in SR learning. Source: [36].

3.3 Core Artificial Intelligence Technologies

3.3.1 LLMs in Education

LLMs, characterized by transformer architectures and self-attention mechanisms, have shifted educational technology from static content delivery toward context-aware generative systems capable of deep semantic understanding. In the aBridgeAI framework, LLMs serve as the primary engine for “fading support” scaffolding, dynamically adjusting the complexity of hints and explanations based on the learner’s demonstrated competence. These models facilitate “System-2 level engagement” by generating automated questioning prompts that require students to explain their reasoning in steps, identify underlying assumptions, or solve problems through alternative methodologies. This approach prevents the “AI as Autopilot” failure mode, where a tool provides complete solutions that reduce germane load and hinder mastery; instead, the LLM enforces the productive mental work necessary for schema construction. Furthermore, LLMs enable the large-scale production of high-quality practice questions and quizzes, satisfying the need for a continuous supply of assessment materials in adaptive learning environments without the high cost and resource constraints of manual question construction [37].

Beyond generative scaffolding, LLMs fundamentally resolve the assessment bottlenecks prevalent in legacy LMS. Traditional platforms rely heavily on rigid RegEx or exact string matching, forcing a systemic dependence on MCQs that primarily stimulate recognition memory. LLMs, however, possess the high-dimensional semantic parsing capabilities required to reliably evaluate open-ended, production-based tasks. By analyzing the semantic intent of a student’s constructed response—whether written or transcribed via voice—the LLM can accurately distinguish between genuine structural comprehension and superficial keyword mimicry, enabling nuanced, automated grading of short-answer questions and complex technical explanations.

Specifically within the aBridgeAI architecture, LLMs are operationalized to drive the context-aware mock interview module. This functionality requires the model to maintain conversational state and context over multi-turn dialogues, dynamically adapting its follow-up inquiries based on the learner’s real-time articulation. To ensure the LLM strictly adheres to these pedagogical objectives rather than defaulting to a subservient, answer-providing assistant, the system employs advanced prompt engineering frameworks, including few-shot prompting and strict persona constraints. By functioning as an adaptive Socratic evaluator, the LLM continuously probes the boundaries of the student’s knowledge, actively identifying and reporting the discrepancies between theoretical recall

and practical application.

3.3.2 Relational Reasoning: Overcoming Document Silos

Traditional RAG pipelines often suffer from "document silos," where the system treats individual documents or text chunks as isolated units of information. This structural blindness limits relational reasoning, making it difficult for an AI to answer complex queries that require linking configuration rules, transactional dependencies, or conceptual hierarchies distributed across multiple sources[38]. aBridgeAI addresses this through a structured lecture ingestion pipeline, where uploaded source materials — spanning documents, videos, and presentation files — are parsed and decomposed into typed knowledge units rather than stored as raw, undifferentiated text chunks. Each unit preserves its relational origin within the lesson it belongs to, allowing the system to reason across source boundaries rather than treating each file as a self-contained silo. Knowledge is represented not as a flat collection of passages but as a structured set of concepts, summaries, and examples that together form a coherent pedagogical context.

At the retrieval layer, the system combines parametric memory — the LLM’s pre-trained knowledge — with non-parametric memory held in a dense vector index built over these structured materials, following the hybrid retrieval architecture described by Lewis et al. [38]. This ensures that AI-generated assessments are grounded in verified, teacher-uploaded content rather than produced purely from the model’s internal weights, reducing the risk of hallucination and improving factual specificity. The generation layer is further constrained by teacher-defined configuration. For quiz generation, instructors can supply additional contextual instructions that narrow the retrieval scope to lesson-relevant material. For interview generation, teacher-defined outcome requirements serve as semantic anchors that guide which regions of the knowledge index are consulted when producing sample questions — without requiring a pre-defined answer key from the teacher. This mirrors the principle of cognitive load management described by Sweller et al. (2019)[16], where scaffolded knowledge structures reduce extraneous cognitive load by narrowing the problem space to what is pedagogically relevant, benefiting both the learner and the generative process itself. Together, these design choices position aBridgeAI’s knowledge layer not as a collection of flat files but as a structured cognitive ecosystem — one where the relational context between materials, lessons, and assessment outcomes is preserved architecturally, enabling more coherent, traceable, and pedagogically grounded AI-generated content[39].

RAG pipelines address the inherent limitations of purely parametric LLMs, such as

their tendency to produce "hallucinations" and their inability to easily expand or revise stored world knowledge. By combining a pre-trained sequence-to-sequence model (parametric memory) with a dense vector index of verified instructional material (non-parametric memory), RAG anchors the AI's output to specific, factual lesson contexts. This architecture ensures that the "supportive information" provided to a learner—which often involves high element interactivity and complex reasoning—is retrieved accurately from a ground-truth source before the LLM generates a response. The retrieval mechanism allows the system to be “human-readable” and “human-writable”, enabling educators to update the knowledge base by simply editing the document index without retraining the underlying model parameters. Ultimately, this hybrid approach generates language that is more factual, specific, and diverse than parametric-only baselines, ensuring the learner is presented with information that is both reliable and contextually relevant to their current learning objective[38].

3.3.3 Vector Databases and High-Dimensional Search

Vector databases solve the problem of “semantic isolation” by embedding knowledge into high-dimensional vector spaces, where information is represented as mathematical coordinates based on semantic meaning. Unlike traditional keyword search, which relies on literal word overlap, high-dimensional search utilizes Maximum Inner Product Search (MIPS) and cosine similarity to identify conceptually related documents. This capability transforms the system from a collection of isolated data points into a “dynamic, interdependent cognitive ecosystem” where semantically related “isolated elements” are sequenced to manage the learner’s working memory. When a student struggles with a concept, the vector database can retrieve “just-in-time” procedural information or supportive theoretical backgrounds by identifying the nearest neighbor concepts in the knowledge space. By mapping the dependencies between item difficulty, memory stability, and semantic relevance, high-dimensional search ensures that new information is integrated into a coherent mental model, effectively automating the “isolated elements effect” to prevent cognitive overload during the integration of complex causal systems[3, 16, 38].

3.4 Technical Foundations and System Design Rationale

3.4.1 Component-Based Interface Design and Cognitive Load

The practical software framework for a learning system must be architected to respect the human cognitive architecture, specifically the capacity and duration limits of

working memory. To support low-latency and highly interactive applications, the frontend architecture utilizes component-based UI rendering, which facilitates the "segmenting effect" by breaking complex, high-element interactivity lessons into manageable, learner-controlled segments. This approach ensures that learners are not overwhelmed by a single continuous unit of information, allowing working memory resources to recover between processing episodes. Furthermore, a strongly-typed backend architecture is necessary to manage the "state-heavy" nature of individualized learner modeling, where the system must track millions of unique parameters—such as item-specific Easiness Factors (EF), Memory Stability (S), and Memory Retrievability (R)—across high-dimensional matrices. By minimizing extraneous load through a simplified, responsive user interface, the framework ensures that the learner’s limited cognitive resources are focused entirely on the germane task of schema construction[2, 3, 16].

3.4.2 Integration of the Adaptive Loop

The system operates through a continuous "use, verify, and correct" feedback loop that synchronizes the web client, vector database, LLM, and SM-2 based algorithms. The loop begins when the web client presents a learning item; once the student provide a self-assessed quality grade (0–5), this "retrievability" data is sent back to the algorithm to verify the accuracy of the predicted interval. If the grade is low, indicating forgetting, the system immediately "corrects" the multidimensional matrices by reducing the stability increase factors and resetting the repetition schedule. Simultaneously, the LLM utilizes RAG to pull "supportive information" from the vector database, identifying semantically related knowledge nodes through high-dimensional search to provide "just-in-time" procedural hints. This integration ensures that the algorithm "learns the learner" in real-time, shifting from a fixed, heuristic model to a personalized forgetting curve that adapts to individual memory decay rates[5, 36].

3.5 Sumary about the Theoretical Background

The theoretical frameworks and technological paradigms examined in this chapter provide the essential blueprint for aBridgeAI. By grounding the system in the principles of CLT and individualized learner modeling, the system establishes a cognitively aware baseline that respects human memory constraints. The integration of a hybrid SM-2 algorithm and dynamic evaluation rubrics translates these psychological principles into an actionable, mathematical engine capable of adaptive spacing. Furthermore, LLMs, RAG

pipelines, and Vector Databases supports the requisite technological infrastructure to maintain semantic context and relational depth at scale.

Together, these theoretical and computational elements form a cohesive ecosystem that directly addresses the limitations of traditional, isolated SRS.

Having established why aBridgeAI is designed the way it is and the core technologies that power it, the focus must now shift to implementation. Chapter 4 – System Requirements and Analysis will translate these theoretical foundations into concrete software requirements, architectural diagrams, and functional specifications, detailing exactly how this cognitive ecosystem is operationalized in practice.

4 System Requirements and Analysis

This chapter provides a comprehensive analysis of the aBridgeAI system. It details the stakeholders and users of the system, business process, functional, non-functional and data requirements, and visualizes the system's behavior through Use Case and Activity diagrams.

4.1 System Overview

This section establishes the foundational context of the aBridgeAI system, outlining its core mission, the entities invested in its success, and the human actors who interact with the platform.

4.1.1 Purpose

The purpose of aBridgeAI is to bridge the gap between theoretical academic curricula and practical industry expectations. By operating as an AI-driven LMS and Mock Interview system, it transforms traditional, passive learning into an active, context-aware application loop. The system aims to automate the generation of assessments and provide personalized interview practices to ensure students are "Job-Ready."

4.1.2 Stakeholders and Users of the System

It is essential to distinguish between stakeholders (those impacted by or invested in the system's outcome) and users (those directly interacting with the software).

Stakeholders

- **Higher Education Institutions and Coding Bootcamps:** The host organisations that adopt aBridgeAI to improve student employability rates, reduce manual assessment overhead, and surface auditable evidence of progression.
- **Accreditation Bodies and Industry Partners:** External entities that consume institution-level reporting on progression and outcomes. These parties do not interact with the system directly but rely on the aggregated metrics it produces to evaluate programme quality.
- **Hiring Companies:** The eventual consumers of the graduate pipeline, who benefit from candidates whose technical and communication competencies have been verified against outcome checklists rather than self-reported coursework.

- **Academic Supervisor and Project Review Committee:** The faculty advisors and reviewers responsible for evaluating the scientific and engineering contributions of this work against academic criteria.

Users of the System

- **Student:** The primary end-user. Consumes multimedia lessons, participates in AI-driven quiz and spaced-repetition quiz review cycles whose intervals are governed by the hybrid SM-2 scheduler, takes AI-conducted mock interviews in voice or text mode, and tracks personal progression via the Knowledge Retention Estimate, Progression Readiness, and Review Compliance Rate metrics. The student receives a binary pass/fail outcome per interview session.
- **Teacher:** Designs lessons, uploads raw materials for AI ingestion, and is responsible for the human-in-the-loop review of all AI-generated artefacts before publication — including editing quiz cards, confirming the per-card expected response time T_{exp} that anchors Q derivation, defining target outcomes for the mock interview, and reviewing the AI-generated interview question bank. Teachers additionally configure the SM-2 unlock parameters ($EF_{\text{min}}^{\text{unlock}}$, coverage threshold τ , prerequisite criteria) per lesson and monitor cohort retention and compliance metrics via the teacher dashboard.
- **Educational Manager:** The administrative authority within a single organisation (e.g. a Dean, Vice Dean, or programme lead). Defines *careers*—the ordered pathways of courses representing a target professional competency—and assigns teachers to courses within those pathways. Onboards students and teachers within the organisation, including via bulk CSV import, and consumes organisation-scoped reports including Career Readiness progression at both the individual and cohort level. The Educational Manager’s authority is bounded by the organisation they administer; they have no cross-tenant visibility.
- **IT Administrator:** The super-user responsible for cross-tenant operations. Provisions new organisations, configures organisation-level limits (storage quotas, concurrent-user caps), invites the initial Educational Manager for each newly provisioned tenant, monitors system health and infrastructure, and enforces platform-wide security and compliance policies. The IT Administrator does not manage course content or intra-organisational user accounts; those responsibilities belong to the Educational Manager.

4.2 Business Process

Traditional educational workflows rely heavily on manual content creation, fixed assessment schedules, and uniform grading criteria, creating systematic bottlenecks in delivering personalised, competency-driven feedback at scale. aBridgeAI reengineers this workflow through a four-stage AI-orchestrated process that separates content preparation, knowledge validation, practical assessment, and longitudinal reporting into distinct, traceable phases.

1. **Content Ingestion (Preparation).** Teachers upload raw learning materials in supported formats including PDF documents, MP4 video recordings, and presentation files. The ingestion pipeline parses each source, extracts typed knowledge units — summaries, concepts, examples, and indexes them in a dense vector store to support semantic retrieval. The resulting structured knowledge base serves as the sole authoritative source for all subsequent AI generation tasks, ensuring that generated assessments are grounded exclusively in teacher-uploaded content.
2. **The Learning Loop (Knowledge Validation).** Students consume published learning materials and undertake AI-generated quizzes to validate baseline knowledge retention. Each quiz card carries a teacher-confirmed expected response time T_{exp} . The system derives the quality response value Q algorithmically from response correctness, hint usage, and the time ratio $\rho = T_{\text{actual}}/T_{\text{exp}}$, updating the student’s per-card Easiness Factor EF via the hybrid SM-2 scheduling model. Students remain in the learning loop — receiving SR reminders at algorithmically determined intervals — until their EF distribution across the lesson’s cards satisfies the teacher-configured unlock condition.
3. **The Application Loop (Practical Assessment).** Upon satisfying the quiz unlock condition, students gain access to the lesson’s interview section. Interview questions are drawn from a pre-generated, teacher-reviewed question bank aligned to teacher-defined outcome requirements. Students engage with the interview in text or voice mode, with each session conducted independently under a dedicated model instance. Upon session completion, the full transcript is submitted to the AI assessment model in a single batch call, which evaluates whether each defined outcome was satisfied. The student receives a binary pass or fail verdict; no rubric, per-outcome breakdown, or model reasoning is exposed. A lesson is considered complete only when both the quiz unlock condition and the interview pass condition are satisfied.

4. **Evaluation and Reporting (Feedback).** The system continuously aggregates per-card SM-2 state into three learner-facing metrics: the Knowledge Retention Estimate $\hat{R}_{s,\ell}$, Progression Readiness, and Review Compliance Rate $\rho_{s,\ell}$. These metrics are surfaced in per-student and class-wide dashboard views accessible to both students and teachers. At the career level, course completion percentages are aggregated into a Career Readiness score reported to the student and the organisation manager. Teachers are additionally notified of cohort-level patterns — including systematic retention gaps and declining compliance rates — enabling intervention at the content or configuration level rather than purely at the individual student level.

The four stages form a closed feedback loop: content ingestion informs quiz generation, quiz performance governs interview access, interview outcomes determine lesson completion, and reporting surfaces the gaps that guide both student remediation and teacher reconfiguration of content and assessment criteria.

4.3 Requirements

4.3.1 User Requirements

User Requirements articulate the functional needs of the system from the perspective of the end-users.

Student Requirements

- **UR-STU-01:** As a Student, I want to consume multimedia course materials and complete spaced-repetition quiz reviews scheduled by the system so that I can build durable retention of lesson content.
- **UR-STU-02:** As a Student, I want to participate in context-aware mock interviews via voice or text so that I can practise articulating my understanding under realistic conditions.
- **UR-STU-03:** As a Student, I want to receive the binary pass/fail outcome of each interview session — without per-outcome scoring or rubric exposure — so that I am evaluated on overall demonstrated competence rather than on isolated answers.
- **UR-STU-04:** As a Student, I want to track my own progress through learner-facing metrics — per-lesson Knowledge Retention Estimate, Progression Readiness,

and Review Compliance Rate — so that I can self-regulate my study pace without depending on instructor intervention.

Teacher Requirements

- **UR-TCH-01:** As a Teacher, I want to upload and organise learning materials so that I can build a structured knowledge base for each lesson.
- **UR-TCH-02:** As a Teacher, I want the system to draft quiz questions and interview prompts from my uploaded materials, and I want to review, edit, approve, or reject each draft before publication, so that AI-generated assessments retain pedagogical accuracy through Human-in-the-Loop oversight.
- **UR-TCH-03:** As a Teacher, I want to confirm the expected response time T_{exp} for every published quiz card and define the outcome criteria for every published interview so that the system can compute response-time ratios and binary pass/fail verdicts against my domain judgment rather than against generic heuristics.
- **UR-TCH-04:** As a Teacher, I want to configure per-lesson spaced-repetition parameters — the minimum Easiness Factor for unlock ($EF_{\text{min}}^{\text{unlock}}$), the coverage threshold (τ), and lesson prerequisites — so that progression stringency can be tuned to the criticality of each lesson.
- **UR-TCH-05:** As a Teacher, I want a cohort monitoring dashboard exposing per-student and class-wide Knowledge Retention Estimates, Progression Readiness distributions, and Review Compliance Rates so that I can identify at-risk learners and detect systemic weak points in my source material.

Educational Manager Requirements

- **UR-MGR-01:** As an Educational Manager (e.g., Dean, Vice Dean), I want to bulk-onboard Students and Teachers via CSV import within my organisation so that I can provision a new cohort without manual per-account creation.
- **UR-MGR-02:** As an Educational Manager, I want to define career pathways by grouping courses into ordered sequences and assigning Teachers to those courses so that my institution’s curriculum structure is reflected in the platform’s progression model.
- **UR-MGR-03:** As an Educational Manager, I want to view organisation-scoped Career Readiness snapshots — aggregated across all enrolled Students in a pathway,

with drill-down to individual progress — so that I can report on cohort outcomes without exposing per-interview rubrics.

- **UR-MGR-04:** As an Educational Manager, I want all my actions to be strictly scoped to my single tenant organisation so that I cannot accidentally or intentionally access data belonging to other institutions on the platform.

IT Administrator Requirements

- **UR-ITA-01:** As an IT Administrator, I want to provision new tenant organisations and invite their initial Educational Manager so that institutions can be onboarded onto the platform without my involvement in their internal user management.
- **UR-ITA-02:** As an IT Administrator, I want to configure organisation-level operational limits (storage quotas, concurrent-user caps, feature-flag toggles) so that I can enforce platform-wide capacity and licensing constraints across all tenants.
- **UR-ITA-03:** As an IT Administrator, I want to monitor cross-tenant system health, infrastructure metrics, and an immutable audit log so that I can ensure platform stability and respond to security or compliance incidents.
- **UR-ITA-04:** As an IT Administrator, I want my authority to exclude course content, assessment configuration, and intra-organisation user data so that pedagogical decisions remain with the Educational Manager and Teachers of each tenant.
- **UR-ITA-05:** As an IT Administrator, I want the platform to enforce centralised authentication and identity controls — including Google SSO, two-factor authentication, session management, and role-based access control — so that user access is uniformly secured across all tenant organisations.

4.3.2 Functional Requirements

The functional requirements of aBridgeAI specify the concrete system behaviours, processes, and user interactions that the platform must support. They are organised into six functional modules: (1) Authentication and Access Control, (2) Organisation and User Management, (3) Course and Material Management, (4) Learning Loop and Adaptive Quiz, (5) Application Loop and Mock Interview, and (6) Analytics, Reporting, and Notifications. Each requirement is traced to one or more User Requirements to ensure comprehensive coverage and end-to-end traceability. The complete Functional Requirements specification is provided in Appendix A.

Table 4.1: Core functional requirements by module.

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
<i>Module 1 — Authentication & Access Control</i>						
FR-1.1	The system shall allow users to authenticate using Google SSO and log out of the system.	All Users	UR-ITA-05	Yes	Yes	High
FR-1.3	The system shall support two-factor authentication using authenticator-app verification codes and one-time recovery codes for additional account protection.	All Users	UR-ITA-05	Yes	Yes	High
<i>Module 2 — Organisation & User Management</i>						
FR-2.1	The system shall allow IT administrators to provision new tenant organisations and to assign the initial Educational Manager role to a user within each organisation.	IT Administrators	UR-ITA-01	Yes	Yes	High
Continued on next page						

Table 4.1 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-2.3	The system shall allow Educational Managers to bulk-enroll students into a course via CSV import, creating new student accounts for unseen email addresses and reporting per-row validation errors without aborting the batch.	Educational Managers	UR-MGR-01	Yes	Yes	High
<i>Module 3 — Course & Material Management</i>						
FR-3.1	The system shall allow Teachers and Educational Managers to manage courses through draft, published, archived, and soft-deleted states, including administrative restoration of soft-deleted courses.	Teachers, Educational Managers	UR-TCH-01	Yes	Yes	High
Continued on next page						

Table 4.1 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-3.3	The system shall allow Teachers to upload learning materials in supported formats using standard or resumable upload flows, manage material meta-data, link materials to lessons, and re-run processing when needed.	Teachers, Educa- tional Managers	UR-TCH-01	Yes	Yes	High
<i>Module 4 — Learning Loop & Adaptive Quiz</i>						
FR-4.1	The system shall allow Teachers to create quiz questions manually or through AI drafting from ingested course content, review them through pending, approved, edited, or rejected states, reuse approved questions, and maintain source references and revision history.	Teachers, Educa- tional Managers	UR-TCH-02	Yes	Yes	High
Continued on next page						

Table 4.1 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-4.5	The system shall gate lesson access using prerequisite completion, the lesson coverage threshold τ over cards meeting the EF unlock threshold, and optional interview-pass requirements.	Students	UR-STU-01	Yes	Yes	High
<i>Module 5 — Application Loop & Mock Interview</i>						
FR-5.1	The system shall allow Teachers to define interview sections by specifying desired outcomes that students must demonstrate to pass, without requiring any expected or model answer.	Teachers, Educational Managers	UR-TCH-03	Yes	Yes	High
FR-5.4	The system shall allow Students to start mock interview sessions in text mode or voice mode as an adaptive turn-taking dialogue with an AI interviewer.	Students	UR-STU-02	Yes	Yes	High
<i>Module 6 — Analytics, Reporting & Notifications</i>						
Continued on next page						

Table 4.1 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-6.7	The system shall maintain an immutable audit log for role assignments, sensitive data changes, and HTTP request events, queryable by administrators for security and compliance review.	IT Admin-istrators	UR-ITA-03	Yes	Yes	High

4.3.3 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of aBridgeAI, specifying how the system behaves rather than what it does. They govern performance, scalability, security, and usability properties that collectively determine the reliability and institutional suitability of the platform. The complete Non-Functional Requirements specification is provided in Appendix B. Representative points are summarised below:

- Performance and latency targets ensure AI interview responses and core learning workflows remain responsive for users.
- Scalability and reliability requirements target modest peak concurrency while prioritising graceful degradation and uptime.
- Security and compliance mandates cover encryption at rest, RBAC, audit logging, and session management.
- Usability and accessibility requirements emphasise responsive UI and WCAG 2.1 AA alignment.

The complete Functional Requirements specification is provided in Appendix B.

Table 4.2: Representative non-functional requirements by domain.

ID	Requirement	Users	Prior-ity
NFR-2.1	The system shall support on average 50 concurrent active users, 10 simultaneous video streams, and 5 concurrent AI interview sessions during peak hours without performance degradation.	Admin	Medium
NFR-3.1	All sensitive data must be encrypted at rest using AES-256, including user passwords (hashed with bcrypt, cost factor ≥ 12), PII, session tokens, API keys, and uploaded learning materials.	All Users, Admin	High
NFR-4.2	The system must comply with WCAG 2.1 AA accessibility standards, supporting full keyboard navigation, sufficient color contrast ratios, and alt text for all visual elements.	Users with Disabilities	Low

4.3.4 Data Requirements

This section describes the data demands of aBridgeAI across its core functional domains. Rather than specifying a physical schema, it identifies what information must be captured, maintained, and related to support each system capability. The requirements are organised by domain and reflect the full lifecycle of content creation, assessment, learner modeling, and progress reporting.

Audit and Traceability

All entities in the system are subject to audit requirements. Every record — regardless of domain — must carry a creation timestamp, the identity of the actor who created it, a last-modified timestamp, and the identity of the actor who last modified it. Where records are soft-deleted rather than physically removed, a deletion timestamp and responsible actor must also be recorded. This base traceability requirement applies uniformly across users, courses, assessments, sessions, and configuration records, and forms the foundation for compliance reporting, dispute resolution, and system monitoring.

Identity, Roles, and Permission Management

The system must maintain a unified identity record for every participant, capturing sufficient information to authenticate the individual, associate them with one or more organisations, and enforce access control across all system functions. Identity data includes credentials, profile attributes, and organisational membership.

Access control in aBridgeAI is fully dynamic: permissions are not hardcoded to roles but are defined and modified at runtime by authorised actors. The system must therefore maintain a permission model that records which actions each role may perform, which roles are assigned to which users within which organisational scope, and which role assignments were granted by whom and when. Four canonical roles exist — administrator, organisation manager, teacher, and student — but the permission sets associated with these roles must be configurable rather than fixed, and role assignments must be scoped to the organisational or course context in which they apply.

The system must support the case where a single user holds different roles in different contexts: a user may be a teacher in one course and a student in another, and their data access and action eligibility must reflect the role active in each specific context rather than a single global assignment.

Organisations, Careers, and Courses

Organisation managers are responsible for defining the structural containers within which learning occurs. The system must capture the definition of each organisation and its associated managers. Within each organisation, managers define *careers* — structured learning pathways representing a target professional competency or qualification. A career groups an ordered collection of courses that together constitute the pathway, and the system must record this membership and ordering so that career-level progress can be computed from course-level outcomes.

Each course must be described by its title, subject domain, and associated tags that enable discovery and categorisation. The system must record which teachers are assigned to deliver the course, which students are enrolled, and the dates on which these assignments were made. Course publication state must be tracked so that content remains invisible to students until explicitly released. Courses are subdivided into lessons, each of which carries an ordering index that determines the sequence in which students encounter the material.

Learning Material and Knowledge Ingestion

Teachers upload source materials — documents, videos, audio recordings, and presentation files — as the raw input from which the system generates structured learning content. The system must record the identity of each uploaded file, its type, its storage location, its association with a specific lesson, and the processing status of the AI ingestion pipeline acting upon it.

The AI ingestion pipeline transforms raw uploads into a structured knowledge representation. The system must store the resulting knowledge units — summaries, concepts, examples, and their relationships — in a form that preserves their origin in specific source files and their association with the lesson they belong to. These knowledge units serve as the retrieval corpus for all subsequent AI generation tasks and must therefore be queryable by semantic similarity as well as by structural membership. The system must record whether each knowledge unit was generated by the AI pipeline or subsequently edited by a teacher, and maintain the full edit history to support content auditing.

Quiz Configuration and Generation

Before quiz generation, teachers configure the parameters that govern both the generation process and the subsequent SM-2 scheduling behaviour. The system must capture the following configuration demands per lesson:

- The desired number of questions.
- The default difficulty distribution across easy, medium, and hard questions.
- Any supplementary instructions provided to the AI generation model to constrain the topic scope or stylistic register of generated questions.
- Whether teacher review is required before questions are published to students.
- The SM-2 scheduling parameters: the initial Easiness Factor EF_0 , the minimum EF required per card before a student satisfies the lesson unlock condition $EF_{\min}^{\text{unlock}}$, the coverage threshold τ defining what proportion of cards must meet this floor, and whether SR reminders are active for this lesson.
- Optional prerequisite criteria: a reference to a prior lesson and the minimum EF that students must have achieved in that lesson before this quiz becomes accessible.

The system must also store, per individual quiz card, a teacher-assigned *expected response time* — the duration within which a student demonstrating confident recall is

expected to answer the question. Unlike a formula-derived baseline, this value is set explicitly by the teacher during the review phase, informed by the teacher’s domain knowledge of the question’s cognitive demand. The expected response time is the primary normalisation baseline for computing the time ratio ρ used in Q derivation, and must therefore be a mandatory field before any card is published. The system must additionally capture a per-card *expected EF ceiling* — the EF value the teacher considers achievable for that card given its difficulty — enabling fine-grained configuration of unlock criteria at the card level rather than only at the lesson level.

Quiz Review and Publication

After generation, each quiz card enters a review workflow. The system must track the review status of every card — pending, approved, edited, or rejected — along with the identity of the reviewing teacher and the timestamp of the review action. Where a teacher edits a generated card, the system must preserve both the original AI-generated content and the edited version to support quality analysis of the generation pipeline over time.

A card may only be published — and therefore made visible to students — once it has been approved or edited by a teacher and its expected response time has been explicitly confirmed. The publication state of individual cards must be tracked independently from the publication state of the lesson as a whole, allowing teachers to publish a subset of cards while continuing to review the remainder.

Student Quiz Interaction and SM-2 State

At the *session level*, the system must capture when the review session began and ended, how many cards were reviewed, and summary statistics including the mean Q value and mean EF across cards reviewed in that session.

At the *answer level*, the system must capture for each individual card response: the answer given, whether it was correct, whether a hint was used, the actual response time in milliseconds, the derived Q value, and the EF and interval values both before and after the SM-2 update. This granular record is necessary both for computing the performance metrics described in Section 5.3.3 and for supporting teacher review of individual student trajectories.

The SM-2 state for each student-card pair — comprising the current EF, the current interval, the repetition count, and the next scheduled due date — must be stored as an independent record isolated from all other students. No student’s scheduling state may be derived from or influenced by the state of any peer on the same card.

Interview Configuration and Question Generation

Teachers define interview sections by specifying a set of *desired outcomes* — descriptions of the competencies or demonstrations the student must satisfy to pass the interview. The system must record each outcome, its association with a specific lesson, and its ordering within the outcome set. Outcomes are the authoritative reference against which both question generation and post-session assessment are performed.

From these outcomes, the AI generation model produces a bank of sample questions. The system must record each generated question, its association with the outcome it is designed to probe, and whether it has been reviewed and approved by the teacher. Critically, no expected answer or model answer is stored alongside interview questions — by design, the assessment is outcome-based rather than answer-keyed, and exposing model answers would undermine the open-ended nature of the interview.

The system must capture teacher-defined interview configuration parameters including the maximum number of attempts permitted per student, the minimum number of outcomes that must be satisfied for a pass verdict, whether EF progression in the associated quiz is locked until the interview is passed, and any supplementary instructions provided to the AI assessment model.

Student Interview Sessions

Each student interview session is an independent interaction between the student and a dedicated model instance. The system must record the session’s association with a specific interview configuration, the student’s identity, the attempt number, the input mode used (voice, text, or both), and the session’s start and end timestamps.

The system must store the full session transcript — either as structured text or as a reference to a stored audio file in the case of voice input — to support post-session AI assessment and to provide an auditable record of the interaction. The transcript must be linked to the specific questions drawn from the question bank during that session, along with the student’s response to each.

At the conclusion of each session, the AI assessment model evaluates the transcript against the defined outcome checklist. The system must store the per-outcome verdict — met or not met — along with the model’s internal reasoning for each verdict. The internal reasoning is retained for audit and quality assurance purposes but is never exposed to the student. The student-facing result is limited to a binary pass or fail at the session level, with no rubric, no per-outcome breakdown, and no score.

Progress Tracking and Career Readiness Reporting

The system must aggregate assessment outcomes into progress representations at multiple levels of granularity for both students and teachers.

At the *lesson level*, progress must reflect the student’s current SM-2 state across quiz cards — specifically the Knowledge Retention Estimate $\hat{R}_{s,\ell}$, the Progression Readiness against the teacher-configured unlock threshold, and the Review Compliance Rate $\rho_{s,\ell}$ — as well as the pass/fail status of the associated interview section if one exists. A lesson is considered complete only when both the quiz unlock condition and the interview pass condition are satisfied.

At the *course level*, progress must be computed as the proportion of lessons completed relative to the total lessons in the course, weighted or unweighted according to teacher configuration. This course completion percentage is surfaced to both the student and the assigned teachers.

At the *career level*, the system must aggregate course completion percentages across all courses belonging to a career to produce a *Career Readiness* score for each enrolled student. This score represents the student’s overall progression through the defined competency pathway and is reported to both the student and the organisation manager. The system must retain historical snapshots of career readiness over time to support longitudinal reporting and to allow managers to track cohort progression trends.

Teacher-facing views must additionally support class-wide aggregations — mean retention estimates, interview pass rates, and compliance rates across all enrolled students — to enable identification of systematic content gaps or cohort-level disengagement patterns that warrant intervention at the course or lesson level rather than at the individual student level.

Notifications and Review Reminders

The system must maintain a notification record for each scheduled or triggered communication to a student or teacher. For SR reminders, the system must store the scheduled delivery time — derived from the next due date of the student’s cards — alongside the recipient, the lesson in scope, and the delivery status. Review reminder notifications are only generated for lessons where the teacher has enabled the repetition reminder flag in the quiz configuration.

Additional notification types include lesson unlock events triggered when a student satisfies the progression criteria, interview result notifications delivered after AI assessment completes, and course announcements initiated by teachers or managers. All notifications

must be associated with the entity that triggered them — card, lesson, session, or course — to support deep-linking from the notification to the relevant system view.

AI Generation and Assessment Logging

Every interaction with an external AI model — whether for knowledge ingestion, quiz generation, interview question generation, or post-session outcome assessment — must be logged with sufficient detail to support cost monitoring, quality auditing, and model performance analysis. The log must capture the triggering actor and entity, the full prompt submitted to the model, the model identifier and version, token counts for both input and output, estimated cost, response latency, and the success or failure status of the call. Where generation fails, the error detail must be retained to support retry logic and failure analysis.

Deduplication lock records must be maintained for generation jobs that should not be triggered concurrently — specifically quiz generation and interview question generation — to prevent duplicate content being produced when a teacher saves a configuration in rapid succession. These lock records carry an expiry time after which a new generation job may proceed.

4.4 System Modeling

This section provides a visual and structured representation of the functional behavior of the aBridgeAI system. The system is modeled using Unified Modeling Language (UML) artifacts, primarily Use Case Diagrams, to describe how each actor interacts with the major functional modules of the platform.

The system is decomposed into six functional modules: Authentication and Access Control, Organization and User Management, Course and Material Management, Learning Loop and Adaptive Quiz, Application Loop and Mock Interview, and Analytics, Reporting and Notifications.

4.4.1 Overall Use Case Diagram

The Overall Use Case Diagram provides a macroscopic view of the aBridgeAI system boundary and the major interactions between the system and its authorised actors. The primary actors include Students, Teachers, Educational Managers, and IT Administrators.

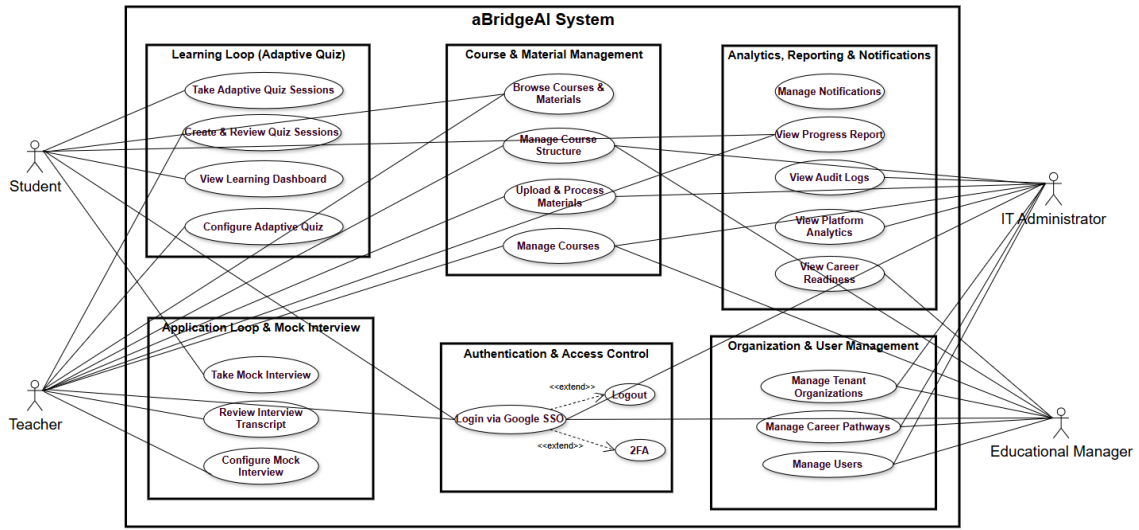


Figure 4.1: Overall Use Case Diagram of the aBridgeAI System

Note. In this diagram, authentication is treated as a prerequisite for all protected use cases rather than as a repeated *«include»* relationship.

4.4.2 Detailed Use Case Diagrams and Use Case Specifications

The detailed use case diagrams decompose the overall system into module-level views. Each module-level diagram is followed by use case specifications that illustrate the most important workflows of that module. To keep the main report concise, only the most critical use cases are described in this section. The complete set of use case specifications is provided in Appendix C.

Authentication and Access Control Module

This module covers Google SSO authentication, logout, authenticated session management, multi-factor authentication, role-based and permission-based access control, and security-related audit events.

The diagram highlights that all users must authenticate before accessing protected features. It also shows that sensitive account-security operations require recent second-factor verification, while access to backend routes and frontend UI actions is controlled by the authenticated user's active roles and effective permissions.

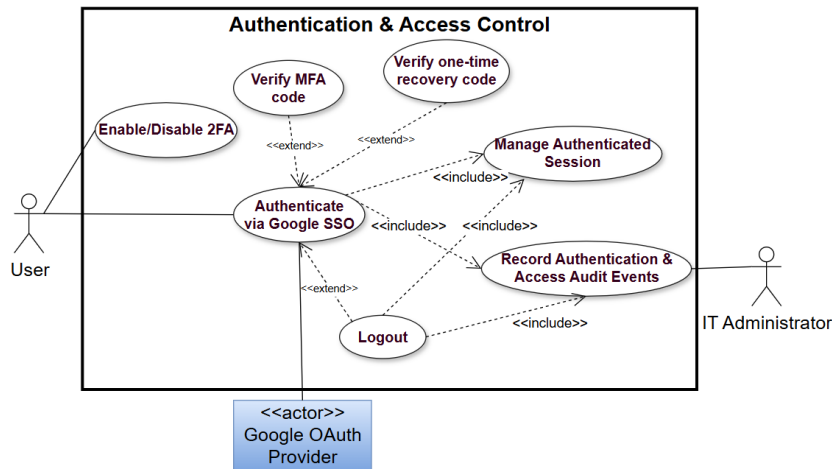


Figure 4.2: Detailed Use Case Diagram: Authentication and Access Control Module

The full set of use case specifications for this module is provided in Appendix C.

Organization and User Management Module

This module supports multi-tenant organisation management, user onboarding, organisation memberships, user profiles, domain-based auto-provisioning, and tenant-scoped access control.

IT Administrators are responsible for provisioning and managing tenant organisations, organisation domains, and account status. Educational Managers manage organisation-level learning operations such as bulk student enrollment, teacher-course assignment, and career pathway definition. The module also ensures that Educational Manager actions and queries remain scoped to their own tenant organisation.

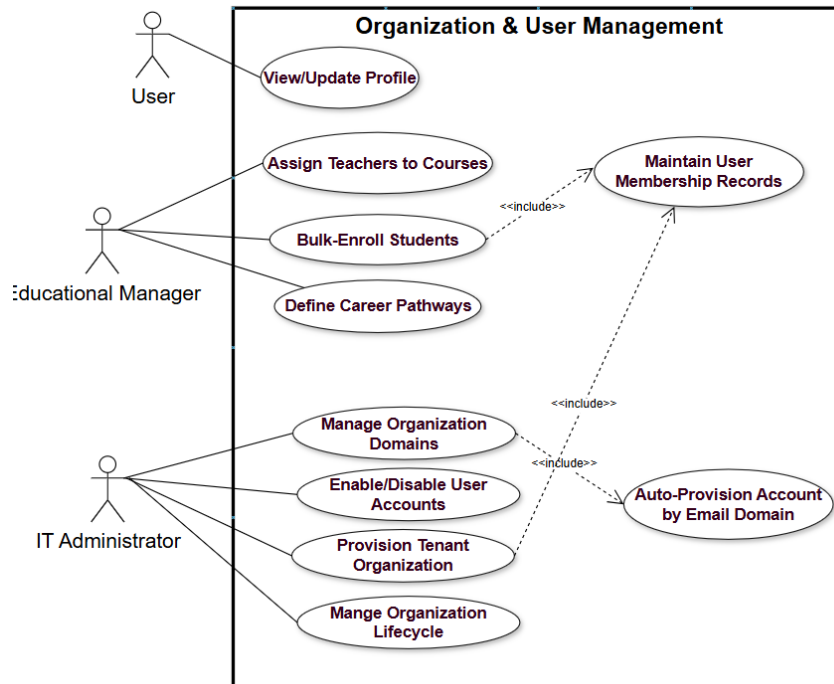


Figure 4.3: Detailed Use Case Diagram: Organization and User Management Module

Representative use case. The following use case specification illustrates tenant organisation management. The full set of use case specifications for this module is provided in Appendix C.

Table 4.3: Representative Use Case Specification: Manage Tenant Organisations

Use Case ID	UC-ORG-01
Use Case Name	Manage Tenant Organisations
Actors	IT Administrator
Related Requirements	FR-2.1, FR-2.2, FR-2.7, FR-2.9
Description	Allows IT Administrators to provision tenant organisations, manage organisation lifecycle states, configure organisation domains, and enable domain-based account auto-provisioning.
Pre-conditions	The user is authenticated as an IT Administrator.
Continued on next page	

Table 4.3 – Continued from previous page

Post-conditions	A tenant organisation is created or updated, and its domain and lifecycle configuration are saved.
Main Flow	1. The IT Administrator opens the Organisation Management dashboard. 2. The administrator creates a new tenant organisation or selects an existing one. 3. The administrator enters or updates the organisation's basic information. 4. For a new organisation, the administrator assigns the initial Educational Manager. 5. The administrator configures the organisation lifecycle status. 6. The administrator attaches one or more email domains to the organisation. 7. The administrator enables or disables auto-provisioning for each domain. 8. The system saves the organisation configuration.
Alternative Flows	3a. Invalid organisation information: 1. The system highlights invalid or missing fields. 2. The organisation is not saved until the fields are corrected. 6a. Domain already attached elsewhere: 1. The system rejects the domain assignment. 2. The administrator is informed that the domain is already registered.

Course and Material Management Module

This module allows Teachers and Educational Managers to manage course lifecycles, organise modules and learning items, configure prerequisites and unlock rules, upload materials, manage material versions, and monitor ingestion progress.

The diagram also shows the student-facing side of the module, where enrolled Students can browse published courses, view course details, and access approved materials. Material visibility is controlled separately from course publication status, ensuring that students

only access content that is both published and explicitly marked visible.

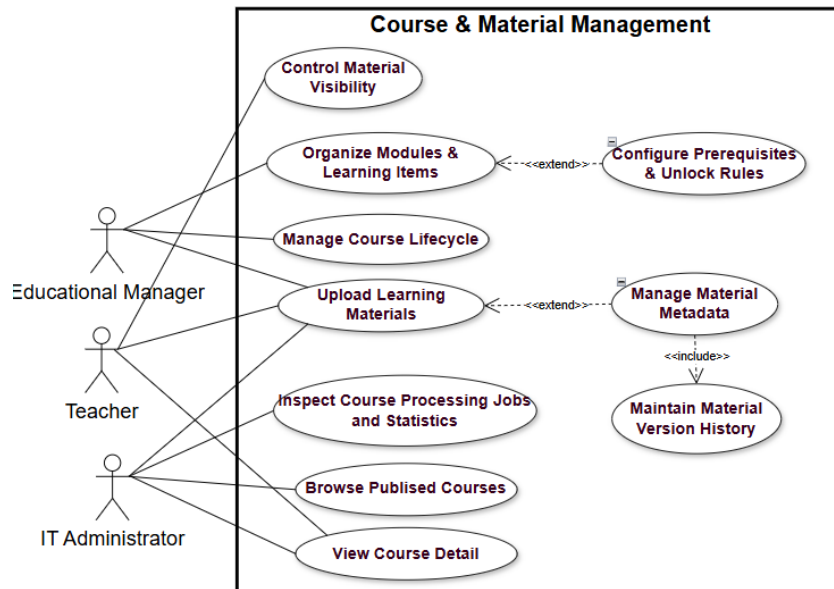


Figure 4.4: Detailed Use Case Diagram: Course and Material Management Module

Representative use case. The following use case specification illustrates the upload and ingestion workflow for learning materials. The full set of use case specifications for this module is provided in Appendix C.

Table 4.4: Representative Use Case Specification: Upload and Process Learning Materials

Use Case ID	UC-COURSE-02
Use Case Name	Upload and Process Learning Materials
Actors	Teacher, Educational Manager
Related Requirements	FR-3.3, FR-3.4, FR-3.5
Description	Allows Teachers to upload supported learning materials, manage metadata, link materials to lessons, monitor ingestion progress, and maintain material version history.
Continued on next page	

Table 4.4 – Continued from previous page

Pre-conditions	The actor has course-authoring permission and has selected a target lesson.
Post-conditions	The material is stored, linked to the lesson, processed by the ingestion pipeline, and versioned when updated.
Main Flow	1. The Teacher opens a lesson in authoring mode. 2. The Teacher selects “Upload Material”. 3. The Teacher uploads a supported file using a standard or resumable upload flow. 4. The system stores the material and records upload metadata. 5. The Teacher links the material to one or more lessons. 6. The system queues the material for ingestion and processing. 7. The system displays processing status, percent completion, and any failure reason. 8. If a replacement file is uploaded, the system creates a new material version and retains prior versions for rollback or re-processing.
Alternative Flows	3a. Unsupported format or failed upload: 1. The system rejects the upload. 2. The Teacher is shown an error message. 7a. Processing fails: 1. The system marks the material version as failed. 2. The failure reason is displayed to the Teacher.

Learning Loop and Adaptive Quiz Module

This module represents the core learning mechanism of aBridgeAI, combining AI-assisted quiz question authoring, teacher review, adaptive quiz attempts, quality-score derivation, and hybrid SM-2 spaced-repetition scheduling.

Teachers create or review quiz questions, configure expected response time T_{exp} , spaced-repetition parameters, unlock thresholds, and prerequisites. Students take quiz sessions, during which correctness, hint usage, and response time are recorded. These values are used to derive the per-card quality score Q and update each student’s spaced-

repetition state. The module also supports lesson access gating, due-card reminders, remediation links, and learning-loop dashboards.

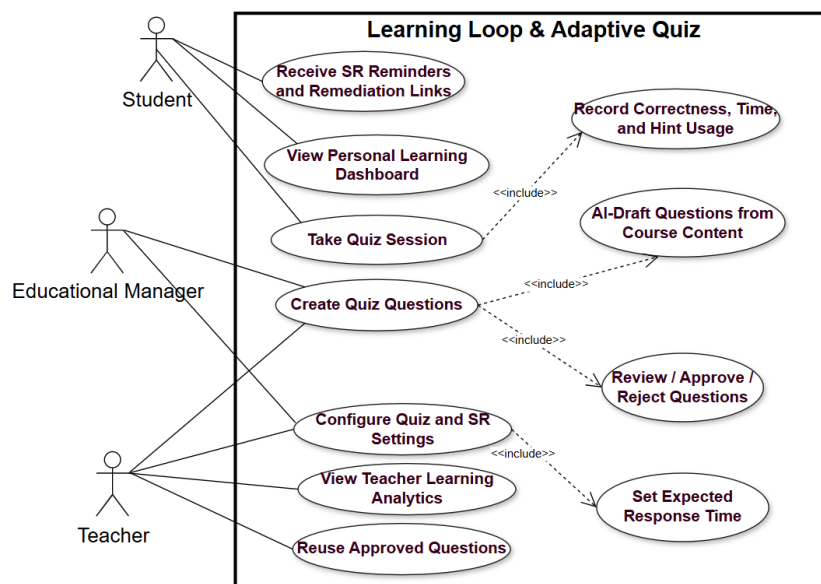


Figure 4.5: Detailed Use Case Diagram: Learning Loop and Adaptive Quiz Module

Representative use case. The following use case specification illustrates the adaptive quiz workflow, including Q derivation and hybrid SM-2 scheduling. The full set of use case specifications for this module is provided in Appendix C.

Table 4.5: Representative Use Case Specification: Configure and Take Adaptive Quiz

Use Case ID	UC-LEARN-02
Use Case Name	Configure and Take Adaptive Quiz
Actors	Teacher, Student
Related Requirements	FR-4.2, FR-4.3, FR-4.4, FR-4.5
Description	Covers quiz configuration, student quiz attempts, quality-score derivation, hybrid SM-2 spaced-repetition updates, and lesson access gating.
Continued on next page	

Table 4.5 – Continued from previous page

Pre-conditions	The quiz contains approved questions, and the Student is enrolled in the course.
Post-conditions	The Student’s answers are recorded, quality score Q is derived, spaced-repetition state is updated, and lesson access is recalculated.
Main Flow	1. The Teacher configures quiz and spaced-repetition settings, including T_{exp}, initial EF, EF unlock threshold, coverage threshold τ, prerequisites, and optional interview-pass requirements. 2. The system blocks publication until T_{exp} is set for every live question. 3. The Student opens an available quiz session. 4. The Student answers quiz cards, optionally using hints. 5. The system records correctness, response time, and hint usage for each answer. 6. The system derives a quality score Q from correctness, hint usage, and $\rho = T_{\text{actual}}/T_{\text{exp}}$. 7. The system updates the Student’s per-card spaced-repetition state using the hybrid SM-2 scheduling model. 8. The system recalculates whether the Student satisfies prerequisite completion and lesson coverage threshold τ. 9. If all unlock conditions are satisfied, the next lesson becomes accessible.
Alternative Flows	2a. Missing expected response time: 1. The system blocks quiz publication. 2. The Teacher is prompted to set T_{exp}. 3a. Retake policy or cooldown prevents attempt: 1. The system blocks the quiz attempt. 2. The Student is shown the remaining cooldown or retake limitation.

Application Loop and Mock Interview Module

This module focuses on transforming learned knowledge into applied interview performance. Teachers define desired outcomes that students must demonstrate, while the system drafts interview questions from ingested course content and associates them with those outcomes.

Students participate in mock interviews through text mode or voice mode. The system persists the interview transcript and session metadata, evaluates the student's responses against teacher-defined desired outcomes, and produces a binary pass-or-fail verdict. Students only see the final verdict, while Teachers can review the transcript and outcome-level assessment for remediation purposes. The module also records interview session activity signals to support post-session integrity review.

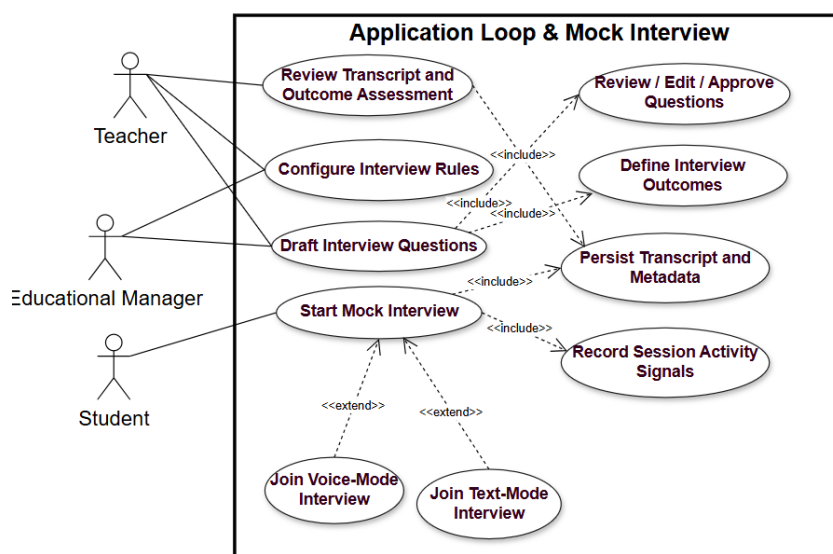


Figure 4.6: Detailed Use Case Diagram: Application Loop and Mock Interview Module

The full set of use case specifications for this module is provided in Appendix C.

Analytics, Reporting and Notifications Module

This module consolidates progress tracking, notification delivery, administrative analytics, audit-log querying, AI cost telemetry, processing-job inspection, and Career Readiness reporting.

Students can view completion progress and receive notifications, while Teachers can monitor roster progress and identify at-risk learners. Educational Managers can view organisation-scoped Career Readiness snapshots and drill down into individual learner

progress. IT Administrators can inspect platform statistics, AI usage, processing jobs, and immutable audit logs for security and compliance review.

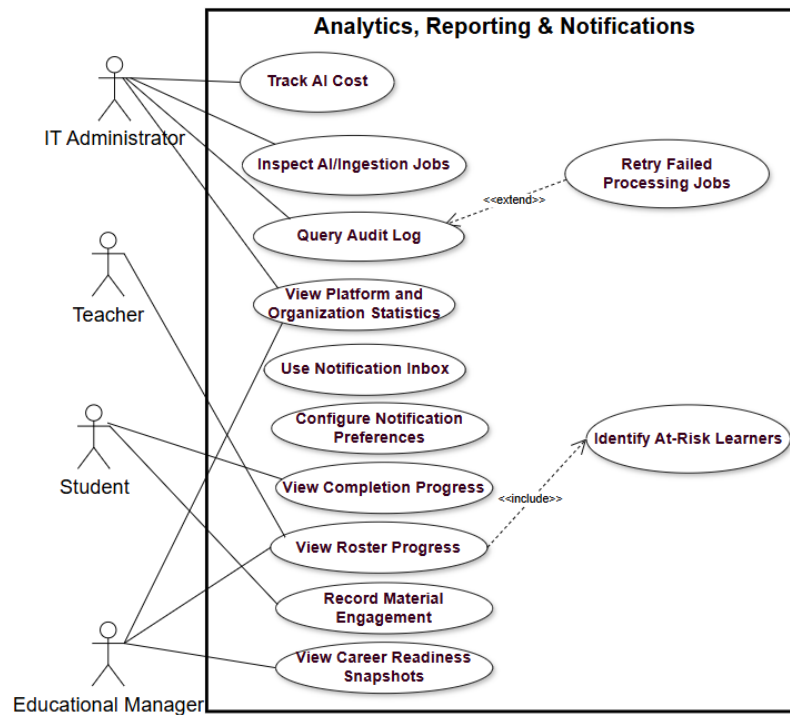


Figure 4.7: Detailed Use Case Diagram: Analytics, Reporting and Notifications Module

The full set of use case specifications for this module is provided in Appendix C.

4.4.3 Activity Diagrams

Activity diagrams model the dynamic behavior of the system by illustrating the flow of activities, decision points, and asynchronous AI-driven processes. The system activity diagrams focus on four major workflows: Authentication, Smart Content Ingestion, Human-in-the-Loop Quiz Generation, and Context-Aware Mock Interviews.

The following activity diagram illustrates the Human-in-the-Loop quiz generation workflow. The full set of activity diagrams is provided in Appendix D.

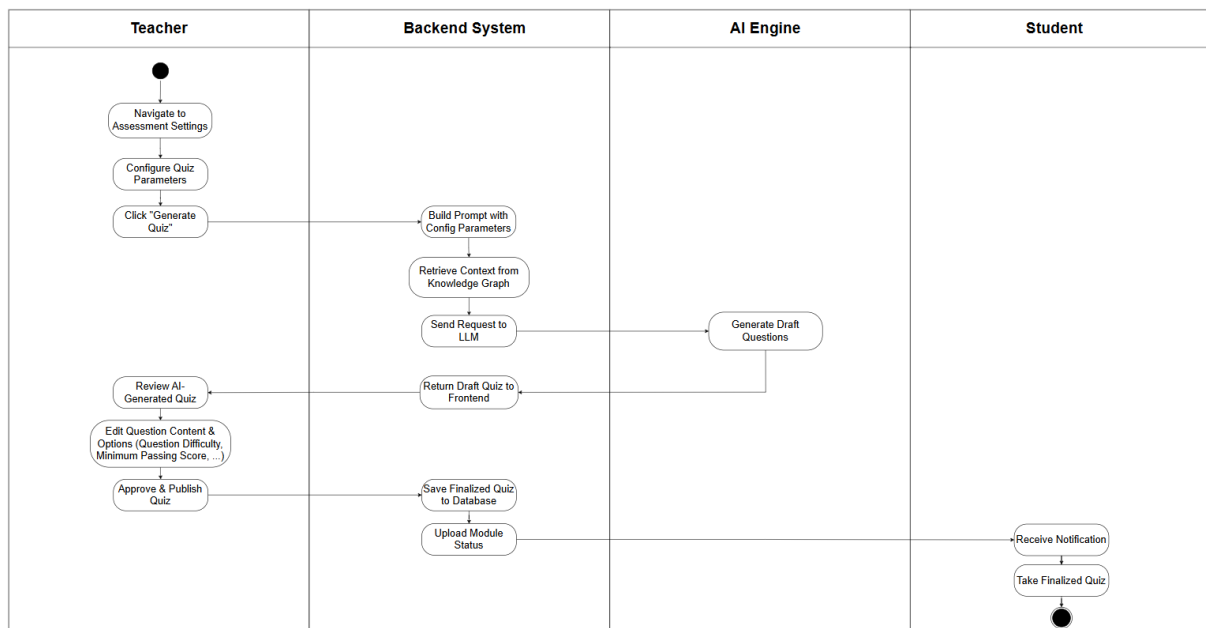


Figure 4.8: Activity Diagram: HITL Quiz Generation and Execution

The following activity diagram details the material ingestion workflow in aBridgeAI. It shows how teachers upload learning materials directly to object storage, how the backend verifies the upload and queues processing, and how the ingestion worker extracts, chunks, embeds, and stores content for retrieval.

Smart Content Ingestion Flow

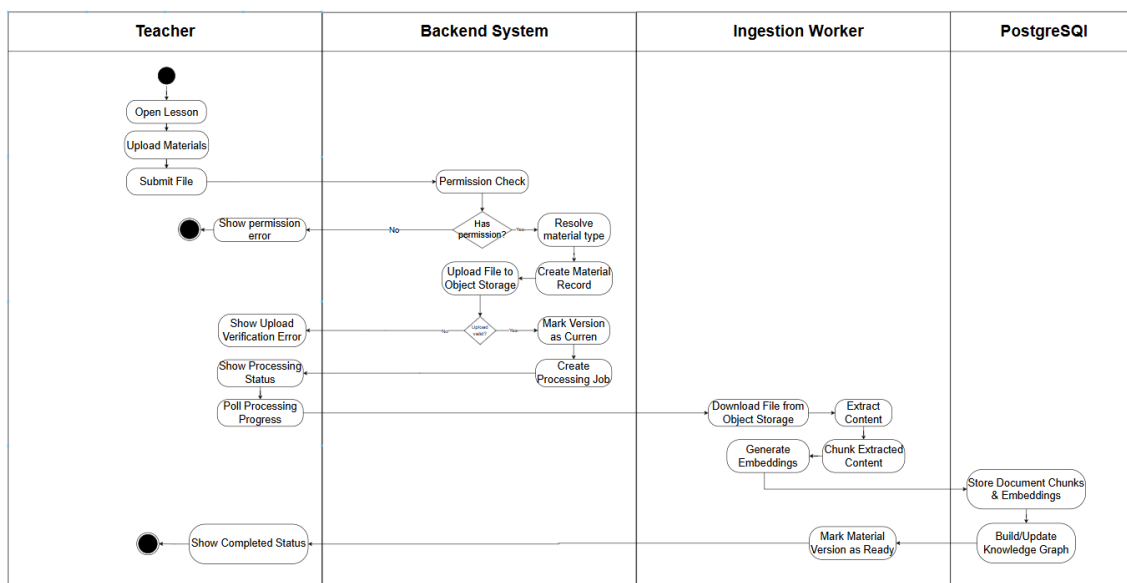


Figure 4.9: Activity Diagram: Smart Content Ingestion and AI Parsing

Detailed activity diagrams are provided in Appendix D.

5 System Design

5.1 Technology Stack Selection

The technology stack for aBridgeAI was selected by evaluating each major system component against its most viable alternatives. Each decision is grounded in the system’s core technical constraints: native AI/ML orchestration, real-time WebRTC voice processing, high-throughput vector search, and graph-based knowledge representation.

5.1.1 Backend Framework

The backend must orchestrate complex AI logic — adaptive SR scheduling, knowledge retention estimation, and real-time interview evaluation — while handling asynchronous LLM API calls and audio streams simultaneously. The orchestration layer requires native access to Python AI libraries (LangChain, scikit-learn, NumPy) without inter-language bridging overhead.

Table 5.1: Backend Framework Comparison

Feature	FastAPI	Django REST	Express.js
Language	Python	Python	JavaScript / Node.js
AI/ML Integration	Native (LangChain, scikit-learn, NumPy)	Native but synchronous ORM bottleneck	Requires Python sidecar microservices
Concurrency Model	Async I/O (asyncio)	Sync-first (WSGI); async support limited	Single-threaded event loop
Performance	High (~180MB cold start, async-native)	Moderate (~250MB, sync overhead)	Moderate (~120MB, CPU-bound limits)
Auto Schema / Docs	Built-in (OpenAPI, Swagger UI)	Requires drf-spectacular add-on	Requires manual setup

Decision: FastAPI. FastAPI is the only framework that combines Python-native

AI library access with a fully asynchronous request model. Django REST’s synchronous ORM creates bottlenecks when streaming LLM responses; Express.js requires a separate Python sidecar to run LangChain or scikit-learn, introducing network latency and operational complexity. FastAPI’s built-in OpenAPI generation also accelerates API contract development across the team.

5.1.2 Frontend Framework

The frontend must support both a standard LMS interface (course catalog, quiz editor, progress dashboards) and a real-time voice interview module with live transcript rendering and session state management. As a fully authenticated SPA with no public-facing SEO requirements, the system does not benefit from Server-Side Rendering (SSR) — making a lightweight CSR-first toolchain the optimal choice.

Table 5.2: Frontend Framework Comparison

Feature	Vite + React + TanStack Query	Next.js (App Router)	Angular
Bundle Size	~42KB (minimal runtime) [40]	~92KB (RSC runtime overhead) [40]	Large (full framework)
HMR Speed	<50ms (native ESM) [41]	Slower (Turbopack improving) [41]	Moderate
Complexity	Low (standard React patterns)	High (RSC boundaries, App Router, Flight payload) [42]	High (Zone.js, decorators)
Data Fetching	Fine-grained (TanStack Query: cache, invalidation, optimistic updates) [43]	Coarse (fetch caching, requires manual React Query integration) [43]	RxJS (steep learning curve)
LiveKit SDK	Official React SDK [44]	Official React SDK [44]	No official SDK
SSR / SEO	Not needed (authenticated SPA)	Built-in (unnecessary overhead for this use case)	Angular Universal (complex)

Decision: Vite + React + TanStack Query. aBridgeAI is a fully authenticated application — all routes require login, so SSR and SEO optimizations provided by Next.js yield no benefit. Next.js App Router introduces significant architectural complexity: React Server Components (RSC) require developers to reason about server/client boundaries throughout the codebase, the Flight payload serialization adds overhead at every server-client crossing, and the framework has accumulated a reputation for instability across major versions [42, 45]. Vite produces a 54% smaller initial bundle (~42KB vs ~92KB) and delivers sub-50ms Hot Module Replacement, directly improving developer iteration speed [40, 41]. TanStack Query replaces Next.js’s coarse fetch caching with fine-grained server-state management — cache invalidation, background refetching, and optimistic updates — using standard React patterns that require no framework-specific mental model [43].

5.1.3 Real-Time Communication

The AI interview feature requires sub-200ms voice latency, built-in Speech-to-Text (STT) integration, and AI agent turn-taking orchestration. The communication layer must handle continuous audio streams without TCP head-of-line blocking.

Table 5.3: Real-Time Communication Comparison

Feature	LiveKit	Twilio	Socket.io
Protocol	WebRTC (UDP)	SIP / WebRTC	WebSockets (TCP)
Voice Latency	Ultra-low (<200ms) [46]	Moderate (cloud routing overhead)	High (TCP head-of-line blocking)
AI Agent SDK	Built-in (Agents SDK, VAD, STT)	Requires manual pipeline	No native support
STT Integration	Integrated (Whisper, Deepgram)	Separate service required	Separate service required
Self-Hostable	Yes (open-source)	No (vendor lock-in)	Yes

Decision: LiveKit. LiveKit is the only self-hostable, open-source WebRTC platform with a dedicated AI Agents SDK that handles turn-taking, Voice Activity Detection (VAD), and Whisper STT out of the box. Twilio introduces vendor lock-in and cloud routing latency. Socket.io’s TCP transport causes head-of-line blocking that makes continuous voice streams unacceptable for real-time AI interviews [46].

5.1.4 Primary Database & Vector Search

The system requires ACID-compliant relational storage for user profiles, enrollment records, and session history, combined with high-throughput vector similarity search for course knowledge graph retrieval during peak concurrent usage.

Table 5.4: Primary Database & Vector Search Comparison

Feature	PostgreSQL + pgvector	Qdrant	Pinecone
Vector Throughput (99% Recall)	471.57 QPS [47]	41.47 QPS [47]	N/A (managed, opaque)
ACID Transactions	Full [48]	None	None
SQL + Vector in one query	Yes (JOIN + <-> operator)	No	No
Operational Overhead	Low (single service)	Moderate (separate cluster)	Low (managed SaaS)
Self-Hostable	Yes	Yes	No (vendor lock-in)

Decision: PostgreSQL with pgvector. PostgreSQL with `pgvector` delivers 11.4× higher vector query throughput than Qdrant at 99% recall [47], while simultaneously providing full ACID compliance for transactional data. The ability to combine vector similarity search with SQL JOINS in a single query eliminates the need for a separate vector database service, significantly reducing operational complexity. Pinecone’s managed-only model introduces vendor lock-in and prevents on-premise deployment.

5.1.5 Graph Database

The Course Knowledge Graph requires efficient multi-hop traversal to identify concept relationships, prerequisite chains, and knowledge gaps for adaptive question generation. These traversal patterns are prohibitively expensive in purely relational models.

Table 5.5: Graph Database Comparison

Feature	Neo4j	ArangoDB
Graph Traversal	Native (Cypher, index-free adjacency)	Native (AQL, multi-model)
Query Language	Cypher (declarative, readable)	AQL (verbose, general-purpose)
Plugin Ecosystem	APOC (graph algorithms, ML, path expansion)	Foxx microservices (custom JS)
Python Driver	Official (neo4j package)	Official (python-arango)
Community & Docs	Large, mature	Moderate

Decision: Neo4j. Neo4j’s index-free adjacency model makes multi-hop traversals (e.g., “find all prerequisite concepts of topic X within 3 hops”) orders of magnitude faster than ArangoDB’s document-oriented storage layer [49]. The APOC plugin library provides ready-made graph algorithms (PageRank, community detection) that directly support knowledge gap analysis. ArangoDB’s multi-model approach adds flexibility but sacrifices graph traversal performance compared to a purpose-built graph engine.

5.1.6 Caching & Task Queue

The system requires a high-speed in-memory store for JWT session tokens, API rate limiting, and background task scheduling (SR card scan, quiz generation jobs). Offloading these workloads from PostgreSQL prevents write contention during peak examination periods.

Table 5.6: Caching & Task Queue Comparison

Feature	Redis	Memcached	RabbitMQ
Throughput	>100K ops/sec [50]	>100K ops/sec	Moderate (AMQP overhead)
Latency	<1ms [50]	<1ms	Higher (protocol overhead)
Data Structures	Rich (lists, sets, sorted sets, streams)	Key-value only	Queue-only
Task Queue Support	Yes (via <code>arq</code> , Celery)	No	Yes (native)
Persistence	Optional (RDB / AOF)	None	Yes

Decision: Redis. Redis uniquely serves three roles in a single service: session cache, rate limiter, and task queue (via `arq`). Memcached matches Redis on raw throughput but lacks the data structures needed for rate limiting (sorted sets) and task queuing (streams/lists). RabbitMQ is purpose-built for message queuing but requires a separate caching solution, doubling operational overhead. Redis’s optional persistence (AOF) also ensures queued jobs survive container restarts [50].

5.1.7 Final Stack Summary

Table 5.7: Selected Technology Stack — aBridgeAI

Component	Technology	Primary Justification
Backend	FastAPI (Python)	Native AI/ML, async I/O, auto OpenAPI
Frontend	Vite + React + TanStack Query	Authenticated SPA — no SSR needed; 54% smaller bundle vs Next.js, sub-50ms HMR
Real-Time	LiveKit	WebRTC UDP, built-in AI Agents SDK
Primary DB	PostgreSQL + pgvector	Full ACID + 11.4× vector throughput vs Qdrant
Graph DB	Neo4j	Native multi-hop traversal for KG
Cache / Queue	Redis	Session cache + rate limit + task queue in one
Object Storage	Garage (S3- compatible)	Self-hostable, no vendor lock-in

5.2 System Architecture

The system architecture of aBridgeAI is designed as a modern, decoupled structure that clearly delineates the **system boundary**—separating internal processing components from external actors and third-party service providers. This separation ensures that the system can evolve independently of its environment while maintaining well-defined interfaces for all external interactions.

5.2.1 Actors and External Environment

Before describing the internal architecture, it is essential to identify all actors and external services that exist *outside* the system boundary and interact with it through defined interfaces.

System Actors (Users)

Four distinct user roles interact with the aBridgeAI system, each with specific access privileges and operational responsibilities:

1. **Student:** The primary end-user who accesses course materials, completes SR review sessions, takes assessments, and participates in AI-driven voice interviews. Students interact with the system exclusively through web browsers or mobile devices.
2. **Instructor:** The content creator and pedagogical authority. Instructors design courses, define learning outcomes, configure assessment parameters (e.g., SR thresholds, interview difficulty levels), and review aggregated student performance analytics including knowledge retention reports.
3. **Education Manager:** The organizational administrator responsible for provisioning the learning environment. This role creates and manages user accounts, establishes course structures, assigns instructors to courses, enrolls students, and oversees institutional-level analytics and reporting.
4. **IT Administrator:** The technical operations role responsible for system health monitoring, infrastructure maintenance, deployment management, and troubleshooting.

External Service Providers

The system integrates with several third-party services that reside outside the system boundary:

- **Identity Provider (Google OAuth 2.0):** Provides delegated authentication via the user's Google account. The platform issues its own session JWTs and persists second-factor enrolment (TOTP) in its internal database.
- **LLM API Provider (OpenAI / Anthropic):** Supplies LLM inference capabilities consumed by the AI Integration Layer via secure API calls.
- **Speech Services (STT/TTS):** Provides voice transcription and synthesis capabilities utilized during real-time AI interview sessions, integrated through the LiveKit agent pipeline.

5.2.2 Internal System Components

The internal architecture is segregated into specialized layers within the system boundary:

- **Frontend Application (Vite + React + TanStack Query):** Serves as the primary client interface, providing a responsive SPA environment. All routes require authentication; there is no public-facing content, making SSR unnecessary. TanStack Query manages server-state with fine-grained cache invalidation and background refetching.
- **Reverse Proxy & API Gateway (Nginx):** Acts as the single ingress point into the system boundary. Handles HTTPS/TLS termination, rate limiting, load balancing, and routes API requests and WebSocket upgrades to internal services.
- **Backend Core & Orchestration (FastAPI):** The central processing hub. Built on Python for native AI/ML ecosystem access, it handles business logic, RESTful API endpoints, async I/O via `asyncio`, SR scheduling (SM-2), knowledge retention computation, and AI pipeline orchestration.
- **Real-Time Voice Agent (LiveKit):** Manages ultra-low latency WebRTC (UDP) audio streams. Provides built-in VAD, speaker interruption handling, and conversational turn-taking for AI-driven interviews. The LiveKit Agents SDK coordinates STT, LLM inference, and TTS within a single real-time session.
- **AI Integration Layer (LangChain):** Constructs conversational chains, manages prompt engineering templates, and executes RAG pipelines for grounded question generation and interview evaluation.
- **Primary Data Store (PostgreSQL + pgvector):** Unified relational and vector database with full ACID compliance. Stores structured data and high-dimensional vector embeddings via `pgvector`, enabling combined SQL + vector similarity queries within single transactions.
- **Graph Database (Neo4j):** Stores the Course Knowledge Graph. Enables native multi-hop traversal for concept relationship queries, prerequisite chain resolution, and knowledge gap analysis via Cypher queries and APOC algorithms.
- **Cache & Task Queue (Redis):** In-memory store serving three roles: JWT session caching, API rate limiting (sorted sets), and background job scheduling via the `arq` worker framework for SR card scans and quiz generation jobs.
- **Object Storage (Garage S3):** Self-hosted S3-compatible object store for course material files, uploaded documents, and media assets. Eliminates vendor lock-in while maintaining AWS S3 API compatibility.

5.2.3 System Boundary Diagram

Figure 5.1 illustrates the high-level system architecture, delineating the system boundary that separates internal components from external actors and third-party service providers.

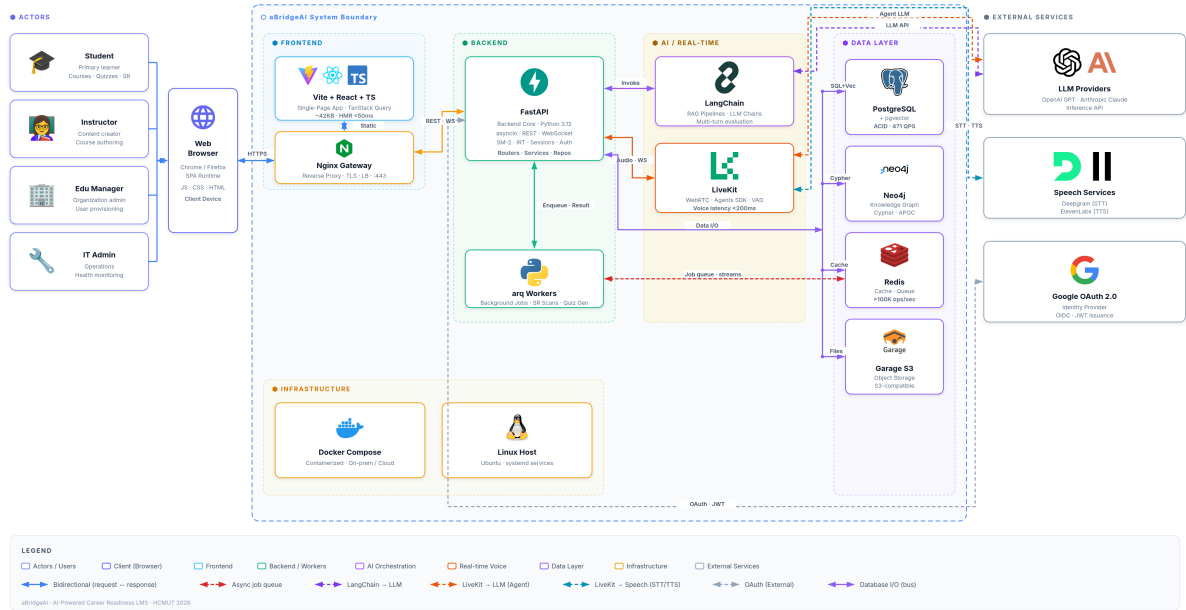


Figure 5.1: aBridgeAI System Architecture Diagram

5.2.4 Deployment Architecture

The aBridgeAI system is deployed as a modular, containerized platform. Incoming traffic is handled by **Nginx**, which serves as the reverse proxy and API gateway. The presentation layer is a **Vite + React** SPA served as static assets, while core application logic runs in a **FastAPI** backend.

For persistence, the system uses **PostgreSQL with pgvector** for relational and vector data, **Neo4j** for the Course Knowledge Graph, **Redis** for session caching and background task queuing, and **Garage** as a self-hosted S3-compatible object store for course materials and media files.

The AI processing layer uses **LangChain** for RAG pipeline orchestration and connects to external **LLM services** (OpenAI / Anthropic). In parallel, **LiveKit** handles real-time WebRTC voice sessions for AI interviews, integrating with external speech services for STT and TTS processing.

This architecture separates concerns across routing, application logic, storage, AI pro-

cessing, and real-time delivery, making the system scalable, maintainable, and deployable on both cloud and on-premise infrastructure.

5.3 Algorithm Design

5.3.1 Hybrid SM-2 and Q Derivation

aBridgeAI adopts SM-2 as the scheduling foundation for quiz-based SR, extending it with two modifications: replacement of self-reported Q with an algorithmically derived value grounded in objective response signals, and introduction of a calibration weight that dampens early EF growth until sufficient retrieval evidence has accumulated. The theoretical motivation for these departures is established in Section 3.2.1; this section specifies their formal implementation.

Teacher-Defined Expected Response Time

Each quiz card carries a teacher-assigned expected response time T_{exp} , set explicitly during the card review phase before publication. This value reflects the teacher’s domain judgment of the duration within which a student demonstrating confident, independent recall should be able to answer the question. T_{exp} is a mandatory field — no card may be published without it — and may be revised by the teacher at any point prior to or during deployment. Because T_{exp} is teacher-defined rather than formula-derived, it incorporates contextual knowledge of the question’s cognitive demand that no surface-feature heuristic could reliably approximate: a teacher setting T_{exp} for a TCP troubleshooting question draws on their understanding of how long a competent student requires to reason through the problem, not merely how long the question takes to read.

Time Ratio

Each student response is normalised against the card’s expected response time via a dimensionless time ratio ρ :

$$\rho = \frac{T_{\text{actual}}}{T_{\text{exp}}}, \quad (5.1)$$

where T_{actual} is the elapsed time in milliseconds between question presentation and answer submission. A value of $\rho < 1$ indicates the student responded faster than expected; $\rho = 1$ indicates exact correspondence with the teacher’s expectation; and $\rho > 1$ indicates the response exceeded the expected window. Because ρ is dimensionless and normalised against a card-specific baseline, it is comparable across questions of differing complexity

— a student answering a hard question in exactly the expected time produces the same ρ as a student answering an easy question in its expected time.

Q Value Assignment

The quality response value $Q \in \{0, 1, 2, 3, 4, 5\}$ is determined by two signals in sequence: first, whether the answer was correct; second, for correct answers, whether a hint was used. Within each branch, Q is resolved as a continuous function of ρ rather than through fixed threshold boundaries, ensuring that the mapping from response latency to recall quality varies smoothly with the ratio rather than producing discontinuous jumps at arbitrary cutoff values.

An incorrect answer yields $Q = 0$ unconditionally, reflecting the position established in Section 3.2.1 that partial credit for failed recall creates an advancement pathway inconsistent with competency-based progression.

For correct answers where a hint was used, the student has demonstrated assisted rather than independent recall. The ceiling is therefore $Q = 2$, with the value resolved by:

$$Q = \max(1, 2 - \lfloor \rho \rfloor), \quad Q \in \{1, 2\}. \quad (5.2)$$

This maps $\rho < 1$ and $\rho \in [1, 2)$ to $Q = 2$, and $\rho \geq 2$ — indicating the student took more than twice the expected time even with a hint — to $Q = 1$. No hint usage can produce $Q \geq 3$, preserving the SM-2 convention that scores of 3 and above require independent retrieval [5].

For correct answers where no hint was used, Q is mapped from ρ over the range $\{3, 4, 5\}$ as:

$$Q = \max(3, 5 - \lfloor 2\rho \rfloor), \quad Q \in \{3, 4, 5\}. \quad (5.3)$$

This assigns $Q = 5$ when $\rho < 0.5$ — the student answered in less than half the teacher’s expected time, consistent with effortless retrieval; $Q = 4$ when $0.5 \leq \rho < 1$ — the student answered within the expected window, consistent with controlled but fluent recall; and $Q = 3$ when $\rho \geq 1$ — the student exceeded the expected time, consistent with laboured reconstruction despite eventual correctness [6]. The floor of 3 ensures that any independent correct response, regardless of latency, is treated as a passing recall event and does not trigger an interval reset. The complete assignment rule is summarised in Table 5.8.

Table 5.8: Q value assignment rules based on correctness, hint usage, and time ratio ρ . Boundaries at $\rho = 1$ and $\rho = 2$ correspond to the teacher-defined expected response time and twice that value respectively; the boundary at $\rho = 0.5$ corresponds to half the expected time.

Correctness	Hint Used	Time Ratio ρ	Q
Incorrect	—	any	0
Correct	Yes	$\rho \geq 2$	1
Correct	Yes	$\rho < 2$	2
Correct	No	$\rho \geq 1$	3
Correct	No	$0.5 \leq \rho < 1$	4
Correct	No	$\rho < 0.5$	5

The boundaries in the no-hint branch — $\rho = 1$ and $\rho = 0.5$ — require no external calibration because they derive directly from the teacher-defined T_{exp} : $\rho = 1$ is exactly the teacher’s competency expectation, and $\rho = 0.5$ is half of it. Similarly, the hint branch boundary at $\rho = 2$ represents twice the expected time, a duration at which continued difficulty despite hint exposure reasonably indicates weak encoding rather than momentary hesitation. All boundaries are therefore interpretable as multiples of T_{exp} rather than as independently chosen constants, and their justification is grounded in the semantics of the teacher-defined baseline rather than in arbitrary numerical choices.

EF Update and Interval Scheduling

The derived Q value is passed into the standard SM-2 EF update rule:

$$EF_{\text{new}} = EF_{\text{old}} + (0.1 - (5 - Q)(0.08 + (5 - Q) \cdot 0.02)), \quad (5.4)$$

with EF bounded below by $EF_{\text{min}} = 1.3$. The review interval follows the standard SM-2 recurrence:

$$I(n) = \begin{cases} 1 & \text{if } n = 1 \\ 6 & \text{if } n = 2 \\ I(n-1) \cdot EF & \text{if } n > 2, \end{cases} \quad (5.5)$$

where n denotes the repetition count. If $Q < 3$, the item is considered failed: I resets to 1 day and n returns to zero regardless of prior history. The EF penalty is applied

before the reset, ensuring that repeated failures progressively reduce the interval growth rate even as the absolute interval restarts.

Interval Jitter

To prevent review storms caused by the simultaneous scheduling of multiple cards to the same date, a jitter factor $\varepsilon \sim \mathcal{U}(-0.1, 0.1)$ is applied multiplicatively to each computed interval:

$$I_{\text{final}} = I(n) \cdot (1 + \varepsilon). \quad (5.6)$$

The magnitude of ε is small enough to have negligible effect on retention outcomes while distributing scheduled reviews across a window of approximately $\pm 10\%$ of the nominal interval, preventing the concentration of review obligations on single calendar dates.

Calibration Weight

SM-2 initialises EF at 2.5 for all items, implicitly assuming average retention ability from the first review. In an institutional context where EF influences lesson unlock eligibility, an overestimated early EF produces intervals that grow too quickly before the algorithm has accumulated sufficient evidence of the student’s retention capacity for that item. To address this, aBridgeAI applies a calibration weight α to positive EF increments during the first three repetitions of any card:

$$EF_{\text{new}} = \begin{cases} EF_{\text{old}} + \alpha \cdot \Delta EF & \text{if } n \leq 3 \text{ and } \Delta EF > 0 \\ EF_{\text{old}} + \Delta EF & \text{otherwise,} \end{cases} \quad (5.7)$$

where ΔEF denotes the increment produced by the standard update rule and $\alpha = 0.6$. Negative increments — produced when $Q < 4$ — are applied at full magnitude regardless of n , ensuring that evidence of forgetting is incorporated immediately while evidence of retention accumulates more conservatively. Once $n > 3$, $\alpha = 1$ and the standard update rule applies without modification. This asymmetric initialisation ensures faster convergence onto the student’s true retention profile than an optimistic starting assumption would permit, consistent with the principle that conservative initial models require fewer corrective cycles than overconfident ones [5].

5.3.2 Individualised Learner Modeling

A foundational design decision in aBridgeAI is the isolation of all SM-2 state variables at the level of the student-card pair rather than the card itself. The Easiness Factor EF ,

review interval I , repetition counter n , and next scheduled due date are maintained as independent records for each unique combination of student and knowledge item. Two students reviewing the same card therefore accumulate entirely separate EF trajectories reflecting their individual retrieval histories, and no student’s scheduling is influenced by the aggregate performance of peers on shared material. This isolation is a theoretical necessity as much as an implementation choice: the same observable response pattern may reflect different underlying knowledge states in different learners depending on prior experience, domain familiarity, and cognitive strategy [27, 32]. Aggregating state across learners would systematically misrepresent individual knowledge trajectories and undermine the validity of any progression gate derived from the model.

Per-Student EF Trajectory

Because EF evolves independently for every student-card pair, each student accumulates a distinct EF trajectory for every card they review. A student who consistently answers a card correctly and quickly — producing high Q values and therefore upward EF drift — will be scheduled to review that card at progressively longer intervals, reflecting growing memory stability. A student who repeatedly fails the same card will see its EF driven toward the floor of 1.3, producing frequent short-interval reviews until stable retention is demonstrated. These trajectories are entirely independent: the high-performing student’s EF on a card has no influence on the struggling student’s schedule for the same card, and vice versa.

The calibration weight $\alpha = 0.6$ applied during the first three repetitions, specified in Section 5.3.1, further ensures that early EF growth is conservative across all student-card pairs. Since EF is initialised at 2.5 for every new card encounter regardless of the student’s general performance profile, the calibration phase provides a card-level adjustment that complements the per-student isolation: each student begins each new card at the same conservative starting point and diverges from there according to their own retrieval evidence.

Lesson-Level EF Distribution

Because EF is tracked at the card level, each student possesses a distribution of EF values across all active cards within a lesson. Let $\mathcal{C}_\ell$ denote the set of active cards in lesson ℓ , and let $EF_{s,c}$ denote the current Easiness Factor of student s on card c . The set of EF values $\{EF_{s,c} \mid c \in \mathcal{C}_\ell\}$ constitutes the student’s memory stability profile for that lesson — a card-granular representation of which knowledge components are well-consolidated and which remain fragile. This profile is the basis from which both the lesson unlock condition

and the Knowledge Retention Estimate are derived.

Lesson Unlock Eligibility

A student satisfies the unlock condition for lesson ℓ — becoming eligible to progress to the subsequent lesson — when the proportion of their cards whose EF meets or exceeds a teacher-configured minimum threshold $EF_{\min}^{\text{unlock}}$ reaches a teacher-configured coverage threshold τ :

$$\frac{|\{c \in \mathcal{C}_\ell \mid EF_{s,c} \geq EF_{\min}^{\text{unlock}}\}|}{|\mathcal{C}_\ell|} \geq \tau. \quad (5.8)$$

The numerator counts the cards in the lesson for which the student’s EF has reached the required floor; the denominator is the total number of active cards in the lesson; and $\tau \in (0, 1]$ is the minimum proportion required before the unlock condition is satisfied. Setting $\tau = 1.0$ requires every card to meet the EF floor before progression, enforcing zero tolerance for weak knowledge components. Lower values of τ permit advancement when a large majority of cards are consolidated, accommodating instructors who wish to allow progression despite isolated difficult items. Both $EF_{\min}^{\text{unlock}}$ and τ are configured per lesson by the teacher, allowing unlock stringency to vary across lessons according to the criticality of the material.

Where a teacher additionally configures an interview section for the lesson, the unlock condition above is necessary but not sufficient: the student must also satisfy the interview pass condition before progression is granted. The combined gate ensures that both retention of factual knowledge — measured by the quiz EF distribution — and demonstrated application of that knowledge — measured by the interview outcome assessment — are required for lesson completion.

5.3.3 Student Performance Metrics

aBridgeAI aggregates per-card SM-2 state into three learner-facing metrics that together characterise a student’s cognitive standing within a lesson. All three are derived exclusively from the student’s own retrieval history, with no dependence on peer performance or instructor-assigned grades. They are surfaced in both per-student and class-wide views: per-student views allow instructors to identify individuals requiring targeted intervention, while class-wide aggregations expose systematic knowledge gaps shared across the cohort that may warrant revision of the source material or quiz configuration rather than individual remediation.

Knowledge Retention Estimate

The *Knowledge Retention Estimate* $\hat{R}_{s,\ell}$ summarises the student’s memory stability profile for lesson ℓ as a single normalised score in $[0, 1]$. Each card’s EF is first normalised against the operational range of the Easiness Factor, then averaged across all active cards in the lesson:

$$\hat{R}_{s,\ell} = \frac{1}{|\mathcal{C}_\ell|} \sum_{c \in \mathcal{C}_\ell} \frac{EF_{s,c} - EF_{\min}}{EF_{\max} - EF_{\min}}, \quad (5.9)$$

where $EF_{\min} = 1.3$ is the SM-2 floor and $EF_{\max} = 2.5$ is the initialisation value, defining the full operational range. A card at the floor contributes 0 to the sum; a card at the ceiling contributes 1. The resulting $\hat{R}_{s,\ell}$ therefore reflects the average proportion of the possible EF range that has been achieved across the lesson’s cards, with values approaching 1 indicating widespread stable retention and values approaching 0 indicating widespread difficulty or recent failure resets.

Cards with repeated $Q = 0$ outcomes are driven toward $EF_{\min}$, pulling $\hat{R}_{s,\ell}$ downward and surfacing persistent knowledge gaps without requiring explicit gap detection logic. This metric is presented to both students and teachers as a continuous progress indicator without exposing the underlying per-card EF values directly, preserving a clean separation between the diagnostic signal and its raw parameters.

Progression Readiness

Progression Readiness is a binary eligibility signal derived directly from the unlock condition specified in Section 5.3.2. It evaluates whether the proportion of the student’s cards meeting $EF_{\min}^{\text{unlock}}$ satisfies the coverage threshold τ , returning either eligible or ineligible. Unlike $\hat{R}_{s,\ell}$, which reflects average stability across all cards, Progression Readiness is sensitive specifically to the distribution of cards relative to the unlock floor: a student with high average $\hat{R}_{s,\ell}$ may nonetheless be ineligible if $\tau = 1.0$ and one card remains below $EF_{\min}^{\text{unlock}}$, while a student with moderate $\hat{R}_{s,\ell}$ may be eligible if the majority of their cards are above the floor and $\tau < 1.0$. The two metrics are therefore complementary: $\hat{R}_{s,\ell}$ describes the overall health of the lesson’s knowledge profile, while Progression Readiness answers the specific question of whether the unlock condition has been met.

Review Compliance Rate

The *Review Compliance Rate* $\rho_{s,\ell}$ measures the proportion of due cards that the student reviewed within a configurable grace window Δt_{grace} following their scheduled date:

$$\rho_{s,\ell} = \frac{|\{c \in \mathcal{C}_\ell \mid t_{\text{reviewed}} \leq t_{\text{due}} + \Delta t_{\text{grace}}\}|}{|\{c \in \mathcal{C}_\ell \mid t_{\text{due}} \leq t_{\text{now}}\}|}, \quad (5.10)$$

where the numerator counts cards reviewed on time and the denominator counts all cards whose due date has passed as of the current moment t_{now} . The grace window Δt_{grace} acknowledges that students cannot review at the exact moment a card becomes due, and is configured at the system level.

This metric exists specifically to address a structural consequence of the no-decay design: as established in Section 3.2.1, aBridgeAI does not apply automatic EF penalties to overdue cards. A student who stops reviewing therefore retains their last recorded EF values indefinitely, and $\hat{R}_{s,\ell}$ remains frozen at its last state — presenting an artificially healthy retention picture to both student and teacher. $\rho_{s,\ell}$ fills this gap: a student whose compliance rate is falling while $\hat{R}_{s,\ell}$ remains stable is exhibiting engagement drift on unreviewed material, a condition the teacher dashboard flags as a warning signal. The integrity of EF as a measure of demonstrated retrieval performance is preserved — only reviews that actually occurred contribute to EF — while disengagement remains visible through a dedicated metric rather than being silently absorbed into a decaying parameter.

The relationship between the three metrics is summarised in Table 5.9.

Table 5.9: Summary of student performance metrics and their roles within the aBridgeAI learner modeling layer.

Metric	Type	What It Measures	Primary Purpose
Knowledge Retention Estimate $\hat{R}_{s,\ell}$	Continuous [0, 1]	Average normalised EF across all cards in the lesson	Diagnostic — overall memory stability health
Progression Readiness	Binary	Proportion of cards meeting $EF_{\text{min}}^{\text{unlock}}$ relative to τ	Gate — lesson unlock eligibility
Review Compliance Rate $\rho_{s,\ell}$	Continuous [0, 1]	Proportion of due cards reviewed within grace window	Signal — engagement drift detection

5.4 API Design

5.4.1 Architecture & Design Philosophy

The aBridgeAI backend is a Python **FastAPI** service exposing a versioned RESTful API at the base path `/api/v1`. It serves as the single data contract between the Vite SPA and the server. The API surface is composed by a single `create_app()` factory that mounts every per-feature router under the shared `API_V1_PREFIX`; routers themselves carry their own per-feature prefix (e.g. `/teacher`, `/me/notifications`, `/admin/processing`). The full endpoint catalogue — 195 routes across 12 features — is documented in Appendix F; the following six principles capture the design intent.

1. **Vertical-slice feature layout.** Each feature owns a self-contained directory under `abridgeai/features/<feature>/` with its own `routers/`, `services/`, `queries/`, and `schemas/` sub-packages. Routers translate HTTP to application calls; they never touch persistence directly. An import-linter contract (*“Routers do not call queries directly”*) enforces the boundary at CI time.
2. **Audience-scoped routers.** Every feature exposes one router per audience. The most common split is `learner` (the calling student, scoped under `/me/...`) versus `authoring` (instructor write operations, under `/teacher/...`), with optional `administration` (`/admin/...`), `assignment` (`/dept/...`), and `management` (`/management/...`) routers. The URL prefix communicates the access context at a glance.
3. **Bearer-token authentication via JWT.** The API uses OAuth-only authentication (Google SSO). On a successful Google callback, the server returns a JSON body containing an `access_token` and a `refresh_token`; the SPA presents the access token as an `Authorization:Bearer<token>` header on every authenticated request. There is no traditional username/password endpoint and no CSRF middleware — the bearer scheme on a header is not vulnerable to cross-site form submission.
4. **MFA gate as a dependency.** When a user has TOTP enabled, the access token is initially issued with a *pre-MFA* marker. A FastAPI dependency on every protected endpoint refuses traffic until the holder of the token completes `POST/auth/me/mfa/challenge` then `POST/auth/me/mfa/verify`; the dependency rejects with HTTP 403 and a stable `mfa_required` error code so the SPA can route to the MFA challenge screen.

5. **JSON-first, snake_case field naming.** All request and response bodies use JSON with `snake_case` field names (Pydantic v2 schemas). IDs are UUID v4, timestamps are ISO 8601 UTC strings, and enumerations are lowercase or lowercase-with-hyphens. Numeric ranges are validated by Pydantic at the FastAPI boundary, returning a 422 with a structured `detail` array.
6. **Async-first request handling.** Every router function is `asyncdef`, every database session is an `AsyncSession` from SQLAlchemy 2.0 `async`, and long-running AI work (quiz generation, interview question regeneration, material reprocessing) is dispatched to an ARQ worker pool over Redis rather than executed in-request. The HTTP layer therefore returns within hundreds of milliseconds even for AI-driven mutations; the SPA polls a `generation-runs/{run_id}` endpoint to surface progress.

The backend deploys behind an HTTPS-terminating reverse proxy and emits security headers (HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy) via a dedicated `SecurityHeadersMiddleware`. Every HTTP request is recorded by an `AuditLogMiddleware` into an `http_audit_log` table, providing a per-request trace for security and performance debugging.

Interactive documentation.

FastAPI auto-generates an OpenAPI 3.1 specification at `/openapi.json` and exposes a Swagger UI at `/docs` (and ReDoc at `/redoc`) for interactive exploration. Figure 5.2 shows the landing view of the live Swagger UI: the title bar, the **Authorize** button (used by reviewers to inject a bearer token before issuing requests), and the first audience-tagged section (`auth`). The full per-tag screenshot catalogue is in Appendix F.

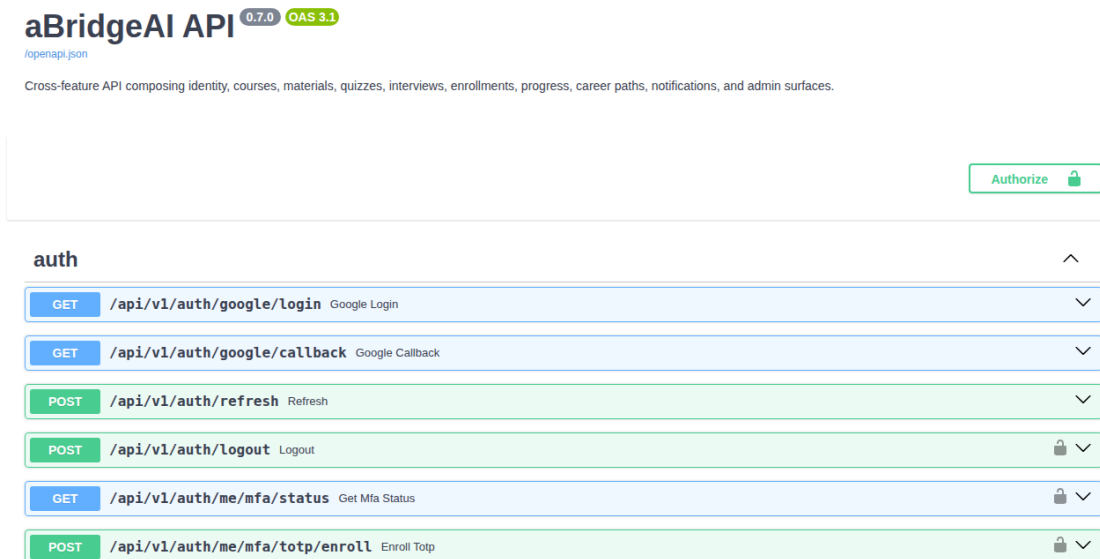


Figure 5.2: Live Swagger UI at `/docs` (rendered from the FastAPI `/openapi.json`): title, authorise control, and the `auth` audience-tag section.

5.4.2 Authentication Strategy

The platform offers a single sign-in path: Google SSO. Username/password, password-reset, and email-verification endpoints intentionally do not exist — this eliminates an entire class of credential-related bugs and shifts identity-proofing to Google’s hardened identity provider.

The application issues an access/refresh token pair as the canonical authentication primitive. The **access token** is a stateless HS256-signed JWT carrying `sub` (user ID), `sid` (session ID), `iat`, and `exp`; it is the only credential validated on every API request. The **refresh token** is a high-entropy opaque random string returned to the client once on login and rotated on every refresh. The two tokens have very different storage profiles, summarised in Table 5.10.

Table 5.10: Token pair issued on successful Google login — backend storage.

Token	Purpose	Lifetime	Backend storage
<code>access_token</code>	Authenticates every API request via <code>Authorization: Bearer</code>	15 minutes	Stateless JWT, never persisted; validated against <code>auth_sessions</code> via the embedded <code>sid</code> claim.

Table 5.10 (*continued*)

Token	Purpose	Lifetime	Backend storage
<code>refresh_token</code>	Rotates the access/refresh pair via <code>POST/auth/refresh</code>	7 days	Raw token returned to client once; only the SHA-256 hash is persisted in <code>auth_sessions.refresh_token_hash</code> . Rotated on every refresh.

Hot-path session cache.

On every authenticated request the backend must verify that the JWT's `sid` still references a live, non-revoked, MFA-cleared session. To keep this check off the database, session state is cached in Redis under `session:{session_id}` with a 60-second TTL. A reverse-index Redis SET at `user_sessions:{user_id}` stores all active session IDs for a user, enabling $O(N)$ cascade revocation when an administrator disables an account (without an unsafe `SCANMATCHsession:*`). The cache is read-through with safe fallback: any Redis error logs a warning and the request transparently falls through to PostgreSQL, so a Redis outage degrades latency rather than availability.

Login flow.

`GET/auth/google/login` returns an authorisation URL plus an opaque `state`; the SPA stores `state` and redirects the browser to Google. Google redirects back with a `code`; the SPA forwards it to `GET/auth/google/callback`, which exchanges the code with Google, finds (or rejects) the application user — the platform requires the email to match a pre-provisioned account; self-signup is not supported — and returns the token pair plus a `requires_mfa` flag. If `requires_mfa` is `true`, the SPA navigates to the MFA challenge screen and exchanges the pre-MFA token for an MFA-completed token via the challenge / verify pair.

MFA.

TOTP-based MFA is exposed as a self-service feature under `/auth/me/mfa`. Once enrolled, every fresh login produces a pre-MFA token; `POST.../challenge` opens a challenge and `POST.../verify` accepts either a TOTP code or a single-use recovery code. Sensitive operations (regenerating recovery codes, disabling MFA) require an inline proof-of-possession check so a stolen access token cannot silently turn off the second factor. The complete MFA endpoint list is in Appendix F.

5.4.3 Error Handling

Errors surface as standard FastAPI `HTTPException` responses with a JSON body of `{detail:...}` and a numeric status code. The application defines a small hierarchy of exceptions (`AppError`, `NotFoundError`, `UnauthorizedError`, `ForbiddenError`, `ConflictError`) raised by the service layer and translated to HTTP at the router boundary. For richer cases (OAuth provisioning, MFA gates, validation failures) the `detail` is an object that carries a stable `error` code so the SPA can branch deterministically:

HTTP/1.1 403 Forbidden

Content-Type: application/json

```
{
  "detail": {
    "error": "oauth_account_not_provisioned",
    "message": "This Google account is not linked to a registered user."
  }
}
```

The full status-code map (200, 201, 202, 204, 400, 401, 403, 404, 409, 422, 503) and the corresponding exception types are tabulated in Appendix F.

5.4.4 Endpoint Surface (overview)

The 195 routes mounted under `/api/v1` are partitioned across 12 features. Table 5.11 summarises the feature breakdown; per-route specifications are catalogued in Appendix F.

Table 5.11: Feature breakdown of the API surface.

Feature	Routes	Audience routers
Identity	16	auth, me, users, mfa
Access control	26	admin, organizations
Courses	48	learner (/me/courses), authoring (/teacher), assignment (/dept), administration (/admin)
Materials	17	authoring (/teacher), learner (/materials)
Quizzes	22	authoring (/teacher), learner

Table 5.11 (*continued*)

Feature	Routes	Audience routers
Interviews	21	authoring (/teacher), learner
Progress	7	learner (/me/progress), authoring (/teacher)
Career paths	2	learner
Spaced repetition	7	learner (/me), teacher
Notifications	7	learner (/me/notifications)
Admin operations	19	stats, users, processing, ai_costs, audit
Health	3	root and /api/v1 (K8s probes + SPA reach)

5.4.5 Pagination, Filtering and Idempotency

Pagination.

Every list endpoint uses cursor pagination via the generic `CursorPage[T]` dataclass at `core/pagination/cursor.py`. The response envelope is `{items:_T[],_next_cursor:_string_|_null}` the cursor is a base64-encoded UUID (or ISO-8601 timestamp for time-ordered feeds) of the last row returned, opaque to the client. Clients pass it back as `?cursor=...` paired with an optional `limit` (default 20, max 100); `next_cursor` being `null` signals end-of-list. The scheme keeps queries on a single index column, avoids the $O(N)$ skip cost of `OFFSET` on deep pages, and tolerates inserts/deletes between fetches without duplicating or skipping rows.

Filtering and sorting.

List endpoints accept feature-specific filters as query parameters (e.g. `status=published`, `role=teacher`, `q=<search>`). Sort order is fixed per endpoint and documented inline; freeform sort keys are deliberately not supported to keep the index strategy predictable.

Idempotency.

Mutating endpoints that may be retried by the SPA (in particular `POST.../attempts` during quiz submission and the bulk `POST.../quizzes/{id}/generate`) accept an optional `Idempotency-Key` header; the backend deduplicates against this key for a 24-hour window. AI generation endpoints additionally return an `Operation-Location` header pointing to `.../generation-runs/{run_id}` for progress polling.

5.4.6 API Design Summary

Table 5.12: API design summary — strategies and rationale.

Concern	Strategy	Rationale
Architecture	Vertical-slice features; routers $\rightarrow$ services $\rightarrow$ queries	Feature ownership; CI-enforced layering boundary.
Authentication	Google OAuth $\rightarrow$ JWT bearer pair (access + re- fresh)	No password surface; identity proofing delegated to Google.
MFA	TOTP + recovery codes; pre-MFA gate on the ac- cess token	Stolen access tokens cannot act before MFA.
Authorisation	Role-scoped URL prefix + per-route dependency check	Clear contextual boundary; cen- tral dependency for permission lookups.
Versioning	URL path <code>/api/v1</code>	Explicit, easy to migrate via a co-existing v2 prefix later.
Error format	FastAPI <code>HTTPException</code> with structured detail	Standard, well-supported in OpenAPI tooling and the SPA's error layer.
Pagination	Cursor (<code>next_cursor</code>) uniformly across every list endpoint	Stable deep traversal; constant cost per page; no row duplica- tion under concurrent writes.
Idempotency	Optional <code>Idempotency-Key</code> header (24-hour win- dow)	Safe SPA retries on quiz submis- sion and AI generation.
Long-running AI	Async dispatch via ARQ + <code>generation-runs/{run_ id}</code>	HTTP layer stays under hun- dreds of milliseconds; progress is observable.

Table 5.12 (*continued*)

Concern	Strategy	Rationale
Object storage	S3-compatible (Garage); presigned multipart uploads	Browser uploads bypass the application server.
Observability	<code>http_audit_log</code> middleware + <code>admin/audit/*</code>	Per-request trace surface.

5.5 Class Diagrams

The Class Diagram for the aBridgeAI platform illustrates the static structure of the backend system, capturing the core domain entities, their attributes, methods, and cardinal relationships. To manage the complexity of the AI-driven system, the architecture is modeled using Domain-Driven Design (DDD) principles, categorized into three distinct packages:

- **User Management Domain:** Implements an inheritance structure where the base abstract `User` class handles core authentication (including 2FA attributes), while `Student` and `Teacher` classes inherit and extend specific role-based functionalities.
- **Course & Ingestion Domain:** Highlights the Composition relationships essential for the RAG pipeline. A `Course` contains multiple `Materials` (storing raw data as BLOB), which are sequentially processed and decomposed into `VectorChunks` containing high-dimensional embeddings.
- **Assessment & Interview Domain:** Maps the execution flow from theory validation (`QuizAttempt`) to practical application (`InterviewSession`). It details how AI interactions (`InterviewQA`) are aggregated to automatically instantiate a personalized `GapReport`.

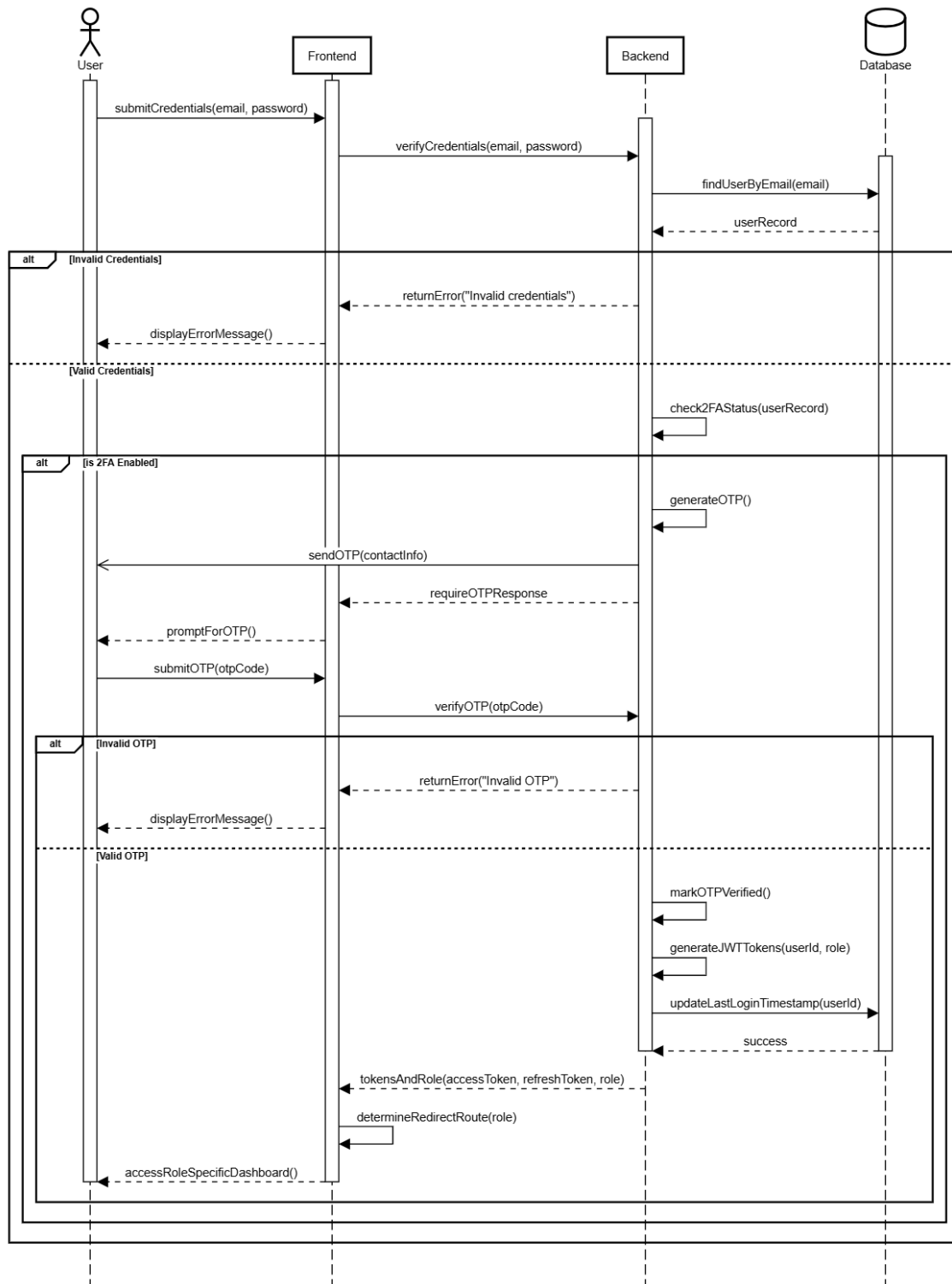


Figure 5.4: Sequence Diagram: Authentication

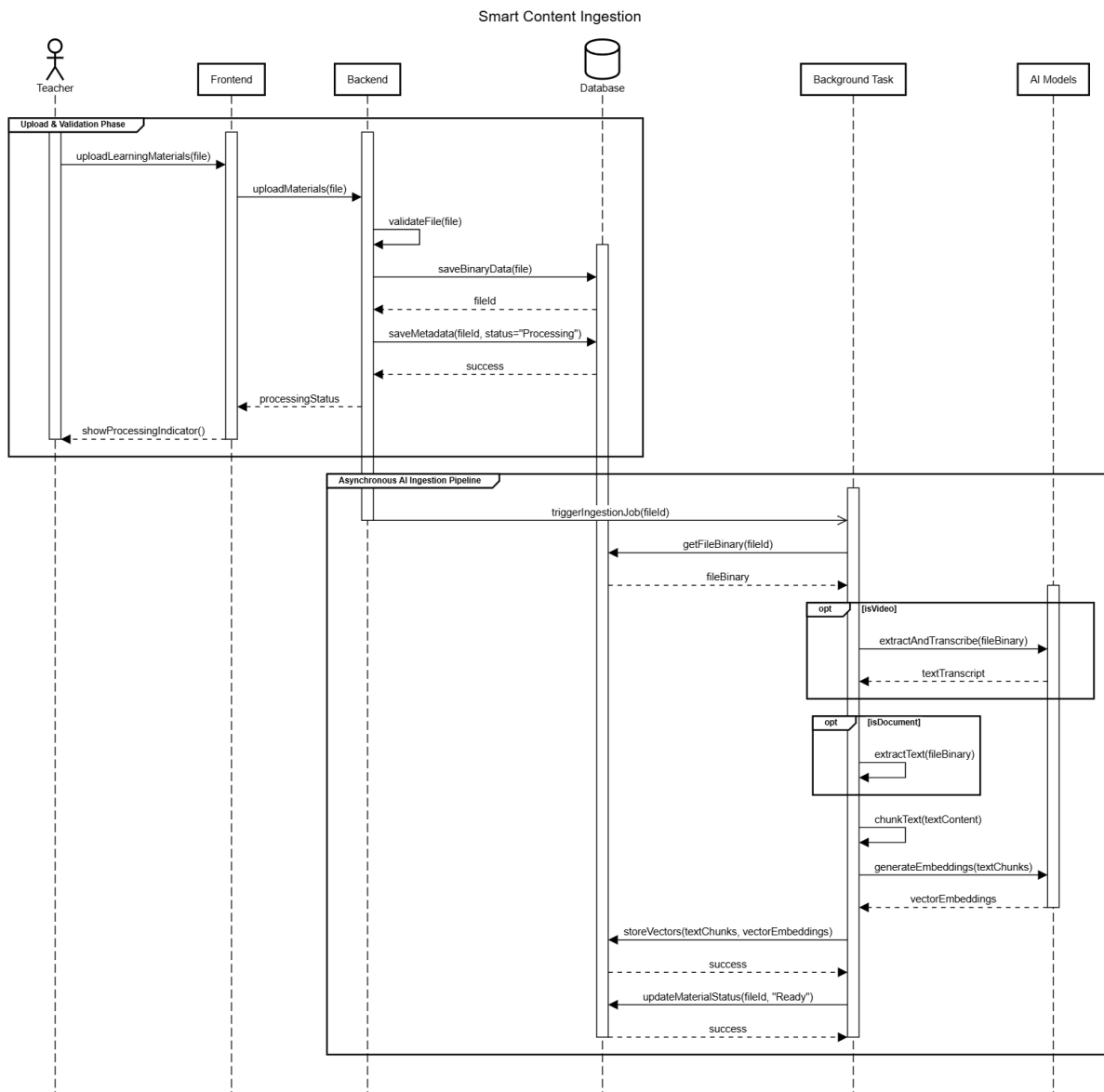


Figure 5.5: Sequence Diagram: Smart Content Ingestion

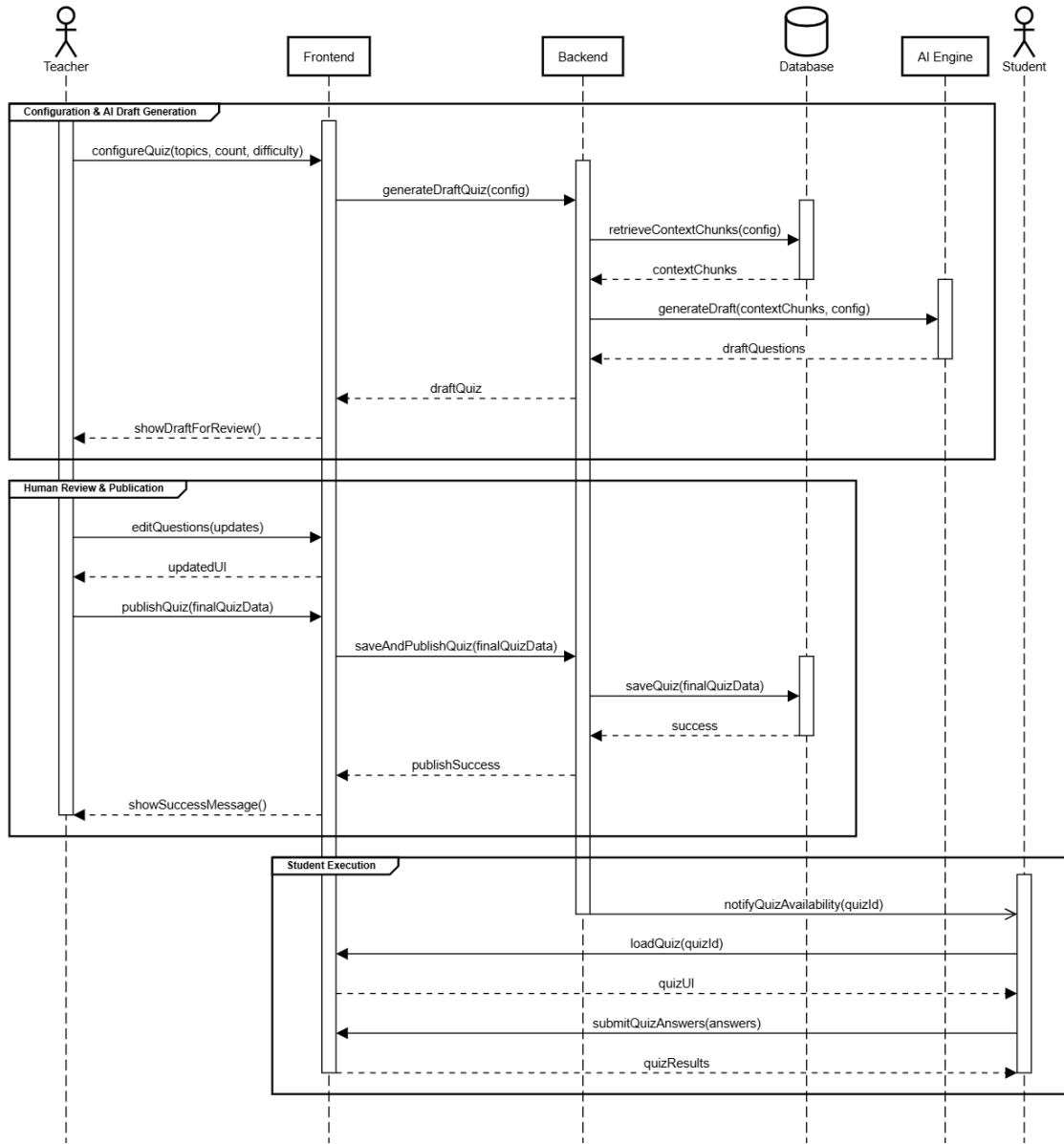


Figure 5.6: Sequence Diagram: HITL Quiz Generation and Execution

5.7 Database Design

This section presents the database design of aBridgeAI through three views: the Entity-Relationship Diagram, the logical design, and the physical design. Together, these views describe how system data is structured, related, and implemented.

5.7.1 Entity-Relationship Diagram

The ERD provides a high-level view of the main entities in the system and the relationships between them.

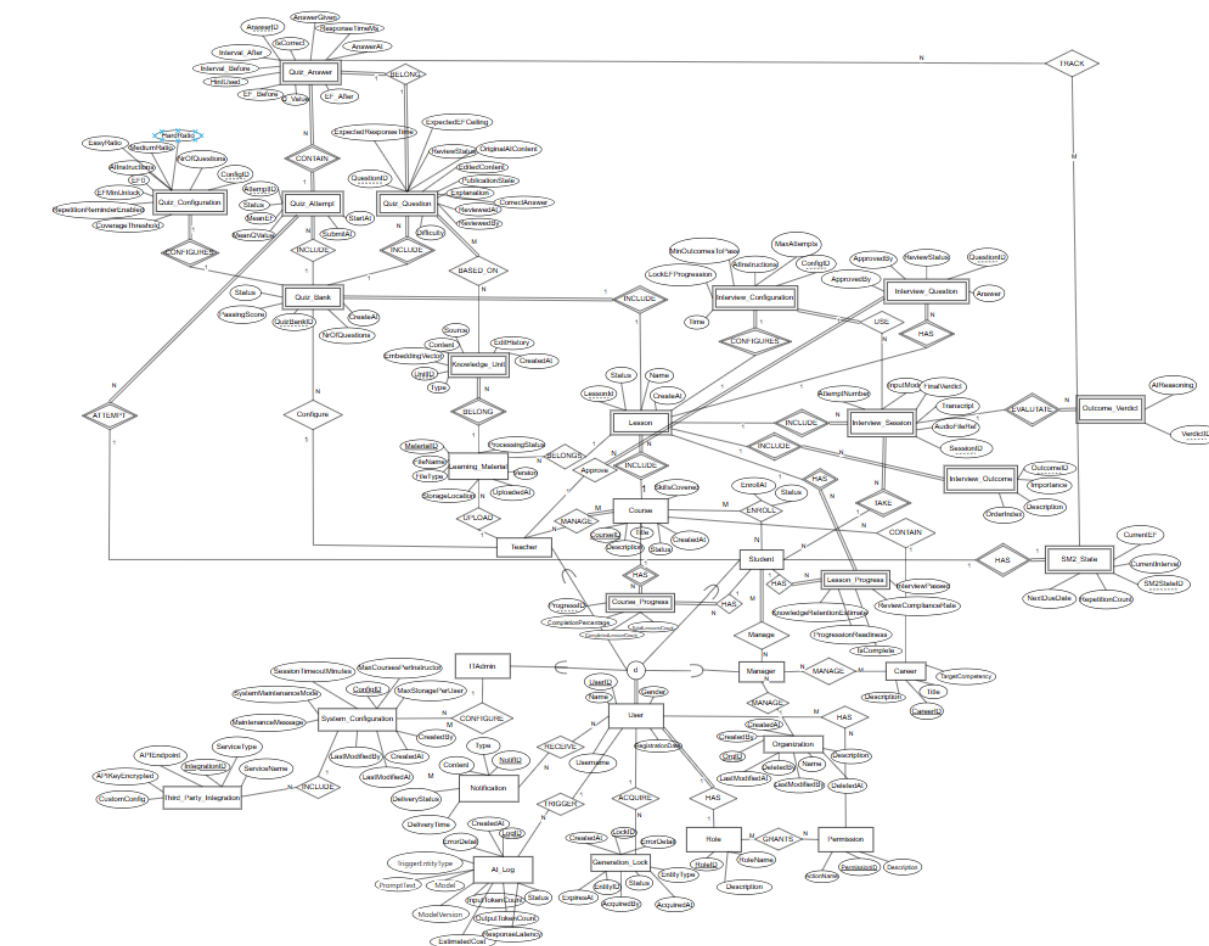


Figure 5.7: Entity-Relationship Diagram

5.8 UI/UX Design

5.8.1 Design Philosophy

aBridgeAI adopts a **light-first, academic** design language inspired by enterprise dashboards (Metronic Demo 6) — dense rows, crisp 1 px borders, restrained 4–10 px corner radii. The interface deliberately reads as a serious learning tool rather than a consumer product. Four guiding principles shape every screen:

1. **Role-based clarity** — students, instructors, and administrators see entirely distinct navigation and accent colours; the same shell component renders all three contexts driven by a per-role `navItems` prop.
2. **Progressive disclosure** — AI features (quiz generation, processing queue, cost telemetry) are surfaced contextually inside their host screens, only when the user has the required role and prerequisites.
3. **Accessibility-first typography** — body copy uses *Be Vietnam Pro*, a humanist sans-serif with full Vietnamese diacritic support, paired with a 14 px base size and 1.5 line-height for sustained reading.
4. **Responsive shell architecture** — a persistent `AppShell` renders a left sidebar that collapses from 256 px (`w-64`) to 64 px (`w-16`, icons only) below the `md` breakpoint or by user toggle.

The implementation uses `shadcn/ui` components on top of **Tailwind CSS v4**; the colour system is defined as CSS custom properties (`var(--primary)`, `var(--surface)`, ...) so that the light and dark themes share a single component layer with full parity.

Internationalisation (i18n).

Every authenticated page exposes a globe icon in the `ContentTopBar` that opens a language selector; the active language is shown next to the icon (*e.g.* “EN”). Locale strings live in `src/locales/<lang>.json` and load lazily, with English as the fallback. Navigation labels and form fields reference an `i18nKey` (such as `nav.dashboard`, `nav.career_paths`) defined in `src/lib/navigation.ts`, so adding a new locale requires only a JSON dictionary — no component changes. The platform was developed with English and Vietnamese as first-class locales, matching the choice of *Be Vietnam Pro* as the primary typeface.

5.8.2 Design Tokens

Table 5.13 consolidates the design tokens that govern the visual system. Full per-rule rationale and the complete spacing, layering, and interaction rules are documented in Appendix H.

Table 5.13: Core design tokens.

Token	Value	Usage
Page background	#f8fafc (slate-50)	Page surface in light mode.
Surface / Card	#ffffff	Cards, panels, dialogs.
Primary accent	#1e40af (blue-800)	Primary actions, active nav, links, chart series 1.
Secondary accent	#3b82f6 (blue-500)	Secondary buttons, focus rings, chart series 2.
CTA / accent	#f59e0b (amber-500)	High-attention CTAs and AI affordances.
Success	#16a34a (green-600)	Published / completed states.
Warning	#ea580c (orange-600)	Drafts, pending review.
Danger	#dc2626 (red-600)	Failed jobs, destructive actions.
Border	#e2e8f0 (slate-200)	1 px borders on cards, inputs, tables.
Typography	Be Vietnam Pro (system-ui fallback)	All body and UI text; full Vietnamese diacritic support.
Component library	shadcn/ui + Radix UI + Lucide React	Dialogs, dropdowns, sheets, tooltips, icons.
Charts	Recharts	Bar, line, area; reuses palette for chart series 1–5.
i18n keys	src/lib/navigation.ts + src/locales/<lang>.json	Translatable nav labels and copy.

Table 5.13 (continued)

Token	Value	Usage
Sidebar widths	64 px collapsed / 256 px expanded	w-16 / w-64 with auto-collapse below md.
Border radius (card)	8 px (rounded-lg)	10 px hard cap project-wide.
Base spacing unit	4 px	All spacing is a multiple of 4 px.
Max modal depth	1 stacked modal	Prevents nested-modal anti-patterns.

5.8.3 Key Screens

This subsection sketches the principal screens by role. Per-screen layouts and interaction details are illustrated in Appendix H.

Public surfaces.

The **Landing Page** (/) is the marketing entry point; it carries a hero, stats strip, category grid, and featured-courses carousel under a dedicated **TopNavBar** that is unique to this route. The **Login** screen (/login) offers Google SSO as the single sign-in path; on success, accounts with MFA enabled are routed to /login/mfa for a TOTP or recovery-code challenge before a session is issued. A **next** query parameter preserves deep links across the auth round-trip.

Student.

The **Student Dashboard** (/dashboard) keeps the layout calm: stat cards summarise enrolled courses, in-progress lessons, and recently earned milestones; a recommendations strip suggests the next course in the active career path. The **Course Catalog** (/courses) opens with a sticky search and filter bar (query, category, difficulty); enrolled courses display an inline progress ring so students can pick up where they left off. The **Course View** (/courses/\$slug/learn) splits into a sticky curriculum accordion on the left and the active lesson's content on the right (video, reading, attached resources, embedded quiz launcher). The **Quiz Session** (/courses/\$slug/quiz/\$quizId) is a focused, distraction-free interface with a countdown timer and a question-navigation grid; on submission students land on the attempt-review screen with correct answers and per-question explanations.

Instructor.

The **Teacher Dashboard** (/teacher) summarises authoring activity (total / published / draft / AI-enabled) and surfaces the most recently edited course with a one-click jump into management. The **Course Manage** screen (/teacher/courses/\$courseId) is the authoring core: modules render as drag-handle cards, expanding to reveal ordered lesson and quiz items, each with a publish-state badge and inline + **Video** / + **Reading** / + **Quiz** quick-add buttons. The **Lesson Editor** (/teacher/courses/\$courseId/lessons/\$lessonId) splits into a Markdown content area with attached resources on the left and a settings sidebar (Visibility, Lesson Type, Duration, Difficulty, Prerequisites, AI Material Hub) on the right.

The **Quiz Editor** (/teacher/courses/\$courseId/quizzes/\$quizId) is structured around three sub-tabs — *Questions*, *Settings*, and *Preview*. The Questions tab lists every item with its type badge, per-question time limit, and approval status; bulk-action controls let the instructor apply uniform timing or bulk-approve. A persistent banner reports how many questions are pending review, enforcing a deliberate approval gate between AI generation and student exposure. An **Open generator** button launches a modal that drives the AI quiz pipeline: source-lesson selection, question count, difficulty band, question types (multiple choice, true/false, short answer, fill in the blank), generation mode (*Topic* balanced spread vs. *Coverage* guaranteeing per-section coverage), and an *Append* or *Replace* policy for existing questions. Generated items land in *Pending Review* state and are visible to students only after explicit instructor approval.

Administrator.

The **Processing Queue** (/admin/processing) gives platform operators direct visibility into the AI background-job pipeline (material ingestion, embedding, quiz generation, validation): stat cards report the current backlog, status-pill filters narrow the table, and failed jobs expose a per-row **Retry** action. The **AI Costs** page (/admin/ai-costs) tracks spend on language-model and embedding APIs across a selectable time range (24 h / 7 d / 30 d); two bar charts break the spend down by *role* (embedding, ideation, chunking enrichment, generation, validation) and by *pipeline stage*, reusing the platform palette so administrators can correlate cost with backlog health on the Processing page without a context switch.

5.8.4 Navigation Architecture

The authenticated experience is built around a single role-agnostic **AppShell** component that orchestrates a fixed-position **SideNavBar** and a sticky **ContentTopBar** around the main content. The shell receives a **navItems** array as a prop (defined in `src/lib/navigation.ts`), so the same layout code serves the student, teacher, and admin route groups; role differentiation is communicated entirely through the **navItems** configuration and the active accent colour. Below the **md** breakpoint the sidebar collapses to icons-only and overlays the page content via a backdrop on toggle, keeping the main column readable on narrow screens.

The **TopNavBar** and **Footer** components are reserved exclusively for the landing page, ensuring a clean visual handoff between the marketing surface and the authenticated product.

5.9 Site Map

The application is a single-page web client built with **Vite** and **TanStack Router**, served as a static bundle from a CDN and consuming the FastAPI backend over HTTPS. All authenticated routes are nested under a single **authenticatedRoute** parent that enforces a session check and an MFA gate before any child route is allowed to mount; deep-linked routes round-trip through `/login` with a **next** query parameter so users land back on the originally requested URL after authentication.

Figure 5.10 summarises the full route tree, organised by the five role groups exposed by the platform: *Public*, *Student*, *Instructor*, *Educational Manager*, and *Administrator*. The complete per-route URL listing — with page name, dynamic parameters, and purpose for each of the 59 routes — is provided in Appendix G.

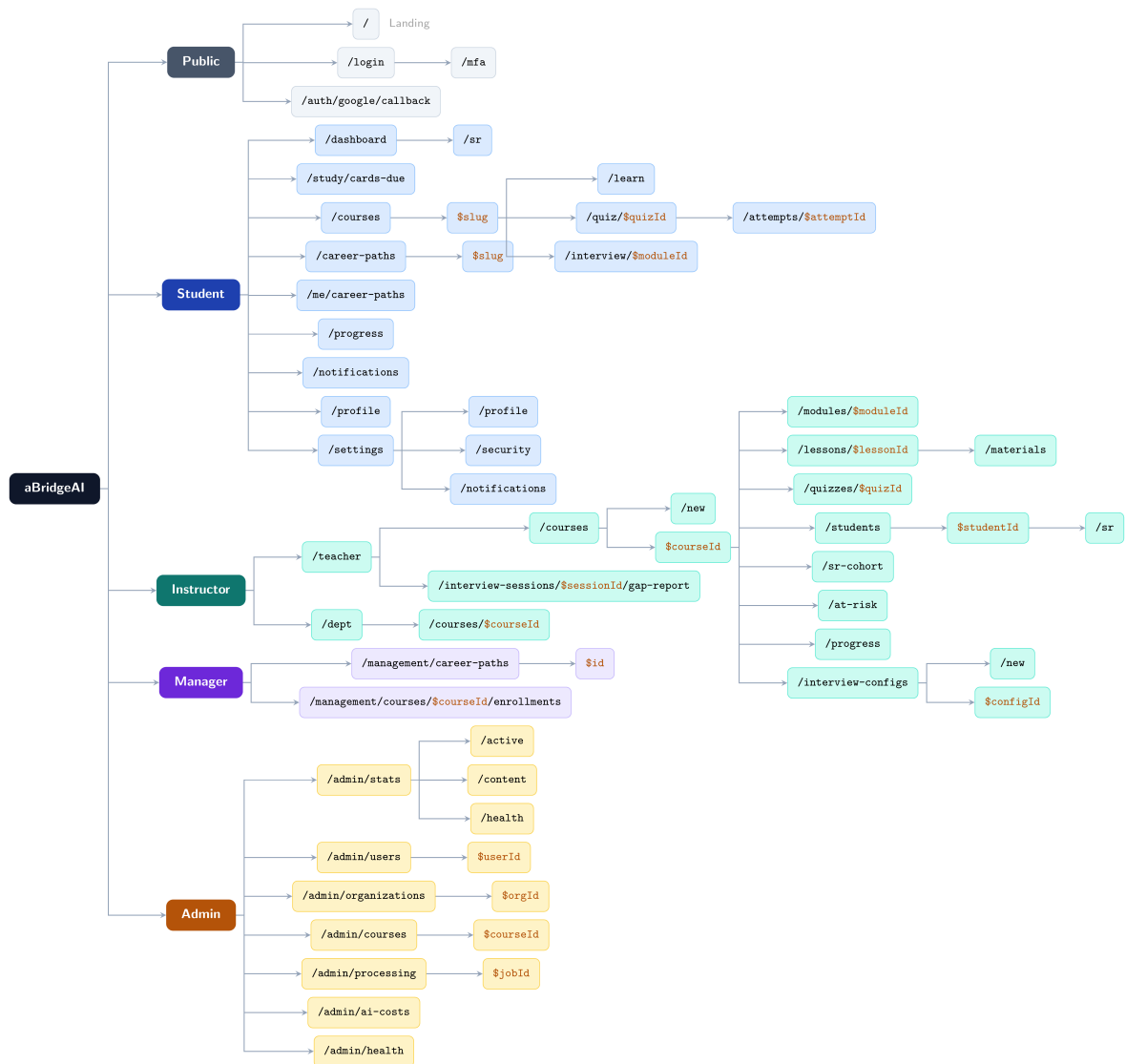


Figure 5.10: Application sitemap — colour-coded by role group: grey (public), blue (student), teal (instructor), violet (educational manager), amber (administrator). Dynamic URL segments are highlighted in amber as `$param`.

The sitemap is intentionally flat within each role group: there are no role-specific subdomains, only path prefixes (`/teacher`, `/admin`, `/management`, `/dept`), so a single deployment serves every persona. Role differentiation is enforced by the `beforeLoad` guard on the parent route plus the role-specific `navItems` array driving the sidebar. This keeps the bundle small (one client-side router for the whole platform) and makes it cheap to add new role-scoped routes — a new admin page is one new `createRoute` call that joins the `authenticatedRoute` children list.

5.10 Test Case Design

Representative test cases are summarized below to illustrate coverage across authentication, learning-loop, and interview workflows. The complete test case specification is provided in Appendix I.

Authentication and Access Control (Representative)

Table 5.14: Representative Test Cases: Authentication & Access Control

ID	Req	Level	Input	Expected Result
TC-1.1.1	FR-1.1	System	User initiates login using Google SSO	System redirects to Google OAuth and authenticates after a successful callback
TC-1.3.1	FR-1.3	System	User with MFA enabled enters a valid authenticator code	MFA verification succeeds and protected features are accessible
TC-1.5.1	FR-1.5	System	User accesses a protected backend route with sufficient permission	Request is allowed and the protected resource is returned

Learning Loop and Adaptive Quiz (Representative)

Table 5.15: Representative Test Cases: Learning Loop & Adaptive Quiz

ID	Req	Level	Input	Expected Result
TC-4.2.1	FR-4.2	System	Teacher configures quiz settings and sets T_{exp} for every live question	Quiz can be published and settings are applied to student attempts
TC-4.3.1	FR-4.3	System	Student submits a quiz answer with response time and optional hint usage	System records correctness, response time, and hint usage for the answer

Continued on next page

Table 5.15 – Continued from previous page

ID	Req	Level	Input	Expected Result
TC-4.4.2	FR-4.4	Integration	Card response produces a derived Q value	Student-card SM-2 state is updated with EF, interval, due date, calibration, and jitter rules

Application Loop and Mock Interview (Representative)

Table 5.16: Representative Test Cases: Application Loop & Mock Interview

ID	Req	Level	Input	Expected Result
TC-5.1.1	FR-5.1	System	Teacher defines desired interview outcomes without model answers	Interview section is saved with outcome-based assessment criteria
TC-5.4.1	FR-5.4	System	Eligible student starts a text-mode interview	AI interviewer begins adaptive turn-taking dialogue
TC-5.6.1	FR-5.6	Integration	Student completes an interview session	Transcript and metadata are persisted and system produces a binary pass-or-fail verdict

6 Implementation and Testing

This chapter describes the concrete implementation of the aBridgeAI backend, covering the development environment, project structure, and the realisation of each major subsystem. The implementation follows a feature-first architecture in which each domain (identity, courses, materials, quizzes, interviews, spaced repetition) is a self-contained package exposing its own models, queries, services, and API routers. Cross-cutting concerns — caching, audit logging, observability, and access control — are isolated in a shared `core` layer consumed by all features.

6.1 Development Environment and Toolchain

The backend is implemented in Python 3.12 using the FastAPI framework for asynchronous HTTP serving. Dependency management is handled by `uv`, which resolves and locks the full dependency graph in `uv.lock` to guarantee reproducible builds across developer machines and CI. The project declares its metadata and tool configuration in a single `pyproject.toml`, covering package discovery, mypy type-checking settings, bandit security rules, and ruff linting profiles.

Runtime dependencies

The core runtime stack consists of:

- **FastAPI + Uvicorn** — ASGI web framework and server.
- **SQLAlchemy 2 (async)** — ORM and query layer over PostgreSQL via the `psycopg` driver.
- **Alembic** — schema migration management; 16 versioned migration files track every DDL change from the baseline schema to the current state.
- **Redis (via arq)** — task queue for background workers and cron jobs.
- **Neo4j (async driver)** — knowledge graph storage for concept-relationship extraction.
- **Pydantic v2** — request/response validation and settings management via `pydantic-settings`.
- **tiktoken** — token counting for chunking budget enforcement.
- **Jinja2** — prompt template rendering for all LLM calls.

Code quality gates

A `.pre-commit-config.yaml` wires 13 hooks that run before every commit: `ruff` for linting and formatting, `mypy` for static type checking, `bandit` for security scanning, and `gitleaks` for secret detection. The same hooks are replicated in the GitHub Actions CI pipeline (`.github/workflows/backend-new-ci.yml`) so violations are caught both locally and on every pull request.

Local infrastructure

A `docker-compose.yml` defines the full local stack: PostgreSQL 16 with the `pgvector` extension, Redis 7, and Neo4j 5. Developers run `docker compose up -d` to bring up all dependencies before starting the application with `uv run uvicorn`.

6.2 Project Structure

The codebase follows a feature-first layout in which every domain is a self-contained package under `abridgeai/features/`. Each feature package contains the same internal layers: `models.py` (SQLAlchemy ORM), `schemas/` (Pydantic DTOs), `queries/` (database access functions), `services/` (business logic), `routers/` (FastAPI route handlers), and optionally `workers/` (background tasks) and `ai/` (AI pipeline stages). This uniform layout makes it straightforward to locate any piece of logic and to add new features without touching unrelated code.

```

abridgeai/
+-- api/                # Application entry point, router registration
+-- core/               # Cross-cutting: config, db, cache, audit, security
|   +-- audit/          # Async audit-log listener and worker actor
|   +-- cache/          # Redis client, decorators, cache-key registry
|   +-- db/             # Soft-delete mixin, conflict mapper, hard-delete guard
|   +-- observability/  # Structured logging, HTTP audit log
|   +-- pagination/     # Cursor-based pagination
|   +-- security/       # JWT creation, token hashing, security headers
+-- ai/                # Shared AI primitives
|   +-- chunking/       # Token-aware, semantic, timestamp-aware chunkers
|   +-- extraction/     # Multimodal extractors (PDF, DOCX, audio, video, ...)
|   +-- knowledge_graph/ # Neo4j builder and retrieval
|   +-- llm/            # LLM gateway, embedding client, pricing, audit
|   +-- prompts/        # Jinja2 prompt loader
|   +-- retrieval/      # pgvector, BM25, RRF fusion, MMR, role filter
+-- features/
|   +-- access_control/  # Organisations, org units, RBAC
|   +-- admin/          # Admin dashboards, AI cost tracking, audit viewer
|   +-- career_paths/   # Career path authoring and learner enrollment
|   +-- courses/        # Course, module, lesson authoring and learner view
|   +-- enrollments/    # Enrollment lifecycle management
|   +-- identity/       # Users, auth sessions, Google OAuth, MFA
|   +-- interviews/     # AI interview generation, session taking, evaluation
|   +-- materials/      # Learning material upload, ingestion pipeline
|   +-- notifications/  # In-app and email notification dispatch
|   +-- progress/       # Lesson progress tracking and analytics
|   +-- quizzes/        # AI quiz generation, taking, grading
|   +-- spaced_repetition/ # SM-2 scheduler, card review, dashboards
+-- infrastructure/     # Third-party adapters: Neo4j, S3/Garage, Google OAuth

```

Figure 6.1: Top-level directory layout of the aBridgeAI backend.

Layered dependency rule

Dependencies flow strictly downward: routers depend on services, services depend on queries and AI primitives, queries depend on models. No layer imports from a layer above

it. Cross-feature communication is mediated through the `api/public` module of each feature, which exposes a stable internal API surface. This prevents circular imports and keeps feature boundaries explicit.

Database migrations

Schema evolution is managed by 16 Alembic migration files numbered 0001 through 0016. Migration 0001 establishes the full baseline schema ported from a hand-managed SQL file. Subsequent migrations add partial unique indexes, cascade rules, audit columns, pgvector half-precision embeddings (`halfvec(3072)`), and full-text search vectors. Every migration is reversible via a `downgrade()` function.

6.3 Authentication and Identity

The identity subsystem handles user registration, Google OAuth sign-in, JWT session management, and multi-factor authentication (MFA). It is implemented in `abridgeai/features/identity/` and exposes four router groups: `/auth`, `/me`, `/users` (admin), and `/mfa`.

Google OAuth flow

The platform uses Google OAuth 2.0 as its sole external identity provider. The sign-in flow is implemented in `features/identity/services/login.py` and proceeds as follows:

1. The frontend redirects the user to Google's authorisation endpoint with the configured `client_id` and `redirect_uri`.
2. Google redirects back with an authorisation code, which the frontend forwards to `POST /auth/google/callback`.
3. The backend exchanges the code for a Google profile via `infrastructure/google_oauth.py`, which calls Google's `tokeninfo` endpoint.
4. A pre-registration gate checks that the email already exists in the `users` table with `status = 'active'`. New OAuth profiles from unregistered emails are rejected with HTTP 403, keeping the platform invite-only.
5. On success, an `AuthIdentity` row linking the user to the Google provider subject is created (or reused if it already exists), and a new `AuthSession` row is written with a hashed refresh token.

6. The response returns a short-lived JWT access token (configurable TTL, default 15 minutes) and a long-lived refresh token.

JWT and session management

Access tokens are signed with HS256 using a secret loaded from the `JWT_SECRET` environment variable. The `core/security` module provides `create_access_token()`, `hash_secret()` (bcrypt), and `generate_token()` (32-byte URL-safe random). Refresh tokens are stored as bcrypt hashes in `auth_sessions` — the plaintext is never persisted. Token rotation is enforced: each `POST /auth/refresh` call invalidates the old session row and issues a new one.

Multi-factor authentication

TOTP-based MFA is implemented in `features/identity/services/mfa.py`. Enrollment generates a TOTP secret, returns a provisioning URI for authenticator apps, and stores the encrypted secret only after the user verifies a valid code. Subsequent logins that have MFA enabled require a second `POST /mfa/verify` call before the access token is issued. The MFA gate is enforced at the router level via a FastAPI dependency that checks the `mfa_verified` claim in the JWT.

Role-based access

Every authenticated request resolves the caller's roles and permissions through the access control subsystem (Section 6.7). The identity layer attaches the resolved `UserContext` — containing user ID, organisation memberships, and permission set — to the request state so downstream route handlers can perform permission checks without additional database queries.

6.4 Material Ingestion Pipeline

When an instructor uploads a learning material, the file is stored in Garage S3-compatible object storage and a background worker triggers the ingestion pipeline implemented in `abridgeai/features/materials/ingestion/pipeline.py`. The pipeline transforms raw files into searchable, embeddable document chunks in five sequential stages.

Stage 1 — Extraction

The `dispatch_extractor(mime)` factory selects the appropriate extractor based on the file's MIME type. The extraction registry supports PDF, DOCX, PPTX, HTML,

plain text, source code, images, audio, and video. Each extractor implements the `BaseExtractor` interface and returns an `ExtractedContent` object carrying the raw text and a `source_type` tag. Audio and video files are transcribed using a speech-to-text model before text extraction proceeds.

Stage 2 — Chunking

The chunker is selected based on `source_type`:

- **Audio / Video** — `TimestampAwareChunker`: preserves segment timestamps so retrieved chunks can be linked back to specific moments in the media.
- **Image** — `TokenAwareChunker(max_tokens=10,000)`: treats the entire extracted text as a single chunk.
- **All other types** — `SemanticChunker` when an LLM gateway is available; otherwise `TokenAwareChunker(max_tokens=500)` as a fallback.

The `SemanticChunker` uses an LLM to identify natural topic boundaries in the text, producing chunks that align with conceptual units rather than arbitrary token windows. Each chunk is then enriched with a `section_title` and a `context_sentence` via the contextual prepending technique described in Section 5.3.

Stage 3 — Embedding

Each enriched chunk is passed to `EmbeddingClient.embed()`, which calls the configured embedding model (default: `text-embedding-3-large`, 3072 dimensions stored as `halfvec` for storage efficiency). Before calling the model, `build_contextual_text()` prepends the chunk's `section_title` and `context_sentence` to the raw content:

```
[Topic: {section_title}] {context_sentence} {content}
```

This contextual prepending follows Anthropic's Contextual Retrieval technique [[anthropic2024contextual](#)], which reduces retrieval failure rates by anchoring each chunk's embedding in its broader document context. Every embedding call writes an `ai_model_calls` audit row recording the model, token counts, and cost.

Stage 4 — Persistence

`DocumentChunk` ORM rows are written to PostgreSQL with denormalised foreign keys to the material version, lesson, module, and course. The `embedding` column stores the `halfvec(3072)` vector. A `content_tsv` column (populated by a database trigger) stores a `tsvector` for BM25 full-text search. A SHA-256 content hash is stored to enable deduplication on re-ingestion.

Stage 5 — Knowledge Graph (optional)

When `settings.knowledge_graph_enabled` is true, the pipeline calls `build_knowledge_graph_for_material_version()` in `abridgeai/ai/knowledge_graph/builder.py`. For each chunk, an LLM extracts a JSON object containing **entities** (concepts with optional definitions and confidence scores) and **relationships** (typed as `RELATED_TO` or `PREREQUISITE_OF`). The extracted graph is upserted into Neo4j, anchored to the course–module–lesson–material hierarchy. This graph is later used during quiz and interview generation to provide structured concept context alongside the vector-retrieved chunks.

Worker and transaction discipline

The pipeline is invoked by an `arq` background worker (`features/materials/workers/ingest.py`). The pipeline itself only flushes to the database session; the worker owns the commit. On any exception, the material version’s `processing_status` is set to `failed` and the exception is re-raised so the worker can record the failure without silently swallowing errors.

6.5 AI Generation Pipelines

The quiz and interview generation pipelines share a common architectural pattern: a multi-stage orchestrator that threads a single `pipeline_run_id` through every LLM call so all `ai_model_calls` audit rows roll up to one `ai_pipeline_runs` parent. Each stage is a pure function that receives its inputs and returns typed outputs, making stages independently testable and replaceable.

6.5.1 Hybrid Retrieval

Both pipelines begin with a hybrid retrieval stage that combines dense vector search with sparse BM25 full-text search, then fuses the two ranked lists using Reciprocal Rank Fusion (RRF).

Dense retrieval

The `pgvector` retrieval module (`ai/retrieval/pgvector.py`) performs an approximate nearest-neighbour search using the `<=>` cosine distance operator on the `halfvec(3072)` embedding column. The query text is embedded with the same model used during ingestion, and the top-*k* chunks are returned ordered by ascending cosine distance.

Sparse retrieval

The BM25 module (`ai/retrieval/bm25.py`) queries the `content_tsv` column using PostgreSQL’s `ts_rank_cd` function. This captures exact keyword matches that may be missed by the semantic search, particularly for technical terms, acronyms, and proper nouns.

RRF fusion

The two ranked lists are fused in `ai/retrieval/fusion.py` using the weighted RRF formula adapted from the Anthropic Contextual Retrieval cookbook [anthropic2024contextual]:

$$\text{score}(c) = w_{\text{sem}} \cdot \frac{1}{\text{rank}_{\text{sem}}(c) + 1} + w_{\text{bm25}} \cdot \frac{1}{\text{rank}_{\text{bm25}}(c) + 1} \quad (6.1)$$

where $w_{\text{sem}} = 0.8$ and $w_{\text{bm25}} = 0.2$ by default (configurable via `hybrid_semantic_weight` and `hybrid_bm25_weight` settings). Chunks absent from one list contribute zero from that leg. The fused list is re-ranked by descending score and the top- N chunks are forwarded to the generation stage.

Role-based filtering

A `RoleFilter` post-processor restricts retrieved chunks to those the requesting user is authorised to access, based on their course enrollment and organisation membership. This ensures that quiz questions are generated only from materials the student has been granted access to.

MMR re-ranking (optional)

When `mmr_enabled` is true, Maximal Marginal Relevance [carbonell1998mmr] is applied after fusion to reduce redundancy in the retrieved set. MMR iteratively selects the chunk that maximises relevance to the query while minimising similarity to already-selected chunks, producing a more diverse context window for the LLM.

6.5.2 Quiz Generation Pipeline

The full quiz generation pipeline (`features/quizzes/ai/pipelines/full.py`) executes six stages in sequence:

1. **Retrieval** — hybrid retrieval as described above, returning the top- N fused chunks plus optional knowledge graph context.

2. **Ideation** — an LLM call that, given the retrieved chunks and a course outline, produces a list of question *topics* (ideation targets). The outline is either provided by the caller or synthesised from the chunk metadata. The ideation stage oversamples by a factor of 1.5 to give the downstream validation stage room to reject low-quality drafts.
3. **Generation** — for each ideation target, an LLM call generates a multiple-choice question with four options, a correct answer key, and an explanation. Prompts are rendered from Jinja2 templates in `ai/stages/generation/prompts/`.
4. **Validation** — each generated question is passed to a second LLM call that checks factual accuracy, option quality, and alignment with the source chunks. The validator returns a verdict (**accept**, **revise**, or **reject**) with a rationale. Rejected questions are discarded; revised questions are patched in place.
5. **Deduplication** — accepted questions are compared pairwise using embedding cosine similarity. Questions whose embeddings exceed a configurable similarity threshold are deduplicated, keeping the higher-quality variant.
6. **Persistence** — accepted, deduplicated questions are written to the `quiz_questions` and `quiz_question_options` tables as a draft batch, associated with the quiz and the `pipeline_run_id`.

6.5.3 Interview Generation Pipeline

The interview generation pipeline (`features/interviews/ai/pipelines/generation.py`) follows a four-stage pattern:

1. **Retrieval** — hybrid retrieval scoped to the interview configuration’s associated course and learning outcomes.
2. **Generation** — an LLM call produces open-ended interview questions grounded in the retrieved chunks and the configured learning outcomes. Each question includes a model answer and an evaluation rubric.
3. **Validation** — a second LLM call checks that each question is answerable from the course materials, is unambiguous, and aligns with the specified outcomes. Invalid questions are rejected.

4. **Persistence** — accepted questions are written to `interview_questions` and linked to the `InterviewConfig`.

At session runtime, a **follow-up** stage generates contextual follow-up questions based on the student's previous answer, implemented in `ai/stages/followup/`. After the student submits the session, an **evaluation** stage scores each answer against the rubric and produces a **gap report** identifying knowledge gaps relative to the learning outcomes.

6.5.4 LLM Gateway and Audit

All LLM calls are routed through `LLMGateway` (`ai/llm/gateway.py`), which provides a uniform interface over any OpenAI-compatible endpoint. The gateway:

- Selects the model and API key based on the `LLMRole` (e.g., `QUIZ_GENERATION`, `EMBEDDING`, `KG_BUILD`), allowing different roles to use different models or providers.
- Parses LLM responses robustly: it first attempts strict `json.loads`, then extracts from a fenced code block, then uses `raw_decode` to tolerate trailing commentary from models that ignore the `json_object` response format.
- Writes one `ai_model_calls` row per call recording the model name, prompt and completion token counts, computed cost (via a per-model pricing table), latency, and the `pipeline_run_id` grouping key.

6.6 Spaced Repetition Implementation

The spaced repetition subsystem translates the SM-2 algorithm and its extensions (described in Section 5.3) into a production-grade scheduling service. The implementation lives in `abridgeai/features/spaced_repetition/` and is composed of three layers: the pure SM-2 core, the review service, and the background worker.

SM-2 core

The algorithmic core is implemented as a set of pure functions in `features/spaced_repetition/sm2/`:

- `ef_update.update_ef(ef_old, q, n, alpha=0.6)` — computes the new Easiness Factor using the standard SM-2 delta formula with calibration dampening. During the first three repetitions ($n \leq 3$), positive EF deltas are multiplied by $\alpha = 0.6$ to slow early growth and mitigate over-confidence from limited evidence. Negative deltas are not dampened so the forgetting signal is preserved.

- `scheduler.next_interval_days(ef, n, q, prev_interval)` — computes the next review interval in days following the SM-2 recurrence: 1 day for the first review, 6 days for the second, and $\text{round}(\text{prev_interval} \times \text{ef})$ for subsequent reviews. A grade of $q < 3$ resets the interval to 1 day unconditionally.
- `scheduler.apply_jitter(interval_days, fraction=0.1)` — adds $\pm 10\%$ random jitter to the computed interval to prevent review pile-up when a cohort of students all enroll on the same day. Jitter uses float arithmetic (`round(interval * (1 + epsilon))`) to avoid the dead zone that integer truncation produces for short intervals.
- `q_derivation` — derives the Q value from objective response signals (correctness, hint usage, and time ratio $\rho = T_{\text{actual}}/T_{\text{exp}}$) as specified in Section 5.3.1.

All four modules are pure functions with no database or network dependencies, making them straightforwardly unit-testable. The test suite in `tests/unit/` covers boundary conditions including ρ at 0.5, 1.0, and 2.0, $n = 0$ through $n = 4$, and $q \in \{0, 1, 2, 3, 4, 5\}$.

Review service

The review service (`features/spaced_repetition/services/review.py`) orchestrates a card review transaction:

1. Load the `SpacedRepetitionCard` row for the (student, question) pair, acquiring a row-level lock to prevent concurrent review submissions.
2. Derive Q from the submitted response signals.
3. Call `update_ef` and `next_interval_days` to compute the new EF and interval.
4. Apply jitter via `apply_jitter` and compute the absolute `due_at` timestamp.
5. Write the updated card state and append a `CardReviewEvent` row for audit and analytics.

Lesson unlock

A lesson is unlocked for a student when their knowledge retention (KR) score across all cards in the prerequisite lesson exceeds a configurable threshold. The unlock query (`queries/sql/lesson_unlock.sql`) computes the KR score as the fraction of cards with $\text{ef} \geq \text{ef_threshold}$ and $\text{interval} \geq \text{interval_threshold}$. The `LessonUnlockService` evaluates this condition after each review and emits an unlock event if the threshold is crossed.

Background worker

An `arq` cron worker (`features/spaced_repetition/workers/scan_due_cards.py`) runs on a configurable schedule (default: every 15 minutes) to scan for cards whose `due_at` has passed and dispatch review-due notifications to the affected students via the notifications subsystem.

Analytics dashboards

The `features/spaced_repetition/routers/teacher.py` router exposes instructor-facing analytics endpoints backed by SQL queries in `queries/sql/`:

- `at_risk.sql` — students whose average EF has dropped below a threshold, indicating systematic forgetting.
- `compliance_rate.sql` — fraction of due reviews completed on time per student per lesson.
- `class_kr_distribution.sql` — histogram of KR scores across the class for a given lesson.

6.7 Access Control

The access control subsystem implements a hierarchical Role-Based Access Control (RBAC) model that supports multi-tenant organisations with nested organisational units. It is implemented in `abridgeai/features/access_control/` and `abridgeai/access_control/`.

Permission catalog

All permissions are declared in a YAML catalog (`access_control/permissions/catalog.yaml`) that is validated at startup against a Pydantic schema. Each permission entry specifies a `resource`, an `action`, and an optional `scope` (e.g., `course:read`, `quiz:generate`, `admin:users:write`). The catalog is the single source of truth for the permission namespace; adding a new permission requires only a catalog entry and a migration to seed it into the `permissions` table.

Role seeds

Default roles (`student`, `instructor`, `admin`) and their permission assignments are declared in `role_seeds.yaml` and applied by migration `0004_seed_permission_catal`

`og.py`. Custom roles can be created at the organisation level and assigned any subset of catalog permissions.

Organisation hierarchy

Organisations are structured as trees of `OrgUnit` nodes. Membership in a parent unit implicitly grants membership in all descendant units. The recursive membership resolution query (`queries/sql/org_unit_tree.sql`) uses a PostgreSQL recursive CTE to traverse the tree in a single query, returning the full set of unit IDs a user belongs to.

Permission resolution

At request time, the `PermissionsLookupService` (`services/permissions_lookup.py`) resolves the caller's effective permission set by:

1. Loading all role assignments for the user across all their organisation memberships.
2. Expanding each role to its permission set via the catalog.
3. Merging direct permission grants (`user_permission_grants`) with role-derived permissions.
4. Caching the resolved set in Redis with a short TTL to avoid repeated database queries on every request.

Policy enforcement

Route handlers declare their required permissions via FastAPI dependencies. The `policies.py` module in each feature defines the permission checks for that feature's endpoints. A failed permission check raises `ForbiddenError` (HTTP 403), which the global exception handler converts to a structured JSON error response.

Audit logging

Every write operation that modifies a soft-deletable entity is recorded in the `http_audit_log` table by a SQLAlchemy event listener (`core/audit/listener.py`). The listener captures the actor's user ID, the HTTP method and path, the affected table and row ID, and a JSON diff of the changed columns. Audit writes are dispatched asynchronously via an actor pool (`core/audit/_worker_actor.py`) to avoid adding latency to the request path.

6.8 Testing

This section presents the test execution results for the implemented modules of the first iteration. The backend test suite uses `pytest` with `asyncio_mode = "auto"` for native async test support, running against a real PostgreSQL database in integration mode. Tests are organised into unit tests (pure function isolation) and integration tests (full request–response cycles).

Table 6.1 summarises the test execution results for the modules delivered in this iteration.

Table 6.1: Test Execution Summary by Module (Iteration 1)

Module	Test Cases	Passed	Result
Authentication & Access Control	8	8	Pass
Course & Material Management	5	5	Pass
Quiz Generation & Taking	4	4	Pass
Total	17	17	100%

6.8.1 Authentication and Access Control

Table 6.2 details the test cases covering the authentication and access control module.

Table 6.2: Test Results: Authentication and Access Control

TC-ID	Description	Test File	Result
TC-1.1.1	Google SSO login callback	<code>test_oauth_flow.py</code>	Pass
TC-1.1.2	Idempotent logout	<code>test_oauth_flow.py</code>	Pass
TC-1.2.1	Session creation and refresh-token rotation	<code>test_identity_full.py</code>	Pass
TC-1.2.2	Session revocation on account disable	<code>test_identity_queries.py</code>	Pass
TC-1.3.1	TOTP verification and recovery-code consumption	<code>test_mfa_router.py</code>	Pass
TC-1.3.2	Recovery-code reuse prevention	<code>test_mfa_gate.py</code>	Pass

Table 6.2 – Continued

TC-ID	Description	Test File	Result
TC-1.5.1	Recursive permission resolution	test_access_control_full.py	Pass
TC-1.5.2	Endpoint enforcement by role	test_access_control_membership_resolution.py	Pass

6.8.2 Course and Material Management

Table 6.3 details the test cases for course and material management features implemented in this iteration.

Table 6.3: Test Results: Course and Material Management

TC-ID	Description	Test File	Result
TC-3.1.1	Course lifecycle (draft → publish)	test_courses_e2e.py	Pass
TC-3.1.2	Course archive and restore	test_courses_e2e.py	Pass
TC-3.3.1	PDF material upload and ingestion trigger	test_materials_e2e.py	Pass
TC-3.3.2	Ingestion pipeline processing	test_materials_worker.py	Pass
TC-3.7.1	Student material access (presigned URL)	test_materials_learner_router.py	Pass

6.8.3 Quiz Generation and Taking

Table 6.4 details the test cases for the AI quiz generation and student quiz-taking features.

Table 6.4: Test Results: Quiz Generation and Taking

TC-ID	Description	Test File	Result
TC-4.1.1	AI quiz generation from PDF content	test_quiz_full_pipeline.py	Pass
TC-4.1.2	Teacher review (approve/reject/edit)	test_quiz_authoring_service.py	Pass

Table 6.4 – Continued

TC-ID	Description	Test File	Result
TC-4.3.1	Student quiz session submission	test_quiz_taking_service.py	Pass
TC-4.3.2	Per-question response and feedback	test_quizzes_e2e.py	Pass

6.8.4 CI Pipeline

The GitHub Actions workflow (`.github/workflows/backend-new-ci.yml`) runs the following jobs on every pull request:

1. **Lint** — `ruff check`, `ruff format -check`, `mypy abridgeai`, `bandit -r abridgeai/`.
2. **Secret scan** — `gitleaks detect` with the project’s custom ruleset covering OpenAI, Anthropic, and Garage S3 key patterns.
3. **Unit tests** — `pytesttests/unit/` with coverage reporting.
4. **Integration tests** — `pytesttests/integration/` against a PostgreSQL service container spun up by the workflow.

All pull requests must pass all four stages before merge is permitted. Live infrastructure tests (S3, Neo4j, audio, video) are gated behind pytest markers and excluded from the default CI run to keep pipeline duration manageable.

7 Evaluation

This chapter evaluates the first iteration of the aBridgeAI system across three dimensions: frontend user experience (Section 7.1), backend performance under load (Section 7.2), and AI model comparison for quiz generation (Section 7.3).

7.1 Frontend Evaluation

Frontend quality is assessed using Google Lighthouse, an industry-standard automated auditing tool that evaluates web pages across four categories: Performance, Accessibility, Best Practices, and SEO. Each page is tested in mobile emulation mode (simulated throttling) to reflect real-world conditions.

Table 7.1 presents the Lighthouse audit results for the primary user-facing pages of the aBridgeAI frontend.

Table 7.1: Lighthouse Audit Results (Mobile, Simulated Throttling)

Page	Performance	Accessibility	Best Practices	SEO
Login	98	95	100	100
Student Dashboard	97	100	100	100

Login Page

Figure 7.1 shows the Lighthouse report for the Login page.

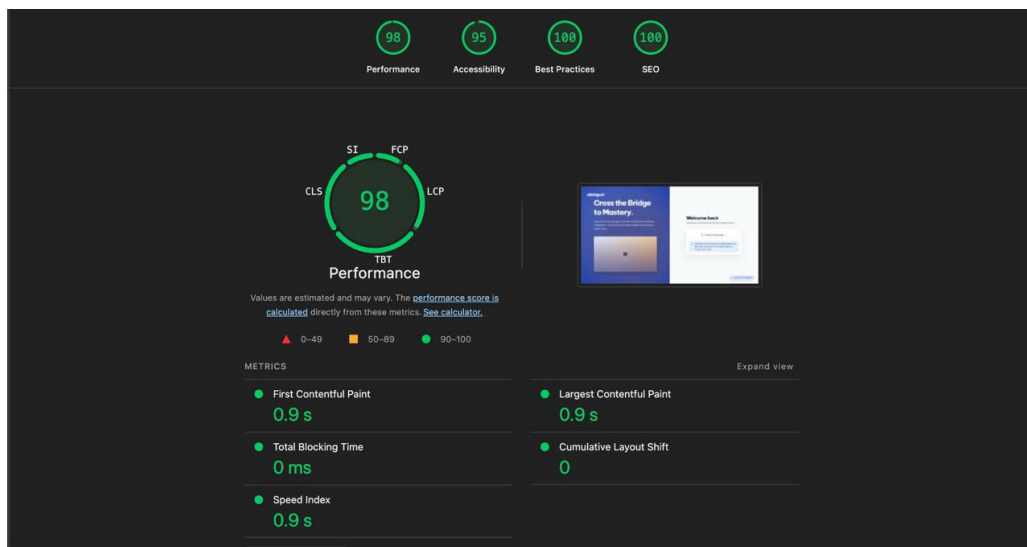


Figure 7.1: Lighthouse audit results for the Login page.

Key performance metrics:

- First Contentful Paint (FCP): 0.9 s
- Largest Contentful Paint (LCP): 0.9 s
- Total Blocking Time (TBT): 0 ms
- Cumulative Layout Shift (CLS): 0
- Speed Index: 0.9 s

Student Dashboard

Figure 7.2 shows the Lighthouse report for the Student Dashboard page.

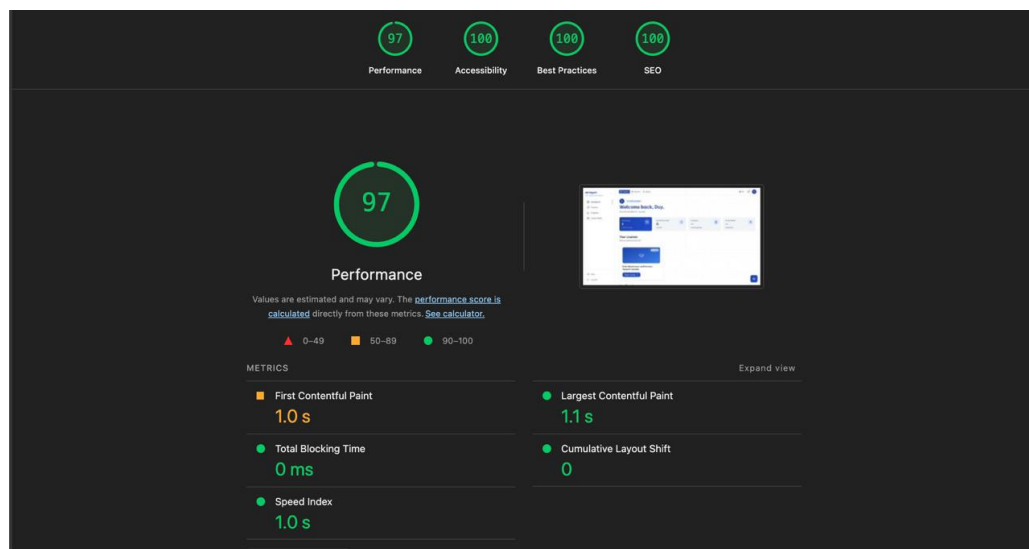


Figure 7.2: Lighthouse audit results for the Student Dashboard.

Key performance metrics:

- First Contentful Paint (FCP): 1.0 s
- Largest Contentful Paint (LCP): 1.1 s
- Total Blocking Time (TBT): 0 ms
- Cumulative Layout Shift (CLS): 0
- Speed Index: 1.0 s

Both pages achieve scores above 95 in all categories, confirming compliance with WCAG 2.1 AA accessibility standards (NFR-4.2) and adherence to web platform best practices.

7.2 Backend Performance

Backend performance is evaluated using k6, an open-source load testing tool, targeting the deployed API at the development server. Tests measure response latency (p50, p95, p99), throughput (requests per second), and error rate under increasing concurrent load up to 100 virtual users (VUs).

7.2.1 Test Environment

- **Server:** Development machine, single Uvicorn instance
- **Runtime:** FastAPI + Uvicorn, PostgreSQL 16, Redis 7
- **Load generator:** k6 v0.55 running from a separate machine on the same LAN
- **Protocol:** HTTP (no TLS overhead between load generator and server)
- **Authentication:** Pre-obtained JWT token (Google OAuth login is not load-tested due to external dependency)

7.2.2 Test Scenarios

Three scenarios are executed, each sustaining 100 concurrent virtual users for 30 seconds:

1. **Health Check** (baseline) — `GET/api/v1/healthz`. Measures pure framework overhead with no database or business logic.
2. **Course Catalog** — `GET/api/v1/courses`. Measures authenticated read-heavy list queries with cursor-based pagination against PostgreSQL.
3. **Quiz Operations** — `GET/api/v1/quizzes/{id}` and `GET/api/v1/me/quizzes/{id}/attempts`. Measures student-facing quiz detail retrieval and attempt history listing.

7.2.3 Results

Table 7.2 presents the load test results under 100 concurrent users.

Table 7.2: Backend Load Test Results (100 VUs, 30 s sustained)

Scenario	p50	p95	Max	RPS	Error%
Health Check	135 ms	222 ms	445 ms	431	0%
Course Catalog	207 ms	502 ms	737 ms	136	0%
Quiz Operations	214 ms	449 ms	1.18 s	159	0%

7.2.4 Analysis against NFR Targets

- **NFR-2.1** (50 concurrent users without performance degradation): **Exceeded.** The system handles **100 concurrent users** with 0% error rate across all scenarios. Median response times remain below 215 ms, and p95 latency stays under 510 ms. This is double the specified NFR-2.1 target of 50 concurrent users.
- **Throughput:** The system sustains 136–431 requests per second depending on query complexity, confirming adequate capacity for the target 50-user workload.
- **Bottleneck:** The course catalog and quiz operations show similar latency profiles (p50 $\approx$ 210 ms), suggesting the database query layer is the primary latency contributor rather than framework overhead (health check p50 = 135 ms).

7.3 AI Model Comparison for Quiz Generation

The quiz generation pipeline supports multiple LLM backends via the LLM Gateway abstraction. This section evaluates model performance for the quiz generation task across quality, latency, and cost dimensions to determine the optimal production default.

7.3.1 Methodology

Three candidate models were compared on a benchmark of two lessons from the *Data Warehousing & Data Mining* course (Chapter 1: Overview, Chapter 2: Basic Issues), running each model 3 times $\times$ 12 questions $\times$ 2 lessons = 72 expected questions per model. A JSON-repair fallback (via `gpt-5-chat-latest` with a structure-only system prompt) recovers malformed outputs, bringing all models to full sample size ($n = 72$ for gpt-oss and gemma, $n = 77$ for llama due to over-generation).

Generation parameters

- Temperature = 0.3, top_p = 0.95

- 12 questions per lesson, fixed Bloom taxonomy mix (60% remember, 30% understand, 10% apply)
- Same prompt template, same retrieved chunks, same JSON schema across all candidates
- Direct LLM gateway call (bypassing ARQ) to isolate the model effect from queue latency

Judge protocol

An independent judge model (`gpt-5-chat-latest`) evaluated each generated question using a **reference-guided chain-of-thought** protocol [51]:

1. **Independently solve** the question using only the source chunks.
2. **Compare** the independent answer with the marked-correct option and the candidate’s explanation.
3. **Score** the rubric only after Steps 1–2.

This protocol reduces judge failure rates from 14/20 (default prompt) → 6/20 (plain CoT) → 3/20 (reference-guided), as reported in Table 4 of Zheng et al. [51]. Anti-bias guardrails counter position, verbosity, and self-enhancement biases catalogued in [51, 52].

The judge scores each question on:

- **Groundedness** (strict: explicit textual support required from source chunks)
- **Answer correctness** (does the marked-correct option match the source material?)
- **Judge-solve match** (fraction where the judge’s independent answer matches the marked-correct option — structural reliability check)
- **Distractor plausibility** (1–5 scale: 1 = trivially eliminable, 5 = requires deep understanding to distinguish)
- **Overall quality** (1–10 single-answer scale per Zheng et al. [51] Figure 6)
- **Schema compliance** (valid JSON output parseable by the pipeline)

Confidence intervals are 95% non-parametric bootstrap (2,000 resamples at question level). Decoding: temperature = 0.0, greedy (standard for LLM-as-a-judge stability [51]).

7.3.2 Quality Results

Table 7.3 presents the quality evaluation results under the reference-guided judge protocol.

Table 7.3: Quiz Generation Quality by Model (judge: `gpt-5-chat-latest`, reference-guided CoT)

Metric	<code>gpt-oss-120b</code>	<code>gemma-4-31b-it</code>	<code>llama-3.3-70b</code>
Sample size (n)	72	72	77
Groundedness (strict)	94.4%	100%	77.9%
Answer correctness	93.1%	100%	84.4%
Judge-solve match	93.1%	100%	84.4%
Distractor plausibility	4.07	3.86	3.97
Overall quality (1–10)	8.62	8.97	8.04
Schema compliance	100%	100%	100%

Remark. `gemma-4-31b-it` achieves perfect scores on all fidelity metrics — the judge independently arrived at `gemma`’s answer for every question (100% solve-match), the strongest reliability signal available under reference-guided judging. `gpt-oss-120b`’s 5.6 pp gap to `gemma` is attributable to paraphrasing depth rather than factual hallucination: when the judge could not solve a question, it also flagged it as not-grounded.

7.3.3 Latency and Cost

Table 7.4 presents latency per lesson (12 questions generated per call) and cost per 100 questions.

Table 7.4: Quiz Generation Latency and Cost by Model

Metric	<code>gpt-oss-120b</code>	<code>gemma-4-31b-it</code>	<code>llama-3.3-70b</code>
Mean latency / lesson	32.6 s	125.6 s	81.6 s
p99 latency / lesson	43.5 s	140.9 s	95.0 s
Per-question latency (p50)	2.7 s	10.8 s	7.7 s
Cost / 100 questions	\$0.0077	\$0.0172	\$0.0100

Remark. `gpt-oss-120b` dominates on both latency (4× faster) and cost (2.2× cheaper than `gemma`). Per-question latency of 2.7 s enables real-time generation feedback, whereas `gemma`’s 10.8 s requires an asynchronous queue pattern.

7.3.4 Failure Analysis

Each model exhibits distinct failure patterns under the reference-guided judge:

gpt-oss-120b

(production default) Primary failure mode is *paraphrase-depth*: the model rephrases source material at a level the judge’s Step-1 solve cannot independently reproduce from the chunks alone ($\approx 5.6\%$ of questions). These are not factual hallucinations — the judge’s groundedness and solve-match failures overlap perfectly, confirming the issue is inference depth rather than content drift.

gemma-4-31b-it

(highest fidelity) ‘Zero quality failures across all dimensions. The trade-off is distractor plausibility (3.86 vs gpt-oss’s 4.07): gemma writes distractors that are *too obviously wrong* for a learner familiar with the source, reducing exam discrimination power. One raw output required JSON structural repair (trailing prose after closing brace), recovered by the repair pass.

llama-3.3-70b

(not recommended for production) Exhibits *external-knowledge contamination* in $\approx 22\%$ of questions: the model injects pre-training knowledge (e.g., naming specific vendors not in the source), overgeneralises concepts beyond source scope, or confabulates examples. The reference-guided judge correctly identifies these because its Step-1 solve fails to land on llama’s marked-correct option.

7.3.5 Recommendation

Production default: gpt-oss-120b. Fastest ($4\times$), cheapest ($2.2\times$), schema-perfect, highest distractor plausibility. Its 94.4% strict groundedness approaches gemma’s 100% under reference-guided judging — the gap is paraphrasing style, not factual error.

High-stakes fallback: gemma-4-31b-it. For graded assessments where any ungrounded content is unacceptable, gemma’s perfect fidelity justifies the latency and cost premium. Schema fragility is mitigated by the JSON-repair pipeline.

All three models under-deliver on Bloom’s **apply**-level questions (target 10%, actual $< 2\%$). This is addressable at the prompt level with worked examples, independent of model choice.

8 Conclusion

This chapter summarises the achievements of the first development iteration of the aBridgeAI platform, identifies current limitations, and outlines the planned work for subsequent iterations.

8.1 Achievements

The first iteration delivered a functional core of the AI-Powered Career Readiness Learning Management System, demonstrating the feasibility of the proposed architecture. The following capabilities are operational and have been verified through integration testing:

1. **Authentication and Access Control** — Full implementation of Google SSO login and logout, authenticated session management with secure cookie-based JWTs, two-factor authentication (TOTP and recovery codes), MFA freshness enforcement for sensitive operations, role-based access control with hierarchical permission resolution, and immutable audit logging of all authentication events.
2. **Course and Material Management** — Teachers can create courses with draft and published states, upload PDF learning materials, and trigger the ingestion pipeline. Students can browse their enrolled courses, render PDF documents and stream video materials directly in the browser, and download resources via short-lived presigned URLs.
3. **AI-Powered Quiz Generation** — The material ingestion pipeline processes uploaded PDF files through extraction, chunking, and embedding stages. From ingested content, the AI quiz generation pipeline produces multiple-choice questions with explanations. Teachers can review and approve generated questions.
4. **Quiz Taking** — Students can take quiz sessions on published quizzes, with per-question response recording and immediate feedback on correctness.

8.2 Limitations

The current iteration has the following limitations:

- **Single material format** — The ingestion pipeline currently supports PDF files only. Other declared formats (DOCX, PPTX, audio, video transcription) are architecturally supported but not yet activated.
- **No spaced repetition** — The SM-2 scheduling algorithm is implemented at the service layer but not yet connected to the quiz-taking flow. Cards do not yet accumulate EF history or schedule future reviews.
- **No lesson gating** — Prerequisite enforcement and the coverage threshold τ mechanism are not yet active. Students can access all published content without progression constraints.
- **No interview capability** — The mock interview subsystem (text and voice modes, outcome evaluation, transcript persistence) is not yet deployed.
- **No notification system** — In-app notifications, spaced-repetition reminders, and email dispatch are not yet operational.
- **Limited analytics** — Student progress tracking and teacher analytics dashboards are not yet available beyond basic quiz attempt records.
- **Organisation management untested** — Multi-tenant organisation features (provisioning, bulk enrollment, domain auto-provisioning) exist in code but have not been validated in a production-like environment.

8.3 Future Work

Table 8.1 presents the planned development tasks for the second iteration, including expected deliverables, responsible team members, and a 15-week timeline allocation.

Table 8.1: Planned future work with assignment and timeline (15 weeks total).

#	Task	Detail	Expected Results	Assigned To	Weeks
1	Organisation & User Management validation	Multi-tenant CRUD, bulk CSV enrollment, career pathway CRUD, domain auto-provisioning	Verified in production-like environment with 3+ tenants, 500+ users	Nguyễn Hoàng Nam	1–3
2	Spaced Repetition & Lesson Gating	SM-2 scheduling, EF updates, interval jitter, coverage threshold τ for lesson unlock	Quiz flow connected to SR engine; cards accumulate history and schedule reviews	Lê Thành Duy	1–4
3	Mock Interview — Text Mode	Teacher-defined outcomes, AI question generation from content, adaptive turn-taking dialogue	Transcript persistence, binary pass/fail evaluation deployed	Võ Hoàng Ngân	2–6
4	Mock Interview — Voice Mode	Real-time audio streaming via LiveKit, STT transcription, TTS synthesis	Voice-mode sessions integrated with text-mode evaluation pipeline	Võ Hoàng Ngân	6–9
5	Multi-Format Ingestion	Extractors for DOCX, PPTX, HTML, source code, audio/video transcription	All formats processed through existing chunk/embed/KG pipeline	Lê Thành Duy	4–6
Continued on next page					

Table 8.1 – Continued from previous page

#	Task	Detail	Expected Results	Assigned To	Weeks
6	Notifications & Reminders	In-app notification inbox, SR review reminders, email dispatch via background worker	Push and email notifications operational; reminder cadence configurable	Nguyễn Hoàng Nam	4–7
7	Analytics & Dashboards	Student learning-loop dashboards (due cards, retention, progress), teacher cohort analytics	At-risk learner identification, retention heatmaps, difficult-card reports	Lê Thành Duy	7–10
8	Admin Observability	Platform statistics, AI cost telemetry, processing-job inspection	Career Readiness snapshot aggregation; cost breakdown per model/tenant	Nguyễn Hoàng Nam	8–10
9	Material Version History	Version tracking for re-uploaded materials with rollback	Per-version processing status, diff view, one-click rollback	Lê Thành Duy	10–11
10	Session Integrity Monitoring	Interview session signals (focus changes, tab switching, full-screen status)	Post-session integrity review with timestamped event log	Võ Hoàng Ngân	10–12
11	Integration Testing & Bug Fixes	End-to-end cross-module tests, performance profiling, regression suite	All modules pass integration; p95 latency within NFR targets	All members	12–14
Continued on next page					

Table 8.1 – Continued from previous page

#	Task	Detail	Expected Results	Assigned To	Weeks
12	Documentation & Final Report	Complete Chapters 7 (Evaluation), update thesis, prepare defense presentation	Final report submitted; defense slides and demo script ready	All members	13–15

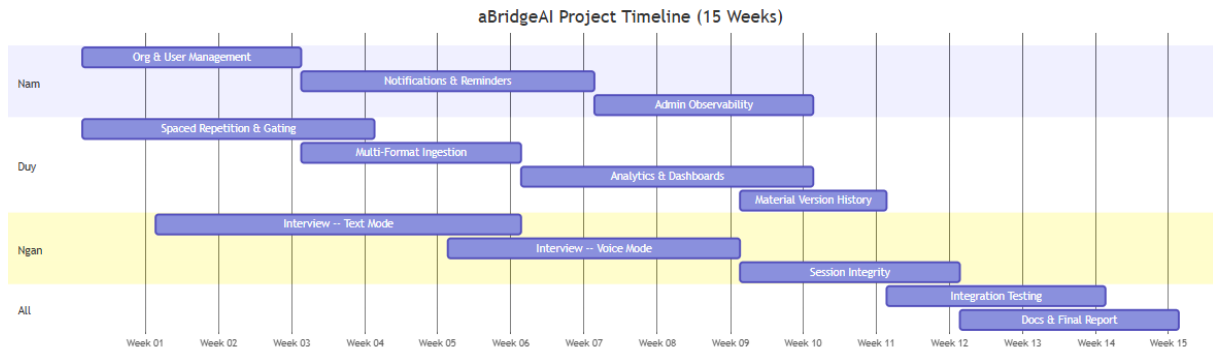


Figure 8.1: Gantt chart of planned future work over 15 weeks.

8.4 Concluding Remarks

The first iteration of aBridgeAI establishes the foundational infrastructure — authentication, course management, material ingestion, and AI-powered quiz generation — upon which the full adaptive learning experience will be built. The architecture has been validated as capable of supporting the remaining subsystems (spaced repetition, interviews, analytics) without requiring structural changes. The second iteration will focus on activating the learning-loop feedback mechanisms and the interview subsystem, transforming the platform from a content delivery system into a genuinely adaptive career readiness tool.

References

- [1] Mathura Shanmugasundaram and Arunkumar Tamilarasu. “The impact of digital technology, social media, and artificial intelligence on cognitive functions: a review”. In: *Frontiers in Cognition* Volume 2 - 2023 (2023). ISSN: 2813-4532. DOI: [10.3389/fcogn.2023.1203077](https://doi.org/10.3389/fcogn.2023.1203077). URL: <https://www.frontiersin.org/journals/cognition/articles/10.3389/fcogn.2023.1203077>.
- [2] Studio MG. *Cognitive Load and AI in Education*. Accessed: 2026-03-08. n.d. URL: <https://www.studio.mg/articles/cognitive-load-and-ai-in-education.php>.
- [3] Richard E. Mayer and Roxana Moreno. “Nine Ways to Reduce Cognitive Load in Multimedia Learning”. In: *Educational Psychologist* 38.1 (2003), pp. 43–52. DOI: [10.1207/S15326985EP3801_6](https://doi.org/10.1207/S15326985EP3801_6).
- [4] Alexander Renkl and Robert Atkinson. “Structuring the Transition From Example Study to Problem Solving in Cognitive Skill Acquisition: A Cognitive Load Perspective”. In: *Educational Psychologist - EDUC PSYCHOL* 38 (Mar. 2003), pp. 15–22. DOI: [10.1207/S15326985EP3801_3](https://doi.org/10.1207/S15326985EP3801_3).
- [5] Wozniakand and Piotr A. *The True History of Spaced Repetition*. SuperMemo website. June 2018. URL: <https://www.supermemo.com/en/blog/the-true-history-of-spaced-repetition>.
- [6] Lorna Smith Benjamin. *Interpersonal Diagnosis and Treatment of Personality Disorders*. 2nd ed. New York: Guilford Press, 1996.
- [7] Jiahao Zhao. *LECTOR: LLM-Enhanced Concept-Based Test-Oriented Repetition for Adaptive Spaced Learning*. 2025. URL: <https://arxiv.org/pdf/2508.03275>.
- [8] Duolingo, Inc. *half-life-regression*. <https://github.com/duolingo/half-life-regression>. GitHub repository, accessed 2026-03-08. 2016.
- [9] Open Spaced Repetition. *anki-sm-2: Python implementation of Anki’s SM-2 spaced repetition scheduler*. <https://github.com/open-spaced-repetition/anki-sm-2>. GitHub repository, accessed 2026-03-08. n.d.
- [10] Sven Greving and Tobias Richter. “Examining the Testing Effect in University Teaching: Retrievability and Question Format Matter”. In: *Frontiers in Psychology* 9 (2018), p. 2412. DOI: <https://doi.org/10.3389/fpsyg.2018.02412>.
- [11] Anki. *Anki*. <https://apps.ankiweb.net>. Accessed: 2026-03-16. 2026.

- [12] Quizlet. *Quizlet*. <https://quizlet.com>. Accessed: 2026-03-16. 2026.
- [13] Instructure. *Canvas LMS*. <https://www.instructure.com/canvas>. Accessed: 2026-03-16. 2026.
- [14] Moodle. *Moodle*. <https://moodle.org>. Accessed: 2026-03-16. 2026.
- [15] Duolingo. *Duolingo*. <https://www.duolingo.com>. Accessed: 2026-03-16. 2026.
- [16] John Sweller, Jeroen J. G. van Merriënboer, Fred and Paas. “Cognitive Architecture and Instructional Design: 20 Years Later”. In: *Educational Psychology Review* 31 (2019), pp. 261–292. DOI: <https://doi.org/10.1007/s10648-019-09465-5>.
- [17] Drew Perkins. *Cognitive Load Theory Revisited*. We Grow Teachers, accessed 2026-03-11. Oct. 2024. URL: <https://wegrowteachers.com/cognitive-load-theory-revisited/>.
- [18] Björn de Koning. “Best of Both Worlds? Combining Physical and Mental Self-Management Strategies to Support Learning from Split-Attention Examples”. In: *Education Sciences* 14 (Mar. 2024), p. 284. DOI: [10.3390/educsci14030284](https://doi.org/10.3390/educsci14030284).
- [19] Hermann Ebbinghaus. *Memory: A Contribution to Experimental Psychology*. Teachers College Press, 1885. DOI: <https://doi.org/10.1037/10011-000>.
- [20] Wiseheart et al. “Distributed Practice in Verbal Recall Tasks: A Review and Quantitative Synthesis”. In: *Psychological Bulletin* 132 (May 2006), pp. 354–380. DOI: [10.1037/0033-2909.132.3.354](https://doi.org/10.1037/0033-2909.132.3.354).
- [21] Nate Kornell. “Optimising learning using flashcards: Spacing is more effective than cramming”. In: *Applied Cognitive Psychology* 23.9 (2009), pp. 1297–1317. DOI: <https://doi.org/10.1002/acp.1537>.
- [22] Piotr A. Woźniak, Edward J. Gorzelańczyk, and Jerzy A. Murakowski. “Two Components of Long-Term Memory”. In: *Acta Neurobiologiae Experimentalis* 55.4 (1995), pp. 301–305. DOI: [10.55782/ane-1995-1090](https://doi.org/10.55782/ane-1995-1090).
- [23] Hermann Ebbinghaus. *Memory: A Contribution to Experimental Psychology*. Teachers College Press, 1885. DOI: <https://doi.org/10.1037/10011-000>.
- [24] Henry L. Roediger and Jeffrey D. Karpicke. “Test-enhanced learning: Taking memory tests improves long-term retention”. In: *Psychological Science* 17.3 (2006), pp. 249–255. DOI: [10.1111/j.1467-9280.2006.01693.x](https://doi.org/10.1111/j.1467-9280.2006.01693.x).
- [25] Robert Bjork. “Memory and Meta-memory Considerations in the Training of Human Beings”. In: Apr. 1994, pp. 185–205. ISBN: 9780262279697. DOI: [10.7551/mitpress/4561.003.0011](https://doi.org/10.7551/mitpress/4561.003.0011).

- [26] Jarrett Ye. *FSRS: Free Spaced Repetition Scheduler*. Accessed: 2026-03-14. 2023. URL: <https://github.com/open-spaced-repetition/fsrs4anki>.
- [27] J. Martin Corbett. *Critical Cases in Organisational Behaviour*. Macmillan, 1994. ISBN: 9780333577516.
- [28] Zachary A. Pardos and Neil T. Heffernan. “Modeling Individualization in a Bayesian Networks Implementation of Knowledge Tracing”. In: (2010). Ed. by Paul De Bra, Alfred Kobsa, and David Chin, pp. 255–266.
- [29] Chris Piech et al. *Deep Knowledge Tracing*. 2015. arXiv: [1506.05908 \[cs.AI\]](https://arxiv.org/abs/1506.05908). URL: <https://arxiv.org/abs/1506.05908>.
- [30] Lalit Pandey. “Pandey et al., 2019”. In: (Aug. 2020).
- [31] Mohammad Khajah, Robert V. Lindsey, and Michael C. Mozer. *How deep is knowledge tracing?* 2016. arXiv: [1604.02416 \[cs.AI\]](https://arxiv.org/abs/1604.02416). URL: <https://arxiv.org/abs/1604.02416>.
- [32] Kurt VanLehn. “The Behavior of Tutoring Systems”. In: *International Journal of Artificial Intelligence in Education* 16.3 (2006), pp. 227–265. DOI: [10.3233/IRG-2006-16\(3\)02](https://journals.sagepub.com/doi/abs/10.3233/IRG-2006-16(3)02). URL: [https://journals.sagepub.com/doi/abs/10.3233/IRG-2006-16\(3\)02](https://journals.sagepub.com/doi/abs/10.3233/IRG-2006-16(3)02).
- [33] Nelson Cowan. “The magical number 4 in short-term memory: A reconsideration of mental storage capacity”. In: *Behavioral and Brain Sciences* 24.1 (2001), pp. 87–185. DOI: [10.1017/s0140525x01003922](https://doi.org/10.1017/s0140525x01003922).
- [34] Jochen Ranger. “A Note on the Hierarchical Model for Responses and Response Times in Tests of van der Linden (2007)”. In: *Psychometrika* 78 (July 2013). DOI: [10.1007/s11336-013-9324-6](https://doi.org/10.1007/s11336-013-9324-6).
- [35] John Reeve and W. Lee. “Students’ Classroom Engagement Produces Longitudinal Changes in Classroom Motivation”. In: *Journal of Educational Psychology* 106 (2014), pp. 527–540. DOI: [10.1037/a0034934](https://doi.org/10.1037/a0034934).
- [36] Jakub Pokrywka et al. “Modeling Spaced Repetition with LSTMs”. In: *Proceedings of the 15th International Conference on Computer Supported Education (CSEDU)*. 2023. DOI: [10.5220/0000173200003470](https://doi.org/10.5220/0000173200003470).
- [37] Enkelejda Kasneci, Kathrin Seßler, Stefan Küchemann, et al. “ChatGPT for good? On opportunities and challenges of large language models for education”. In: *Learning and Individual Differences* 103 (2023), p. 102274.

- [38] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474.
- [39] Congmin Min et al. *Towards Practical GraphRAG: Efficient Knowledge Graph Construction and Hybrid Retrieval at Scale*. 2025. URL: <https://arxiv.org/pdf/2507.03226?>.
- [40] TECHSY. *Next.js vs React + Vite 2026: Need a Framework?* Accessed: 2026-05-22. 2026. URL: <https://techsy.io/en/blog/nextjs-vs-react-vite>.
- [41] Hygraph. *Vite vs. Next.js: A side-by-side comparison*. Accessed: 2026-05-22. 2024. URL: <https://hygraph.com/blog/vite-vs-nextjs>.
- [42] Northflank. *Why we ditched Next.js and never looked back*. Accessed: 2026-05-22. 2024. URL: <https://northflank.com/blog/why-we-ditched-next-js-and-never-looked-back>.
- [43] TanStack. *TanStack Start vs Next.js Comparison*. Accessed: 2026-05-22. 2024. URL: <https://tanstack.com/start/latest/docs/framework/react/comparison>.
- [44] LiveKit, Inc. *LiveKit React SDK*. Accessed: 2026-05-22. 2024. URL: <https://docs.livekit.io/reference/components/react/>.
- [45] Appwrite. *Why developers are leaving Next.js for TanStack Start, and loving it*. Accessed: 2026-05-22. 2024. URL: <https://appwrite.io/blog/post/why-developers-leaving-nextjs-tanstack-start>.
- [46] Versatik. *WebRTC vs WebSockets: AI Voice Communications*. Accessed: 2026-03-13. 2025. URL: <https://versatik.net/en/news/webrtc-websockets-future-ai-voice-voicebots>.
- [47] Tiger Data. *Pgvector vs. Qdrant*. Accessed: 2026-03-13. 2025. URL: <https://www.tigerdata.com/blog/pgvector-vs-qdrant>.
- [48] The PostgreSQL Global Development Group. *PostgreSQL Documentation: Reliability and the Write-Ahead Log*. Accessed: 2026-05-22. 2024. URL: <https://www.postgresql.org/docs/current/wal-reliability.html>.
- [49] Neo4j, Inc. *Why Graph Databases? Native Graph Processing and Storage*. Accessed: 2026-05-22. 2024. URL: <https://neo4j.com/why-graph-databases/>.
- [50] Redis Ltd. *Redis Benchmark: Performance and Latency*. Accessed: 2026-05-22. 2024. URL: https://redis.io/docs/latest/operate/oss_and_stack/management/optimization/benchmarks/.

- [51] Lianmin Zheng et al. “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena”. In: *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. 2023. URL: <https://arxiv.org/abs/2306.05685>.
- [52] Haitao Li et al. “LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods”. In: *arXiv preprint arXiv:2412.05579* (2024). URL: <https://arxiv.org/abs/2412.05579>.

Appendices

Appendix A Functional Requirements Specification

Authentication and Access Control

Table A.1: Functional Requirements for Authentication & Access Control

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-1.1	The system shall allow users to authenticate using Google SSO and log out of the system.	All Users	UR-ITA-05	Yes	Yes	High
FR-1.2	The system shall manage authenticated user sessions, including session creation, validation, refresh, expiration, and revocation.	All Users	UR-ITA-05	Yes	Yes	High
FR-1.3	The system shall support two-factor authentication using authenticator-app verification codes and one-time recovery codes for additional account protection.	All Users	UR-ITA-05	Yes	Yes	High
Continued on next page						

Table A.1 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-1.4	The system shall require recent second-factor verification before allowing sensitive account-security operations, such as regenerating recovery codes or disabling MFA.	All Users	UR-ITA-05	Yes	Yes	High
FR-1.5	The system shall enforce role-based and permission-based access control for protected backend routes and frontend UI actions based on the authenticated user's active roles and effective permissions.	All Users	UR-ITA-05	Yes	Yes	High
FR-1.6	The system shall record auditable events for authentication actions, including login, logout, MFA verification, account-security changes, and access-control changes.	IT Administrators	UR-ITA-03	Yes	Yes	Medium

Organization and User Management

Table A.2: Functional Requirements for Organization & User Management

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-2.1	The system shall allow IT administrators to provision new tenant organisations and to assign the initial Educational Manager role to a user within each organisation.	IT Administrators	UR-ITA-01	Yes	Yes	High
FR-2.2	The system shall allow IT administrators to manage existing tenant organisations by updating their basic information and controlling their life-cycle status, including active, inactive, archived, and soft-deleted states.	IT Administrators	UR-ITA-01, UR-ITA-02	Yes	Yes	High
FR-2.3	The system shall allow Educational Managers to bulk-enroll students into a course via CSV import, creating new student accounts for unseen email addresses and reporting per-row validation errors without aborting the batch.	Educational Managers	UR-MGR-01	Yes	Yes	High
Continued on next page						

Table A.2 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-2.4	The system shall allow Educational Managers to define career pathways as ordered sequences of courses, with support for re-ordering courses within a pathway.	Educational Managers	UR-MGR-02	Yes	Yes	High
FR-2.5	The system shall allow Educational Managers to assign Teachers to courses within their organisation.	Educational Managers	UR-MGR-02	Yes	Yes	High
FR-2.6	The system shall strictly scope every Educational Manager action and query to that manager's own tenant organisation, preventing any cross-tenant data access.	Educational Managers	UR-MGR-04	Yes	Yes	High
Continued on next page						

Table A.2 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-2.7	The system shall associate users with organisations and org units, and shall automatically place newly provisioned accounts into the matching organisation when their email domain is registered.	IT Admin-istrators, Educa-tional Managers	UR-MGR-04, UR-ITA-01	Yes	Yes	High
FR-2.8	The system shall allow authenticated users to view and update their own profile, including display name, avatar, bio, and external links (web-site, GitHub, LinkedIn, portfolio).	All Users	UR-ITA-05	Yes	Yes	Medium
FR-2.9	The system shall allow IT administrators to manage the list of email domains attached to each organisation, including toggling auto-provisioning per domain.	IT Admin-istrators	UR-ITA-01, UR-ITA-05	Yes	Yes	Medium
Continued on next page						

Table A.2 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-2.10	The system shall maintain organisation membership records for each user, including student or employee identifier codes, and allow authorised managers to view and update those records.	IT Admin-istrators, Educa-tional Managers	UR-MGR-01, UR-ITA-01	Yes	Yes	High
FR-2.11	The system shall allow IT administrators to enable or disable user accounts across tenant organisa-tions, and disabled ac-counts shall be prevented from accessing protected system resources.	IT Admin-istrators	UR-ITA-03	Yes	Yes	High
Continued on next page						

Table A.2 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-2.12	The system shall allow authorised administrators to organise each tenant organisation into faculties, including creating, renaming, and archiving a faculty together with its subordinate units, and shall present an organisation's units either as a flat list or as a nested tree with descendant counts.	IT Admin-istrators, Educa-tional Managers	UR-ITA-01, UR-MGR-04	Yes	Yes	Medium
FR-2.13	The system shall maintain faculty staff affilia-tions as a many-to-many relationship that autho-rised deans may grant or revoke in bulk, and shall require a teacher to be ac-tive staff of the faculty owning a course before that teacher may be as-signed to it.	Faculty Deans, IT Adminis-trators	UR-MGR-02, UR-MGR-04	Yes	Yes	Medium
Continued on next page						

Table A.2 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-2.14	The system shall allow Educational Managers to issue per-course invitation codes with an optional maximum number of uses and an optional expiry date, to amend or deactivate them, and to revoke a code while retaining its record for audit purposes.	Educational Managers	UR-MGR-01	Yes	Yes	Low
FR-2.15	The system shall maintain a course enrolment lifecycle of active, completed, dropped, and waitlisted states together with the origin of each enrolment, shall record an unenrolment as a dropped enrolment rather than deleting it, and shall reactivate the existing record when a previously dropped student is re-enrolled.	Educational Managers, Teachers	UR-MGR-01	Yes	Yes	High

Course and Material Management

Table A.3: Functional Requirements for Course & Material Management

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-3.1	The system shall allow Teachers and Educational Managers to manage courses through draft, published, archived, and soft-deleted states, including administrative restoration of soft-deleted courses.	Teachers, Educational Managers	UR-TCH-01	Yes	Yes	High
FR-3.2	The system shall allow authoring roles to organise course content into ordered modules and learning items, including lessons, quizzes, and interviews, with configurable prerequisites and unlock rules.	Teachers, Educational Managers	UR-TCH-01	Yes	Yes	High
FR-3.3	The system shall allow Teachers to upload learning materials in supported formats using standard or resumable upload flows, manage material metadata, link materials to lessons, and re-run processing when needed.	Teachers, Educational Managers	UR-TCH-01	Yes	Yes	High
Continued on next page						

Table A.3 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-3.4	The system shall maintain version histories for updated materials, expose the current approved version to learners, and retain prior versions for re-processing or rollback.	Teachers, Educa- tional Managers	UR-TCH-01	Yes	Yes	Medium
FR-3.5	The system shall expose ingestion progress for each material version, including processing status, percent completion, and failure reasons, both per material and aggregated per lesson.	Teachers, Educa- tional Managers	UR-TCH-01	Yes	Yes	Medium
FR-3.6	The system shall allow Teachers to control material and resource visibility independently from course publication status, ensuring that students can access only materials that are both course-published and marked visible to students.	Teachers, Educa- tional Managers	UR-TCH-01	Yes	Yes	Medium
Continued on next page						

Table A.3 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-3.7	The system shall allow enrolled Students to browse the published catalogue of courses in their organisation, view course detail, and stream or download approved materials via short-lived presigned URLs.	Students	UR-STU-01	Yes	Yes	High
FR-3.8	The system shall allow IT administrators to view courses across organisations, restore soft-deleted courses, inspect processing-job history, and view aggregate course statistics.	IT Admin-istrators	UR-ITA-03	Yes	Yes	Medium
Continued on next page						

Table A.3 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-3.9	The system shall allow authorised managers to staff a course with multiple Course Instructors and multiple Teaching Assistants, where a single teacher may hold both titles, shall treat the re-assignment of an already active teacher as no change, shall revoke an assignment by expiring it rather than deleting it, and shall notify a newly assigned teacher.	Educational Managers, Faculty Deans	UR-MGR-02	Yes	Yes	High
FR-3.10	The system shall expose a pre-publication readiness checklist for a course reporting staffing adequacy against the configured minimum and maximum teacher counts, the number of gradeable curriculum units, and the presence of learning outcomes.	Teachers, Educational Managers	UR-TCH-01	Yes	Yes	Medium
Continued on next page						

Table A.3 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-3.11	The system shall allow authorised managers to clone an existing course at a selectable depth — the course shell with its learning outcomes, the shell together with the module skeleton and module prerequisites, or the full curriculum — creating the copy as a draft with a uniquely generated slug, and shall deliberately exclude learner runtime data such as enrolments, attempts, and grades from the copy.	Educational Managers, Faculty Deans	UR-TCH-01	Yes	Yes	Medium
FR-3.12	The system shall derive a knowledge map of concepts and their relationships for each lesson from that lesson’s ingested materials, shall allow Teachers to curate the map, and shall expose it to Students only after the Teacher has published it.	Teachers, Students	UR-TCH-01, UR-STU-01	Yes	Yes	Medium
Continued on next page						

Table A.3 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-3.13	The system shall allow Teachers to select a text-preprocessing mode for each material, shall quarantine extracted units that are unsafe to index together with the reason and the number of occurrences, shall allow Teachers to restore or confirm each quarantined unit, and shall expose a preview of the resulting content chunks.	Teachers	UR-TCH-01	Yes	Yes	Medium
FR-3.14	The system shall recover ingestion work that was interrupted, reconciling processing jobs left pending or running with no live background task, finalising abandoned generation runs, and periodically removing orphaned uploads.	IT Admin-istrators	UR-ITA-03	Yes	Yes	Medium
Continued on next page						

Table A.3 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-3.15	The system shall provide per-lesson discussion topics that course managers may open, edit, close, and moderate, on which enrolled Students and course managers may post and edit comments, and shall refuse new comments on a closed topic from users who cannot manage the course.	Students, Teachers, Educa- tional Managers	UR-STU- 01, UR-TCH- 05	Yes	Yes	Medium

Learning Loop and Adaptive Quiz

Table A.4: Functional Requirements for Learning Loop & Adaptive Quiz

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-4.1	The system shall allow Teachers to create quiz questions manually or through AI drafting from ingested course content, review them through pending, approved, edited, or rejected states, reuse approved questions, and maintain source references and revision history.	Teachers, Educational Managers	UR-TCH-02	Yes	Yes	High
FR-4.2	The system shall require Teachers to configure quiz and spaced-repetition settings — including expected response time T_{exp} , question metadata, initial EF, EF unlock threshold, coverage threshold τ , prerequisites, and optional interview-pass requirements — and shall block quiz publication until T_{exp} is set on every live question.	Teachers, Educational Managers	UR-TCH-03, UR-TCH-04	Yes	Yes	High
Continued on next page						

Table A.4 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-4.3	The system shall allow Students to take quiz sessions subject to teacher-configured retake policies and per-card cooldown rules, while recording correctness, response time, and hint usage for each answer.	Students	UR-STU-01	Yes	Yes	High
FR-4.4	The system shall derive a per-card quality score Q from correctness, hint usage, and the time ratio $\rho = T_{\text{actual}}/T_{\text{exp}}$, then update each student's per-card spaced-repetition state using the hybrid SM-2 scheduling model, which applies a calibration weight α on early-rep positive EF deltas, enforces an EF floor of 1.3, and adds per-card interval jitter.	Students	UR-STU-01	Yes	Yes	High
Continued on next page						

Table A.4 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-4.5	The system shall gate lesson access using prerequisite completion, the lesson coverage threshold τ over cards meeting the EF unlock threshold, and optional interview-pass requirements.	Students	UR-STU-01	Yes	Yes	High
FR-4.6	The system shall deliver in-app spaced-repetition reminders for due cards and remediation notifications after failed answers, linking students back to relevant source materials when available.	Students	UR-STU-01	Yes	Yes	Medium
FR-4.7	The system shall provide Teachers with learning-loop analytics, including cohort retention distribution, difficult-card lists, at-risk-student lists, and per-student spaced-repetition details.	Teachers	UR-TCH-05	Yes	Yes	Medium
Continued on next page						

Table A.4 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-4.8	The system shall provide Students with personal learning-loop dashboards, including due-card lists, per-lesson knowledge-retention estimates, progression readiness, and course-level progress overview.	Students	UR-STU-04	Yes	Yes	Medium
FR-4.9	The system shall route answers that cannot be graded automatically — code answers always, and short-answer or fill-in-the-blank answers whose exact-match grading failed — into a per-quiz manual grading queue, shall withhold the attempt score while any such answer is unmarked, and shall recompute the score and the pass decision once every answer carries a mark.	Teachers, Students	UR-TCH-02, UR-TCH-05	Yes	Yes	High
Continued on next page						

Table A.4 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-4.10	The system shall maintain a single grade of record for each student on each quiz, derived from that quiz's grading method of highest, first, last, or average attempt, shall refresh it whenever an attempt is graded or regraded, and shall expose the resulting grade-book to Teachers.	Teachers, Students	UR-TCH-05	Yes	Yes	High
FR-4.11	The system shall allow a Teacher to correct an answer key and preview the consequences as a dry run reporting every changed answer and the number of affected attempts, and shall apply the correction only upon explicit commit, recomputing the affected scores and recording the operation for audit.	Teachers	UR-TCH-02	Yes	Yes	Medium
Continued on next page						

Table A.4 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-4.12	The system shall allow Teachers to grant per-student and per-group assessment accommodations that override a quiz’s availability window, due date, time limit, attempt ceiling, retake permission, and cooldown, shall resolve them by student-over-group-over-quiz precedence, and shall apply the resolved policy when the Student starts an attempt.	Teachers, Students	UR-TCH-03, UR-STU-01	Yes	Yes	Medium
FR-4.13	The system shall allow Teachers to define score bands on a quiz mapping a range of percentage scores to a feedback message shown to the Student on attempt review, and shall withhold that feedback whenever the score itself is not disclosed to the Student.	Teachers, Students	UR-TCH-03	Yes	Yes	Low
Continued on next page						

Table A.4 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-4.14	The system shall maintain a course-scoped curated question bank of reusable items with a draft, approved, and archived lifecycle, shall demote an approved item to draft when its content changes, shall skip rather than reject duplicates when copying existing questions into the bank, and shall admit only approved items when importing from the bank into a quiz.	Teachers	UR-TCH-02	Yes	Yes	Medium
FR-4.15	The system shall import quiz questions from, and export them to, interchange files in the Moodle GIFT and Moodle XML formats, shall admit imported questions as pending review, and shall support bulk approval and bulk assignment of expected response time.	Teachers	UR-TCH-02	Yes	Yes	Low
Continued on next page						

Table A.4 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-4.16	The system shall record browser integrity signals reported during a quiz attempt in an append-only store, accepting focus loss, tab switching, full-screen exit, warnings, and connection loss, shall accept such reports only while the attempt is in progress and never allow their ingestion to interrupt the attempt, shall inform the Student that monitoring is active, and shall present the resulting timeline together with an append-only assessment audit trail to the Teacher.	Students, Teachers	UR-TCH-05	Yes	Yes	Medium
Continued on next page						

Table A.4 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-4.17	The system shall produce per-question item analysis for a quiz, including the facility index and the discrimination index with an explanation when the latter cannot be computed, together with per-student response reports, and shall deliver both as structured data, comma-separated, or spreadsheet downloads.	Teachers	UR-TCH-05	Yes	Yes	Medium
FR-4.18	The system shall close timed quiz attempts whose deadline passes without submission, honouring the quiz's overdue handling policy by grading the work completed, extending the deadline by a configured grace period, or abandoning the attempt without a grade.	Students	UR-STU-01	Yes	Yes	High

Application Loop and Mock Interview

Table A.5: Functional Requirements for the Application Loop & Mock Interview

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-5.1	The system shall allow Teachers to define interview sections by specifying desired outcomes that students must demonstrate to pass, without requiring any expected or model answer.	Teachers, Educational Managers	UR-TCH-03	Yes	Yes	High
FR-5.2	The system shall draft interview questions from ingested course content, associate each question with one or more desired outcomes, and allow Teachers to review, edit, approve, or reject them before publication.	Teachers, Educational Managers	UR-TCH-02	Yes	Yes	High
FR-5.3	The system shall allow Teachers to configure interview session rules, including maximum attempts, cooldown periods, and whether related quiz progression remains locked until the interview is passed.	Teachers, Educational Managers	UR-TCH-03	Yes	Yes	High
Continued on next page						

Table A.5 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-5.4	The system shall allow Students to start mock interview sessions in text mode or voice mode as an adaptive turn-taking dialogue with an AI interviewer.	Students	UR-STU-02	Yes	Yes	High
FR-5.5	The system shall support voice-mode interview sessions through real-time audio streaming, speech-to-text transcription, and text-to-speech responses, while processing raw audio transiently without persistent archival.	Students	UR-STU-02	Yes	Yes	High
FR-5.6	The system shall persist the interview transcript and session metadata, and evaluate the student's responses against teacher-defined desired outcomes to produce a binary pass-or-fail verdict.	Students, Teachers	UR-STU-03, UR-TCH-03	Yes	Yes	High
Continued on next page						

Table A.5 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-5.7	The system shall expose only the binary pass-or-fail verdict to Students — without per-outcome rubric scores — while allowing Teachers to review the transcript and outcome-level assessment for remediation purposes.	Students, Teachers	UR-STU-03, UR-TCH-03	Yes	Yes	High
FR-5.8	The system shall record interview session activity signals, such as browser focus changes, tab switching, full-screen status, and session events, to support post-session integrity review.	Students, Teachers	UR-STU-02, UR-TCH-03	Yes	Yes	Medium
Continued on next page						

Table A.5 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-5.9	The system shall maintain a course-scoped interview question bank in which one logical question carries sibling variants covering complementary interview angles, shall detect near-duplicate questions before they are added, and shall require Teacher approval of generated variants before they may be used in a published interview.	Teachers, Educa- tional Managers	UR-TCH-02	Yes	Yes	Medium
Continued on next page						

Table A.5 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-5.10	The system shall generate a gap report for a completed interview session summarising the observed discrepancies together with an actionable study plan of topics linked to the relevant lessons and suggested resources, shall withhold criterion-level scores and internal assessment rationale from the Student, and shall allow the Teacher to attach remediation notes.	Students, Teachers	UR-STU-03, UR-TCH-03	Yes	Yes	Medium

Analytics, Reporting and Notifications

Table A.6: Functional Requirements for Analytics, Reporting & Notifications

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-6.1	The system shall allow Students to view their per-lesson and per-course completion progress and record material engagement separately from spaced-repetition metrics.	Students	UR-STU-04	Yes	Yes	Medium
FR-6.2	The system shall allow Teachers to view course-level roster progress and identify at-risk learners based on course-progress signals.	Teachers, Educational Managers	UR-TCH-05	Yes	Yes	Medium
FR-6.3	The system shall provide an in-app notification inbox with unread-count, mark-read, dismiss actions, and configurable notification preferences by category and channel.	All Users	UR-STU-04, UR-ITA-05	Yes	Yes	Medium
Continued on next page						

Table A.6 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-6.4	The system shall provide authorised administrators with platform and organisation-level statistics, including active users, content volume, and system-health indicators.	IT Admin-istrators, Educa-tional Managers	UR-ITA-03	Yes	Yes	Medium
FR-6.5	The system shall track AI-related operational cost telemetry, including usage summaries by user, pipeline, and recent model calls.	IT Admin-istrators	UR-ITA-03	Yes	Yes	Medium
FR-6.6	The system shall allow administrators to inspect ingestion and AI process-ing jobs, view failed jobs, and manually retry failed processing tasks.	IT Admin-istrators	UR-ITA-03	Yes	Yes	Medium
Continued on next page						

Table A.6 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-6.7	The system shall maintain an immutable audit log for role assignments, sensitive data changes, and HTTP request events, queryable by administrators for security and compliance review.	IT Admin-istrators	UR-ITA-03	Yes	Yes	High
FR-6.8	The system shall produce organisation-scoped Career Readiness snapshots that aggregate pathway-level learner outcomes with drill-down to individual progress while excluding per-interview rubric details.	Educational Managers	UR-MGR-03	Yes	Yes	Low
Continued on next page						

Table A.6 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-6.9	The system shall let each user maintain notification preferences as a matrix of category against delivery channel in which a preference is enabled unless explicitly disabled, shall consult the recipient's preference for the relevant category before sending an e-mail, shall always create the in-app notification, and shall deliver e-mail asynchronously so that no request depends upon mail delivery.	All Users	UR-STU-04, UR-ITA-05	Yes	Yes	Medium

Learning Programs and Path Governance

Table A.7: Functional Requirements for Learning Programs & Path Governance

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-7.1	The system shall allow Faculty Deans and Educational Managers to define a learning programme within a faculty of their own organisation as an ordered selection of published career pathways, and shall reject any pathway that is unpublished, archived, or owned by another organisation.	Faculty Deans, Educational Managers	UR-MGR-02	Yes	Yes	High
FR-7.2	The system shall manage a learning programme through draft, published, and archived states, shall refuse publication when the programme has no draft version, no attached pathway, or a pathway that has since become unavailable, and shall treat an archived programme as immutable and closed to enrolment.	Faculty Deans, Educational Managers	UR-MGR-02	Yes	Yes	High
Continued on next page						

Table A.7 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-7.3	The system shall version a learning programme, creating a new draft version rather than modifying a published one, shall carry forward the pathway selection with each pathway pinned to a specific published pathway version, and shall retain every enrolled student on the programme version they enrolled upon.	Faculty Deans, Ed- ucational Managers	UR-MGR-02	Yes	Yes	High
Continued on next page						

Table A.7 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-7.4	The system shall allow authorised operators to enrol Students into a published learning programme individually or by comma-separated file import, creating accounts for unknown e-mail addresses, reporting per-row failures without aborting the batch, de-duplicating repeated addresses, and reinstating a previously withdrawn student rather than duplicating the enrolment.	Faculty Deans, Educational Managers	UR-MGR-01	Yes	Yes	High
FR-7.5	The system shall enforce an organisation-configurable ceiling on the number of learning-programme enrolments a Student may hold concurrently, and shall reject an enrolment that would exceed it, reporting the limit and the current count.	Faculty Deans, Educational Managers	UR-MGR-01, UR-MGR-04	Yes	Yes	Medium
Continued on next page						

Table A.7 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-7.6	The system shall require an enrolled Student to select exactly one initial career pathway from those pinned in their programme version, shall grant pathway access at the pinned version, and shall permit that initial selection only once.	Students	UR-STU-01	Yes	Yes	High
FR-7.7	The system shall allow a Student with an active enrolment to request a change of career pathway with a mandatory justification, shall permit at most one open request per enrolment, shall reject a request targeting the current pathway or a pathway outside the enrolled programme version, and shall refuse a request once the Student has exhausted the switch allowance defined by their programme version.	Students	UR-STU-01	Yes	Yes	Medium
Continued on next page						

Table A.7 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-7.8	The system shall progress a pathway-change request through pending, acknowledged, approved, rejected, cancelled, and invalidated states, shall restrict the decision to a Faculty Dean owning the programme, shall forbid a reviewer from deciding a request concerning their own enrolment, and shall require a rejection to cite a reason drawn from a fixed vocabulary with supporting detail whenever the reason is recorded as other.	Faculty Deans, Students	UR-MGR-03	Yes	Yes	Medium
Continued on next page						

Table A.7 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-7.9	The system shall, upon approving a pathway change, capture a snapshot of the outgoing attempt's completed courses and overall progress, close that attempt, open a new attempt pinned to the target pathway version, carry over access to courses shared by both pathways while releasing access that is no longer justified, and count the change against the Student's switch allowance.	Faculty Deans, Students	UR-MGR-03, UR-STU-01	Yes	Yes	High
FR-7.10	The system shall preserve a Student's earned course completions as immutable awards that survive a pathway change, so that results already achieved continue to count towards the new pathway.	Students	UR-STU-01	Yes	Yes	High
Continued on next page						

Table A.7 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-7.11	The system shall allow authorised operators to withdraw a Student from a learning programme with a mandatory reason, shall refuse to withdraw a completed enrolment, and shall cancel any open pathway-change request as part of the withdrawal.	Faculty Deans, Educational Managers	UR-MGR-01	Yes	Yes	Medium
FR-7.12	The system shall notify a Student when their pathway-change request is acknowledged, approved, or rejected, and shall notify the owning Faculty Deans when a request is filed.	Students, Faculty Deans	UR-STU-04, UR-MGR-03	Yes	Yes	Medium
FR-7.13	The system shall complete a Student's programme enrolment once every course of their pinned pathway version is satisfied.	Students	UR-STU-01	Yes	Yes	Medium

Platform Configuration and Policy Documents

Table A.8: Functional Requirements for Platform Configuration & Policy Documents

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-8.1	The system shall expose a catalogue of runtime-tunable settings, each declaring its key, grouping, data type, default value, description, and permitted range, so that authorised administrators may retune system behaviour without a redeployment.	IT Administrators	UR-ITA-03	Yes	Yes	Medium
FR-8.2	The system shall resolve the effective value of every setting by a fixed precedence of organisation override, deployment-wide override, environment configuration, and code default, and shall report all candidate values alongside the effective value and its source.	IT Administrators, Educational Managers	UR-ITA-03	Yes	Yes	Medium
Continued on next page						

Table A.8 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-8.3	The system shall allow an authorised administrator to preview a pending configuration change as a validated dry run reporting the current and proposed values and the scope affected, before any change is applied.	IT Admin-istrators	UR-ITA-03	Yes	Yes	Medium
FR-8.4	The system shall require a written reason for every configuration change and shall record each application, clearance, and roll-back in an append-only change history identifying the setting, the scope, the previous and new values, and the acting administrator.	IT Admin-istrators	UR-ITA-03	Yes	Yes	Medium
Continued on next page						

Table A.8 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-8.5	The system shall allow an authorised administrator to roll back a recorded configuration change, restoring the value it replaced or removing the override so that the next precedence layer resumes, and shall record the rollback as a new history entry rather than rewriting history.	IT Admin-istrators	UR-ITA-03	Yes	Yes	Medium
FR-8.6	The system shall continue to operate on environ-ment configuration and code defaults when a stored setting is unreach-able or no longer satisfies its declared constraints, and shall not fail the call-ing operation.	IT Admin-istrators	UR-ITA-03	Yes	Yes	Medium
Continued on next page						

Table A.8 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-8.7	The system shall maintain policy documents, each classified as legal or academic, whose content is held in independently numbered versions progressing from draft to published, so that re-issuing a document never invalidates an existing reference to it.	IT Admin-istrators	UR-ITA-05	Yes	Yes	Low
FR-8.8	The system shall serve the index of published policies and the current text of any published policy without requiring authentication, shall substitute the English version when the requested language is unavailable, and shall allow administrators to restrict a policy's audience to selected roles or leave it public.	All Users, IT Admin-istrators	UR-ITA-05	Yes	Yes	Low

Appendix B Non-Functional Requirements Specification

Performance and Latency

Table B.1: Non-Functional Requirements for Performance & Latency

ID	Requirement	Users	Feasible	Testable	Priority
NFR-1.1	The system shall transcribe and index uploaded video lectures at an average processing speed of no more than 40 seconds per minute of video content (e.g., a 15-minute lecture completes within approximately 10 minutes on average).	Teachers	Yes	Yes	High
NFR-1.2	The system shall complete quiz generation within 5 minutes on average per lesson.	Teachers	Yes	Yes	High
NFR-1.3	The system shall complete interview question bank generation within 15 minutes on average per lesson, producing a minimum of 20 candidate questions covering all teacher-defined target outcomes.	Teachers	Yes	Yes	High
NFR-1.4	Dashboard pages must load within 2 seconds on average on a standard connection.	All Users	Yes	Yes	Medium
Continued on next page					

Table B.1 – Continued from previous page

ID	Requirement	Users	Feasi- ble	Test- able	Prior- ity
NFR-1.5	Video streaming must start within 3 seconds on average after the user initiates playback.	All Users	Yes	Yes	Medium
NFR-1.6	Search results must return within 0.5 seconds on average.	All Users	Yes	Yes	Medium

Scalability and Reliability

Table B.2: Non-Functional Requirements for Scalability & Reliability

ID	Requirement	Users	Feasi- ble	Test- able	Prior- ity
NFR-2.1	The system shall support on average 50 concurrent active users, 10 simultaneous video streams, and 5 concurrent AI interview sessions during peak hours without performance degradation.	Admin	Yes	Yes	Medium
NFR-2.2	The system shall efficiently store on average 10GB of course content using standard MP4 compression and optimize database queries for an average response time under 0.5 seconds with 10,000 student records.	Admin	Yes	Yes	Low

Continued on next page

Table B.2 – Continued from previous page

ID	Requirement	Users	Feasi- ble	Test- able	Prior- ity
NFR-2.3	The system shall maintain a 95% uptime target on average and feature graceful degradation (e.g., video streaming continues even if the AI interview module temporarily fails).	All Users, Admin	Partial	Yes	Medium
NFR-2.4	The system shall provide fault tolerance through redundant AI servers, database replication across availability zones, and automatic retry logic for transient network issues.	Admin	Partial	Yes	Low

Security and Compliance

Table B.3: Non-Functional Requirements for Security & Compliance

ID	Requirement	Users	Feasi- ble	Test- able	Prior- ity
NFR-3.1	All sensitive data must be encrypted at rest using AES-256, including user passwords (hashed with bcrypt, cost factor ≥ 12), personally identifiable information (PII), session tokens, API keys, and uploaded learning materials.	All Users, Admin	Yes	Yes	High
Continued on next page					

Table B.3 – Continued from previous page

ID	Requirement	Users	Feasi- ble	Test- able	Prior- ity
NFR-3.2	The system must enforce Role-Based Access Control (RBAC) following the principle of least privilege, ensuring users can only access resources and perform actions authorized for their assigned role.	Managers, Admin	Yes	Yes	High
NFR-3.3	The system must maintain immutable audit logs for all sensitive operations, including authentication events, permission changes, data modifications, and administrative actions, with each log entry capturing the actor, timestamp, action type, and affected resource.	Managers, Admin	Yes	Yes	High
NFR-3.4	The system must enforce a 30-minute idle session timeout, automatically logging out users after the period of inactivity and requiring re-authentication.	All Users	Yes	Yes	High

Usability and Accessibility

Table B.4: Non-Functional Requirements for Usability & Accessibility

ID	Requirement	Users	Feasible	Testable	Priority
NFR-4.1	The User Interface must be mobile-responsive and maintain a consistent design system.	Students, Teachers	Yes	Yes	High
NFR-4.2	The system must comply with WCAG 2.1 AA accessibility standards, supporting full keyboard navigation (no mouse required), sufficient color contrast ratios, and alt text for all visual elements.	Users with Disabilities	Partial	Yes	Low
NFR-4.3	The system shall only support Chromium-based browsers (e.g., Google Chrome, Microsoft Edge, Brave, Opera).	All Users	Yes	Yes	Medium

Appendix C Use Case Specification

C.1 Authentication and Access Control Module

Table C.1: Use Case Specification: Authenticate and Log Out

Use Case ID	UC-AUTH-01
Use Case Name	Authenticate and Log Out
Actors	Anonymous Visitor for login; Authenticated User for logout
Related Requirements	FR-1.1, FR-1.2, FR-1.3
Description	Allows a pre-registered user to authenticate using Google SSO, complete multi-factor authentication when required, establish an authenticated session, and securely log out of the system.
Pre-conditions	• The user has been provisioned in the system and the account is active. • The user owns a Google account associated with the provisioned system account. • For logout, the user is currently authenticated with an active session.
Post-conditions	• If authentication succeeds and MFA is not required, the user receives an authenticated session and is redirected to the appropriate dashboard. • If MFA is required, the user must complete the second-factor challenge before accessing protected system features. • If the user logs out, the active session is revoked and can no longer be used to access protected resources.
Continued on next page	

Table C.1 – Continued from previous page

Main Flow	1. The user navigates to the Login page. 2. The user selects “Continue with Google”. 3. The system redirects the user to the Google SSO authentication flow. 4. Google authenticates the user and redirects back to the aBridgeAI system. 5. The system validates the authentication response and resolves the corresponding local user account. 6. The system verifies that the account is active and allowed to access the platform. 7. The system creates an authenticated session and issues the required access credentials. 8. If MFA is required, the system redirects the user to the MFA verification screen. 9. If MFA is not required or has been completed, the system redirects the user to the dashboard according to their role and permissions. 10. When the user selects “Log out”, the system revokes the active session and redirects the user to the Login page.
Continued on next page	

Table C.1 – Continued from previous page

Alternative Flows	4a. Google SSO authentication fails: 1. The system rejects the authentication attempt. 2. The user is returned to the Login page with an authentication error. 5a. User account is not provisioned: 1. The system cannot match the Google identity to a provisioned local account. 2. The system denies access and instructs the user to contact an administrator. 6a. User account is inactive or disabled: 1. The system refuses to create an authenticated session. 2. The user is informed that the account is not allowed to access the system. 8a. MFA verification is required: 1. The system prompts the user to enter an authenticator-app code or a one-time recovery code. 2. If the code is valid, the system marks the session as MFA-verified and redirects the user to the dashboard. 3. If the code is invalid or expired, the system rejects the verification attempt and remains on the MFA screen. 10a. Session has already expired before logout: 1. The system treats the logout request as completed. 2. The user is redirected to the Login page.
-------------------	--

Table C.2: Use Case Specification: Manage Multi-Factor Authentication

Use Case ID	UC-AUTH-02
Use Case Name	Manage Multi-Factor Authentication
Actors	All Users
Related Requirements	FR-1.3, FR-1.4
Continued on next page	

Table C.2 – Continued from previous page

Description	Allows users to verify their identity using an authenticator-app code or a one-time recovery code, and requires recent second-factor verification before sensitive account-security operations.
Pre-conditions	The user is authenticated through Google SSO and either has MFA enabled or is performing an MFA-sensitive account-security operation.
Post-conditions	The user completes MFA verification, or the requested sensitive operation is blocked until verification is refreshed.
Main Flow	1. The user completes Google SSO. 2. The system detects that MFA is enabled for the account. 3. The system prompts the user to enter a verification code from an authenticator application. 4. The user enters the verification code. 5. The system validates the code and marks the session as MFA-verified. 6. The user accesses protected features normally. 7. If the user attempts a sensitive operation, such as regenerating recovery codes or disabling MFA, the system checks whether the second-factor verification is still fresh. 8. If verification is fresh, the operation is allowed.
Alternative Flows	4a. Invalid verification code: 1. The system rejects the code. 2. The user remains on the MFA challenge screen. 4b. User chooses recovery code: 1. The user enters a one-time recovery code. 2. The system validates the code and marks it as used. 7a. Verification is no longer fresh: 1. The system blocks the sensitive operation. 2. The user is required to complete second-factor verification again.

Table C.3: Use Case Specification: Access Protected Features

Use Case ID	UC-AUTH-03
Use Case Name	Access Protected Features
Actors	All Users, IT Administrators
Related Requirements	FR-1.5, FR-1.6, FR-1.8
Description	Ensures that access to protected backend routes and frontend UI actions is controlled according to the authenticated user's active roles and effective permissions.
Pre-conditions	The user is authenticated and has an active session.
Post-conditions	The requested action is either allowed or denied based on the user's effective permissions, and relevant auditable events are recorded.
Main Flow	1. The user attempts to access a protected page or perform a protected action. 2. The frontend retrieves the user's active roles and effective permissions. 3. The frontend renders only the UI actions permitted for that user. 4. The user submits a protected backend request. 5. The backend validates the session and checks the required permission. 6. If the permission is present, the system completes the requested operation. 7. For authentication, MFA, account-security, or access-control actions, the system records an auditable event.
Continued on next page	

Table C.3 – Continued from previous page

Alternative Flows	5a. Permission missing: 1. The system denies the request. 2. No protected data is returned. 5b. Session invalid or revoked: 1. The system rejects the request. 2. The user is redirected to authenticate again.
--------------------------	---

Table C.4: Use Case Specification: Manage User Account Access

Use Case ID	UC-AUTH-04
Use Case Name	Manage User Account Access
Actors	IT Administrator
Related Requirements	FR-1.7, FR-1.8
Description	Allows IT Administrators to search users, view user details, enable or disable user accounts, and ensure that sensitive account-access changes are recorded as auditable events.
Pre-conditions	The actor is authenticated as an IT Administrator and has account-management permission.
Post-conditions	The selected user's account status is updated, active sessions are revoked when required, and the account-access change is recorded for audit review.
Continued on next page	

Table C.4 – Continued from previous page

Main Flow	1. The IT Administrator opens the account management page. 2. The administrator searches for a user by name, email, organisation, or status. 3. The system displays matching users within the administrator's permitted scope. 4. The administrator opens a user's detail page. 5. The administrator reviews account status, assigned roles, organisation membership, and recent security-related information. 6. The administrator enables or disables the selected account. 7. If the account is disabled, the system revokes all active sessions for that account. 8. The system records the account-access change as an auditable event.
Alternative Flows	2a. No matching user found: 1. The system displays an empty search result. 2. No account state is changed. 3a. User is outside administrator scope: 1. The system hides the user record or denies access to the detail page. 6a. Administrator attempts an invalid status transition: 1. The system rejects the update and displays a validation message. 8a. Audit logging fails: 1. The system prevents the sensitive change from being completed or flags the operation for administrative review according to the audit policy.

C.2 Organization and User Management Module

Table C.5: Use Case Specification: Manage Tenant Organisations

Use Case ID	UC-ORG-01
Use Case Name	Manage Tenant Organisations
Actors	IT Administrator
Related Requirements	FR-2.1, FR-2.2, FR-2.7, FR-2.9
Description	Allows IT Administrators to provision tenant organisations, manage organisation lifecycle states, configure organisation domains, and enable domain-based account auto-provisioning.
Pre-conditions	The user is authenticated as an IT Administrator.
Post-conditions	A tenant organisation is created or updated, and its domain and lifecycle configuration are saved.
Main Flow	1. The IT Administrator opens the Organisation Management dashboard. 2. The administrator creates a new tenant organisation or selects an existing one. 3. The administrator enters or updates the organisation's basic information. 4. For a new organisation, the administrator assigns the initial Educational Manager. 5. The administrator configures the organisation lifecycle status. 6. The administrator attaches one or more email domains to the organisation. 7. The administrator enables or disables auto-provisioning for each domain. 8. The system saves the organisation configuration.
Continued on next page	

Table C.5 – Continued from previous page

Alternative Flows	3a. Invalid organisation information: 1. The system highlights invalid or missing fields. 2. The organisation is not saved until the fields are corrected. 6a. Domain already attached elsewhere: 1. The system rejects the domain assignment. 2. The administrator is informed that the domain is already registered.
--------------------------	--

Table C.6: Use Case Specification: Manage Users and Memberships

Use Case ID	UC-ORG-02
Use Case Name	Manage Users and Memberships
Actors	Educational Manager, IT Administrator, All Users
Related Requirements	FR-2.3, FR-2.6, FR-2.8, FR-2.10
Description	Supports user onboarding, organisation membership management, tenant-scoped manager actions, and authenticated user profile updates.
Pre-conditions	The actor is authenticated and has the required organisation-management or profile-management permission.
Post-conditions	User membership records, profile data, or account status are updated according to the actor's permission scope.
Continued on next page	

Table C.6 – Continued from previous page

Main Flow	1. The Educational Manager opens the User Management area for their organisation. 2. The manager uploads a CSV file to bulk-enroll students into a course. 3. The system validates each row and creates accounts for unseen email addresses. 4. The system reports per-row validation errors without aborting the whole batch. 5. The manager views or updates organisation membership records, including student or employee identifier codes. 6. The system ensures that every manager query and action is scoped to the manager's tenant organisation. 7. Separately, an authenticated user may view and update their own profile information.
Alternative Flows	3a. CSV contains invalid rows: 1. The system imports valid rows. 2. The system reports invalid rows with validation errors. 6a. Manager attempts cross-tenant access: 1. The system denies the request. 2. No cross-tenant data is returned. 7a. Profile data is invalid: 1. The system rejects invalid profile fields or external links. 2. The user is prompted to correct the profile information.

Table C.7: Use Case Specification: Manage Career Pathways and Teacher Assignments

Use Case ID	UC-ORG-03
Use Case Name	Manage Career Pathways and Teacher Assignments
Actors	Educational Manager
Continued on next page	

Table C.7 – Continued from previous page

Related Requirements	FR-2.4, FR-2.5, FR-2.6
Description	Allows Educational Managers to define career pathways as ordered course sequences and assign teachers to courses within their organisation.
Pre-conditions	The Educational Manager is authenticated and belongs to the tenant organisation being managed.
Post-conditions	Career pathways and teacher-course assignments are saved within the organisation scope.
Main Flow	1. The Educational Manager opens the Career Pathway Management page. 2. The manager creates a new pathway or edits an existing pathway. 3. The manager selects courses and arranges them into the intended order. 4. The manager saves the pathway. 5. The manager opens the course assignment page. 6. The manager selects a teacher within the organisation. 7. The manager assigns the teacher to a course. 8. The system saves the assignment and grants the teacher course-authoring access.
Alternative Flows	3a. Invalid course reference: 1. The system rejects the pathway update. 2. The manager is prompted to choose valid courses. 6a. Teacher outside organisation: 1. The system blocks the assignment. 2. The manager is informed that the teacher is outside their organisation scope.

Table C.8: Use Case Specification: Organise Faculties and Staff Courses

Use Case ID	UC-ORG-04
Use Case Name	Organise Faculties and Staff Courses
Actors	IT Administrator and Faculty Dean for the faculty structure; Educational Manager for course staffing
Related Requirements	FR-2.12, FR-2.13, FR-3.9, FR-3.10
Description	Allows administrators to organise an organisation into faculties and to maintain faculty staff affiliations, and allows managers to staff a course with instructors and teaching assistants subject to those affiliations.
Pre-conditions	• The operator is authenticated and holds organisation-unit management rights, or dean rights over the faculty concerned. • For course staffing, the course exists and the candidate teacher is active staff of the faculty owning the course.
Post-conditions	• The organisation's faculties and their staff affiliations reflect the changes. • Course staffing records the instructor and assistant titles held by each teacher. • A revoked assignment is expired rather than deleted, preserving the audit trail.
Continued on next page	

Table C.8 – Continued from previous page

Main Flow	1. The administrator opens the organisation's structure page. 2. The administrator creates a faculty, or renames, re-codes, or archives an existing one. 3. The system presents the organisation's units as a flat list or as a nested tree with descendant counts. 4. The administrator assigns members into a faculty in bulk, or detaches them, and the system reports how many rows were affected. 5. A Faculty Dean grants or revokes staff affiliations for the faculty, which may span several faculties per member. 6. An Educational Manager opens a course and requests the list of assignable teachers, which the system scopes to the course's organisation and marks those already assigned. 7. The manager assigns a teacher, marking them as Course Instructor, Teaching Assistant, or both. 8. The system records the assignment and notifies the teacher with a link to the course. 9. The manager reviews the course readiness checklist, which reports staffing against the configured minimum and maximum, the gradeable units, and the learning outcomes. 10. To remove a teacher, the manager revokes one or several assignments and the system expires them.
Continued on next page	

Table C.8 – Continued from previous page

Alternative Flows	2a. Faculty archived while it still has subordinate units: 1. The system archives the subordinate units together with the faculty. 3a. A unit's parent is missing or archived: 1. The system presents the unit as a root so that it is never hidden from the operator. 7a. Teacher is not staff of the owning faculty: 1. The system refuses the assignment. 7b. Teacher already actively assigned: 1. The system treats the request as no change and does not send a further notification. 8a. Course is archived: 1. The system records the assignment without notifying the teacher.
-------------------	---

Table C.9: Use Case Specification: Manage Course Enrolment and Invitation Codes

Use Case ID	UC-ORG-05
Use Case Name	Manage Course Enrolment and Invitation Codes
Actors	Educational Manager, Teacher
Related Requirements	FR-2.3, FR-2.14, FR-2.15
Description	Allows an Educational Manager to populate and maintain a course roster by direct selection or file import and to issue invitation codes, and allows a Teacher to adjust an individual enrolment.
Pre-conditions	• The operator is authenticated and holds course enrolment rights for the course. • The course exists.
Continued on next page	

Table C.9 – Continued from previous page

Post-conditions	• Accepted Students hold an active enrolment on the course. • Rejected rows are reported individually without aborting the batch. • An unenrolment is recorded as a dropped enrolment retaining its history.
Main Flow	1. The manager opens the course roster. 2. The manager enrolls Students by selecting accounts, entering e-mail addresses, or uploading a comma-separated file. 3. The system creates accounts for unknown e-mail addresses and enrolls each accepted Student. 4. The system reports the number enrolled, the accounts created, and the per-row failures. 5. The manager issues an invitation code for the course, optionally limiting the number of uses and setting an expiry date. 6. The system records the code together with its usage counter and active state. 7. The manager amends a code's expiry, usage limit, or active state, or revokes it. 8. The system deactivates a revoked code while retaining its record for audit. 9. A Teacher adjusts an individual enrolment on the roster, dropping or reactivating it. 10. The manager unenrolls a Student, and the system records the enrolment as dropped.
Continued on next page	

Table C.9 – Continued from previous page

Alternative Flows	2a. Neither accounts nor addresses supplied: 1. The system rejects the request as requiring at least one identifier. 3a. Malformed or duplicate file row: 1. The system records the row number and the reason and continues with the remaining rows. 10a. Student was previously dropped: 1. The system reactivates the existing enrolment rather than creating a duplicate. 5a. Duplicate code string: 1. The system reports the conflict and the manager supplies a different code.
-------------------	---

C.3 Course and Material Management Module

Table C.10: Use Case Specification: Manage Course Structure and Lifecycle

Use Case ID	UC-COURSE-01
Use Case Name	Manage Course Structure and Lifecycle
Actors	Teacher, Educational Manager, IT Administrator
Related Requirements	FR-3.1, FR-3.2, FR-3.6, FR-3.8
Description	Allows authorised users to manage course lifecycle states, organise course content into ordered modules and learning items, configure prerequisites and unlock rules, and control material visibility.
Pre-conditions	The actor is authenticated and has course-authoring or administrative permission.
Post-conditions	The course structure, lifecycle state, visibility settings, and unlock rules are saved.
Continued on next page	

Table C.10 – Continued from previous page

Main Flow	1. The Teacher or Educational Manager opens the Course Management dashboard. 2. The actor creates or selects a course. 3. The actor manages the course lifecycle state, such as draft, published, archived, or soft-deleted. 4. The actor organises modules and learning items, including lessons, quizzes, and interviews. 5. The actor configures prerequisites and unlock rules. 6. The actor configures material and resource visibility for students. 7. The system saves the course configuration. 8. If the actor is an IT Administrator, the actor may inspect course processing-job history or restore soft-deleted courses.
Alternative Flows	5a. Invalid prerequisite configuration: 1. The system rejects the configuration. 2. The actor is prompted to correct the prerequisite rule. 8a. Restore permission missing: 1. The system blocks course restoration. 2. The actor is shown an unauthorised message.

Table C.11: Use Case Specification: Upload and Process Learning Materials

Use Case ID	UC-COURSE-02
Use Case Name	Upload and Process Learning Materials
Actors	Teacher, Educational Manager
Related Requirements	FR-3.3, FR-3.4, FR-3.5
Description	Allows Teachers to upload supported learning materials, manage metadata, link materials to lessons, monitor ingestion progress, and maintain material version history.
Continued on next page	

Table C.11 – Continued from previous page

Pre-conditions	The actor has course-authoring permission and has selected a target lesson.
Post-conditions	The material is stored, linked to the lesson, processed by the ingestion pipeline, and versioned when updated.
Main Flow	1. The Teacher opens a lesson in authoring mode. 2. The Teacher selects “Upload Material”. 3. The Teacher uploads a supported file using a standard or resumable upload flow. 4. The system stores the material and records upload metadata. 5. The Teacher links the material to one or more lessons. 6. The system queues the material for ingestion and processing. 7. The system displays processing status, percent completion, and any failure reason. 8. If a replacement file is uploaded, the system creates a new material version and retains prior versions for rollback or re-processing.
Alternative Flows	3a. Unsupported format or failed upload: 1. The system rejects the upload. 2. The Teacher is shown an error message. 7a. Processing fails: 1. The system marks the material version as failed. 2. The failure reason is displayed to the Teacher. 8a. Teacher re-runs processing: 1. The Teacher selects “Re-run processing”. 2. The system queues a new processing job for the material version.

Table C.12: Use Case Specification: Browse Courses and Access Materials

Use Case ID	UC-COURSE-03
Use Case Name	Browse Courses and Access Materials
Actors	Student
Related Requirements	FR-3.7
Description	Allows enrolled Students to browse published courses in their organisation, view course details, and stream or download approved materials.
Pre-conditions	The Student is authenticated, enrolled in the course, and the course is published.
Post-conditions	The Student accesses only approved and visible learning materials.
Main Flow	1. The Student opens the course catalogue. 2. The system displays published courses available to the Student's organisation and enrollment status. 3. The Student opens a course detail page. 4. The system displays modules, lessons, learning outcomes, tags, and visible lesson resources. 5. The Student selects an approved material. 6. The system generates a short-lived presigned URL. 7. The Student streams or downloads the material.
Alternative Flows	2a. Course unpublished or unavailable: 1. The course is not displayed to the Student. 5a. Material hidden or not approved: 1. The material is not accessible to the Student. 6a. Presigned URL expires: 1. The system issues a new URL if the Student remains authorised.

Table C.13: Use Case Specification: Curate Ingested Content and Reuse a Course

Use Case ID	UC-COURSE-04
Use Case Name	Curate Ingested Content and Reuse a Course
Actors	Teacher for curation; Educational Manager for cloning; Student as the reader of the published knowledge map
Related Requirements	FR-3.11, FR-3.12, FR-3.13, FR-3.14
Description	Allows a Teacher to control how uploaded material is preprocessed, to review and resolve quarantined content, to curate and publish the lesson knowledge map, and allows a manager to reuse an existing course by cloning it.
Pre-conditions	• The Teacher is authenticated and holds authoring rights on the course. • At least one material has been uploaded and ingested for the lesson concerned.
Post-conditions	• The material's preprocessing mode and the resolution of each quarantined unit are recorded. • Students see the lesson knowledge map only after the Teacher has published it. • A cloned course exists as a draft with a unique identifier and carries no learner runtime data.
Continued on next page	

Table C.13 – Continued from previous page

Main Flow	1. The Teacher opens a material and reviews its preprocessing report. 2. The Teacher selects the preprocessing mode appropriate to the document. 3. The system reports the units it quarantined as unsafe to index, each with a reason and its number of occurrences. 4. The Teacher restores a quarantined unit or confirms its exclusion. 5. The Teacher previews the resulting content chunks. 6. The Teacher opens the lesson knowledge map derived from the ingested material and curates its concepts and relationships. 7. The Teacher publishes the curated map, after which Students see it on the lesson reading view. 8. The Teacher may later unpublish the map, after which the panel is hidden from Students. 9. To reuse a course, a manager selects “Clone” and chooses the depth: the shell with its learning outcomes, the shell with the module skeleton and module prerequisites, or the full curriculum. 10. The system creates the copy as a draft in the same organisation and faculty with a generated unique identifier, excluding enrolments, attempts, and grades.
Continued on next page	

Table C.13 – Continued from previous page

Alternative Flows	3a. Nothing was quarantined: 1. The system reports a clean extraction and no action is required. 7a. Map has never been published: 1. The system reports the map as unpublished and the Student view hides the panel rather than reporting an error. 1a. Ingestion was interrupted: 1. The system reconciles the affected processing job and resumes or fails it explicitly, rather than leaving it pending indefinitely.
-------------------	--

Table C.14: Use Case Specification: Participate in a Lesson Discussion

Use Case ID	UC-COURSE-05
Use Case Name	Participate in a Lesson Discussion
Actors	Course manager, being the course owner or an assigned teacher, opens and moderates topics; enrolled Student and course manager post comments
Related Requirements	FR-3.15, FR-6.9
Description	Allows a course manager to open discussion topics on a lesson and moderate them, and allows enrolled Students and course managers to comment, with participants notified of new activity.
Pre-conditions	• The participant is authenticated. • A Student participant is enrolled on the course owning the lesson. • For commenting, the topic is open, or the participant can manage the course.
Continued on next page	

Table C.14 – Continued from previous page

Post-conditions	• Topics and comments are stored with their authorship. • A removed topic or comment is withdrawn from view while its record is retained. • Prior participants and, for a Student comment, the course managers are notified.
Main Flow	1. The participant opens the Discussion area of a lesson. 2. The system lists the lesson's topics, newest first, indicating whether the viewer may open new topics. 3. A course manager opens a new topic with a title and an optional body. 4. A participant opens a topic and the system lists its comments oldest first with each author's display name. 5. The participant posts a comment. 6. The system notifies every prior commenter on the topic, excluding the comment's own author. 7. Where the author is not a course manager, the system additionally notifies the course managers. 8. The author edits or removes their own comment; a course manager may remove any comment as moderation. 9. A course manager edits the topic, closes it to further discussion, or removes it together with its comments.
Continued on next page	

Table C.14 – Continued from previous page

Alternative Flows	5a. Topic is closed: 1. The system refuses the comment from a participant who cannot manage the course. 2. A course manager may still post. 6a. Notification cannot be delivered: 1. The system records the comment regardless and does not fail the participant's action. 1a. Participant is not enrolled and cannot manage the course: 1. The system reports the discussion as unavailable without disclosing whether it exists. 5b. Recipient has disabled the discussion notification category for e-mail: 1. The system creates the in-app notification and omits the e-mail.
-------------------	---

C.4 Learning Loop and Adaptive Quiz Module

Table C.15: Use Case Specification: Author and Review Quiz Questions

Use Case ID	UC-LEARN-01
Use Case Name	Author and Review Quiz Questions
Actors	Teacher, Educational Manager
Related Requirements	FR-4.1
Description	Allows Teachers to create quiz questions manually or through AI drafting from ingested course content, review drafted questions, reuse approved questions, and maintain source references and revision history.
Continued on next page	

Table C.15 – Continued from previous page

Pre-conditions	The relevant course materials have been ingested, and the actor has quiz-authoring permission.
Post-conditions	Quiz questions are created, reviewed, and stored with source references and revision history.
Main Flow	1. The Teacher opens the Quiz Authoring page for a lesson or module. 2. The Teacher chooses to create questions manually or generate them from ingested content. 3. If AI drafting is selected, the system drafts questions from source materials and attaches source references. 4. The Teacher reviews each question. 5. The Teacher edits, approves, rejects, or reuses approved questions from the question bank. 6. The system stores the final question state and revision history.
Alternative Flows	3a. Source content unavailable: 1. The system blocks AI drafting. 2. The Teacher is informed that the source material is not ready. 5a. Question rejected: 1. The question is excluded from student-facing quiz publication.

Table C.16: Use Case Specification: Configure and Take Adaptive Quiz

Use Case ID	UC-LEARN-02
Use Case Name	Configure and Take Adaptive Quiz
Actors	Teacher, Student
Related Requirements	FR-4.2, FR-4.3, FR-4.4, FR-4.5
Continued on next page	

Table C.16 – Continued from previous page

Description	Covers quiz configuration, student quiz attempts, quality-score derivation, hybrid SM-2 spaced-repetition updates, and lesson access gating.
Pre-conditions	The quiz contains approved questions, and the Student is enrolled in the course.
Post-conditions	The Student's answers are recorded, quality score Q is derived, spaced-repetition state is updated, and lesson access is recalculated.
Main Flow	1. The Teacher configures quiz and spaced-repetition settings, including T_{exp}, initial EF, EF unlock threshold, coverage threshold τ, prerequisites, and optional interview-pass requirements. 2. The system blocks publication until T_{exp} is set for every live question. 3. The Student opens an available quiz session. 4. The Student answers quiz cards, optionally using hints. 5. The system records correctness, response time, and hint usage for each answer. 6. The system derives a quality score Q from correctness, hint usage, and $\rho = T_{\text{actual}}/T_{\text{exp}}$. 7. The system updates the Student's per-card spaced-repetition state using the hybrid SM-2 scheduling model. 8. The system recalculates whether the Student satisfies prerequisite completion and lesson coverage threshold τ. 9. If all unlock conditions are satisfied, the next lesson becomes accessible.
Continued on next page	

Table C.16 – Continued from previous page

Alternative Flows	2a. Missing expected response time: 1. The system blocks quiz publication. 2. The Teacher is prompted to set T_{exp}. 3a. Retake policy or cooldown prevents attempt: 1. The system blocks the quiz attempt. 2. The Student is shown the remaining cooldown or retake limitation. 8a. Unlock conditions not satisfied: 1. The next lesson remains locked. 2. The system shows the remaining requirements.
-------------------	--

Table C.17: Use Case Specification: View Learning Loop Dashboards and Notifications

Use Case ID	UC-LEARN-03
Use Case Name	View Learning Loop Dashboards and Notifications
Actors	Student, Teacher
Related Requirements	FR-4.6, FR-4.7, FR-4.8
Description	Provides spaced-repetition reminders, remediation notifications, Teacher learning-loop analytics, and Student learning dashboards.
Pre-conditions	Quiz activity and spaced-repetition data exist for the course or student.
Post-conditions	The actor receives relevant learning-loop insights, reminders, or remediation links.
Continued on next page	

Table C.17 – Continued from previous page

Main Flow	1. A Student has due cards or recently failed a card. 2. The system sends an in-app spaced-repetition reminder or remediation notification. 3. The Student opens the personal learning dashboard. 4. The system displays due-card lists, retention estimates, progression readiness, and course-level progress. 5. The Teacher opens course learning-loop analytics. 6. The system displays cohort retention distribution, difficult-card lists, at-risk students, and per-student spaced-repetition details.
Alternative Flows	1a. No due cards exist: 1. The system displays an empty due-card state. 2a. No reliable source reference exists: 1. The remediation notification is sent without a direct source-material link. 6a. No quiz activity exists: 1. The Teacher dashboard displays an empty state.

Table C.18: Use Case Specification: Grade Open Responses and Maintain the Gradebook

Use Case ID	UC-LEARN-04
Use Case Name	Grade Open Responses and Maintain the Gradebook
Actors	Teacher for grading and regrading; Student as the recipient of the resulting grade
Related Requirements	FR-4.9, FR-4.10, FR-4.11, FR-4.13
Description	Allows a Teacher to mark answers that cannot be graded automatically, to maintain a single grade of record per Student, and to correct an answer key by previewing and then committing a regrade.
Continued on next page	

Table C.18 – Continued from previous page

Pre-conditions	• The Teacher is authenticated and holds authoring rights on the quiz's course. • At least one attempt has been submitted.
Post-conditions	• Every answer requiring human judgement carries a mark, and the attempt's score and pass decision are computed. • Each Student holds one grade of record for the quiz, derived from the quiz's grading method. • A committed regrade updates the affected scores and is recorded for audit.
Continued on next page	

Table C.18 – Continued from previous page

Main Flow	1. The Teacher opens the quiz’s grading queue, which lists every answer awaiting a human mark. 2. The system withholds the headline score of any attempt that still contains an unmarked answer. 3. The Teacher records a mark and optional feedback for each open response. 4. Once no answer on an attempt remains unmarked, the system computes the attempt’s score, evaluates it against the pass threshold, and records the attempt as graded. 5. The system reduces the Student’s completed attempts to a single grade of record using the quiz’s grading method of highest, first, last, or average. 6. The system reflects the refreshed grade in the Student’s course completion. 7. Where the Teacher discovers an incorrect answer key, the Teacher corrects the question and requests a regrade preview. 8. The system re-grades the affected answers against the corrected definition and reports every changed answer together with the number of affected attempts, without altering any stored result. 9. The Teacher reviews the preview and commits it. 10. The system applies the changes, recomputes the affected attempts and grades of record, and records the operation. 11. The Student opens the attempt review and sees the score together with the feedback band matching it.
Continued on next page	

Table C.18 – Continued from previous page

Alternative Flows	1a. Nothing awaits grading: 1. The queue is empty because every answer was graded automatically. 8a. Preview reports no change: 1. The Teacher abandons the regrade. 11a. Score is not disclosed to the Student by the review policy: 1. The system withholds the band feedback together with the score. 6a. Course completion side effect fails: 1. The system retains the grade and repairs the completion state by a subsequent reconciliation.
--------------------------	--

Table C.19: Use Case Specification: Administer Assessment Delivery and Integrity

Use Case ID	UC-LEARN-05
Use Case Name	Administer Assessment Delivery and Integrity
Actors	Teacher for accommodations, the question bank, and reports; Student as the subject of delivery and monitoring
Related Requirements	FR-4.12, FR-4.14, FR-4.15, FR-4.16, FR-4.17, FR-4.18
Description	Allows a Teacher to grant assessment accommodations, to build and reuse a question bank including interchange-file import and export, to review integrity signals and item analysis, while the system closes timed attempts that are never submitted.
Pre-conditions	• The Teacher is authenticated and holds authoring rights on the quiz's course. • For reports, at least one attempt has been completed.
Continued on next page	

Table C.19 – Continued from previous page

Post-conditions	• An accommodated Student's attempt honours the resolved availability window, time limit, attempt ceiling, and cooldown. • Reusable questions are held in the course question bank with an approval state. • Integrity signals and the assessment audit trail are available to the Teacher. • A timed attempt is finalised even when the Student never submits it.
Continued on next page	

Table C.19 – Continued from previous page

Main Flow	1. The Teacher grants a Student or a group an accommodation overriding any of the availability window, due date, time limit, attempt ceiling, retake permission, and cooldown. 2. When the Student starts an attempt, the system resolves the effective policy by Student over group over quiz precedence and applies it. 3. The system displays the Student's own window, time limit, and attempt ceiling on the quiz detail. 4. During the attempt the client reports integrity signals such as tab switching and focus loss, and the Student is informed that monitoring is active. 5. Where the Student does not submit before the deadline, the system closes the attempt according to the quiz's overdue handling policy, either grading the work completed, extending the deadline by the configured grace period, or abandoning the attempt without a grade. 6. The Teacher curates the course question bank, authoring items as drafts and approving them for reuse. 7. The Teacher copies existing quiz questions into the bank, and the system skips items whose content already has a bank copy. 8. The Teacher imports approved bank items into a quiz, or imports questions from an interchange file, which arrive pending review. 9. The Teacher exports the quiz's questions as an interchange file. 10. The Teacher reviews the integrity timeline for an attempt and the append-only assessment audit trail for the quiz. 11. The Teacher opens the item analysis report and reviews the facility and discrimination indices per question, and downloads the reports.
Continued on next page	

Table C.19 – Continued from previous page

Alternative Flows	2a. Several group accommodations apply: 1. The system combines them into the single most generous set. 6a. An approved bank item is edited: 1. The system returns the item to draft so that it re-enters review. 8a. Bank item is not approved: 1. The system refuses to import it into the quiz. 11a. Discrimination index cannot be computed: 1. The system reports the reason, distinguishing insufficient attempts from an absence of variance, rather than omitting the figure silently.
-------------------	---

C.5 Application Loop and Mock Interview Module

Table C.20: Use Case Specification: Configure Mock Interview

Use Case ID	UC-INTERVIEW-01
Use Case Name	Configure Mock Interview
Actors	Teacher, Educational Manager
Related Requirements	FR-5.1, FR-5.2, FR-5.3
Description	Allows Teachers to define desired interview outcomes, generate and review interview questions, and configure interview session rules.
Pre-conditions	The actor has authoring permission for the course or module.
Post-conditions	The interview configuration, desired outcomes, reviewed questions, and session rules are saved.
Continued on next page	

Table C.20 – Continued from previous page

Main Flow	1. The Teacher opens the Mock Interview configuration page. 2. The Teacher defines interview sections and desired outcomes that students must demonstrate to pass. 3. The system does not require the Teacher to provide expected or model answers. 4. The Teacher drafts interview questions from ingested course content. 5. The system associates each question with one or more desired outcomes. 6. The Teacher reviews, edits, approves, or rejects drafted questions. 7. The Teacher configures session rules, including maximum attempts, cooldown periods, and whether related quiz progression remains locked until the interview is passed. 8. The system saves the interview configuration for publication.
Alternative Flows	2a. No desired outcomes defined: 1. The system blocks interview publication. 2. The Teacher is prompted to define at least one desired outcome. 5a. Question cannot be mapped to an outcome: 1. The system flags the question for review. 7a. Invalid session rule: 1. The system rejects the rule and displays validation feedback.

Table C.21: Use Case Specification: Take Mock Interview

Use Case ID	UC-INTERVIEW-02
Use Case Name	Take Mock Interview
Actors	Student
Continued on next page	

Table C.21 – Continued from previous page

Related Requirements	FR-5.4, FR-5.5, FR-5.6, FR-5.8
Description	Allows Students to participate in text-mode or voice-mode mock interview sessions with an adaptive AI interviewer, while the system records transcript, metadata, and session activity signals.
Pre-conditions	The Student is authenticated, enrolled, and eligible to access the interview.
Post-conditions	The interview transcript, session metadata, and activity signals are persisted for evaluation and review.
Main Flow	1. The Student selects an available mock interview. 2. The Student chooses text mode or voice mode. 3. The system starts an adaptive turn-taking dialogue with the AI interviewer. 4. The AI interviewer presents a question. 5. The Student responds by typing or speaking. 6. In voice mode, the system streams audio, transcribes speech, and generates text-to-speech responses. 7. The system processes raw audio transiently without persistent archival. 8. The interview continues until the session is completed. 9. The system persists the interview transcript and session metadata. 10. The system records session activity signals such as browser focus changes, tab switching, full-screen status, and session events.
Continued on next page	

Table C.21 – Continued from previous page

Alternative Flows	1a. Student is not eligible: 1. The system denies access to the interview. 2. The Student is shown the unmet requirement. 2a. Microphone permission denied: 1. The system shows guidance or allows fallback to text mode. 10a. Integrity signal detected: 1. The system records the event for post-session review.
-------------------	---

Table C.22: Use Case Specification: Review Interview Result

Use Case ID	UC-INTERVIEW-03
Use Case Name	Review Interview Result
Actors	Student, Teacher
Related Requirements	FR-5.6, FR-5.7
Description	Evaluates the Student's interview responses against teacher-defined desired outcomes and controls result visibility for Students and Teachers.
Pre-conditions	The interview session has been completed and the transcript has been persisted.
Post-conditions	A binary pass-or-fail verdict is produced, shown to the Student, and the transcript with outcome-level assessment is available to Teachers for remediation review.
Continued on next page	

Table C.22 – Continued from previous page

Main Flow	1. The Student completes the interview session. 2. The system evaluates the transcript against teacher-defined desired outcomes. 3. The system produces a single binary pass-or-fail verdict. 4. The Student opens the interview result page. 5. The system displays only the binary verdict to the Student. 6. The Teacher opens the completed interview session. 7. The system displays the transcript and outcome-level assessment to the Teacher for remediation purposes.
Alternative Flows	1a. Interview incomplete: 1. The system does not produce a final verdict. 5a. Student attempts to access outcome-level details: 1. The system hides per-outcome rubric details. 6a. Teacher does not manage the course: 1. The system denies access to the transcript and assessment.

Table C.23: Use Case Specification: Curate the Interview Question Bank

Use Case ID	UC-INTERVIEW-04
Use Case Name	Curate the Interview Question Bank
Actors	Teacher, Educational Manager
Related Requirements	FR-5.9, FR-5.2
Description	Allows a Teacher to build a reusable interview question bank in which one logical question carries sibling variants covering complementary interview angles, and to approve those variants before they are used in a published interview.
Continued on next page	

Table C.23 – Continued from previous page

Pre-conditions	• The Teacher is authenticated and holds authoring rights on the course. • Course material has been ingested where generation is to be used.
Post-conditions	• Bank items exist grouped by logical question with their sibling variants. • Only Teacher-approved variants are available to a published interview. • Near-duplicate questions are reported before they enter the bank.
Main Flow	1. The Teacher opens the course’s interview question bank. 2. The Teacher adds a question manually, or requests generation from the ingested material. 3. The system groups generated questions as one logical question with sibling variants covering complementary angles. 4. The system checks a proposed question against the existing bank and reports near-duplicates. 5. The Teacher reviews each variant and approves, edits, or rejects it. 6. The Teacher may request regeneration of a specific question or remove a variant or a whole logical group. 7. The Teacher imports approved bank items into an interview configuration. 8. The system admits only approved items.
Alternative Flows	2a. Insufficient ingested material: 1. The system declines to generate rather than producing ungrounded questions. 4a. Proposed question duplicates an existing one: 1. The system reports the duplicate and the Teacher amends or abandons it. 7a. Item is not approved: 1. The system excludes it from the import.

Table C.24: Use Case Specification: Review the Interview Gap Report

Use Case ID	UC-INTERVIEW-05
Use Case Name	Review the Interview Gap Report
Actors	Student as the recipient of the remediation plan; Teacher for the full assessment and notes
Related Requirements	FR-5.10, FR-5.7
Description	Allows a Student to receive a qualitative gap report with an actionable study plan after an interview, while the Teacher additionally reviews criterion-level assessment and attaches remediation notes.
Pre-conditions	• The interview session has been completed and evaluated. • The Student owns the session; the Teacher holds authoring rights on the course.
Post-conditions	• The Student sees the discrepancy summary and a study plan of topics with lesson links and suggested resources. • Criterion-level scores and internal assessment rationale remain visible only to the Teacher. • Teacher notes are recorded against the report.
Continued on next page	

Table C.24 – Continued from previous page

Main Flow	1. The Student opens the result of a completed interview session. 2. The system presents the pass or fail verdict together with the discrepancy summary. 3. The system presents the study plan as a list of topics, each linked to the relevant lesson with suggested resources where available. 4. The Teacher opens the same session and additionally reviews the criterion-level assessment and the transcript. 5. The Teacher attaches remediation notes to the report. 6. The system records the notes and makes them available on subsequent reviews.
Alternative Flows	3a. The course does not support a topic that would otherwise be suggested: 1. The system omits the suggestion rather than inventing coverage the course does not have. 2a. No gap report has been generated: 1. The system presents the verdict alone. 4a. Student attempts to view criterion-level detail: 1. The system withholds it, exposing only the verdict and the qualitative remediation.

C.6 Analytics, Reporting and Notifications Module

Table C.25: Use Case Specification: Manage Notifications

Use Case ID	UC-ANALYTICS-01
Use Case Name	Manage Notifications
Actors	All Users
Continued on next page	

Table C.25 – Continued from previous page

Related Requirements	FR-6.3
Description	Allows users to view in-app notifications, manage unread state, dismiss notifications, and configure notification preferences by category and channel.
Pre-conditions	The user is authenticated.
Post-conditions	Notification state or preferences are updated.
Main Flow	1. The user opens the notification inbox. 2. The system displays notifications and unread count. 3. The user marks notifications as read or dismisses them. 4. The user opens notification preferences. 5. The user configures notification preferences by category and channel. 6. The system saves the preferences and applies them to future notifications.
Alternative Flows	5a. Invalid preference combination: 1. The system rejects the configuration. 2. The user is prompted to correct the preference.

Table C.26: Use Case Specification: View Progress and Career Readiness Reports

Use Case ID	UC-ANALYTICS-02
Use Case Name	View Progress and Career Readiness Reports
Actors	Student, Teacher, Educational Manager
Related Requirements	FR-6.1, FR-6.2, FR-6.8
Description	Provides student completion progress, teacher roster progress, at-risk learner identification, and organisation-scoped Career Readiness snapshots.
Continued on next page	

Table C.26 – Continued from previous page

Pre-conditions	The actor is authenticated and has permission to view the selected report scope.
Post-conditions	The actor views progress, risk, or Career Readiness information according to their role and organisation scope.
Main Flow	1. A Student opens the progress page. 2. The system displays per-lesson and per-course completion progress separately from spaced-repetition metrics. 3. A Teacher opens the roster progress view for a course. 4. The system displays course-level roster progress and at-risk learners. 5. An Educational Manager opens the Career Readiness report for an organisation pathway. 6. The system displays aggregated pathway-level learner outcomes. 7. The manager drills down into individual progress where permitted. 8. The report excludes per-interview rubric details.
Alternative Flows	2a. Student has no activity: 1. The system displays zero progress instead of an error. 4a. No roster activity exists: 1. The system displays an empty state. 7a. Manager attempts to access another organisation: 1. The system denies access.

Table C.27: Use Case Specification: Monitor Platform Operations

Use Case ID	UC-ANALYTICS-03
Use Case Name	Monitor Platform Operations
Actors	IT Administrator, Educational Manager
Continued on next page	

Table C.27 – Continued from previous page

Related Requirements	FR-6.4, FR-6.5, FR-6.6, FR-6.7
Description	Allows authorised administrators to view platform and organisation-level statistics, inspect AI cost telemetry, inspect processing jobs, retry failed jobs, and query immutable audit logs.
Pre-conditions	The actor is authenticated and has administrative reporting or audit permissions.
Post-conditions	The actor obtains operational visibility into platform health, AI usage, processing jobs, or audit records.
Main Flow	1. The administrator opens the Operations dashboard. 2. The system displays platform or organisation-level statistics according to the actor's scope. 3. The administrator opens AI cost telemetry. 4. The system displays usage summaries by user, pipeline, and recent model calls. 5. The administrator opens ingestion or AI processing jobs. 6. The system displays job status and failed job details. 7. The administrator retries a failed processing job when appropriate. 8. The administrator opens the audit log. 9. The system displays immutable audit records for role assignments, sensitive data changes, and HTTP request events.
Alternative Flows	2a. Actor has organisation-only scope: 1. The system filters statistics to the actor's organisation. 7a. Failed job cannot be retried: 1. The system blocks retry and explains the missing material or configuration. 8a. Actor lacks audit permission: 1. The system denies access to audit logs.

C.7 Learning Programs and Path Governance Module

Table C.28: Use Case Specification: Define and Publish a Learning Programme

Use Case ID	UC-PROGRAM-01
Use Case Name	Define and Publish a Learning Programme
Actors	Faculty Dean, Educational Manager
Related Requirements	FR-7.1, FR-7.2, FR-7.3
Description	Allows a Faculty Dean or Educational Manager to define a learning programme within a faculty of their organisation as an ordered selection of published career pathways, revise it under version control, and publish it for enrolment.
Pre-conditions	• The operator is authenticated and holds learning-programme management rights. • The target faculty belongs to the operator's own organisation. • At least one published career pathway exists in the organisation.
Post-conditions	• The programme exists with a version pinning each selected pathway to a specific published pathway version. • A published programme is open for student enrolment. • Students already enrolled remain attached to the programme version they enrolled upon.
Continued on next page	

Table C.28 – Continued from previous page

Main Flow	1. The operator opens the Learning Programmes page for their organisation. 2. The operator selects “Create programme” and chooses the owning faculty. 3. The system offers the organisation’s career pathways, marking those that cannot be selected together with the reason. 4. The operator enters the programme name, description, and the permitted number of pathway switches, and arranges the chosen pathways in order. 5. The operator saves the programme, which the system creates as a draft together with version one. 6. The operator reviews the draft and selects “Publish”. 7. The system verifies that every pinned pathway and pathway version is still available. 8. The system promotes the draft to published, records the publisher and the publication time, and opens the programme for enrolment. 9. To revise a published programme, the operator edits it and the system opens a new draft version carrying forward the existing pathway pins.
Continued on next page	

Table C.28 – Continued from previous page

Alternative Flows	3a. No published pathway is available: 1. The system marks every pathway as not selectable. 2. The operator is directed to publish a career pathway first. 4a. Faculty outside the operator’s organisation: 1. The system rejects the programme. 2. The operator is informed that the faculty is out of scope. 4b. Duplicate programme identifier: 1. The system reports that the identifier is already taken within the organisation. 2. The operator supplies a different one. 7a. A pinned pathway has become unavailable: 1. The system refuses to publish and names the offending pathway. 2. The operator corrects the pathway selection and retries. 9a. Programme is archived: 1. The system refuses the edit because an archived programme is immutable.
-------------------	---

Table C.29: Use Case Specification: Enrol Students onto a Learning Programme

Use Case ID	UC-PROGRAM-02
Use Case Name	Enrol Students onto a Learning Programme
Actors	Educational Manager, Faculty Dean
Related Requirements	FR-7.4, FR-7.5, FR-7.11
Description	Allows an authorised operator to place Students onto a published learning programme individually or by file import, and to withdraw a Student from the programme.
Continued on next page	

Table C.29 – Continued from previous page

Pre-conditions	• The operator is authenticated and holds learning-programme enrolment rights for the programme’s organisation. • The programme is published and has a published version.
Post-conditions	• Each accepted Student holds an enrolment awaiting initial pathway selection, pinned to the current published version. • Rejected rows are reported individually and do not prevent the remaining rows from being enrolled. • A withdrawn Student’s enrolment records the withdrawal reason and any open pathway-change request is cancelled.
Main Flow	1. The operator opens the programme’s Students tab. 2. The operator either selects Students individually or uploads a comma-separated file of e-mail addresses with optional names. 3. The system validates each entry against the Student role, the programme’s organisation, and the concurrent-enrolment ceiling. 4. For an unknown e-mail address in a file import, the system creates a Student account in the programme’s organisation. 5. The system creates one enrolment per accepted Student awaiting initial pathway selection. 6. The system reports the outcome as the number enrolled, the accounts created, those already enrolled, and the per-row failures. 7. To remove a Student, the operator selects “Withdraw” and supplies a reason. 8. The system records the withdrawal, snapshots and cancels the active pathway attempt, and releases the access it granted.
Continued on next page	

Table C.29 – Continued from previous page

Alternative Flows	3a. A selected user is not a Student: 1. The system rejects the whole hand-picked batch and names the offending users. 3b. Student already enrolled: 1. The system reports the existing enrolment and skips the row. 3c. Concurrent-enrolment ceiling reached: 1. The system refuses the enrolment and reports the limit together with the Student's current count. 3d. Previously withdrawn Student: 1. The system reinstates the existing enrolment onto the current published version instead of creating a duplicate. 2a. Malformed file row: 1. The system records the row number and the reason, and continues with the remaining rows. 7a. Enrolment is already completed: 1. The system refuses the withdrawal.
--------------------------	---

Table C.30: Use Case Specification: Select and Change Career Pathway

Use Case ID	UC-PROGRAM-03
Use Case Name	Select and Change Career Pathway
Actors	Student for selection and request; Faculty Dean for the decision
Related Requirements	FR-7.6, FR-7.7, FR-7.8, FR-7.9, FR-7.10, FR-7.12, FR-7.13
Description	Allows a Student to choose their initial career pathway within an enrolled learning programme and subsequently to request a change of pathway, which an owning Faculty Dean acknowledges and then approves or rejects.
Continued on next page	

Table C.30 – Continued from previous page

Pre-conditions	• The Student holds an enrolment on a published learning programme. • For a change request, the enrolment is active and the Student has not exhausted the permitted number of switches. • For a decision, the reviewer is a Faculty Dean owning the programme and is not the subject of the request.
Post-conditions	• Upon initial selection the enrolment becomes active with an open pathway attempt pinned to the chosen pathway version. • Upon approval the outgoing attempt is closed with a progress snapshot, a new attempt is opened on the target pathway version, and the switch is counted against the allowance. • Course completions already earned remain valid and continue to count towards the new pathway. • Upon rejection the enrolment and the current pathway are unchanged and the recorded reason is visible to the Student.
Continued on next page	

Table C.30 – Continued from previous page

Main Flow	1. The Student opens their programme enrolment and reviews the pathways offered by their programme version. 2. The Student selects one pathway as their initial choice. 3. The system activates the enrolment, opens a pathway attempt pinned to that pathway version, and grants access to the pathway's courses. 4. To change pathway later, the Student selects a different pathway and submits a written justification. 5. The system verifies that the target pathway belongs to the enrolled programme version, differs from the current pathway, and that the switch allowance has not been exhausted. 6. The system records the request as pending and notifies the owning Faculty Deans. 7. A Faculty Dean opens the programme's review tab and acknowledges the request, which the system marks as in progress and notifies the Student. 8. The Faculty Dean approves or rejects the request. 9. On approval the system re-verifies the enrolment, snapshots the outgoing attempt's completed courses and overall progress, closes it, and opens a new attempt on the target pathway version. 10. The system carries over access to courses shared by both pathways, releases access that is no longer justified, and counts the switch against the allowance. 11. The system notifies the Student of the outcome. 12. When every course of the pinned pathway version is satisfied, the system completes the enrolment.
Continued on next page	

Table C.30 – Continued from previous page

Alternative Flows	2a. Initial pathway already chosen: 1. The system refuses, because the initial pathway may be selected only once. 5a. Target pathway equals the current pathway, or lies outside the enrolled version, or is archived: 1. The system rejects the request and explains the reason. 5b. A request is already open: 1. The system refuses a second request until the open one is decided or cancelled. 5c. Switch allowance exhausted: 1. The system refuses the request and reports that the limit has been reached. 8a. Rejection: 1. The Faculty Dean selects a reason from the defined vocabulary. 2. Where the reason is recorded as other, the system requires explanatory text. 3. The system records the decision and notifies the Student. 8b. Reviewer is the subject of the request: 1. The system refuses the decision as self-approval. 9a. Target pathway became unavailable before approval: 1. The system marks the request invalidated and leaves the enrolment unchanged. 4a. Student cancels: 1. While the request is still open the Student withdraws it. 2. The system retains the cancelled request in the visible history.
-------------------	---

C.8 Platform Configuration and Policy Documents Module

Table C.31: Use Case Specification: Tune Runtime Configuration with Preview and Rollback

Use Case ID	UC-CONFIG-01
Use Case Name	Tune Runtime Configuration with Preview and Rollback
Actors	IT Administrator for deployment-wide settings; Educational Manager for organisation-scoped settings
Related Requirements	FR-8.1, FR-8.2, FR-8.3, FR-8.4, FR-8.5, FR-8.6
Description	Allows an authorised administrator to retune system behaviour at run time without a redeployment, previewing the effect of a change before applying it, recording the justification, and rolling a change back when necessary.
Pre-conditions	• The administrator is authenticated and holds configuration rights for the scope being changed. • The setting being changed exists in the published settings catalogue.
Post-conditions	• The effective value of the setting reflects the applied override according to the documented precedence. • The change, clearance, or rollback is recorded with its reason and the acting administrator. • The new value takes effect on the next operation that reads the setting.
Continued on next page	

Table C.31 – Continued from previous page

Main Flow	1. The administrator opens the Settings console at either deployment-wide or organisation scope. 2. The system lists the catalogue, showing for each setting its type, permitted range, the value held at each precedence layer, and the effective value together with its source. 3. The administrator selects a setting and enters a proposed value. 4. The administrator selects “Preview”. 5. The system validates the proposed value against the setting’s type and range and reports the current value, the proposed value, and the scope affected. 6. The administrator supplies a written reason and applies the change. 7. The system stores the override, records the change in the history, and invalidates its resolution cache. 8. To restore inheritance, the administrator clears the override with a reason and the next precedence layer resumes. 9. To undo a change, the administrator opens the change history and selects “Roll back”. 10. The system restores the value the change had replaced, or removes the override when there was no previous value, and records the rollback as a new history entry.
Continued on next page	

Table C.31 – Continued from previous page

Alternative Flows	5a. Proposed value fails validation: 1. The system rejects the preview and reports the permitted type and range. 6a. Reason omitted or too short: 1. The system refuses the change until an adequate reason is supplied. 7a. Setting requires a restart to take effect: 1. The system records the change and indicates that a restart is required. 2a. Stored value no longer satisfies its constraints: 1. The system disregards the stored value, falls through to the next precedence layer, and continues to operate. 2b. Configuration store unreachable: 1. The system operates on environment configuration and code defaults without failing the calling operation.
--------------------------	---

Table C.32: Use Case Specification: Publish and Read Policy Documents

Use Case ID	UC-CONFIG-02
Use Case Name	Publish and Read Policy Documents
Actors	IT Administrator for authoring; any reader, including an unauthenticated visitor, for reading
Related Requirements	FR-8.7, FR-8.8
Description	Allows an administrator to maintain versioned legal and academic policy documents and to publish them, while any reader may consult the current published text without authenticating.
Continued on next page	

Table C.32 – Continued from previous page

Pre-conditions	• For authoring, the administrator is authenticated and holds policy administration rights. • For reading, no precondition applies.
Post-conditions	• The published version becomes the current text for its language and any previously published version in that language is archived. • The document remains reachable by its stable identifier across re-issues. • No record of reader acceptance is created, since reading a policy is a read-only act.
Main Flow	1. The administrator opens the Policies console and creates a policy, giving it an identifier, a category of legal or academic, a title, and a language. 2. The system creates the policy together with an empty first draft version. 3. The administrator edits the draft's title, body, and change note. 4. The administrator selects "Publish". 5. The system records the publication time and publisher and archives any previously published version in that language. 6. The administrator optionally restricts the policy's audience to selected roles, or leaves it public. 7. To re-issue the document, the administrator opens the next draft version, which the system seeds from the latest existing version. 8. A reader opens the policy index and selects a document. 9. The system returns the current published text for the reader's language together with its version number and publication date.
Continued on next page	

Table C.32 – Continued from previous page

Alternative Flows	1a. Identifier already in use: 1. The system reports the conflict and the administrator supplies a different identifier. 9a. Requested language has no published version: 1. The system returns the English version instead. 9b. Policy does not exist or has never been published: 1. The system reports that the document is not available.
-------------------	---

Appendix D Activity Diagrams Specification

D.1 Authentication Flow

This activity diagram details the user authentication workflow in aBridgeAI. It shows how users log in through Google SSO, how the system verifies the local account and MFA requirement, creates an authenticated session, resolves roles and permissions, and handles logout.

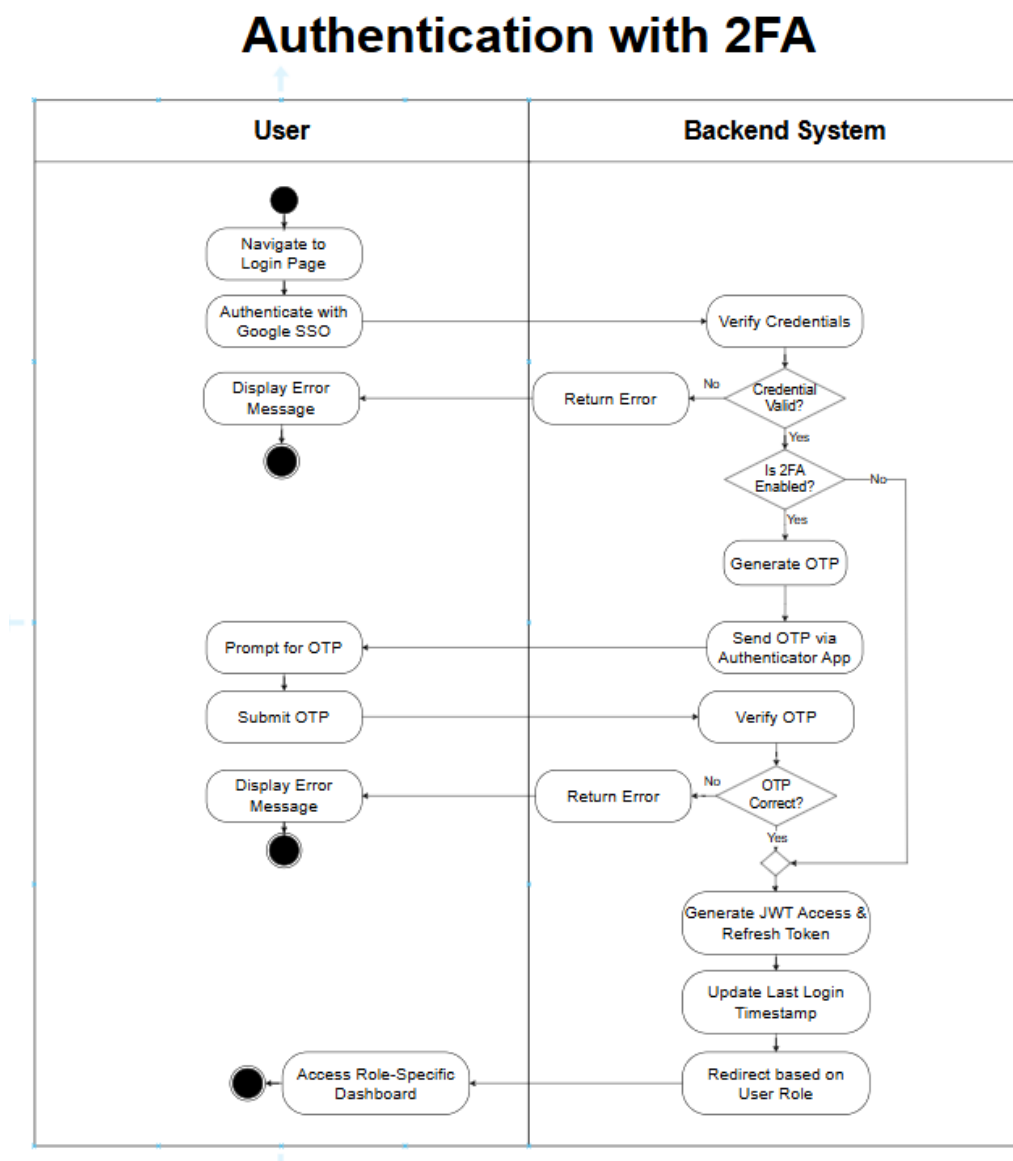


Figure D.1: Activity Diagram: User Login with 2FA

D.2 Smart Content Ingestion Flow

This activity diagram details the material ingestion workflow in aBridgeAI. It shows how teachers upload learning materials directly to object storage, how the backend verifies the upload and queues processing, and how the ingestion worker extracts, chunks, embeds, and stores content for retrieval.

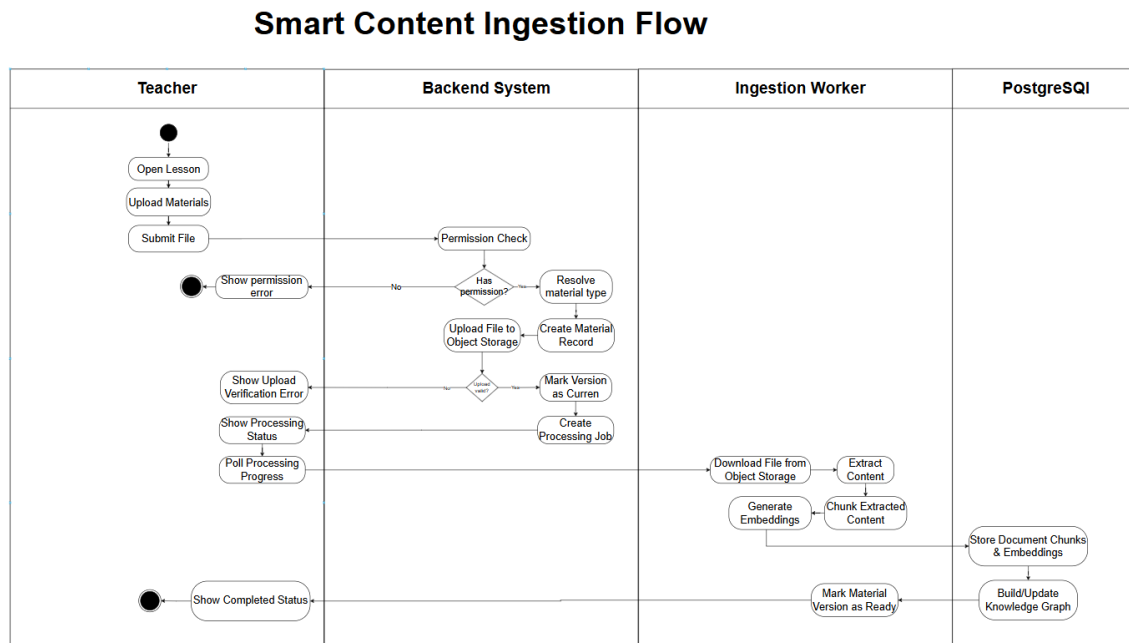


Figure D.2: Activity Diagram: Smart Content Ingestion and AI Parsing

D.3 Human-in-the-Loop Quiz Generation Flow

This diagram maps the assessment creation lifecycle. It demonstrates the "Human-in-the-Loop" architecture where the teacher configures parameters, the AI generates a draft assessment based on the Knowledge Graph, and the teacher performs the final review and pedagogical adjustments before publishing the quiz to the students.

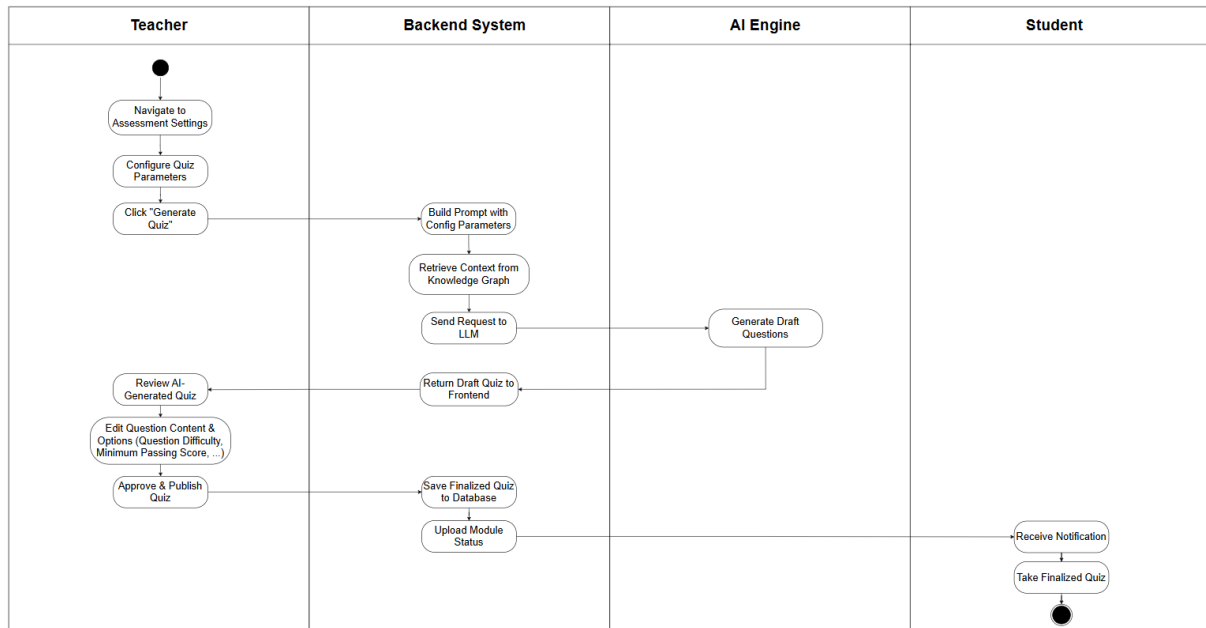


Figure D.3: Activity Diagram: HITL Quiz Generation and Execution

D.4 Context-Aware Mock Interview Flow

This diagram maps the core application loop where a student undergoes an AI-driven mock interview. It details the real-time interaction cycle: the AI generating context-specific questions, the student providing text or voice answers, and the system evaluating the response to determine follow-up questions. It also includes alternative paths for handling AI latency and connection timeouts.

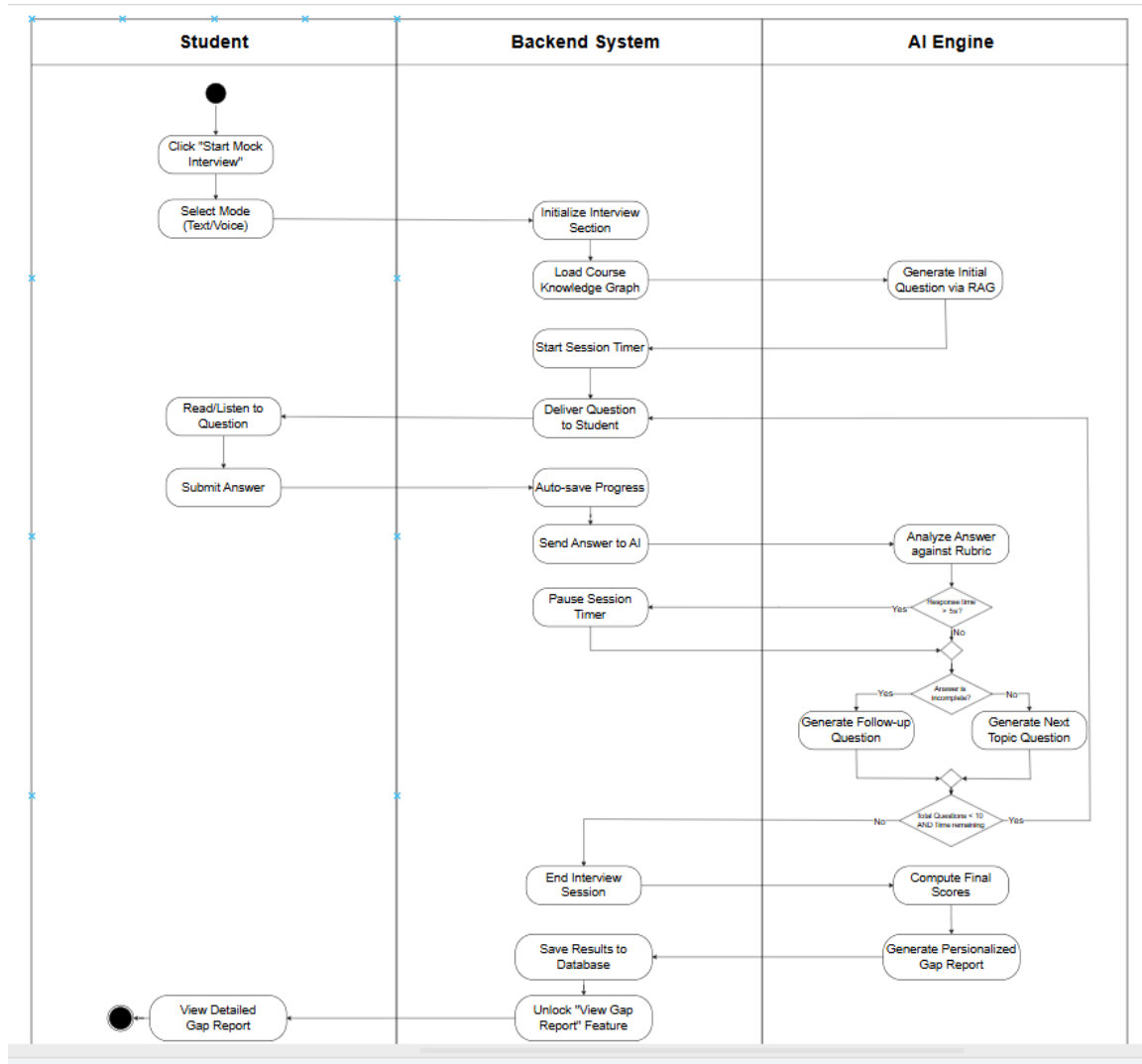


Figure D.4: Activity Diagram: Context-Aware Mock Interview

Appendix E Sequence Diagrams Specification

E.0.1 Authentication

This sequence diagram maps the execution flow of the user authentication process.

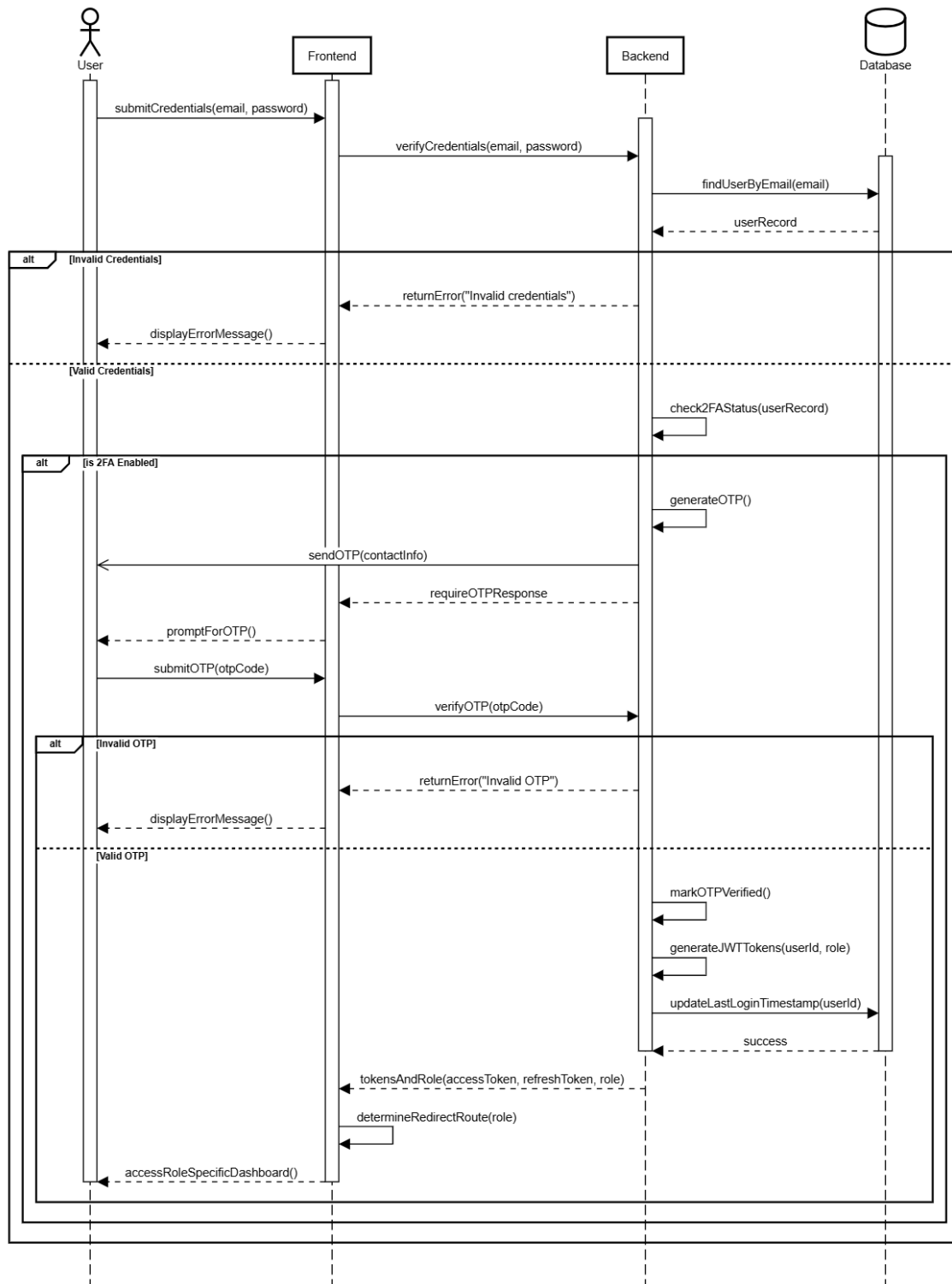


Figure E.1: Sequence Diagram: Authentication

E.0.2 Smart Content Ingestion

This sequence details the interactions when a teacher uploads course materials. It highlights the asynchronous chunking, embedding generation, and vector storage processes required to build the RAG pipeline.

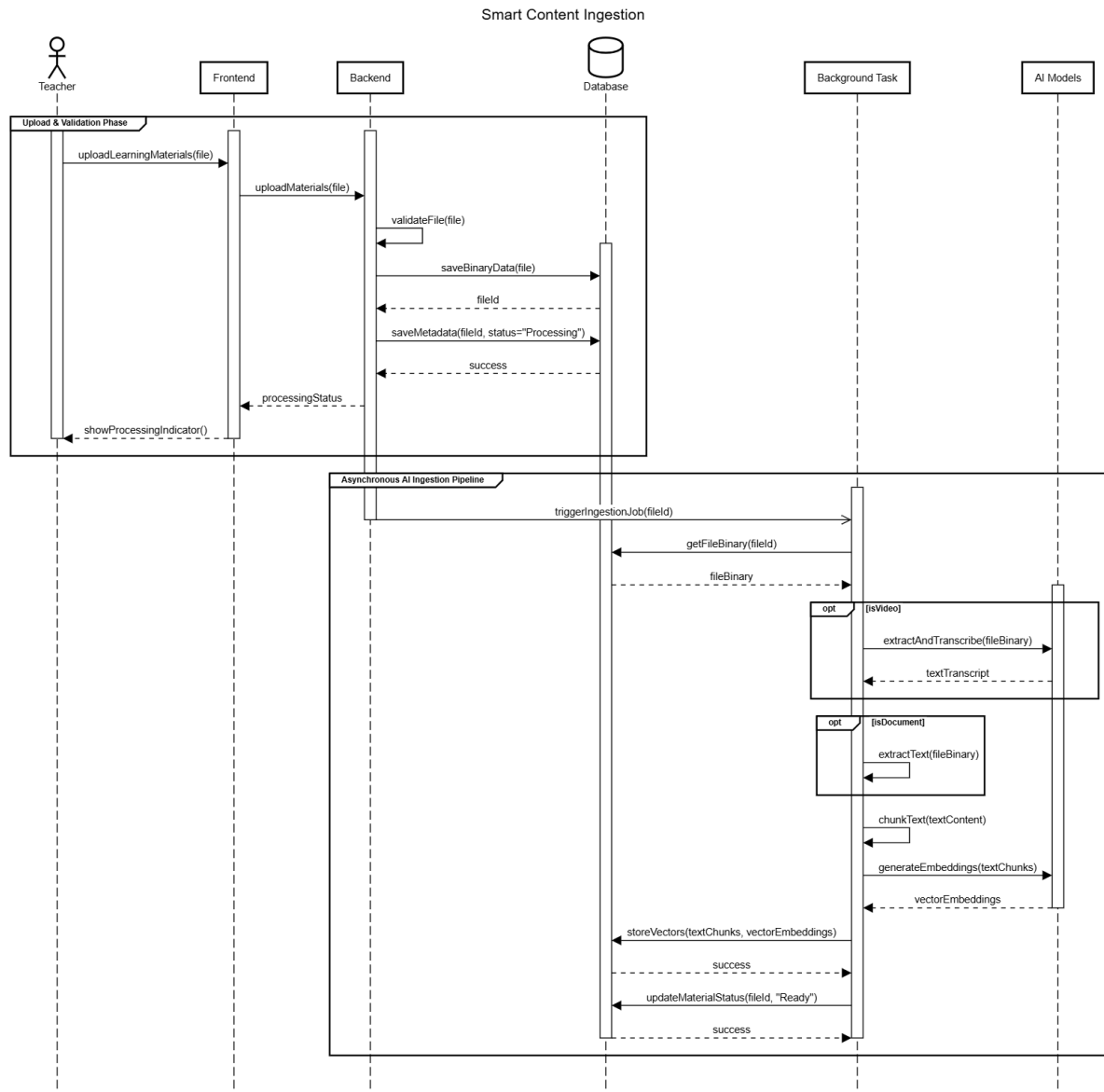


Figure E.2: Sequence Diagram: Smart Content Ingestion

E.0.3 Human-in-the-Loop Quiz Generation

This activity diagram maps the assessment creation lifecycle, emphasizing the "Human-in-the-Loop" architectural pattern. While the AI Engine performs the compu-

tationally heavy task of generating questions via RAG based on the Course Knowledge Graph, the Teacher retains full control. The workflow includes a review and editing phase, allowing the instructor to adjust question content, difficulty levels, and passing thresholds before publishing the final quiz to the students.

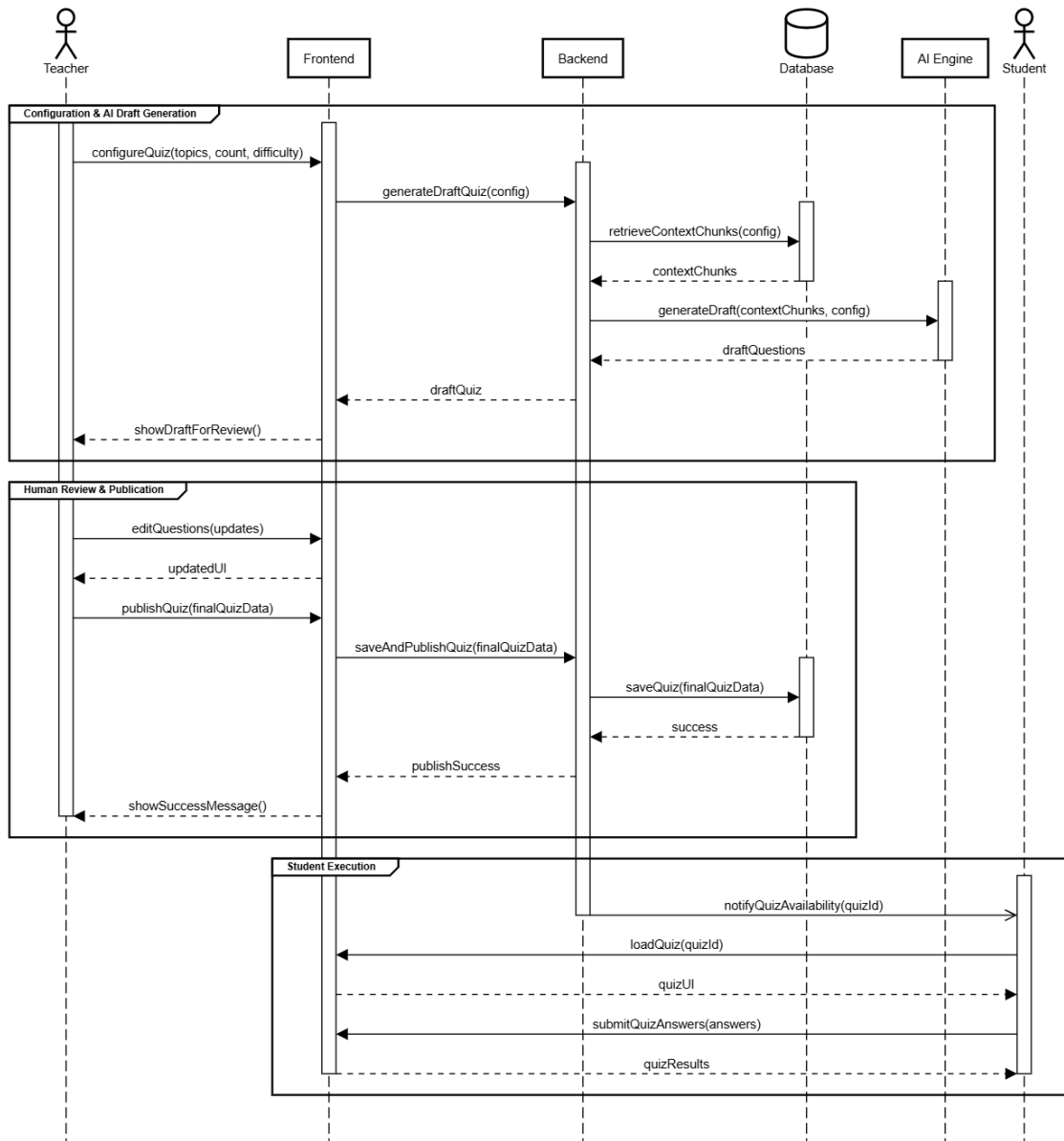


Figure E.3: Sequence Diagram: HITL Quiz Generation and Execution

E.0.4 Context-Aware Mock Interview Session

This sequence maps the real-time interaction during a mock interview. It demonstrates how the backend retrieves semantic context from the Vector Database (Qdrant)

and communicates with the LLM to generate dynamic, curriculum-aligned questions and simultaneously evaluate student responses.

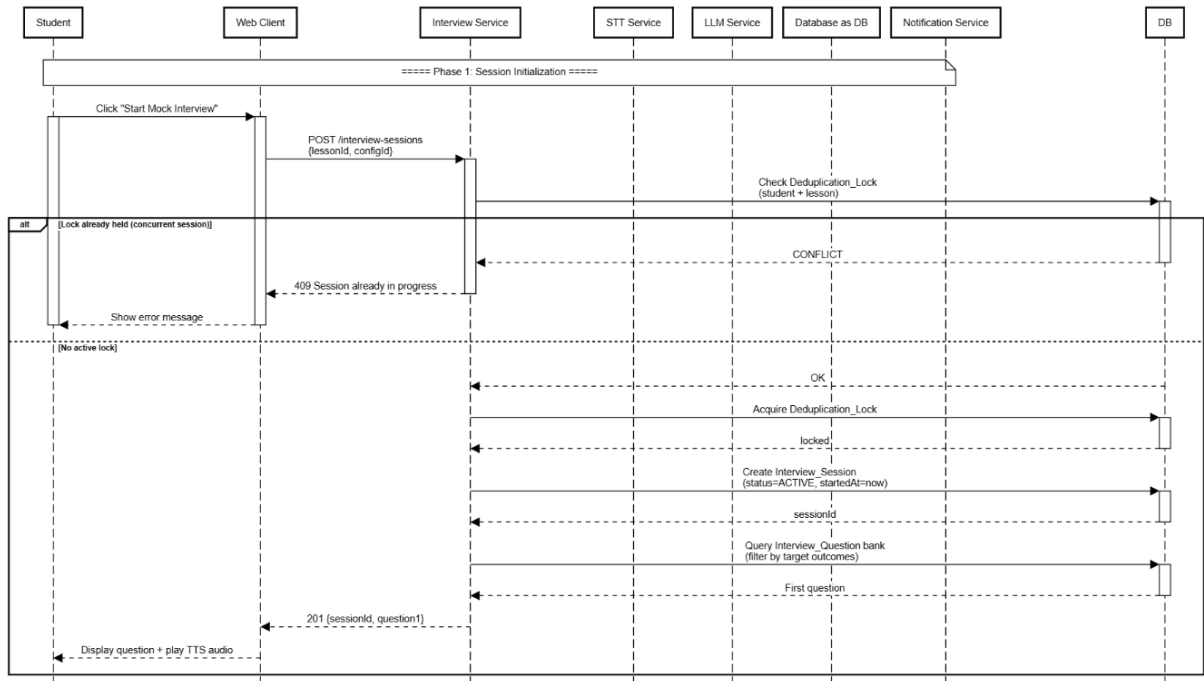


Figure E.4: Sequence Diagram: Mock Interview (Phase 1)

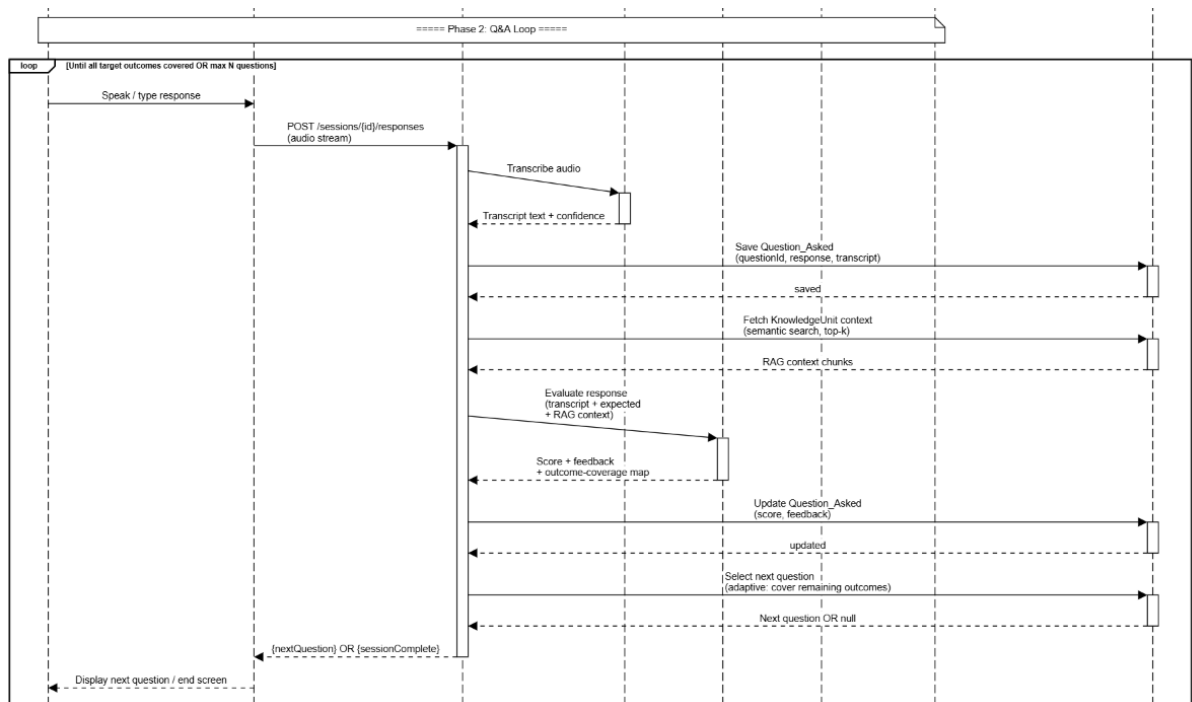


Figure E.5: Sequence Diagram: Mock Interview (Phase 2)

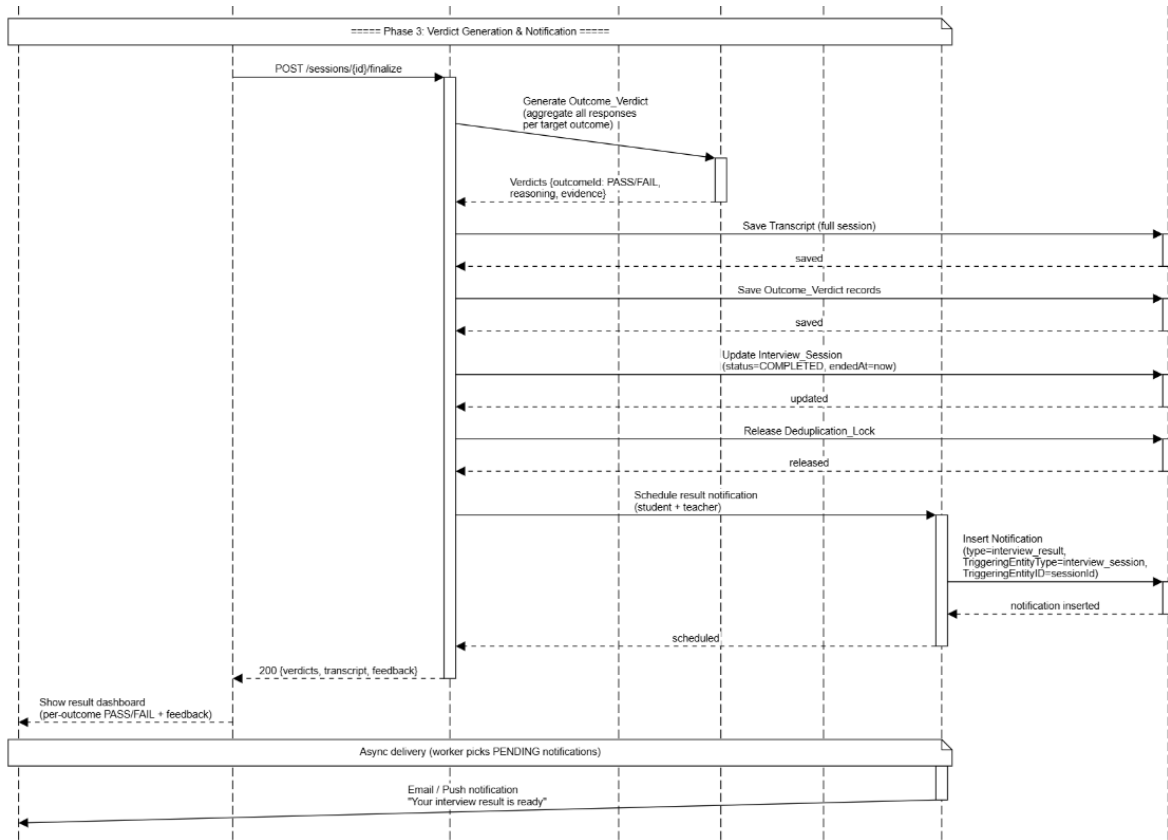


Figure E.6: Sequence Diagram: Mock Interview (Phase 3)

Appendix F API Designs Specification

F.1 Architecture & Design Philosophy

The aBridgeAI backend is a Python FastAPI service exposing a versioned RESTful API at the base path `/api/v1`. It serves as the single data contract between the Vite + TanStack Router single-page application and the server. The API surface is composed by a single `create_app()` factory that mounts every per-feature router under the shared `API_V1_PREFIX`; routers themselves carry their own per-feature prefix (e.g. `/teacher`, `/me/notifications`, `/admin/processing`). The platform follows six guiding principles:

1. **Vertical-slice feature layout** — each feature owns a self-contained directory under `abridgeai/features/<feature>/` with its own `routers/`, `services/`, `queries/`, and `schemas/` sub-packages. Routers translate HTTP to application calls; they never touch persistence directly. An import-linter contract (*“Routers do not call queries directly”*) enforces the boundary at CI time.
2. **Audience-scoped routers** — a feature exposes one router per audience. The most common split is `learner` (the calling student, scoped under `/me/...`) versus `authoring` (instructor write operations, under `/teacher/...`), with optional `administration` (under `/admin/...`), `assignment` (department-level, under `/dept/...`) and `management` (educational-manager, under `/management/...`) routers. The URL prefix communicates the access context at a glance.
3. **Bearer-token authentication via JWT** — the API uses OAuth-only authentication (Google SSO). On a successful Google callback, the server returns a JSON body containing an `access_token` and a `refresh_token`; the SPA stores them and presents the access token as an `Authorization:Bearer<token>` header on every authenticated request. There is no traditional username/password endpoint and no CSRF middleware — the bearer scheme on a header is not vulnerable to cross-site form submission.
4. **MFA gate as a dependency** — when a user has TOTP enabled, the access token is initially issued with a *pre-MFA* marker. A FastAPI dependency on every protected endpoint refuses traffic until the holder of the token completes `POST/auth/me/mfa/challenge` followed by `POST/auth/me/mfa/verify`; the dependency rejects with HTTP 403 and a stable `mfa_required` error code so the SPA can route to the MFA challenge screen.

5. **JSON-first, snake_case field naming** — all request and response bodies use JSON with `snake_case` field names (Pydantic v2 schemas). IDs are UUID v4, timestamps are ISO 8601 UTC strings, and enumerations are lowercase or lowercase-with-hyphens. Numeric ranges (`min_length`, `max_length`, `ge/le`) are validated by Pydantic at the FastAPI boundary, returning a 422 with a structured `detail` array.
6. **Async-first request handling** — every router function is `asyncdef`, every database session is an `AsyncSession` from SQLAlchemy 2.0 `async`, and long-running AI work (quiz generation, interview question regeneration, material reprocessing) is dispatched to an ARQ worker pool over Redis rather than executed in-request. The HTTP layer therefore returns within hundreds of milliseconds even for AI-driven mutations; the SPA polls a `generation-runs/{run_id}` endpoint to surface progress.

The backend deploys behind an HTTPS-terminating reverse proxy and emits security headers (HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy) via a dedicated `SecurityHeadersMiddleware`. Every HTTP request is recorded by an `AuditLogMiddleware` into the `http_audit_log` table with method, path, status, actor, and request ID, providing a per-request trace for security and performance debugging.

F.2 Authentication Strategy

The platform offers a single sign-in path: Google SSO. Username/password, password-reset, and email-verification endpoints intentionally do not exist — this eliminates an entire class of credential-related bugs and shifts identity-proofing to Google’s hardened identity provider.

Token Pair.

The application issues an access/refresh token pair as the canonical authentication primitive. The **access token** is a stateless HS256-signed JWT carrying `sub` (user ID), `sid` (session ID), `iat`, and `exp`. It is the only credential validated on every API request; the JWT decode is performed in memory and the embedded `sid` is then resolved against `auth_sessions` (with a Redis hot-path cache) to detect revocation and MFA gate state. The **refresh token** is a high-entropy opaque random string (URL-safe, 32 bytes) returned to the client once on login; the server persists only its SHA-256 hash in `auth_sessions.refresh_token_hash`, never the raw value. The access token has a 15-minute lifetime; the refresh token has a 7-day lifetime tied to the session row’s `expires_at`. Refresh rotates both tokens (a new random refresh value, a new JWT) on every call.

Table F.1: Token Pair — backend storage

Token	Purpose	Lifetime	Backend storage
<code>access_token</code>	Authenticates every API request via <code>Authorization: Bearer</code>	15 minutes	Stateless JWT, never persisted on the server; <code>sid</code> claim resolved against <code>auth_sessions</code> on each request.
<code>refresh_token</code>	Rotates the access/refresh pair via <code>POST/auth/refresh</code>	7 days	Raw value returned to client once; only SHA-256 hash stored in <code>auth_sessions.refresh_token_hash</code> (UNIQUE NOT NULL). Rotated on every refresh.

Hot-path session cache.

On every authenticated request the backend must verify that the JWT's `sid` still references a live, non-revoked, MFA-cleared session. To keep this check off PostgreSQL the session state is cached in Redis under `session:{session_id}` with a 60-second TTL — short enough that an admin-triggered revoke (which also invalidates the cache) takes effect within seconds even if pub/sub invalidation fails. A reverse-index Redis SET at `user_sessions:{user_id}` stores the active session IDs for a user, enabling $O(N)$ cascade revocation when an administrator disables an account without an unsafe `SCANMATCH session:*`. The cache is read-through with safe fallback: any `RedisError` or connection failure logs a warning and the request transparently falls through to the source database, so a Redis outage degrades latency rather than availability.

Login Flow.

1. `GET/auth/google/login` returns `{authorization_url,state}`; the SPA stores `state` and redirects the browser to `authorization_url`.
2. Google redirects back to `/auth/google/callback?code=...&state=...`. The SPA verifies the `state` matches and forwards the `code` to the backend.
3. `GET/auth/google/callback` exchanges the code with Google, finds or rejects the application user (the platform requires the email to match a pre-provisioned account

— self-signup is not supported), and returns `{access_token, refresh_token, expires_in, requires_mfa, user}`.

4. If `requires_mfa` is `true` the SPA navigates to the MFA challenge screen and calls `POST/auth/me/mfa/challenge` then `POST/auth/me/mfa/verify`. On successful verification the backend re-issues a token pair flagged as MFA-completed.

Refresh Flow.

The SPA submits the current refresh token to `POST/auth/refresh`; the server validates the session row, rotates the refresh token, and returns a new pair. Token rotation is mandatory — a refresh token cannot be reused twice.

Logout.

`POST/auth/logout` invalidates the refresh-token row server-side; the SPA discards the local pair. Subsequent requests that present the discarded pair receive HTTP 401.

F.3 Multi-Factor Authentication

TOTP-based MFA is exposed as a self-service feature under `/auth/me/mfa`. Enrollment is two-step: `POST.../totp/enroll` generates a TOTP secret and returns the `otpauth://` URL plus a transient `factor_id`; the user scans the QR in an authenticator app and confirms with `POST.../totp/verify` supplying the first generated code. Successful verification produces a one-time set of recovery codes (`MfaRecoveryCodesResponse`) that the SPA must persist for the user.

Once enrolled, every fresh login produces a pre-MFA token. `POST.../challenge` returns a `challenge_id`; `POST.../verify` accepts either a 6–8 digit TOTP code or a single-use recovery code. Both `.../recovery-codes/regenerate` and `.../disable` require recent MFA verification — they enforce the same code/recovery-code proof-of-possession check inline so a stolen access token cannot silently turn off the second factor.

Table F.2: MFA endpoints

Method	Path	Description
GET	<code>/auth/me/mfa/status</code>	Whether the caller has TOTP enrolled and the session's <code>mfa_verified_at</code> .

Table F.2 (*continued*)

Method	Path	Description
POST	/auth/me/mfa/totp/enroll	Provision a fresh TOTP secret and return the otpauth URL.
POST	/auth/me/mfa/totp/verify	Confirm enrollment with the first valid code; emit recovery codes.
POST	/auth/me/mfa/challenge	Open a challenge during a fresh login.
POST	/auth/me/mfa/verify	Verify a TOTP code or recovery code against the open challenge.
POST	/auth/me/mfa/recovery-codes /regenerate	Wipe and re-issue recovery codes (requires inline proof).
POST	/auth/me/mfa/disable	Disable MFA; requires inline proof-of-possession.

F.4 Error Handling

Errors surface as standard FastAPI `HTTPException` responses with a JSON body of `{detail:...}` and a numeric status code. The application defines a small hierarchy of exceptions (`AppError`, `NotFoundError`, `UnauthorizedError`, `ForbiddenError`, `ConflictError`) raised by the service layer and translated to HTTP at the router boundary. For richer cases (OAuth provisioning, MFA gates, validation failures) the `detail` is an object that carries a stable `error` code so the SPA can branch deterministically:

HTTP/1.1 403 Forbidden

Content-Type: application/json

```
{
  "detail": {
    "error": "oauth_account_not_provisioned",
    "message": "This Google account is not linked to a registered user."
  }
}
```

Table F.3: Common HTTP status codes

Code	Name	Used For
200	OK	Successful read or successful synchronous mutation.
201	Created	Resource created; body returns the new resource.
202	Accepted	AI generation accepted and queued (response carries a <code>run_id</code>).
204	No Content	Successful mutation with no payload (e.g. delete).
400	Bad Request	Domain-level validation failure raised as <code>AppError</code> .
401	Unauthorized	Missing or invalid bearer token; carries <code>WWW-Authenticate:Bearer</code> .
403	Forbidden	RBAC denial, MFA required, or account not provisioned.
404	Not Found	<code>NotFoundError</code> — entity does not exist or is invisible to caller.
409	Conflict	<code>ConflictError</code> — e.g. slug collision, optimistic-lock violation.
422	Unprocessable Entity	Pydantic validation failure on the request body or query string.
503	Service Unavailable	Upstream provider error (Google OAuth, AI provider, Redis).

F.5 Pagination, Filtering and Idempotency

Pagination.

Every list endpoint uses **cursor pagination**. The shared `core/pagination/cursor.py` module exposes a generic `CursorPage[T]` dataclass; all list responses are wrapped

as `{items: T[], next_cursor: string | null}`. The cursor is opaque to the client — typically a base64-encoded UUID of the last row returned, or an ISO-8601 `created_at` timestamp for time-ordered feeds (notifications, audit trails). Clients pass it back as `?cursor=...` (paired with an optional `limit`, default 20, max 100) to fetch the next page; `next_cursor` being `null` signals end-of-list. Cursor pagination keeps queries on a single index column, scales to deep traversal without the $O(N)$ skip cost of `OFFSET`, and is not perturbed by inserts or deletes between page fetches — the user never sees a duplicated or skipped row mid-scroll.

Filtering and sorting.

List endpoints accept feature-specific filters as query parameters (e.g. `status=published, role=teacher, q=<search>`). Sort order is fixed per endpoint and documented inline; freeform sort keys are deliberately not supported to keep the index strategy predictable.

Idempotency.

Mutating endpoints that may be retried by the SPA (in particular `POST.../attempts` during quiz submission and the bulk `POST.../quizzes/{id}/generate`) accept an optional `Idempotency-Key` header; the backend deduplicates against this key for a 24-hour window. AI generation endpoints additionally return an `Operation-Location` header pointing to `.../generation-runs/{run_id}` for progress polling.

F.6 Endpoint Catalogue

The following tables enumerate every endpoint exposed by the backend, grouped by feature and audience. All paths are relative to `/api/v1`. The *Auth* column reads “Bearer” when an authenticated session is required (the default), “Public” for anonymous endpoints, and “Bearer + role” when an additional role check applies.

F.6.1 Identity — Authentication & Profile

Table F.4: Identity endpoints (Google OAuth, refresh, logout, profile).

Method	Path	Auth	Description
GET	/auth/google/login	Public	Return the Google OAuth authorisation URL plus an opaque state .
GET	/auth/google/callback	Public	Exchange the Google authorisation code for an access/refresh pair.
POST	/auth/refresh	Public*	Rotate the refresh token; returns a fresh token pair.
POST	/auth/logout	Bearer	Invalidate the current refresh-token row server-side.
GET	/users/me	Bearer	Return the authenticated user's profile.
PATCH	/users/me/profile	Bearer	Update display name, avatar, locale, etc.
GET	/users/me/permissions	Bearer	Return effective roles and permission flags.
GET	/users	Bearer	Search users by name or email (auto-complete; admin-scoped).
GET	/users/{user_id}	Bearer	Return a public profile by ID.

*The refresh token itself is the credential; no Bearer header is required.

F.6.2 Access Control — Roles, Grants & Organisations

Table F.5: Access-control endpoints (admin-scoped).

Method	Path	Auth	Description
GET	/admin/permissions	Bearer + admin	List the permission catalogue.
GET	/admin/roles	Bearer + admin	List system and tenant roles with their permissions.
GET	/admin/users/{user_id}/assignments	Bearer + admin	List role assignments for a user.
POST	/admin/users/{user_id}/assignments	Bearer + admin	Assign a role to a user (optionally org-scoped).
DELETE	/admin/users/{user_id}/assignments/{assignment_id}	Bearer + admin	Revoke a role assignment.
GET	/admin/users/{user_id}/grants	Bearer + admin	List ad-hoc permission grants for a user.
POST	/admin/users/{user_id}/grants	Bearer + admin	Issue an ad-hoc permission grant.
DELETE	/admin/users/{user_id}/grants/{grant_id}	Bearer + admin	Revoke an ad-hoc grant.
GET	/admin/organizations	Bearer + admin	List tenant organisations.
POST	/admin/organizations	Bearer + admin	Create a tenant organisation.
GET	/admin/organizations/{org_id}	Bearer + admin	Get an organisation.
PATCH	/admin/organizations/{org_id}	Bearer + admin	Update organisation settings.
DELETE	/admin/organizations/{org_id}	Bearer + admin	Soft-delete an organisation.

Table F.5 (*continued*)

Method	Path	Auth	Description
GET	/admin/organizations/{org_id}/domains	Bearer + admin	List email domains owned by the organisation.
POST	/admin/organizations/{org_id}/domains	Bearer + admin	Add an owned email domain.
PATCH	/admin/organization-domains/{domain_id}	Bearer + admin	Update a domain (rename, mark verified).
DELETE	/admin/organization-domains/{domain_id}	Bearer + admin	Remove an owned domain.
GET	/admin/organizations/{org_id}/units	Bearer + admin	List org units (faculties, departments).
POST	/admin/organizations/{org_id}/units	Bearer + admin	Create an org unit.
GET	/admin/org-units/{unit_id}	Bearer + admin	Get an org unit.
PATCH	/admin/org-units/{unit_id}	Bearer + admin	Update an org unit.
DELETE	/admin/org-units/{unit_id}	Bearer + admin	Delete an org unit.
GET	/admin/organizations/{org_id}/memberships	Bearer + admin	List user memberships in an organisation.
POST	/admin/organizations/{org_id}/memberships	Bearer + admin	Add a user to an organisation.
PATCH	/admin/organization-memberships/{membership_id}	Bearer + admin	Update membership status.
DELETE	/admin/organization-memberships/{membership_id}	Bearer + admin	Remove a membership.

F.6.3 Courses — Learner

Table F.6: Course-learner endpoints (the calling user’s own enrolments).

Method	Path	Auth	Description
GET	/me/courses/courses	Bearer	List the caller’s enrolled courses.
GET	/me/courses/courses/by-slug/{slug}	Bearer	Resolve a course by its public slug.
GET	/me/courses/courses/{course_id}	Bearer	Get course detail.
GET	/me/courses/courses/{course_id}/content	Bearer	Get the full module/lesson tree.
GET	/me/courses/courses/{course_id}/tags	Bearer	Get the course’s tags.
GET	/me/courses/courses/{course_id}/outcomes	Bearer	Get the course’s learning outcomes.
GET	/me/courses/courses/{course_id}/modules	Bearer	List modules with publish state.
GET	/me/courses/modules/{module_id}	Bearer	Get a module.
GET	/me/courses/modules/{module_id}/items	Bearer	List ordered lesson/quiz items.
GET	/me/courses/modules/{module_id}/lessons	Bearer	List lessons in a module.
GET	/me/courses/lessons/{lesson_id}	Bearer	Get a lesson.
GET	/me/courses/lessons/{lesson_id}/resources	Bearer	List downloadable lesson resources.
GET	/me/courses/lesson-resources/{resource_id}/download-url	Bearer	Issue a presigned S3 URL for a resource.

F.6.4 Courses — Authoring (Teacher)

Table F.7: Course-authoring endpoints (teacher role).

Method	Path	Auth		Description
POST	/teacher/courses	Bearer teacher	+	Create a course.
GET	/teacher/courses/check-slug	Bearer teacher	+	Probe slug availability.
GET	/teacher/courses	Bearer teacher	+	List courses owned or co-taught by the caller.
GET	/teacher/courses/{course_id}	Bearer teacher	+	Get a course (authoring view).
GET	/teacher/courses/{course_id}/content	Bearer teacher	+	Get the full module/lesson/quiz tree.
GET	/teacher/courses/{course_id}/roster	Bearer teacher	+	List enrolled students.
PATCH	/teacher/courses/{course_id}	Bearer teacher	+	Update course metadata.
POST	/teacher/courses/{course_id}/publish	Bearer teacher	+	Publish the course.
POST	/teacher/courses/{course_id}/archive	Bearer teacher	+	Archive the course.
POST	/teacher/courses/{course_id}/modules	Bearer teacher	+	Add a module.
PATCH	/teacher/modules/{module_id}	Bearer teacher	+	Update a module.
PUT	/teacher/modules/{module_id}/prerequisites	Bearer teacher	+	Set module prerequisites.

Table F.7 (*continued*)

Method	Path	Auth		Description
PUT	/teacher/modules/{module_id}/items/reorder	Bearer teacher	+	Reorder module items (drag-and-drop).
PATCH	/teacher/module-items/{module_item_id}	Bearer teacher	+	Update a module-item entry.
DELETE	/teacher/module-items/{module_item_id}	Bearer teacher	+	Remove a module-item entry.
GET	/teacher/modules/{module_id}/lessons	Bearer teacher	+	List lessons within a module.
POST	/teacher/modules/{module_id}/lessons	Bearer teacher	+	Create a lesson.
GET	/teacher/lessons/{lesson_id}	Bearer teacher	+	Get a lesson.
GET	/teacher/lessons/{lesson_id}/outline	Bearer teacher	+	Get the structural outline of the lesson.
GET	/teacher/lessons/{lesson_id}/resources	Bearer teacher	+	List downloadable resources.
PATCH	/teacher/lessons/{lesson_id}	Bearer teacher	+	Update a lesson.
POST	/teacher/lessons/{lesson_id}/resources	Bearer teacher	+	Attach a resource.
DELETE	/teacher/lesson-resources/{resource_id}	Bearer teacher	+	Detach a resource.
GET	/teacher/lesson-resources/{resource_id}/download-url	Bearer teacher	+	Issue a presigned download URL.

F.6.5 Courses — Department & Administration

Table F.8: Course department-assignment and administration endpoints.

Method	Path	Auth	Description
GET	/dept/courses	Bearer + dept	List courses for the caller's department.
GET	/dept/courses/{course_id}/teachers	Bearer + dept	List teachers assigned to a course.
POST	/dept/courses/{course_id}/teachers	Bearer + dept	Assign a teacher to a course.
DELETE	/dept/courses/{course_id}/teachers/{user_id}	Bearer + dept	Unassign a teacher.
GET	/dept/courses/{course_id}/roster	Bearer + dept	Read the enrolled-student roster.
GET	/dept/org-units/{org_unit_id}/courses	Bearer + dept	List courses owned by an org unit.
GET	/admin/courses	Bearer + admin	Cross-organisation course listing.
POST	/admin/courses/{course_id}/restore	Bearer + admin	Restore a soft-deleted course.
GET	/admin/courses/{course_id}/audit	Bearer + admin	Course-level audit trail.
GET	/admin/courses/{course_id}/processing	Bearer + admin	Per-course AI processing history.
GET	/admin/courses/_stats	Bearer + admin	Cross-org course aggregates.

F.6.6 Materials

Material upload uses a two-phase protocol so large files can be streamed directly from the browser to the object store (Garage S3-compatible) without traversing the application server. The teacher first calls `init-upload` to receive a presigned URL or a multipart upload ID; the SPA then uploads the bytes; finally it calls `complete` (or `.../multipart/complete`) to register the version and trigger AI processing.

Table F.9: Materials endpoints.

Method	Path	Auth		Description
POST	/teacher/lessons/{lesson_id}/materials/init-upload	Bearer teacher	+	Begin an upload (single-part or multipart).
POST	/teacher/materials/{material_id}/versions/{version_id}/multipart/parts	Bearer teacher	+	Issue per-part presigned URLs.
POST	/teacher/materials/{material_id}/versions/{version_id}/multipart/complete	Bearer teacher	+	Finalise a multipart upload.
POST	/teacher/materials/{material_id}/versions/{version_id}/multipart/abort	Bearer teacher	+	Abort a multipart upload.
POST	/teacher/materials/{material_id}/versions/{version_id}/complete	Bearer teacher	+	Finalise a single-part upload and queue processing.
GET	/teacher/lessons/{lesson_id}/materials	Bearer teacher	+	List materials attached to a lesson.

Table F.9 (*continued*)

Method	Path	Auth		Description
POST	/teacher/lessons/{lesson_id}/materials/link	Bearer teacher	+	Link an existing material into another lesson.
GET	/teacher/lessons/{lesson_id}/processing-summary	Bearer teacher	+	Aggregate AI-processing state for a lesson.
GET	/teacher/materials/{material_id}	Bearer teacher	+	Get material detail.
GET	/teacher/materials/{material_id}/processing-summary	Bearer teacher	+	Per-material processing state.
GET	/teacher/materials/{material_id}/stream-url	Bearer teacher	+	Issue a pre-signed streaming URL (PDF/video).
PATCH	/teacher/materials/{material_id}	Bearer teacher	+	Update material metadata.
POST	/teacher/materials/{material_id}/reprocess	Bearer teacher	+	Re-run AI ingestion on the latest version.
DELETE	/teacher/materials/{material_id}	Bearer teacher	+	Soft-delete a material.
GET	/materials/{material_id}	Bearer		Learner-facing material descriptor.

Table F.9 (continued)

Method	Path	Auth	Description
GET	/materials/{material_id}/stream-url	Bearer	Learner-facing presigned streaming URL.
GET	/materials/{material_id}/chunks/preview	Bearer	Preview the first N text chunks for review.

F.6.7 Quizzes

Quiz authoring follows a draft / generate / review / publish pipeline. `POST.../quizzes/{quiz_id}/generate` accepts a generation spec (source lessons, count, mode, types) and returns a `run_id` immediately; the SPA polls `.../generation-runs/{run_id}` to surface progress. Generated questions land in a *Pending Review* state and are visible to students only after explicit instructor approval.

Table F.10: Quizzes endpoints.

Method	Path	Auth	Description
POST	/teacher/courses/{course_id}/quizzes	Bearer teacher	Create a quiz on a module.
GET	/teacher/quizzes/{quiz_id}	Bearer teacher	Get authoring view of a quiz.
PATCH	/teacher/quizzes/{quiz_id}	Bearer teacher	Update quiz metadata.
POST	/teacher/quizzes/{quiz_id}/publish	Bearer teacher	Publish the quiz.
POST	/teacher/quizzes/{quiz_id}/questions/bulk-set-expected-time	Bearer teacher	Apply a uniform per-question time.

Table F.10 (*continued*)

Method	Path	Auth		Description
DELETE	/teacher/quizzes/{quiz_id}	Bearer teacher	+	Delete a quiz.
POST	/teacher/quizzes/{quiz_id}/generate	Bearer teacher	+	Queue an AI generation run; returns run_id.
GET	/teacher/quizzes/{quiz_id}/generation-runs/latest	Bearer teacher	+	Latest generation-run status.
GET	/teacher/quizzes/{quiz_id}/generation-runs/{run_id}	Bearer teacher	+	Specific run status.
POST	/teacher/quizzes/{quiz_id}/questions	Bearer teacher	+	Add a question manually.
PATCH	/teacher/quizzes/{quiz_id}/questions/{question_id}	Bearer teacher	+	Update or approve a question.
DELETE	/teacher/quizzes/{quiz_id}/questions/{question_id}	Bearer teacher	+	Delete a question.
POST	/teacher/quizzes/{quiz_id}/questions/{question_id}/regenerate	Bearer teacher	+	Regenerate a single AI question.
GET	/teacher/courses/{course_id}/question-bank	Bearer teacher	+	Browse the course-level question bank.
POST	/teacher/quizzes/{quiz_id}/questions/import	Bearer teacher	+	Import from the question bank.

Table F.10 (*continued*)

Method	Path	Auth	Description
GET	/quizzes/{quiz_id}	Bearer	Learner view (questions only, no answers).
POST	/quizzes/{quiz_id}/attempts	Bearer	Start a new attempt.
POST	/attempts/{attempt_id}/answers	Bearer	Submit one or more answers.
POST	/attempts/{attempt_id}/submit	Bearer	Finalise the attempt; trigger scoring.
GET	/attempts/{attempt_id}	Bearer	Attempt detail.
GET	/attempts/{attempt_id}/review	Bearer	Per-question review with correct answers.
GET	/me/quizzes/{quiz_id}/attempts	Bearer	Caller's attempt history for the quiz.

F.6.8 Interviews

Table F.11: Interview endpoints (AI mock-interview surface).

Method	Path	Auth	Description
POST	/teacher/courses/{course_id}/interview-configs	Bearer + teacher	Create an interview configuration.

Table F.11 (*continued*)

Method	Path	Auth		Description
GET	/teacher/interview-configs/{config_id}	Bearer teacher	+	Get a configuration.
PATCH	/teacher/interview-configs/{config_id}	Bearer teacher	+	Update a configuration.
POST	/teacher/interview-configs/{config_id}/publish	Bearer teacher	+	Publish the configuration.
DELETE	/teacher/interview-configs/{config_id}	Bearer teacher	+	Delete a configuration.
POST	/teacher/interview-configs/{config_id}/generate	Bearer teacher	+	Queue AI question generation.
GET	/teacher/interview-configs/{config_id}/generation-runs/{run_id}	Bearer teacher	+	Generation-run status.
POST	/teacher/interview-configs/{config_id}/questions	Bearer teacher	+	Add a question manually.
PATCH	/teacher/interview-configs/{config_id}/questions/{question_id}	Bearer teacher	+	Update or approve a question.
DELETE	/teacher/interview-configs/{config_id}/questions/{question_id}	Bearer teacher	+	Delete a question.
POST	/teacher/interview-configs/{config_id}/questions/{question_id}/regenerate	Bearer teacher	+	Regenerate a single question.
POST	/teacher/interview-configs/{config_id}/outcomes	Bearer teacher	+	Add a desired outcome.
PATCH	/teacher/interview-configs/{config_id}/outcomes/{outcome_id}	Bearer teacher	+	Update an outcome.

Table F.11 (*continued*)

Method	Path	Auth	Description
GET	/teacher/interview-sessions/{session_id}/gap-report	Bearer + teacher	Per-student gap report (instructor view).
GET	/interview-configs/{config_id}	Bearer	Learner-facing config view.
POST	/interview-configs/{config_id}/sessions	Bearer	Start an interview session.
GET	/interview-sessions/{session_id}	Bearer	Get session state.
POST	/interview-sessions/{session_id}/respond	Bearer	Submit a response to the current question.
POST	/interview-sessions/{session_id}/finish	Bearer	Finalise the session; trigger AI scoring.
GET	/interview-sessions/{session_id}/gap-report	Bearer	Per-student gap report (learner view).
GET	/me/interview-sessions	Bearer	Caller's interview-session history.

F.6.9 Progress

Table F.12: Progress endpoints.

Method	Path	Auth	Description
GET	/me/progress/lessons/{lesson_id}	Bearer	Caller's progress for a lesson.
GET	/me/progress/courses/{course_id}	Bearer	Caller's progress for a course.
POST	/me/progress/material-engagement	Bearer	Record a material-engagement event (heart-beat).
POST	/me/progress/lessons/{lesson_id}/complete	Bearer	Mark a lesson complete.
POST	/me/progress/lessons/{lesson_id}/uncomplete	Bearer	Undo lesson completion.
GET	/teacher/courses/{course_id}/progress/roster	Bearer teacher +	Per-student progress roster for a course.
GET	/teacher/courses/{course_id}/progress/at-risk	Bearer teacher +	At-risk student list (low completion / quiz score).

F.6.10 Career Paths

Table F.13: Career-path endpoints.

Method	Path	Auth	Description
GET	/career-paths	Bearer	List published career paths.
GET	/career-paths/{slug}	Bearer	Career-path detail with ordered course list.

F.6.11 Spaced Repetition

Table F.14: Spaced-repetition endpoints (FSRS scheduler).

Method	Path	Auth	Description
GET	/me/cards-due	Bearer	Cards due for review across all courses.
GET	/me/lessons/{lesson_id}/sr-summary	Bearer	Per-lesson SR summary (retention, due count).
GET	/me/courses/{course_id}/sr-overview	Bearer	Per-course SR overview.
GET	/teacher/courses/{course_id}/lessons/{lesson_id}/cohort-kr	Bearer teacher +	Cohort-level retention distribution for a lesson.
GET	/teacher/courses/{course_id}/lessons/{lesson_id}/difficult-cards	Bearer teacher +	Cards the cohort is struggling with.

Table F.14 (*continued*)

Method	Path	Auth	Description
GET	/teacher/courses/{course_id}/at-risk	Bearer teacher	Students flagged by SR signals.
GET	/teacher/courses/{course_id}/students/{student_id}/sr-detail	Bearer teacher	A specific student's per-card SR data.

F.6.12 Notifications

Table F.15: Notification endpoints.

Method	Path	Auth	Description
GET	/me/notifications	Bearer	List the caller's notifications.
GET	/me/notifications/unread-count	Bearer	Unread-count badge.
PATCH	/me/notifications/{notification_id}/read	Bearer	Mark one notification read.
POST	/me/notifications/mark-all-read	Bearer	Mark every notification read.
DELETE	/me/notifications/{notification_id}	Bearer	Delete a notification.
GET	/me/notification-preferences	Bearer	Read per-category, per-channel preferences.
PATCH	/me/notification-preferences/{category}/{channel}	Bearer	Update one preference cell.

F.6.13 Admin Operations

Table F.16: Admin operations endpoints.

Method	Path	Auth	Description
GET	/admin/stats/overview	Bearer + admin	Platform-level overview KPIs.
GET	/admin/stats/active-users	Bearer + admin	Active-user time series.
GET	/admin/stats/content	Bearer + admin	Content-volume metrics.
GET	/admin/stats/health	Bearer + admin	Aggregated infrastructure-health probes.
GET	/admin/users	Bearer + admin	Cross-org user listing with filters.
GET	/admin/users/{user_id}	Bearer + admin	Get a user (admin view).
POST	/admin/users/{user_id}/disable	Bearer + admin	Disable an account.
POST	/admin/users/{user_id}/enable	Bearer + admin	Re-enable an account.
GET	/admin/processing/queue	Bearer + admin	Live ARQ queue depth and worker state.
GET	/admin/processing/jobs	Bearer + admin	Recent AI background jobs.
GET	/admin/processing/jobs/{job_id}	Bearer + admin	Job detail and per-step log.
POST	/admin/processing/jobs/{job_id}/retry	Bearer + admin	Re-enqueue a failed job.

Table F.16 (continued)

Method	Path	Auth	Description
GET	/admin/ai/costs/summary	Bearer + admin	Period-bounded cost summary.
GET	/admin/ai/costs/by-user	Bearer + admin	Cost breakdown by user.
GET	/admin/ai/costs/by-pipeline	Bearer + admin	Cost breakdown by pipeline stage.
GET	/admin/ai/costs/recent	Bearer + admin	Recent cost events.
GET	/admin/audit/role-changes	Bearer + admin	Role-change audit.
GET	/admin/audit/data-changes	Bearer + admin	Data-change audit (DB triggers).
GET	/admin/audit/http	Bearer + admin	HTTP-request audit log.

F.6.14 Health

The application mounts the health router both at the root (`/healthz`) for K8s liveness/readiness probes and under `/api/v1/healthz` so the SPA can probe through the same reverse-proxy path it uses for API traffic.

Table F.17: Health-check endpoints.

Method	Path	Auth	Description
GET	/healthz	Public	Liveness probe; returns 200 if the process is up.
GET	/healthz/deep	Public	Deep probe; checks DB and Redis connectivity.

Table F.17 (*continued*)

Method	Path	Auth	Description
GET	/readyz	Public	Readiness probe; returns 200 once the app is ready to accept traffic.

F.7 API Design Summary

Table F.18: API design summary — chosen strategies and their rationale.

Concern	Strategy	Rationale
Architecture	Vertical-slice features under <code>abridgeai/features/</code>	Feature ownership; routers, services, queries, and schemas live together.
Layering	Routers $\rightarrow$ services $\rightarrow$ queries; enforced by <code>import-linter</code>	Keeps HTTP and persistence concerns separate; testable in isolation.
Authentication	Google OAuth $\rightarrow$ JWT bearer pair (access + refresh)	No password surface; identity proofing delegated to Google.
MFA	TOTP + recovery codes; pre-MFA gate on the access token	Prevents stolen access tokens from acting before the second factor is presented.
Authorisation	Role-scoped URL prefix + per-route dependency check	Clear contextual boundary; central dependency for permission lookups.
Versioning	URL path <code>/api/v1</code>	Explicit, debuggable, easy to migrate via co-existing v2 prefix later.

Table F.18 (continued)

Concern	Strategy	Rationale
Error format	FastAPI <code>HTTPException</code> with structured detail	Standard, well-supported in OpenAPI tooling and the SPA's error layer.
Pagination	Cursor (<code>next_cursor</code>) uniformly across every list endpoint	Stable deep traversal; constant cost per page; no row duplication under concurrent writes.
Idempotency	Optional <code>Idempotency-Key</code> header (24-hour window)	Safe SPA retries on quiz submission and AI generation.
Long-running AI	Async dispatch via ARQ + <code>generation-runs/{run_id}</code> polling	HTTP layer stays under hundreds of milliseconds; progress is observable.
Object storage	S3-compatible (Garage); presigned multipart uploads	Browser uploads bypass the application server; no proxy bandwidth tax.
Observability	<code>http_audit_log</code> middleware + <code>admin/audit/*</code> endpoints	Per-request trace surface for security and performance debugging.
Security headers	HSTS, CSP, X-Frame-Options, etc. via <code>SecurityHeadersMiddleware</code>	Defence-in-depth at the transport layer.

F.8 Swagger UI Screenshot Catalogue

This catalogue captures the live Swagger UI rendered by FastAPI from `/openapi.json` at deployment time. Every endpoint visible below is callable directly from the browser using the **Authorize** control (Bearer token injected once, then reused across requests). The screenshots are organised by audience-scoped **tag** so each figure corresponds to a coherent slice of the API surface; the tag name appears as the section heading inside the screenshot. For the parameter and schema details that the screenshots elide, refer back to the per-feature endpoint tables earlier in this appendix.

GET	/api/v1/users/me	Read Me	🔒	▼
PATCH	/api/v1/users/me/profile	Update My Profile	🔒	▼
GET	/api/v1/users/me/permissions	Read My Permissions	🔒	▼
GET	/api/v1/me/roles	Read My Roles	🔒	▼
mfa				^
GET	/api/v1/auth/me/mfa/status	Get Mfa Status	🔒	▼
POST	/api/v1/auth/me/mfa/totp/enroll	Enroll Totp	🔒	▼
POST	/api/v1/auth/me/mfa/totp/verify	Verify Totp Enrollment	🔒	▼
POST	/api/v1/auth/me/mfa/challenge	Create Mfa Challenge	🔒	▼
POST	/api/v1/auth/me/mfa/verify	Verify Mfa Challenge	🔒	▼
POST	/api/v1/auth/me/mfa/recovery-codes/regenerate	Regenerate Recovery Codes	🔒	▼
POST	/api/v1/auth/me/mfa/disable	Disable Mfa	🔒	▼
admin				^
GET	/api/v1/users	List Users	🔒	▼
GET	/api/v1/users/{user_id}	Get User	🔒	▼
GET	/api/v1/admin/permissions	List Permissions	🔒	▼
GET	/api/v1/admin/roles	List Roles	🔒	▼

Figure F.3: Swagger UI — Administration — part 1. First half of the administrative endpoint surface (user disable/enable, audit subset).

GET	/api/v1/admin/users/{user_id}/assignments	List User Assignments	🔒 ▼
POST	/api/v1/admin/users/{user_id}/assignments	Create User Assignment	🔒 ▼
DELETE	/api/v1/admin/users/{user_id}/assignments/{assignment_id}	Revoke User Assignment	🔒 ▼
GET	/api/v1/admin/users/{user_id}/grants	List User Grants	🔒 ▼
POST	/api/v1/admin/users/{user_id}/grants	Create User Grant	🔒 ▼
DELETE	/api/v1/admin/users/{user_id}/grants/{grant_id}	Revoke User Grant	🔒 ▼
GET	/api/v1/admin/organizations/{org_id}/memberships	List Org Memberships	🔒 ▼
POST	/api/v1/admin/organizations/{org_id}/memberships	Add Org Membership	🔒 ▼
GET	/api/v1/admin/organizations	List Organizations Endpoint	🔒 ▼
POST	/api/v1/admin/organizations	Create Organization Endpoint	🔒 ▼
GET	/api/v1/admin/organizations/{org_id}	Get Organization Endpoint	🔒 ▼
PATCH	/api/v1/admin/organizations/{org_id}	Patch Organization Endpoint	🔒 ▼
DELETE	/api/v1/admin/organizations/{org_id}	Delete Organization Endpoint	🔒 ▼
GET	/api/v1/admin/organizations/{org_id}/domains	List Domains Endpoint	🔒 ▼
POST	/api/v1/admin/organizations/{org_id}/domains	Create Domain Endpoint	🔒 ▼
PATCH	/api/v1/admin/organization-domains/{domain_id}	Patch Domain Endpoint	🔒 ▼
DELETE	/api/v1/admin/organization-domains/{domain_id}	Delete Domain Endpoint	🔒 ▼

Figure F.4: Swagger UI — Administration — part 2. Remaining administrative endpoints (continued from previous figure).

GET	/api/v1/admin/organizations/{org_id}/units	List Units Endpoint	🔒	▼
POST	/api/v1/admin/organizations/{org_id}/units	Create Unit Endpoint	🔒	▼
GET	/api/v1/admin/org-units/{unit_id}	Get Unit Endpoint	🔒	▼
PATCH	/api/v1/admin/org-units/{unit_id}	Patch Unit Endpoint	🔒	▼
DELETE	/api/v1/admin/org-units/{unit_id}	Delete Unit Endpoint	🔒	▼
PATCH	/api/v1/admin/organization-memberships/{membership_id}	Patch Membership Endpoint	🔒	▼
DELETE	/api/v1/admin/organization-memberships/{membership_id}	Delete Membership Endpoint	🔒	▼
GET	/api/v1/admin/stats/overview	Get Overview	🔒	▼
GET	/api/v1/admin/stats/active-users	Get Active Users	🔒	▼
GET	/api/v1/admin/stats/content	Get Content Breakdown	🔒	▼
GET	/api/v1/admin/stats/health	Get Health	🔒	▼
GET	/api/v1/admin/audit/role-changes	Get Role Changes	🔒	▼
GET	/api/v1/admin/audit/data-changes	Get Data Changes	🔒	▼
GET	/api/v1/admin/audit/http	Search Http Audit	🔒	▼
GET	/api/v1/admin/processing/queue	Get Queue Depth	🔒	▼
GET	/api/v1/admin/processing/jobs	List Jobs	🔒	▼
GET	/api/v1/admin/processing/jobs/{job_id}	Get Job Detail	🔒	▼
POST	/api/v1/admin/processing/jobs/{job_id}/retry	Retry Job	🔒	▼
GET	/api/v1/admin/users	List Users	🔒	▼

Figure F.5: Swagger UI — Access control & roles. Role assignments, permission grants, organisation-scoped role queries.

GET	/api/v1/admin/users/{user_id}	Get User Detail	🔒 ▼
POST	/api/v1/admin/users/{user_id}/disable	Disable User	🔒 ▼
POST	/api/v1/admin/users/{user_id}/enable	Enable User	🔒 ▼
GET	/api/v1/admin/ai/costs/summary	Get Summary	🔒 ▼
GET	/api/v1/admin/ai/costs/by-user	Get By User	🔒 ▼
GET	/api/v1/admin/ai/costs/by-pipeline	Get By Pipeline	🔒 ▼
GET	/api/v1/admin/ai/costs/recent	Get Recent	🔒 ▼
access_control			⬆
GET	/api/v1/admin/permissions	List Permissions	🔒 ▼
GET	/api/v1/admin/roles	List Roles	🔒 ▼
GET	/api/v1/admin/users/{user_id}/assignments	List User Assignments	🔒 ▼
POST	/api/v1/admin/users/{user_id}/assignments	Create User Assignment	🔒 ▼

Figure F.6: Swagger UI — Organisations. Organisational unit CRUD, hierarchy walk, member listing.

DELETE	/api/v1/admin/users/{user_id}/assignments/{assignment_id}	Revoke User Assignment	🔒 ▼
GET	/api/v1/admin/users/{user_id}/grants	List User Grants	🔒 ▼
POST	/api/v1/admin/users/{user_id}/grants	Create User Grant	🔒 ▼
DELETE	/api/v1/admin/users/{user_id}/grants/{grant_id}	Revoke User Grant	🔒 ▼
GET	/api/v1/admin/organizations/{org_id}/memberships	List Org Memberships	🔒 ▼
POST	/api/v1/admin/organizations/{org_id}/memberships	Add Org Membership	🔒 ▼
GET	/api/v1/admin/organizations	List Organizations Endpoint	🔒 ▼
POST	/api/v1/admin/organizations	Create Organization Endpoint	🔒 ▼
GET	/api/v1/admin/organizations/{org_id}	Get Organization Endpoint	🔒 ▼
PATCH	/api/v1/admin/organizations/{org_id}	Patch Organization Endpoint	🔒 ▼

Figure F.7: Swagger UI — Courses — learner. Published-course catalogue, enrolled-course lookup, and lesson tree for the calling student.

DELETE	/api/v1/admin/organizations/{org_id}	Delete Organization Endpoint	🔒	▼
GET	/api/v1/admin/organizations/{org_id}/domains	List Domains Endpoint	🔒	▼
POST	/api/v1/admin/organizations/{org_id}/domains	Create Domain Endpoint	🔒	▼
PATCH	/api/v1/admin/organization-domains/{domain_id}	Patch Domain Endpoint	🔒	▼
DELETE	/api/v1/admin/organization-domains/{domain_id}	Delete Domain Endpoint	🔒	▼
GET	/api/v1/admin/organizations/{org_id}/units	List Units Endpoint	🔒	▼
POST	/api/v1/admin/organizations/{org_id}/units	Create Unit Endpoint	🔒	▼
GET	/api/v1/admin/org-units/{unit_id}	Get Unit Endpoint	🔒	▼
PATCH	/api/v1/admin/org-units/{unit_id}	Patch Unit Endpoint	🔒	▼
DELETE	/api/v1/admin/org-units/{unit_id}	Delete Unit Endpoint	🔒	▼
PATCH	/api/v1/admin/organization-memberships/{membership_id}	Patch Membership Endpoint	🔒	▼
DELETE	/api/v1/admin/organization-memberships/{membership_id}	Delete Membership Endpoint	🔒	▼
organizations				^
GET	/api/v1/admin/organizations	List Organizations Endpoint	🔒	▼
POST	/api/v1/admin/organizations	Create Organization Endpoint	🔒	▼
GET	/api/v1/admin/organizations/{org_id}	Get Organization Endpoint	🔒	▼
PATCH	/api/v1/admin/organizations/{org_id}	Patch Organization Endpoint	🔒	▼
DELETE	/api/v1/admin/organizations/{org_id}	Delete Organization Endpoint	🔒	▼

Figure F.8: Swagger UI — Courses — authoring. Instructor course creation, module/lesson editing, publish/archive lifecycle, roster, prerequisites, item reordering.

GET	/api/v1/admin/organizations/{org_id}/domains	List Domains Endpoint	🔒	▼
POST	/api/v1/admin/organizations/{org_id}/domains	Create Domain Endpoint	🔒	▼
PATCH	/api/v1/admin/organization-domains/{domain_id}	Patch Domain Endpoint	🔒	▼
DELETE	/api/v1/admin/organization-domains/{domain_id}	Delete Domain Endpoint	🔒	▼
GET	/api/v1/admin/organizations/{org_id}/units	List Units Endpoint	🔒	▼
POST	/api/v1/admin/organizations/{org_id}/units	Create Unit Endpoint	🔒	▼
GET	/api/v1/admin/org-units/{unit_id}	Get Unit Endpoint	🔒	▼
PATCH	/api/v1/admin/org-units/{unit_id}	Patch Unit Endpoint	🔒	▼
DELETE	/api/v1/admin/org-units/{unit_id}	Delete Unit Endpoint	🔒	▼
PATCH	/api/v1/admin/organization-memberships/{membership_id}	Patch Membership Endpoint	🔒	▼
DELETE	/api/v1/admin/organization-memberships/{membership_id}	Delete Membership Endpoint	🔒	▼
courses-learner				^

Figure F.9: Swagger UI — Courses — department & administration. Department-scoped course assignment to teachers, cross-organisation course listing, soft-delete restoration, and audit views.

GET	/api/v1/courses	List Courses	🔒	▼
GET	/api/v1/courses/by-slug/{slug}	Get Course By Slug	🔒	▼
GET	/api/v1/courses/{course_id}	Get Course	🔒	▼
GET	/api/v1/courses/{course_id}/content	Get Course Content	🔒	▼
GET	/api/v1/courses/{course_id}/tags	List Course Tags	🔒	▼
GET	/api/v1/courses/{course_id}/outcomes	List Course Outcomes	🔒	▼
GET	/api/v1/courses/{course_id}/modules	List Course Modules	🔒	▼
GET	/api/v1/modules/{module_id}	Get Module	🔒	▼
GET	/api/v1/modules/{module_id}/items	List Module Items	🔒	▼
GET	/api/v1/modules/{module_id}/lessons	List Module Lessons	🔒	▼
GET	/api/v1/lessons/{lesson_id}	Get Lesson	🔒	▼

Figure F.10: Swagger UI — Materials. Two-phase upload protocol (presigned init / multipart parts / complete), processing summaries, learner-facing presigned streaming URLs.

GET	/api/v1/lessons/{lesson_id}/resources	List Lesson Resources	🔒	▼
GET	/api/v1/lesson-resources/{resource_id}/download-url	Get Lesson Resource Download Url	🔒	▼
GET	/api/v1/me/courses	List My Enrolled Courses	🔒	▼
courses-authoring				^
POST	/api/v1/teacher/courses	Create Course	🔒	▼
GET	/api/v1/teacher/courses	List Authoring Courses	🔒	▼
GET	/api/v1/teacher/courses/check-slug	Check Course Slug	🔒	▼
GET	/api/v1/teacher/courses/{course_id}	Get Authoring Course	🔒	▼
PATCH	/api/v1/teacher/courses/{course_id}	Update Course	🔒	▼
GET	/api/v1/teacher/courses/{course_id}/content	Get Authoring Course Content	🔒	▼
GET	/api/v1/teacher/courses/{course_id}/roster	Get Authoring Course Roster	🔒	▼
POST	/api/v1/teacher/courses/{course_id}/publish	Publish Course	🔒	▼
POST	/api/v1/teacher/courses/{course_id}/archive	Archive Course	🔒	▼
POST	/api/v1/teacher/courses/{course_id}/modules	Create Module	🔒	▼
PATCH	/api/v1/teacher/modules/{module_id}	Update Module	🔒	▼
PUT	/api/v1/teacher/modules/{module_id}/prerequisites	Set Module Prerequisites	🔒	▼
PUT	/api/v1/teacher/modules/{module_id}/items/reorder	Reorder Module Items	🔒	▼

Figure F.11: Swagger UI — Quizzes. Authoring (draft/generate/review/publish) and learner (start attempt, submit, results) endpoints; AI generation through generation-runs/{run_id} polling.

PATCH	/api/v1/teacher/module-items/{module_item_id}	Update Module Item	🔒	▼
DELETE	/api/v1/teacher/module-items/{module_item_id}	Delete Module Item	🔒	▼
GET	/api/v1/teacher/modules/{module_id}/lessons	List Module Lessons For Authoring	🔒	▼
POST	/api/v1/teacher/modules/{module_id}/lessons	Create Lesson	🔒	▼
GET	/api/v1/teacher/lessons/{lesson_id}	Get Authoring Lesson	🔒	▼
PATCH	/api/v1/teacher/lessons/{lesson_id}	Update Lesson	🔒	▼
GET	/api/v1/teacher/lessons/{lesson_id}/outline	Get Lesson Outline	🔒	▼
GET	/api/v1/teacher/lessons/{lesson_id}/resources	List Authoring Lesson Resources	🔒	▼
POST	/api/v1/teacher/lessons/{lesson_id}/resources	Create Lesson Resource	🔒	▼
DELETE	/api/v1/teacher/lesson-resources/{resource_id}	Delete Lesson Resource	🔒	▼
GET	/api/v1/teacher/lesson-resources/{resource_id}/download-url	Get Authoring Resource Download Url	🔒	▼
courses-assignment				^
GET	/api/v1/dept/courses	List Dept Courses	🔒	▼
GET	/api/v1/dept/courses/{course_id}/teachers	List Course Teachers	🔒	▼
POST	/api/v1/dept/courses/{course_id}/teachers	Assign Teacher	🔒	▼
DELETE	/api/v1/dept/courses/{course_id}/teachers/{user_id}	Remove Teacher	🔒	▼
GET	/api/v1/dept/courses/{course_id}/roster	Get Course Roster	🔒	▼

Figure F.12: Swagger UI — Interviews. Mock-interview authoring (slots, question banks, regeneration) and learner endpoints (start/submit/feedback).

GET	/api/v1/dept/org-units/{org_unit_id}/courses	List Org Unit Courses	🔒	▼
courses-admin				^
GET	/api/v1/admin/courses	List All Courses	🔒	▼
POST	/api/v1/admin/courses/{course_id}/restore	Restore Soft Deleted Course	🔒	▼
GET	/api/v1/admin/courses/{course_id}/audit	Get Course Audit	🔒	▼
GET	/api/v1/admin/courses/{course_id}/processing	List Course Processing Jobs	🔒	▼
GET	/api/v1/admin/courses/_stats	Get Course Stats	🔒	▼
materials-learner				^
GET	/api/v1/materials/{material_id}	Get Material	🔒	▼

Figure F.13: Swagger UI — Enrollments. Learner self-enrolment and department-assigned enrolment management.

GET	/api/v1/materials/{material_id}/stream-url	Get Material Stream Url	🔒	▼
GET	/api/v1/materials/{material_id}/chunks/preview	Get Material Chunks Preview	🔒	▼
materials-authoring ^				
POST	/api/v1/teacher/lessons/{lesson_id}/materials/init-upload	Init Upload	🔒	▼
POST	/api/v1/teacher/materials/{material_id}/versions/{version_id}/multipart/parts	Fetch Multipart Parts	🔒	▼
POST	/api/v1/teacher/materials/{material_id}/versions/{version_id}/multipart/complete	Complete Multipart	🔒	▼
POST	/api/v1/teacher/materials/{material_id}/versions/{version_id}/multipart/abort	Abort Multipart	🔒	▼
POST	/api/v1/teacher/materials/{material_id}/versions/{version_id}/complete	Complete Upload	🔒	▼

Figure F.14: Swagger UI — Progress. Learner progress tracking and instructor progress dashboards.

GET	/api/v1/teacher/lessons/{lesson_id}/materials	List Materials	🔒	▼
POST	/api/v1/teacher/lessons/{lesson_id}/materials/link	Link Existing Material	🔒	▼
GET	/api/v1/teacher/lessons/{lesson_id}/processing-summary	Lesson Processing Summary	🔒	▼
GET	/api/v1/teacher/materials/{material_id}	Get Material	🔒	▼
PATCH	/api/v1/teacher/materials/{material_id}	Update Material	🔒	▼
DELETE	/api/v1/teacher/materials/{material_id}	Delete Material	🔒	▼
GET	/api/v1/teacher/materials/{material_id}/processing-summary	Processing Summary	🔒	▼
GET	/api/v1/teacher/materials/{material_id}/stream-url	Get Material Stream Url	🔒	▼
POST	/api/v1/teacher/materials/{material_id}/reprocess	Reprocess Material	🔒	▼
POST	/api/v1/materials/upload-url	Request Upload Url	🔒	▼
quizzes-learner ^				
GET	/api/v1/quizzes/{quiz_id}	Get Published Quiz	🔒	▼
POST	/api/v1/quizzes/{quiz_id}/attempts	Start Attempt	🔒	▼
POST	/api/v1/attempts/{attempt_id}/answers	Record Answer	🔒	▼
POST	/api/v1/attempts/{attempt_id}/submit	Submit Attempt	🔒	▼
GET	/api/v1/attempts/{attempt_id}	Get Attempt	🔒	▼

Figure F.15: Swagger UI — Career paths. Published career-path catalogue (learner) and authoring routes (instructor/admin).

GET	/api/v1/attempts/{attempt_id}/review	Get Attempt Review	🔒	▼
GET	/api/v1/me/quizzes/{quiz_id}/attempts	List My Attempts	🔒	▼
quizzes-authoring				^
POST	/api/v1/teacher/courses/{course_id}/quizzes	Create Quiz Under Course	🔒	▼
GET	/api/v1/teacher/quizzes/{quiz_id}	Get Quiz Authoring	🔒	▼
PATCH	/api/v1/teacher/quizzes/{quiz_id}	Update Quiz	🔒	▼
DELETE	/api/v1/teacher/quizzes/{quiz_id}	Delete Quiz	🔒	▼
POST	/api/v1/teacher/quizzes/{quiz_id}/publish	Publish Quiz	🔒	▼
POST	/api/v1/teacher/quizzes/{quiz_id}/questions/bulk-set-expected-time	Bulk Set Expected Time	🔒	▼

Figure F.16: Swagger UI — Notifications & stats. Learner notification inbox (cursor-paginated by `created_at`) and admin platform-wide statistics.

POST	/api/v1/teacher/quizzes/{quiz_id}/generate	Start Generation	🔒	▼
GET	/api/v1/teacher/quizzes/{quiz_id}/generation-runs/latest	Get Latest Quiz Generation Run	🔒	▼
GET	/api/v1/teacher/quizzes/{quiz_id}/generation-runs/{run_id}	Get Generation Run	🔒	▼
POST	/api/v1/teacher/quizzes/{quiz_id}/questions	Create Question	🔒	▼
PATCH	/api/v1/teacher/quizzes/{quiz_id}/questions/{question_id}	Update Question	🔒	▼
DELETE	/api/v1/teacher/quizzes/{quiz_id}/questions/{question_id}	Delete Question	🔒	▼
POST	/api/v1/teacher/quizzes/{quiz_id}/questions/{question_id}/regenerate	Regenerate Question	🔒	▼
GET	/api/v1/teacher/courses/{course_id}/question-bank	List Question Bank	🔒	▼
POST	/api/v1/teacher/quizzes/{quiz_id}/questions/import	Import Questions From Bank	🔒	▼
interviews-learner				^
GET	/api/v1/interview-configs/{config_id}	Get Interview For Taking	🔒	▼
POST	/api/v1/interview-configs/{config_id}/sessions	Start Session	🔒	▼
GET	/api/v1/interview-sessions/{session_id}	Get Session	🔒	▼
POST	/api/v1/interview-sessions/{session_id}/respond	Respond To Session	🔒	▼
POST	/api/v1/interview-sessions/{session_id}/finish	Finish Session	🔒	▼

Figure F.17: Swagger UI — Audit & processing. Cross-feature audit log queries and AI-processing run inspection (admin).

POST	/api/v1/teacher/sessions/{session_id}/cancel-lesson-session		
GET	/api/v1/interview-sessions/{session_id}/gap-report	Get Gap Report	🔒 ▼
GET	/api/v1/me/interview-sessions	List My Sessions	🔒 ▼
interviews-authoring			^
POST	/api/v1/teacher/courses/{course_id}/interview-configs	Create Interview Config	🔒 ▼
GET	/api/v1/teacher/interview-configs/{config_id}	Get Interview Config	🔒 ▼
PATCH	/api/v1/teacher/interview-configs/{config_id}	Update Interview Config	🔒 ▼
DELETE	/api/v1/teacher/interview-configs/{config_id}	Delete Interview Config	🔒 ▼
POST	/api/v1/teacher/interview-configs/{config_id}/publish	Publish Interview Config	🔒 ▼
POST	/api/v1/teacher/interview-configs/{config_id}/generate	Start Generation	🔒 ▼
GET	/api/v1/teacher/interview-configs/{config_id}/generation-runs/{run_id}	Get Generation Run	🔒 ▼
POST	/api/v1/teacher/interview-configs/{config_id}/questions	Create Question	🔒 ▼
PATCH	/api/v1/teacher/interview-configs/{config_id}/questions/{question_id}	Update Question	🔒 ▼

Figure F.18: Swagger UI — Spaced repetition. Learner card-due queue, review submission, lesson-unlock checks, and instructor analytics.

DELETE	/api/v1/teacher/interview-configs/{config_id}/questions/{question_id}	Delete Question	🔒	▼
POST	/api/v1/teacher/interview-configs/{config_id}/questions/{question_id}/regenerate	Regenerate Question	🔒	▼
POST	/api/v1/teacher/interview-configs/{config_id}/outcomes	Create Outcome	🔒	▼
PATCH	/api/v1/teacher/interview-configs/{config_id}/outcomes/{outcome_id}	Update Outcome	🔒	▼
GET	/api/v1/teacher/interview-sessions/{session_id}/gap-report	Get Session Gap Report Authoring	🔒	▼
enrollments-learner				^
GET	/api/v1/me/enrollments	List My Enrollments	🔒	▼
GET	/api/v1/me/enrollments/{course_id}	Get My Enrollment Detail	🔒	▼
enrollments-assignment				^
GET	/api/v1/dept/courses/{course_id}/enrollments	List Dept Course Enrollments	🔒	▼
POST	/api/v1/management/courses/{course_id}/enrollments/bulk	Manager Bulk Enroll	🔒	▼
DELETE	/api/v1/management/courses/{course_id}/enrollments/{user_id}	Manager Unenroll	🔒	▼
POST	/api/v1/management/courses/{course_id}/enrollments/import-csv	Manager Csv Import	🔒	▼
GET	/api/v1/management/courses/{course_id}/invitation-codes	List Invitation Codes	🔒	▼
POST	/api/v1/management/courses/{course_id}/invitation-codes	Create Invitation Code	🔒	▼
PATCH	/api/v1/management/invitation-codes/{code_id}	Update Invitation Code	🔒	▼
DELETE	/api/v1/management/invitation-codes/{code_id}	Delete Invitation Code	🔒	▼

Figure F.19: Swagger UI — AI costs & health. AI-cost telemetry endpoints and Kubernetes liveness/readiness probes (/healthz, /readyz).

PipelineStage >	Expand all	object
ProcessingJobOut >	Expand all	object
ProcessingJobRow >	Expand all	object
ProcessingProgress >	Expand all	object
QuestionBankEntry >	Expand all	object
QuestionBankImportRequest >	Expand all	object
QueueDepthOut >	Expand all	object
QuizAttemptAnswerInput >	Expand all	object
QuizAttemptAnswerRead >	Expand all	object
QuizAttemptRead >	Expand all	object
QuizAttemptReviewOption >	Expand all	object
QuizAttemptReviewQuestion >	Expand all	object
QuizAttemptReviewRead >	Expand all	object
QuizAttemptStart >	Expand all	object
QuizAuthoring >	Expand all	object
QuizForAuthoringPublic >	Expand all	object
QuizForTakingPublic >	Expand all	object

Figure F.20: Swagger UI — Schemas — part 1. Pydantic v2 schemas exposed in OpenAPI components.schemas (first third of 223 schemas).

QuizGenerationRequest >	Expand all	object
QuizGenerationRunRead >	Expand all	object
QuizPublic >	Expand all	object
QuizQuestionAuthoring >	Expand all	object
QuizQuestionOptionAuthoring >	Expand all	object
QuizQuestionOptionPublic >	Expand all	object
QuizQuestionPublic >	Expand all	object
QuizSummaryPublic >	Expand all	object
RecentCallOut >	Expand all	object
RefreshTokenRequest >	Expand all	object
ReprocessOut >	Expand all	object
ResourceDownloadUrlResponse >	Expand all	object
RoleAssignmentCreate >	Expand all	object
RoleAssignmentRead >	Expand all	object
RoleBreakdown >	Expand all	object
RoleChangeRow >	Expand all	object
RoleRead >	Expand all	object

Figure F.21: Swagger UI — Schemas — part 2. Pydantic v2 schemas (continued).

RoleWithPermissionsRead >	Expand all	object
RosterEntry >	Expand all	object
RosterProgressRead >	Expand all	object
SlugAvailability >	Expand all	object
StageBreakdown >	Expand all	object
StreamUrlResponse >	Expand all	object
StudentCareerEnrollmentAuthoring >	Expand all	object
StudentLessonSummaryRead >	Expand all	object
StudentPathProgressAuthoring >	Expand all	object
StudentProgressRow >	Expand all	object
StudentSrDetailLessonRead >	Expand all	object
StudentSrDetailRead >	Expand all	object
StudentSrDetailReviewRead >	Expand all	object
StudyPlanItem >	Expand all	object
SummaryOut >	Expand all	object
TagAuthoring >	Expand all	object
TagPublic >	Expand all	object

Figure F.22: Swagger UI — Schemas — part 3. Pydantic v2 schemas (final third).

Appendix G Site Map Specification

This appendix provides the complete site map of the aBridgeAI frontend application, organised by user role. Each entry specifies the URL path, page title, and primary purpose. Routes marked with parameters (e.g. `$slug`) are dynamic segments resolved at runtime.

G.1 Public Routes (Unauthenticated)

Table G.1: Public (unauthenticated) routes.

Path	Page	Purpose
<code>/</code>	Landing Page	Marketing page with platform overview and CTA.
<code>/login</code>	Login	Google SSO authentication entry point.
<code>/login/mfa</code>	MFA Verification	TOTP or recovery code verification after initial login.
<code>/auth/google/callback</code>	OAuth Callback	Handles Google OAuth redirect and token exchange.

G.2 Student Routes

Table G.2: Student-facing routes.

Path	Page	Purpose
<code>/dashboard</code>	Dashboard	Overview of enrolled courses, due reviews, and progress summary.
<code>/dashboard/sr</code>	SR Dashboard	Spaced repetition metrics: due cards, retention, streaks.
<code>/study/cards-due</code>	Study Cards	Review due flashcards from spaced repetition queue.

Table G.2 – Continued

Path	Page	Purpose
/courses	Course Catalog	Browse and search available courses.
/courses/\$slug	Course Detail	View course structure, modules, and lessons.
/courses/\$slug/learn	Course Learn	Read lesson content, stream materials, view resources.
/courses/\$slug/quiz/\$quizId	Quiz Session	Take a quiz attempt with timed questions.
/courses/\$slug/quiz/\$quizId/attempts/\$attemptId	Quiz Review	Review past attempt with correct answers and explanations.
/courses/\$slug/interview/\$moduleId	Interview Session	Conduct AI mock interview for a module.
/progress	Progress	Overall learning progress and completion tracking.
/career-paths	Career Paths	Browse available career pathways.
/career-paths/\$slug	Career Path Detail	View courses in a pathway and progression status.
/me/career-paths	My Career Paths	View enrolled career pathways and progress.
/notifications	Notifications	In-app notification inbox.
/profile	Profile	View own public profile.
/settings	Settings Hub	Navigation hub for account settings.
/settings/profile	Profile Settings	Edit display name, avatar, bio, and links.
/settings/security	Security Settings	MFA setup, recovery codes, session management.
/settings/notifications	Notification Prefs	Configure notification preferences by category.

G.3 Teacher / Instructor Routes

Table G.3: Teacher and instructor routes.

Path	Page	Purpose
/teacher	Overview	Teacher dashboard with course summary.
/teacher/courses	My Courses	List of courses the teacher manages.
/teacher/courses/new	Create Course	Form to create a new course.
/teacher/courses/\$courseId	Course Manage	Course authoring: modules, settings, publishing.
/teacher/courses/\$courseId/modules/\$moduleId	Module Manage	Edit module: ordering, items, prerequisites.
/teacher/courses/\$courseId/lessons/\$lessonId	Lesson Manage	Edit lesson content, notes, and settings.
/teacher/courses/\$courseId/lessons/\$lessonId/materials	Lesson Materials	Upload, manage, and re-process materials.
/teacher/courses/\$courseId/quizzes/\$quizId	Quiz Editor	Manage questions, review AI drafts, configure settings.
/teacher/courses/\$courseId/interview-configs/new	New Interview Config	Create interview section with desired outcomes.
/teacher/courses/\$courseId/interview-configs/\$configId	Interview Config	Edit interview configuration and rules.
/teacher/interview-sessions/\$sessionId/gap-report	Gap Report	Review student interview transcript and evaluation.
/teacher/courses/\$courseId/students	Student Roster	View enrolled students and progress.

Table G.3 – Continued

Path	Page	Purpose
/teacher/courses/\$courseId/students/\$studentId	Student Detail	Individual student progress and activity.
/teacher/courses/\$courseId/students/\$studentId/sr	Student SR Detail	Student's spaced repetition card-level data.
/teacher/courses/\$courseId/sr-cohort	SR Cohort	Cohort-level retention distribution and analytics.
/teacher/courses/\$courseId/at-risk	At-Risk Students	Students flagged for low retention or engagement.
/teacher/courses/\$courseId/progress	Course Progress	Aggregate course completion analytics.
/dept	Department Courses	View all courses in the department.
/dept/courses/\$courseId	Dept Course Detail	Read-only view of a department course.

G.4 Educational Manager Routes

Table G.4: Educational Manager routes.

Path	Page	Purpose
/management/courses/\$courseId/enrollments	Course Enrollments	Bulk enroll students via CSV, manage roster.
/management/career-paths	Career Paths Mgmt	Create and manage career pathways.
/management/career-paths/\$id	Career Path Edit	Edit pathway details and course ordering.

G.5 IT Administrator Routes

Table G.5: IT Administrator routes.

Path	Page	Purpose
/admin/stats	Admin Dashboard	Platform-level statistics overview.
/admin/stats/active	Active Users	Real-time active user metrics.
/admin/stats/content	Content Stats	Content volume and growth metrics.
/admin/stats/health	System Health	Infrastructure health indicators.
/admin/users	Users	Manage all user accounts across tenants.
/admin/users/\$userId	User Detail	View/edit individual user, disable account.
/admin/organizations	Organizations	Manage tenant organisations.
/admin/organizations/\$orgId	Org Detail	Edit org settings, domains, lifecycle.
/admin/courses	Courses	Cross-org course overview and restoration.
/admin/courses/\$courseId	Course Detail	Inspect course processing history and stats.
/admin/processing	Processing Jobs	View ingestion and AI pipeline job queue.
/admin/processing/\$jobId	Job Detail	Inspect individual job logs and retry.
/admin/ai-costs	AI Costs	AI usage telemetry by user and pipeline.
/admin/health	Health Check	Backend service health and connectivity.

G.6 Route Summary

Table G.6: Route count by role.

Role / Section	Routes
Public (unauthenticated)	4
Student	19
Teacher / Instructor	19
Educational Manager	3
IT Administrator	14
Total unique routes	59

Appendix H UI/UX Design Specification

H.1 UI/UX Design

H.1.1 Design Philosophy

aBridgeAI adopts a **light-first, academic** design language inspired by enterprise dashboards (Metronic) — dense rows, crisp 1 px borders, restrained 4–10 px corner radii. The interface visually positions the platform as a serious learning tool rather than a consumer product. Four guiding principles drive every screen:

1. **Role-based clarity** — students, instructors and administrators see entirely distinct navigation structures and accent colours, preventing cognitive overload from features that are not relevant to the active role.
2. **Progressive disclosure** — advanced AI capabilities (quiz generation, processing queue, cost telemetry) are surfaced contextually inside their host screens rather than buried in a separate “AI” section, but appear only when the user has the required role and prerequisites.
3. **Accessibility-first typography** — body copy uses *Be Vietnam Pro*, a humanist sans-serif designed with full Vietnamese diacritic support, paired with a 14 px base size and 1.5 line-height for sustained reading.
4. **Responsive shell architecture** — a persistent `AppShell` component renders a left sidebar that collapses from 256 px (`w-64`) to 64 px (`w-16`, icons only) below the `md` breakpoint. The same shell is reused across all authenticated routes; role differentiation is driven entirely by a `navItems` prop and accent colour.

The implementation uses **shadcn/ui** components on top of **Tailwind CSS v4**, with the colour system defined as CSS custom properties (`var(--primary)`, `var(--surface)`, ...) so that the light and dark themes share a single component layer with full parity.

H.1.2 Internationalisation (i18n)

The interface ships with multilingual support out of the box. Every page exposes a globe icon in the top-right corner of the `ContentTopBar` that opens a language selector; the active language is shown next to the icon (*e.g.* “EN”). Locale strings live in `src/locales/<lang>.json` and are loaded lazily, with English as the fallback. Every navigation

label, page heading, and form field references an `i18nKey` (for example `nav.dashboard`, `nav.career_paths`) defined in `src/lib/navigation.ts`, so adding a new locale only requires translating the JSON dictionary — no component changes are needed. The platform was developed with English and Vietnamese as the first-class locales, matching the choice of Be Vietnam Pro as the primary typeface.

H.1.3 Design Rules & Constraints

This subsection documents the formal design rules applied consistently across the interface. They serve as the single source of truth for all UI decisions and are enforced via the design tokens defined in `src/app.css`.

Colour Rules

Table H.1: Colour System Rules

Rule	Value	Rationale
Page background	#f8fafc (slate-50)	Soft neutral surface that reduces eye strain without resorting to pure white.
Surface / Card	ffffff	One-level elevation above the page; combined with a 1px slate-200 border for crisp separation.
Primary accent	#1e40af (blue-800)	Academic blue used for primary actions, active navigation, links, and chart series 1.
Secondary accent	#3b82f6 (blue-500)	Lighter complementary blue for secondary buttons, focus rings, and chart series 2.
Call-to-action / accent	#f59e0b (amber-500)	Reserved for high-attention CTAs (“Generate”, “Save changes”) and AI-related affordances.
Success	#16a34a (green-600)	Published states, completed jobs, success toasts.

Table H.1 (continued)

Rule	Value	Rationale
Warning	#e a 5 8 0 c (orange-600)	Drafts, pending review, soft warnings that do not block the user.
Danger / Destructive	#dc2626 (red-600)	Failed jobs, validation errors, destructive actions (Archive, Delete).
Muted text	#475569 (slate-600)	Captions and secondary labels; never used for primary content.
Border	#e2e8f0 (slate-200)	1 px borders on cards, inputs, and table rows; the only accepted divider treatment.

The dark theme keeps the same primary blue and amber accents but swaps surfaces to slate-900 / slate-800 and inverts text to slate-50 / slate-200, ensuring component-level parity without a separate stylesheet.

Typography Rules

Table H.2: Typography Rules

Element	Specification	Rule
Typeface	Be Vietnam Pro (sans-serif)	All body, heading and UI text; system-ui fallback. Chosen for full Vietnamese diacritic support.
Body size	14px / 1.5 line-height	Optimised for sustained reading on desktop; legibility verified down to 12px minimum.
Page heading (h1)	2xl–3xl, semibold, slate-900	One per page; communicates the primary context. Letter-spacing $-0.01em$.
Section heading (h2)	xl, semibold	At most four per scrollable area; introduces a self-contained section.
Card heading (h3)	lg, semibold	Card-level titles; truncated to two lines with ellipsis when overflowing.

Table H.2 (*continued*)

Element	Specification	Rule
Label / Caption	xs, slate-500, uppercase tracking	Reserved for metadata such as status badges and stat-card legends.
Minimum size	12 px	No text below 12px is permitted anywhere in the interface.

Spacing & Layout Rules

- **Base unit:** 4px (Tailwind `spacing-1`). All spacing values must be multiples of 4px.
- **Content padding:** minimum 16px (`p-4`) on cards and panels; 24px (`p-6`) on top-level page containers.
- **Sidebar width:** fixed 256px (`w-64`) when expanded; 64px (`w-16`) when collapsed; never overlays content on desktop, only on mobile via a backdrop.
- **Content max-width:** 1280px (`max-w-7xl`); content must never span the full viewport width on large screens.
- **Border radius:** `rounded-md` (6px) for inputs, `rounded-lg` (8px) for cards, `rounded-xl` (10px) for elevated surfaces; values above 10px are forbidden by CI.
- **Grid system:** 12-column CSS grid; dashboards use predefined `grid-cols-2` or `grid-cols-3` layouts for stat cards.
- **Responsive breakpoints:** sm 640px, md 768px, lg 1024px, xl 1280px.

Layering & Modal Rules

The interface enforces a strict layering hierarchy to avoid accidental interaction blocking:

Table H.3: Z-Index Layering Rules

Layer	z-index	Rule
Page content (sticky bars)	10	Per-page sticky elements such as filter and search bars.
ContentTopBar	20	Global sticky header; always above page content.
Mobile backdrop	30	Sidebar overlay backdrop on mobile; sits above all page content.
Sidebar	40	Fixed sidebar; always wins above the backdrop.
Dropdowns / Tooltips	50	Radix UI popovers; float above the layout.
Sheets / Drawers	60	Mobile slide-in panels.
Modals / Dialogs	70	Maximum one modal open at a time, always with a 50 % backdrop.
Toast / Notifica- tions	80	System feedback; never blocks primary actions.

Maximum modal depth: a single stacked modal at any point. Confirmation dialogs triggered from inside a modal must close the parent first or use inline expansion — nested “modal-on-modal” patterns are forbidden.

Interaction & State Rules

- **Loading:** every data-fetching operation must show a skeleton or spinner within 200 ms.
- **Hover:** interactive elements (buttons, links, cards) must produce a visible hover state (background tint or underline) within a 150 ms transition.
- **Disabled:** disabled controls render at 50 % opacity with `cursor-not-allowed` and a tooltip explaining the prerequisite.
- **Focus rings:** all keyboard-focusable elements must show a visible `ring-2ring-primary` focus ring for accessibility.
- **Empty:** every list, table or grid renders a descriptive empty-state with an action when no data is present.

- **Errors:** form validation errors are displayed inline beneath the field; alert boxes alone are not acceptable.
- **Success:** destructive or irreversible actions (publish, delete, submit) confirm completion with a toast.

H.1.4 Key Screen Designs

The following subsections present the primary screen designs grouped by role. Screens are sequenced by the typical user flow within each role.

Landing Page

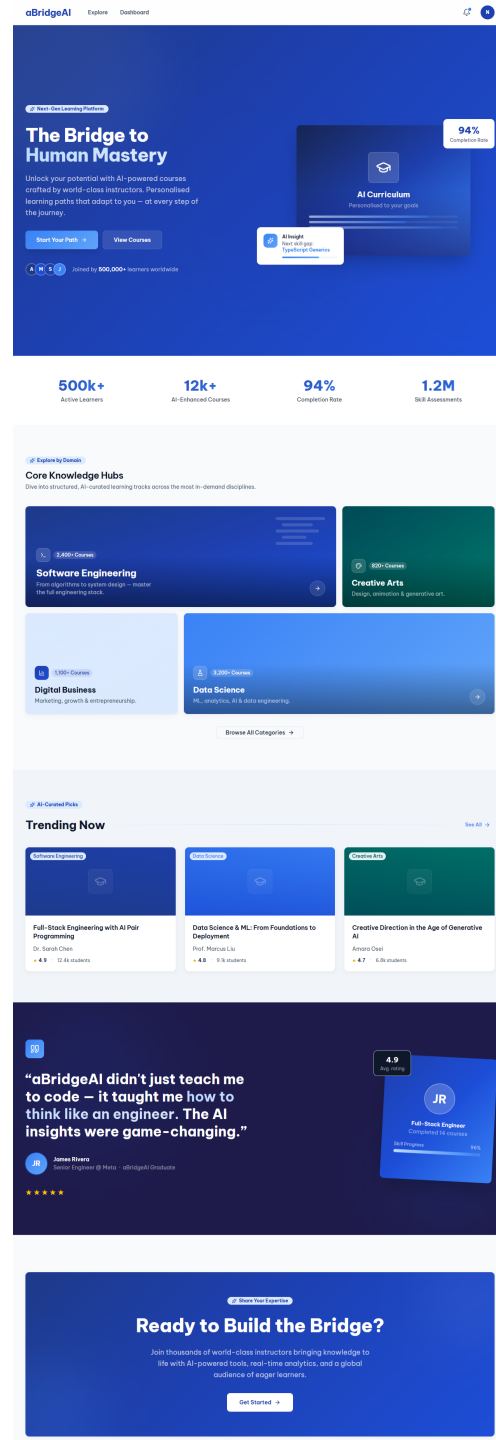


Figure H.1: Landing Page — Marketing entry point with hero section and call-to-action.

The landing page is the public marketing surface. It carries a hero headline introducing the AI-powered career-readiness theme, a stats strip, a curated category grid, and a featured-courses carousel. Navigation links to the catalog, login, and sign-up flow are exposed in a horizontal **TopNavBar** that is unique to this page. The visual treatment is intentionally distinct from the authenticated shell so that crossing the login boundary feels like a deliberate handoff between marketing and product.

Authentication

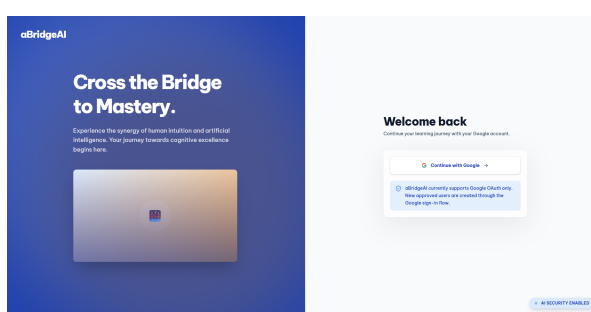


Figure H.2: Login — Google SSO entry point.

The login screen offers Google SSO as the single sign-in path; on successful redirect, the application checks whether the account has MFA configured and, if so, routes the user to `/login/mfa` for a TOTP or recovery-code challenge before issuing a session. A `next` query parameter preserves the originally requested route so deep links survive the auth round-trip.

Student Dashboard

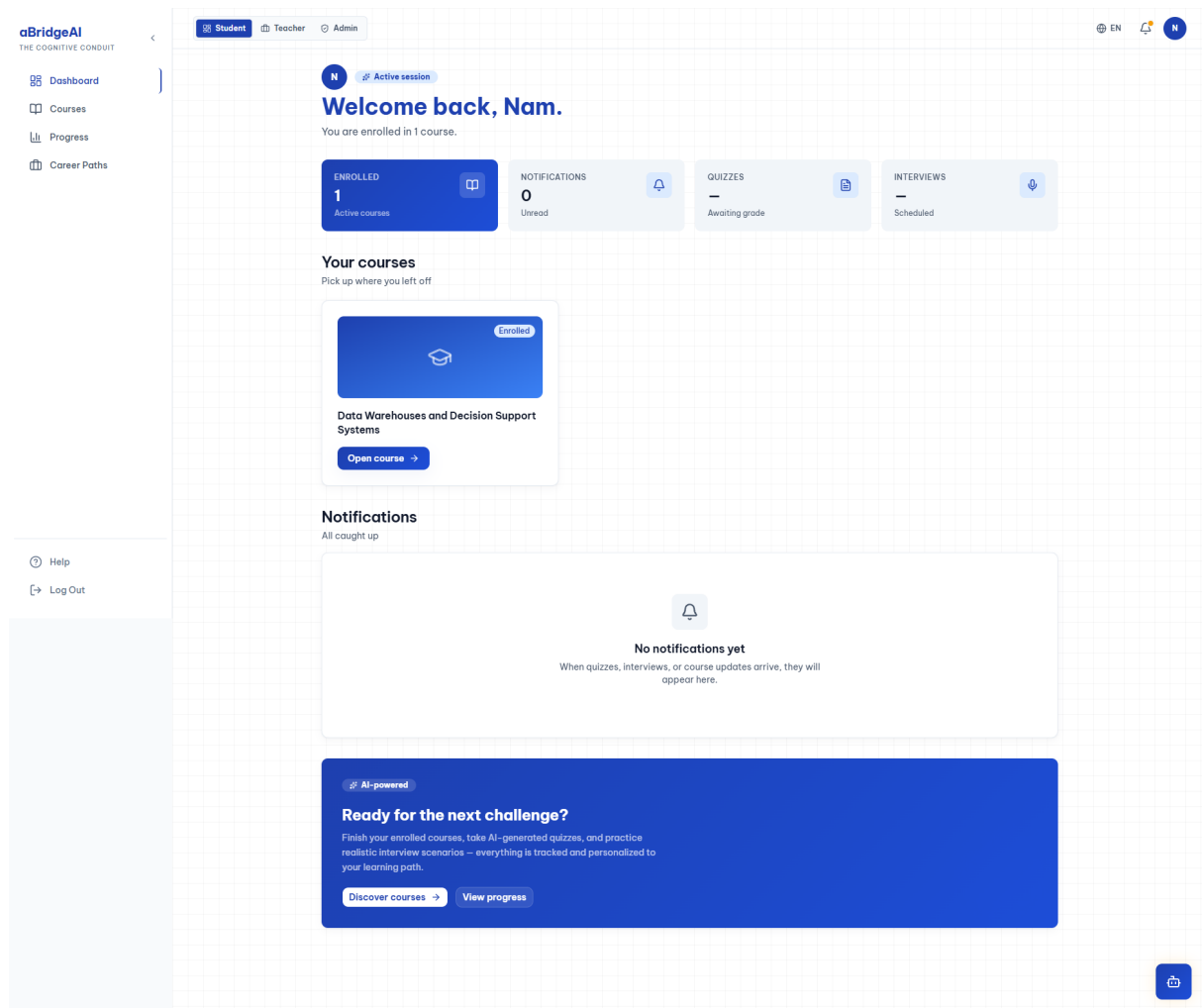


Figure H.3: Student Dashboard — Overview of enrolled courses, due reviews, and recent activity.

The student dashboard surfaces only the information needed to decide what to study next. Stat cards summarise enrolled courses, in-progress lessons and recently earned badges; a recommendations strip suggests the next course in the active career path. The layout is intentionally calm — there is no AI orb or floating action button — to keep students focused on lesson work rather than ambient prompts.

Course Catalog

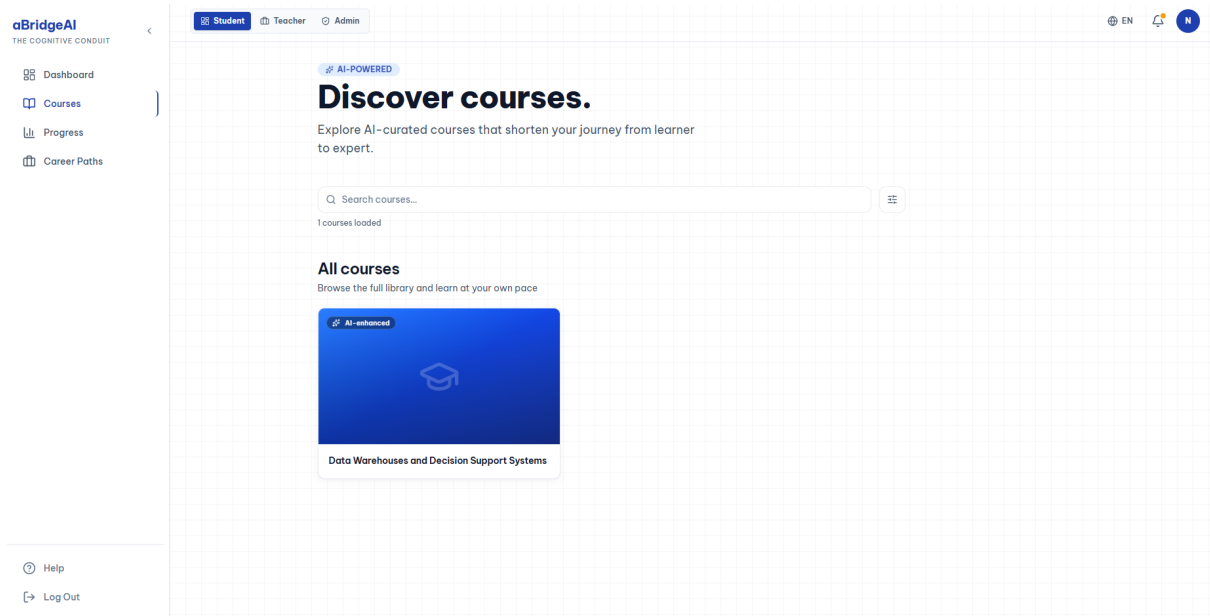


Figure H.4: Course Catalog — Search, filter and enrollment status across the full course library.

The catalog opens with a sticky search and filter bar (query, category, difficulty), followed by a card grid that shows enrollment status and progress inline. Already-enrolled courses display a progress ring so students can pick up where they left off without leaving the catalog page.

Course View

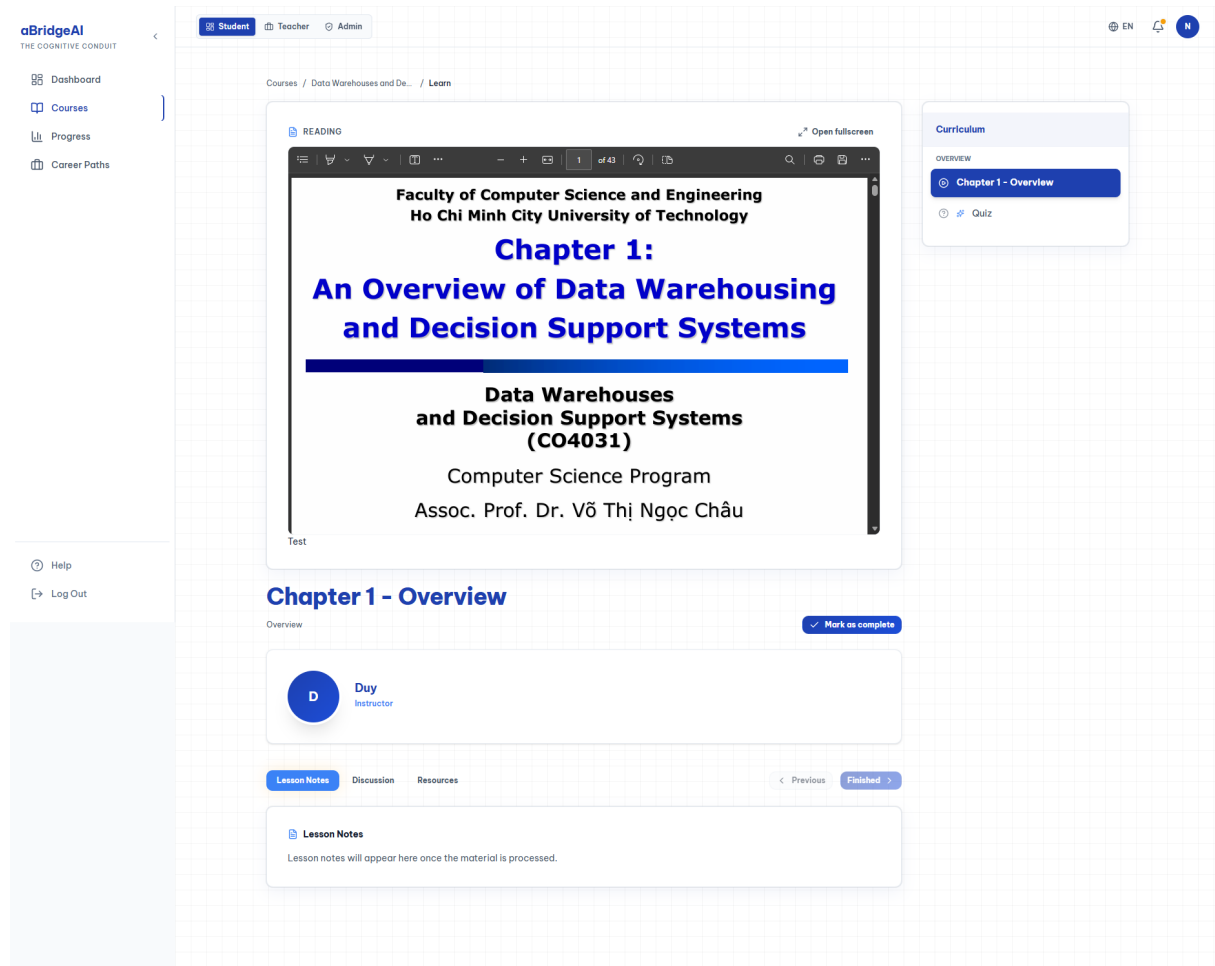


Figure H.5: Course View — Curriculum accordion with lesson state icons.

The course view is the primary learning environment. The left panel hosts a sticky progress card and a curriculum accordion; each lesson displays its completion state (completed / active / locked / pending) via iconography. The right panel renders the active lesson's content — video, reading, attached resources, or an embedded quiz launcher.

Quiz Session

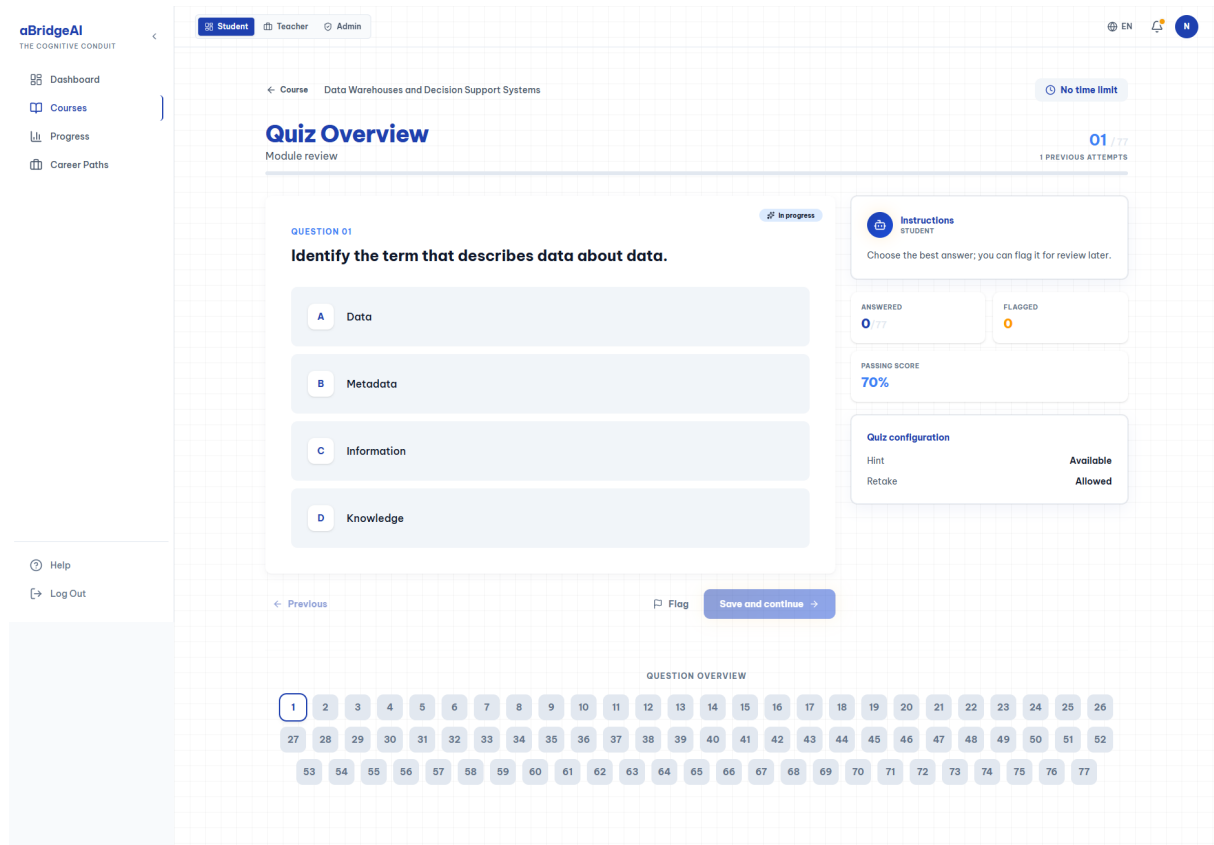


Figure H.6: Quiz Session — Timed multiple-choice interface with question navigation.

The quiz session is a focused, distraction-free interface. A countdown timer at the top creates temporal urgency; the bottom question grid allows non-linear navigation between flagged or unanswered items. After submission, students are redirected to the attempt-review screen at `/courses/$slug/quiz/$quizId/attempts/$attemptId`, which renders correct answers and per-question explanations.

Teacher Dashboard

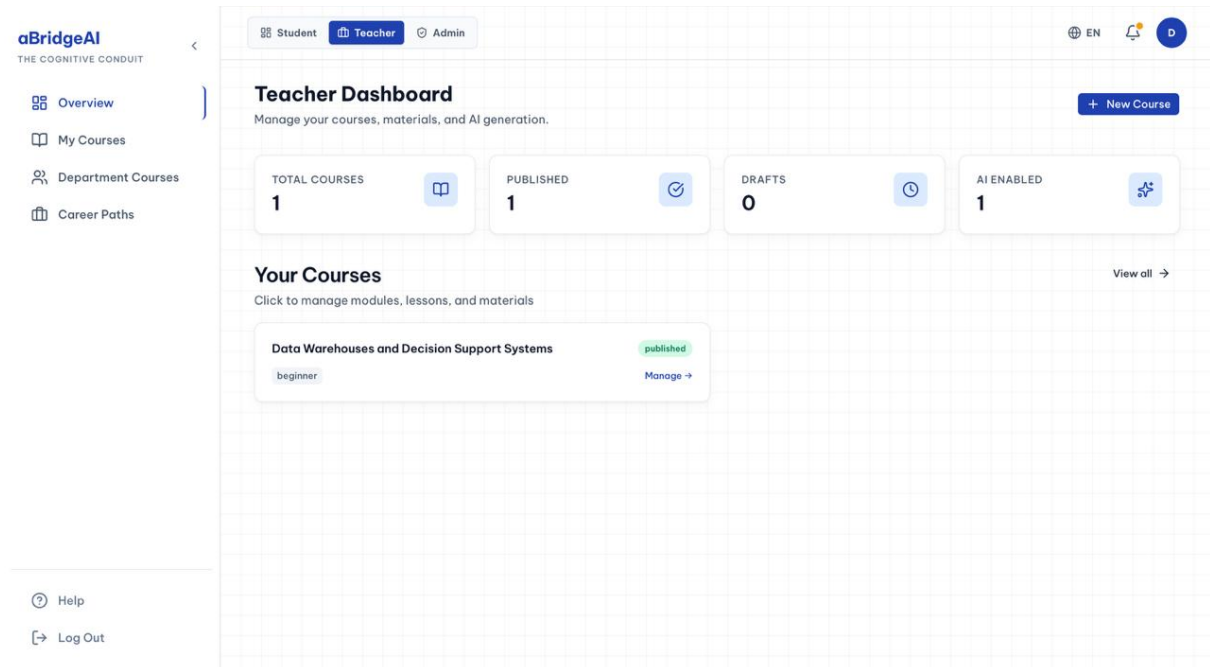


Figure H.7: Teacher Dashboard — Operational overview with course health stats and quick course creation.

The teacher dashboard is the entry screen for the `/teacher` route group. Four stat cards summarise the instructor’s authoring activity — total courses, published, drafts, and AI-enabled — followed by a “Your Courses” strip that surfaces the most recently edited course with its difficulty tag and publish state. A primary + **New Course** button anchors the top-right corner, and a **Manage** link on each course card jumps directly into the course-management view. The role switcher (Student / Teacher / Admin) at the top of the page is a development affordance for users that hold multiple roles; production accounts only see the tabs they are entitled to.

Course Management

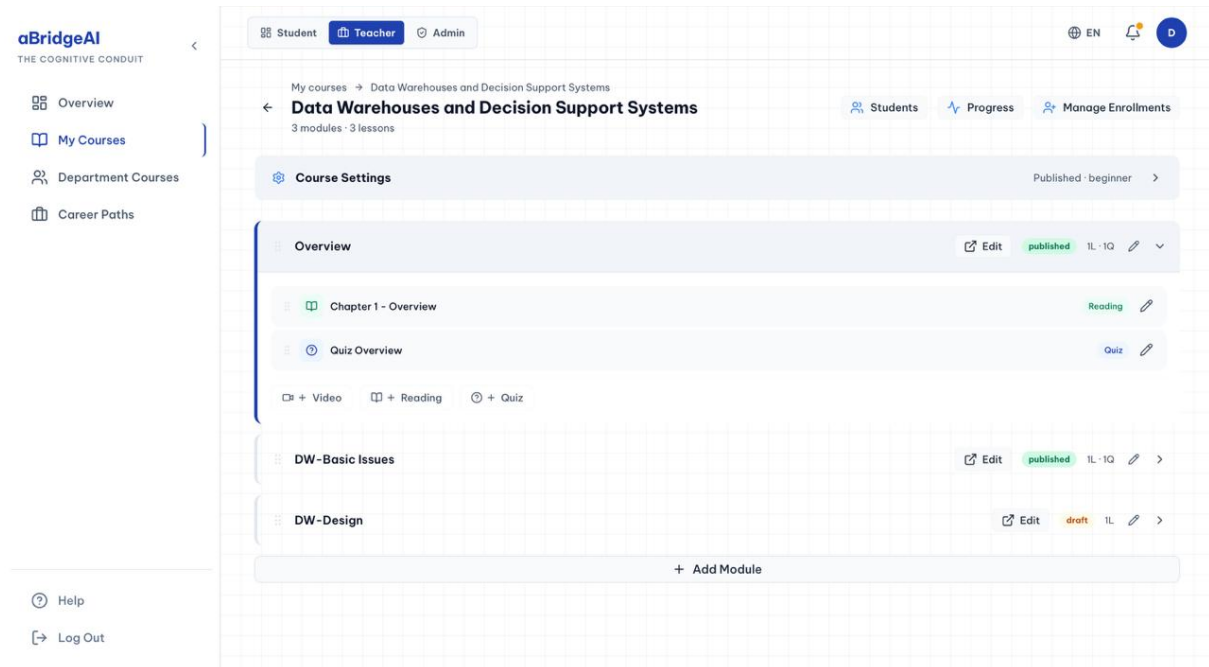


Figure H.8: Course Manage — Module and lesson tree with drag-handle reordering and inline quick-add.

The course-management page (`/teacher/courses/$courseId`) is the authoring core of the platform. Modules render as collapsible cards in a drag-handle list; expanding a module reveals its ordered lesson and quiz items, each carrying a publish-state badge. Inline **+ Video**, **+ Reading** and **+ Quiz** buttons let the instructor grow the curriculum without leaving the page. A right-hand *Course Settings* panel exposes the publish toggle, difficulty, and Students / Progress / Manage Enrollments quick links. A bottom **+ Add Module** button extends the structure.

Lesson Editor

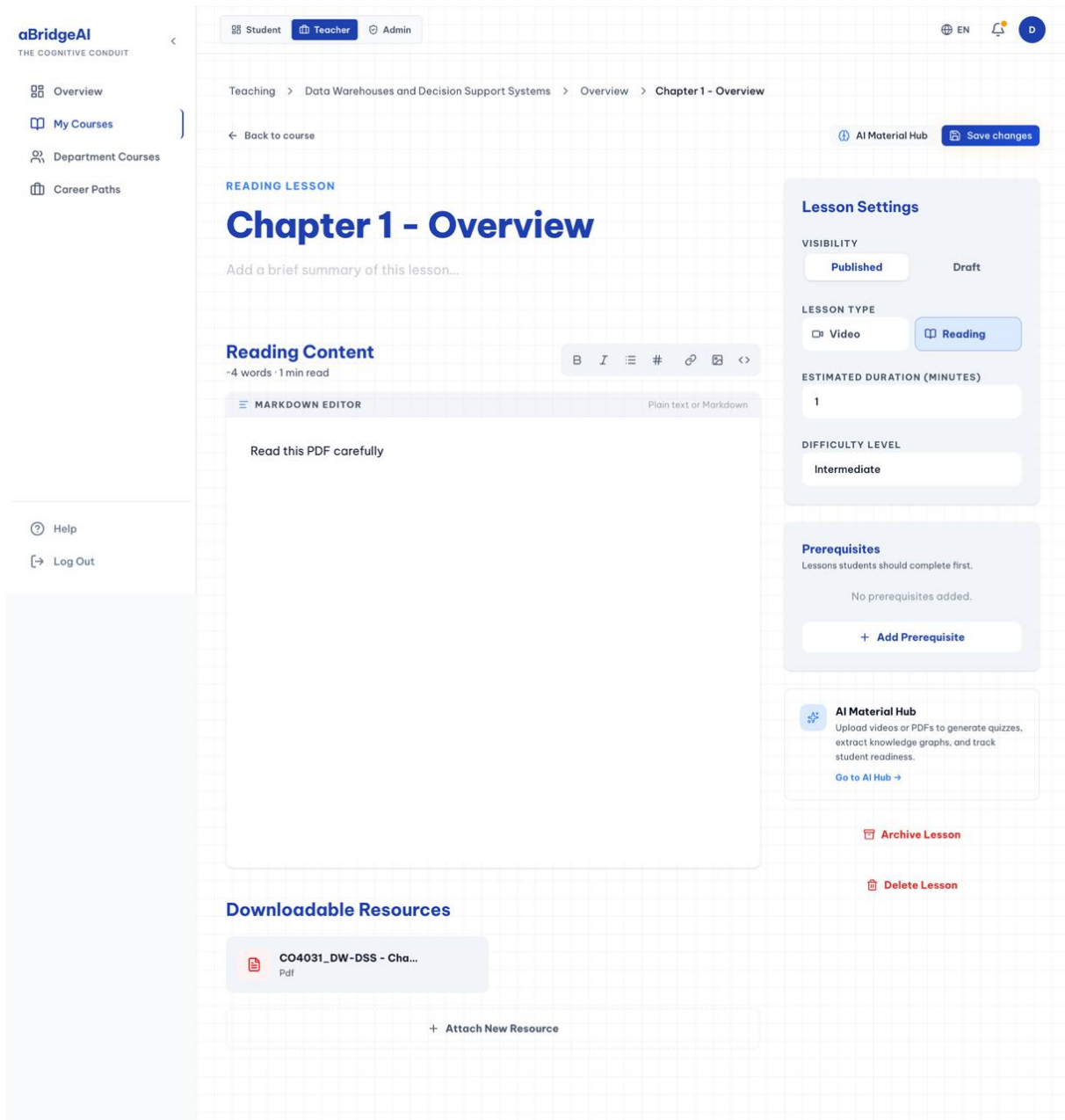


Figure H.9: Lesson Editor — Markdown content editor with attached resources and right-hand settings panel.

The lesson editor (`/teacher/courses/$courseId/lessons/$lessonId`) is split into a content area and a settings sidebar. The left side shows a Markdown editor for reading lessons (with bold, italic, list, heading, link, image, and code controls) and a *Downloadable Resources* block for attached PDFs and other files. The right sidebar exposes Visibility

(Published / Draft), Lesson Type (Video / Reading), Estimated Duration, Difficulty Level, and a Prerequisites picker. An *AI Material Hub* card promotes uploading source material so the platform can auto-generate quizzes from it. Destructive actions (Archive, Delete) are pushed to the bottom of the sidebar to reduce mis-clicks. A persistent **Save changes** button anchors the top-right corner.

Quiz Editor

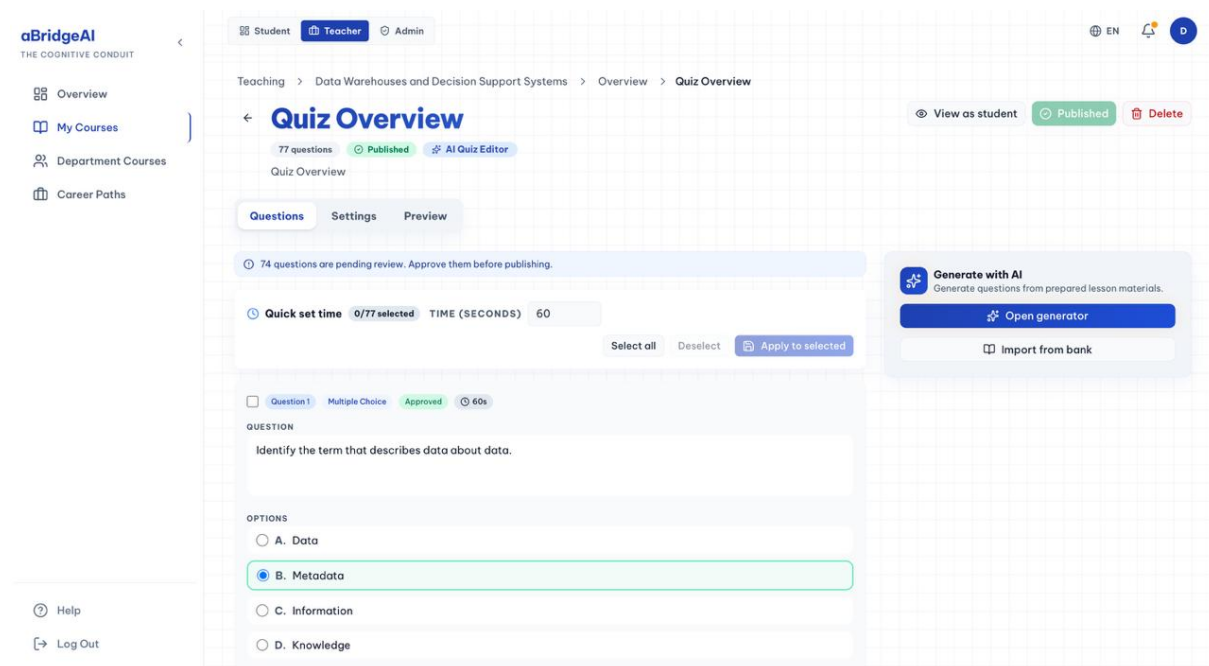


Figure H.10: Quiz Editor — Question list with bulk timing controls and an AI-generation side panel.

The quiz editor (`/teacher/courses/$courseId/quizzes/$quizId`) is structured around three sub-tabs: **Questions** (active by default), **Settings**, and **Preview**. The Questions tab lists every item with its type badge (Multiple Choice, True/False, ...), per-question time limit, and approval status. A bulk-action toolbar lets instructors select multiple questions and apply a uniform time limit, approve in bulk, or delete. A persistent banner reports how many questions are pending review (e.g. “74 questions are pending review — approve them before publishing”), enforcing a deliberate approval gate between AI generation and student exposure.

AI Quiz Generation

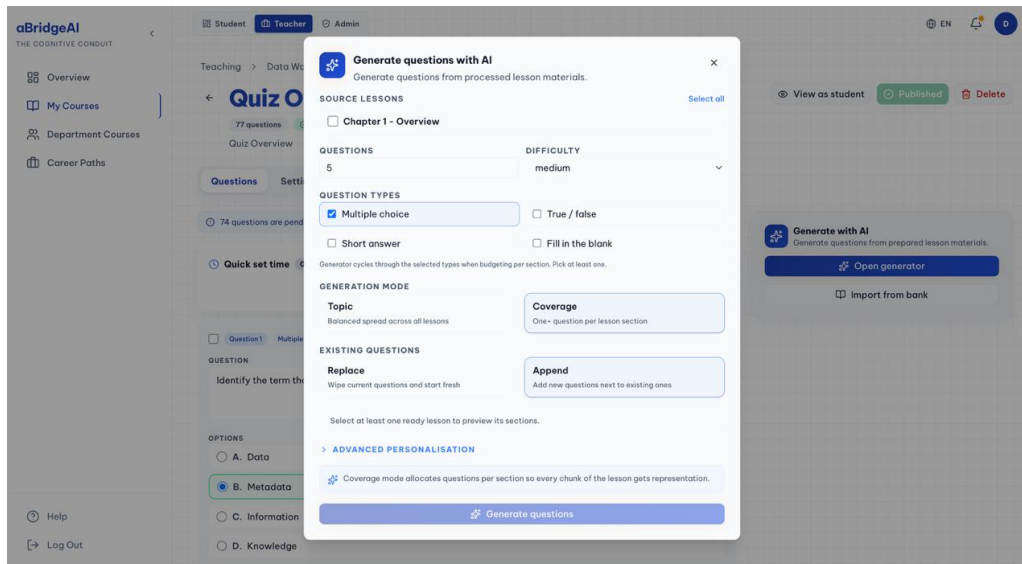


Figure H.11: AI Quiz Generation modal — Source-lesson selection, question count, mode and append/replace controls.

The **Open generator** button on the quiz editor opens a modal that drives the AI quiz pipeline. The instructor selects one or more source lessons (only lessons whose materials have been processed are eligible), the desired number of questions, the difficulty band, and the question types (Multiple choice, True/False, Short answer, Fill in the blank). A generation-mode selector trades off two strategies: *Topic* produces a balanced spread across all selected lessons, while *Coverage* guarantees at least one question per section. An existing-questions selector decides between *Replace* (wipe and regenerate) and *Append* (add to the current bank). An advanced personalisation block, collapsed by default, exposes per-attempt prompt controls. Generated questions land in the editor in a *Pending Review* state and only become visible to students after explicit instructor approval.

Processing Queue

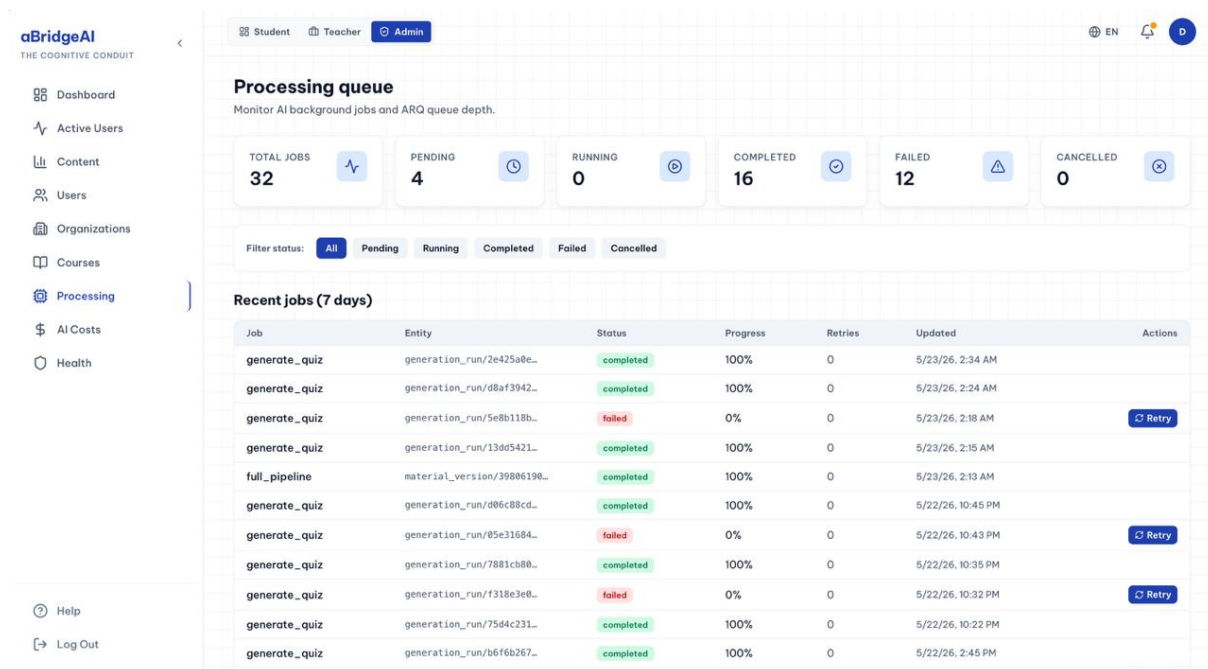


Figure H.12: Admin Processing Queue — Background-job visibility with retry actions for failed runs.

The processing queue (`/admin/processing`) gives platform administrators direct visibility into the AI background-job pipeline (material ingestion, embedding, quiz generation, validation). Stat cards report the current backlog (Total, Pending, Running, Completed, Failed, Cancelled). A status-pill filter (*All*, *Pending*, *Running*, *Completed*, *Failed*, *Cancelled*) narrows the table, which lists the job type (`generate_quiz`, `full_pipeline`, ...), the target entity ID, status, progress, retry count, last update, and a per-row **Retry** action for failed jobs. This is the operations console for the worker tier; it complements the per-course *AI Material Hub* indicator visible to instructors.

AI Costs

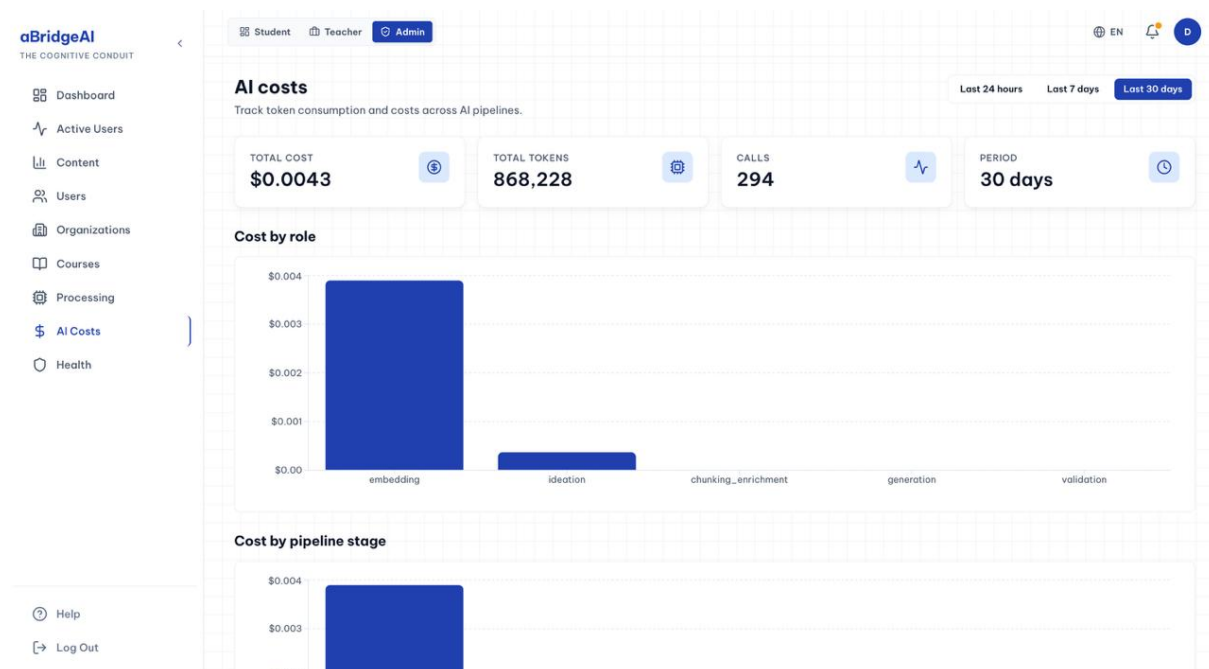


Figure H.13: Admin AI Costs — Token consumption and spend across pipeline roles and stages.

The AI Costs page (`/admin/ai-costs`) exposes platform-wide spend on language-model and embedding APIs. A time-range selector (Last 24 hours, Last 7 days, Last 30 days) refilters every metric on the page. Summary cards report Total Cost, Total Tokens, total API Calls, and the active period. Two bar charts break the spend down by *role* (embedding, ideation, chunking enrichment, generation, validation) and by *pipeline stage*. The chart series reuse the platform palette (academic blue + amber) so administrators can correlate cost with backlog health on the Processing page without a context switch.

H.1.5 Navigation Architecture

Table H.4: Navigation Components by Context

Component	Context	Function
TopNavBar	Landing page only	Horizontal marketing nav: Explore, Login, Sign Up. Distinct from the authenticated shell.
SideNavBar	Authenticated (all)	Always rendered. Collapses to <code>w-16</code> (icons only) below the <code>md</code> breakpoint or by user toggle; expands to <code>w-64</code> with full labels. Receives a role-specific <code>navItems</code> array as a prop.
ContentTopBar	Authenticated (all)	Sticky page header carrying the current page title, language selector (<code>i18n</code>), notification bell, and user avatar with menu.
AppShell	Authenticated (all)	Orchestrates <code>SideNavBar</code> + <code>ContentTopBar</code> + main content into a responsive layout. Receives a <code>navItems</code> array, making it fully reusable across student, teacher, and admin route groups with no per-role layout code.
Footer	Landing page only	Marketing footer with product links and legal copy.

The `AppShell` component is the structural keystone of the authenticated experience. The shell itself is role-agnostic; role differentiation is communicated through the `navItems` prop (defined in `src/lib/navigation.ts`) and through the active accent colour on the sidebar. Below the `md` breakpoint the sidebar overlays the page content via a backdrop rather than pushing it, keeping the main column readable on narrow screens.

H.1.6 Design System Summary

Table H.5: Design System Tokens

Token Category	Value	Usage
Page background	#f8fafc (slate-50)	Page surface in light mode.
Surface / Card	#ffffff	Cards, panels, dialogs.
Primary accent	#1e40af (blue-800)	Primary actions, active nav, links, chart series 1.
Secondary accent	#3b82f6 (blue-500)	Secondary buttons, focus rings, chart series 2.
CTA / accent	#f59e0b (amber-500)	High-attention CTAs, AI affordances.
Success	#16a34a (green-600)	Published / completed state.
Warning	#ea580c (orange-600)	Drafts, pending review.
Danger	#dc2626 (red-600)	Failed jobs, destructive actions.
Border	#e2e8f0 (slate-200)	1 px borders on cards, inputs, tables.
Typography	Be Vietnam Pro (system-ui fallback)	All body and UI text; full Vietnamese diacritic support.
Component library	shadcn/ui + Radix UI	Dialogs, dropdowns, sheets, tooltips.
Icons	Lucide React	Single iconography source across the platform.
Charts	Recharts	Bar, line, area; reuses palette for chart series 1–5.
i18n keys	<code>src/lib/navigation.ts</code> + <code>src/locales/<lang>.json</code>	Translatable nav labels and copy.
Max modal depth	1 stacked modal	Prevents nested-modal anti-patterns.
Base spacing unit	4 px	All spacing is a multiple of 4 px.

Table H.5 (*continued*)

Token Category	Value	Usage
Border radius (card)	8 px (rounded-1g)	Consistent card rounding; 10 px hard cap project-wide.
Sidebar widths	64 px collapsed / 256 px expanded	w-16 / w-64 ; collapse driven by viewport width and user toggle.

Appendix I Test Case Specifications

Authentication and Access Control

Table I.1: Test Cases: Authentication & Access Control

ID	Req	Level	Input	Expected Result	Exception
TC-1.1.1	FR-1.1	System	User initiates login using Google SSO	System redirects the user to Google OAuth and authenticates the user after a successful callback	Login fails if the OAuth callback is invalid or the Google account is not authorised
TC-1.1.2	FR-1.1	System	Authenticated user clicks logout	User is logged out and redirected to the login page	Logout remains idempotent if the session has already expired
TC-1.2.1	FR-1.2	System	User completes Google SSO successfully	System creates an authenticated session and issues valid access and refresh tokens	Session is not created if user identity cannot be mapped or provisioned
TC-1.2.2	FR-1.2	Integration	User sends a request with an expired access token and valid refresh token	System refreshes the session and rotates the refresh token	Refresh is rejected if the refresh token is expired, revoked, or reused

Continued on next page

Table I.1 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-1.2.3	FR-1.2	System	IT administrator disables an active user account	All active sessions for that account are revoked and protected resources become inaccessible	Historical user data remains preserved after account disablement
TC-1.3.1	FR-1.3	System	User with MFA enabled logs in and enters a valid authenticator-app code	MFA verification succeeds and the user can access protected features	Invalid or expired verification codes are rejected
TC-1.3.2	FR-1.3	System	User enters a valid one-time recovery code	Recovery code is accepted and marked as used	Used recovery codes cannot be reused
TC-1.4.1	FR-1.4	System	User attempts to regenerate recovery codes after recent MFA verification	Operation is allowed and new recovery codes are generated	Operation is blocked if MFA verification is outside the freshness window
TC-1.4.2	FR-1.4	System	User attempts to disable MFA without recent second-factor verification	System requires re-verification before allowing the operation	MFA cannot be disabled through direct API bypass
Continued on next page					

Table I.1 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-1.5.1	FR-1.5	System	User accesses a protected backend route with sufficient permission	Request is allowed and the protected resource is returned	Request is denied if the user lacks the required permission
TC-1.5.2	FR-1.5	System	Frontend renders actions for a user with limited permissions	UI hides or disables actions outside the user's effective permissions	Hidden actions remain protected server-side if called directly
TC-1.6.1	FR-1.6	System	User performs login, logout, and MFA verification actions	System records corresponding auditable events	Audit logging failure does not expose sensitive authentication data
TC-1.6.2	FR-1.6	System	Administrator changes account-security or access-control settings	System records actor, action, target, and timestamp in the audit log	Audit records cannot be modified by non-authorised users

Organization and User Management

Table I.2: Test Cases: Organization & User Management

ID	Req	Level	Input	Expected Result	Exception
TC-2.1.1	FR-2.1	System	IT administrator provisions a new tenant organisation with required details and an initial Educational Manager	Organisation is created and the selected user receives the initial manager role	Provisioning fails if required organisation information is missing or invalid
TC-2.2.1	FR-2.2	System	IT administrator updates an existing organisation's basic information	Updated information is saved and reflected in organisation views	Duplicate or invalid organisation metadata is rejected
TC-2.2.2	FR-2.2	System	IT administrator changes an organisation status to inactive, archived, or soft-deleted	Organisation lifecycle state is updated and access rules follow the new state	Invalid lifecycle transitions are rejected
TC-2.3.1	FR-2.3	System	Educational Manager uploads a CSV containing valid and invalid student rows	Valid student accounts are created or enrolled, and invalid rows are reported without aborting the whole batch	Duplicate or malformed email rows are listed in the validation report

Continued on next page

Table I.2 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-2.4.1	FR-2.4	System	Educational Manager creates a career pathway with an ordered sequence of courses	Career pathway is saved and course order is preserved	Pathway creation is rejected if required course references are invalid
TC-2.4.2	FR-2.4	System	Educational Manager reorders courses within a pathway	New order is saved and reflected in pathway views	Reordering fails if the manager does not belong to the organisation
TC-2.5.1	FR-2.5	System	Educational Manager assigns a Teacher to a course within their organisation	Teacher is linked to the course and can access authoring features	Assignment fails if the teacher is inactive or outside the organisation
TC-2.6.1	FR-2.6	System	Educational Manager queries users, courses, or reports from their own organisation	System returns only data within the manager's tenant organisation	Cross-tenant records are not returned
TC-2.6.2	FR-2.6	System	Educational Manager attempts to access another tenant's data via direct URL or API call	Access is denied	No partial cross-tenant data is leaked
Continued on next page					

Table I.2 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-2.7.1	FR-2.7	Integration	New account is provisioned using an email domain registered to an organisation	Account is automatically associated with the matching organisation	Account remains unassigned or requires manual review if no matching domain exists
TC-2.8.1	FR-2.8	System	Authenticated user updates display name, avatar, bio, or external links	Profile changes are saved and reflected in the user's account page	Invalid external link formats are rejected
TC-2.9.1	FR-2.9	System	IT administrator adds, removes, or toggles auto-provisioning for an organisation email domain	Domain settings are updated and used for future account provisioning	Duplicate domain registration is rejected
TC-2.10.1	FR-2.10	System	Authorised manager updates a user's organisation membership and identifier code	Membership record is saved and visible in organisation membership views	Update is denied if the manager lacks organisation-management permission
TC-2.11.1	FR-2.11	System	IT administrator disables a user account across tenant organisations	Disabled user can no longer access protected system resources	User records and historical activity remain preserved

Course and Material Management

Table I.3: Test Cases: Course & Material Management

ID	Req	Level	Input	Expected Result	Exception
TC-3.1.1	FR-3.1	System	Teacher creates a course in draft state with required metadata	Course is saved as draft and is not visible to students until published	Course creation fails if required metadata is missing
TC-3.1.2	FR-3.1	System	Authorised user archives or restores a soft-deleted course	Course lifecycle state is updated correctly	Restore fails if the user lacks administrative permission
TC-3.2.1	FR-3.2	System	Teacher creates ordered modules and learning items with prerequisites	Structure is saved and displayed in the configured order	Circular or invalid prerequisite rules are rejected
TC-3.3.1	FR-3.3	System	Teacher uploads a supported learning material using a standard or resumable upload flow	Material is stored, linked to the lesson, and queued for processing	Unsupported formats or failed uploads are rejected with an error message
TC-3.3.2	FR-3.3	System	Teacher re-runs processing for an existing material	Processing pipeline is restarted and new processed artefacts are generated	Re-run is blocked if the material is missing or inaccessible
Continued on next page					

Table I.3 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-3.4.1	FR-3.4	System	Teacher uploads a replacement version of an existing material	New version becomes current while prior versions remain available for rollback or re-processing	Invalid replacement files do not overwrite the current version
TC-3.5.1	FR-3.5	System	Teacher views ingestion status for a material version	System displays processing status, percent completion, and failure reason if applicable	Failed processing jobs show actionable error information
TC-3.6.1	FR-3.6	System	Teacher marks a material as hidden while the course remains published	Students cannot access the hidden material	Teachers can still view hidden material in authoring mode
TC-3.7.1	FR-3.7	System	Enrolled student browses published courses and opens approved material	Student can view course details and stream or download approved material via a pre-signed URL	Unpublished or invisible materials are not accessible
TC-3.8.1	FR-3.8	System	IT administrator opens course processing history and aggregate statistics	System displays processing jobs, status history, and aggregate course statistics	Access is denied to users without administrative permission

Learning Loop and Adaptive Quiz

Table I.4: Test Cases: Learning Loop & Adaptive Quiz

ID	Req	Level	Input	Expected Result	Exception
TC-4.1.1	FR-4.1	Integration	Teacher drafts quiz questions from ingested course content	Questions are generated from source content, retain source references, and enter review state	Questions without valid source references are flagged for review
TC-4.1.2	FR-4.1	System	Teacher edits, approves, rejects, or reuses quiz questions	Question state and revision history are updated correctly	Students cannot access unapproved questions
TC-4.2.1	FR-4.2	System	Teacher configures quiz settings and sets T_{exp} for every live question	Quiz can be published and settings are applied to student attempts	Publication is blocked if any live question has no T_{exp}
TC-4.3.1	FR-4.3	System	Student submits a quiz answer with response time and optional hint usage	System records correctness, response time, and hint usage for the answer	Attempts are blocked when retake policy or cooldown rules are violated
TC-4.4.1	FR-4.4	Unit	System receives correctness, hint usage, T_{actual} , and T_{exp} for a card response	System derives the correct Q value using the configured mapping	Invalid or missing timing data is rejected or handled by a defined fallback
Continued on next page					

Table I.4 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-4.4.2	FR-4.4	Integration	Card response produces a derived Q value	Student-card SM-2 state is updated with EF, interval, due date, calibration, and jitter rules	EF never falls below the configured floor
TC-4.5.1	FR-4.5	System	Student satisfies prerequisite completion and EF coverage threshold τ	Dependent lesson becomes accessible	Lesson remains locked if prerequisite or coverage threshold is not satisfied
TC-4.6.1	FR-4.6	Integration	A student's card becomes due for review	System delivers an in-app spaced-repetition reminder	Notification is not delivered if the card has already been reviewed
TC-4.6.2	FR-4.6	Integration	Student fails a card linked to source material	System sends remediation notification linking to relevant material when available	Notification omits material link if no reliable source reference exists
TC-4.7.1	FR-4.7	System	Teacher opens learning-loop analytics for a course	System displays cohort retention, difficult cards, at-risk students, and per-student SR details	Empty state is shown if no quiz activity exists
Continued on next page					

Table I.4 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-4.8.1	FR-4.8	System	Student opens personal learning dashboard	System displays due cards, retention estimate, progression readiness, and course-level progress	Dashboard handles courses with no due cards gracefully

Application Loop and Mock Interview

Table I.5: Test Cases: Application Loop & Mock Interview

ID	Req	Level	Input	Expected Result	Exception
TC-5.1.1	FR-5.1	System	Teacher defines desired interview outcomes without model answers	Interview section is saved with outcome-based assessment criteria	Publication is blocked if no desired outcomes are defined
TC-5.2.1	FR-5.2	Integration	Teacher drafts interview questions from ingested course content	Questions are associated with desired outcomes and enter teacher review	Questions not mapped to any desired outcome are flagged
TC-5.2.2	FR-5.2	System	Teacher edits, approves, or rejects drafted interview questions	Reviewed question set is saved for publication	Students cannot access rejected or unpublished questions
Continued on next page					

Table I.5 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-5.3.1	FR-5.3	System	Teacher configures maximum attempts, cooldown, and quiz-progression lock	Interview rules are saved and enforced for student sessions	Invalid attempt limits or cooldown values are rejected
TC-5.4.1	FR-5.4	System	Eligible student starts a text-mode interview	AI interviewer begins adaptive turn-taking dialogue	Student is denied entry if interview prerequisites are not satisfied
TC-5.4.2	FR-5.4	System	Eligible student starts a voice-mode interview	Voice-mode interview starts and supports turn-taking interaction	System falls back or shows guidance if microphone permission is denied
TC-5.5.1	FR-5.5	Integration	Student speaks during a voice-mode interview	Audio is streamed, transcribed, and used to generate spoken AI responses	Raw audio is not persistently archived after processing
TC-5.6.1	FR-5.6	Integration	Student completes an interview session	Transcript and metadata are persisted, and system produces a binary pass-or-fail verdict based on desired outcomes	Evaluation is not triggered for incomplete sessions
Continued on next page					

Table I.5 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-5.7.1	FR-5.7	System	Student opens completed interview result	Student sees only the binary pass-or-fail verdict	Per-outcome rubric scores are not exposed to the student
TC-5.7.2	FR-5.7	System	Teacher opens a completed interview session	Teacher can review transcript and outcome-level assessment for remediation	Teacher access is denied for courses they do not manage
TC-5.8.1	FR-5.8	System	Student switches tabs, exits fullscreen, or changes browser focus during interview	System records session activity signals for post-session integrity review	Integrity signals do not directly expose private non-session content

Analytics, Reporting and Notifications

Table I.6: Test Cases: Analytics, Reporting & Notifications

ID	Req	Level	Input	Expected Result	Exception
TC-6.1.1	FR-6.1	System	Student opens course progress page after viewing materials and completing lessons	System displays per-lesson and per-course completion progress separately from SR metrics	Courses with no activity show zero progress instead of an error
Continued on next page					

Table I.6 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-6.2.1	FR-6.2	System	Teacher opens course roster progress view	System displays roster progress and at-risk learners based on course-progress signals	Empty state is shown if no learners have activity
TC-6.3.1	FR-6.3	System	User opens notification inbox with unread notifications	System displays unread count and allows mark-read and dismiss actions	Dismissed notifications no longer appear in the active inbox
TC-6.3.2	FR-6.3	System	User updates notification preferences by category and channel	Preferences are saved and applied to future notifications	Invalid channel/category combinations are rejected
TC-6.4.1	FR-6.4	System	Authorised administrator opens statistics dashboard	System displays active users, content volume, and system-health indicators according to the caller's scope	Non-authorised users cannot access administrative statistics
TC-6.5.1	FR-6.5	System	IT administrator opens AI usage dashboard	System displays usage summaries by user, pipeline, and recent model calls	Missing cost data is shown as unavailable rather than zero if not computed
Continued on next page					

Table I.6 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-6.6.1	FR-6.6	System	Administrator opens failed ingestion or AI processing jobs and selects retry	Failed job details are shown and retry is queued successfully	Retry is blocked if the underlying material or configuration no longer exists
TC-6.7.1	FR-6.7	System	IT administrator queries audit log by actor, event type, or date range	Matching audit events are returned without allowing modification	Non-authorised users cannot read audit logs
TC-6.8.1	FR-6.8	System	Educational Manager opens Career Readiness report for a pathway	System displays organisation-scoped pathway outcomes and drill-down to individual progress	Per-interview rubric details are excluded from the manager report

Non-Functional Requirement Test Cases

Table I.7: Test Cases: Non-Functional Requirements

ID	Req	Category	Input	Expected Result	Exception
TC-NFR-1.1	NFR-1.1	Performance	Upload a 15-minute video lecture and start ingestion	Video transcription and indexing complete within approximately 10 minutes on average	Processing failure or unsupported video format is reported to the teacher
Continued on next page					

Table I.7 – Continued from previous page

ID	Req	Category	Input	Expected Result	Exception
TC-NFR-1.2	NFR-1.2	Performance	Teacher generates quiz questions for a lesson with processed materials	Quiz generation completes within 5 minutes on average	Generation failure is logged and the teacher can retry
TC-NFR-1.3	NFR-1.3	Performance	Teacher generates an interview question bank for a lesson with defined target outcomes	Question bank generation completes within 15 minutes and produces at least 20 candidate questions covering all defined outcomes	Missing or invalid target outcomes prevent generation
TC-NFR-1.4	NFR-1.4	Performance	User opens major dashboard pages under normal network conditions	Dashboard pages load within 2 seconds on average	Slow external services degrade gracefully without breaking page rendering
TC-NFR-1.5	NFR-1.5	Performance	User starts playback of a course video	Video streaming starts within 3 seconds on average	Playback failure displays a clear error message
TC-NFR-1.6	NFR-1.6	Performance	User performs a course or material search query	Search results are returned within 0.5 seconds on average	Empty result set is displayed without error
Continued on next page					

Table I.7 – Continued from previous page

ID	Req	Category	Input	Expected Result	Exception
TC-NFR-2.1	NFR-2.1	Load	Simulate 50 active users, 10 video streams, and 5 AI interview sessions concurrently	System remains usable without major Performance degradation	Excess requests are handled without crashing core services
TC-NFR-2.2	NFR-2.2	Load	Populate the system with 10GB of course content and 10,000 student records	Storage remains efficient and common database queries respond under 0.5 seconds on average	Large queries are paginated or rejected safely
TC-NFR-2.3	NFR-2.3	Reliability	Temporarily disable the AI interview module while other services remain active	Core LMS features such as video streaming and course access continue operating	Affected AI features display an unavailable status instead of triggering a system-wide failure
TC-NFR-2.4	NFR-2.4	Reliability	Trigger a transient network or AI service failure during background processing	System retries or fails gracefully according to retry policy	Permanent failure is recorded with an error reason
TC-NFR-3.1	NFR-3.1	Security	Inspect stored sensitive data, uploaded materials, credentials, and tokens	Sensitive data is encrypted or securely hashed according to specification	Plaintext secrets or sensitive values are treated as test failures
Continued on next page					

Table I.7 – Continued from previous page

ID	Req	Category	Input	Expected Result	Exception
TC-NFR-3.2	NFR-3.2	Security	Users with different roles attempt to access restricted resources	Access follows least-privilege RBAC rules	Unauthorised direct API access is denied
TC-NFR-3.3	NFR-3.3	Security	Perform sensitive actions including login, permission change, and data modification	Immutable audit log records actor, timestamp, action type, and affected resource	Audit records cannot be edited by unauthorised users
TC-NFR-3.4	NFR-3.4	Security	Leave an authenticated session idle for 30 minutes	System automatically logs the user out and requires re-authentication	Active interaction correctly resets the idle timer
TC-NFR-4.1	NFR-4.1	Usability	Open the system on desktop, tablet, and mobile screen sizes	UI remains responsive and visually consistent across supported layouts	Critical actions remain accessible on smaller screens
TC-NFR-4.2	NFR-4.2	Accessibility	Navigate major pages using keyboard-only navigation and inspect contrast ratios and alt text	Interface satisfies key WCAG 2.1 AA checks including keyboard access, contrast, and image alternatives	Non-compliant components are recorded for accessibility remediation
Continued on next page					

Table I.7 – Continued from previous page

ID	Req	Category	Input	Expected Result	Exception
TC-NFR-4.3	NFR-4.3	Compatibility	Open the system using supported Chromium-based browsers	System functions correctly on Chrome, Edge, Brave, and Opera	Unsupported browsers display a compatibility warning if major features fail